CuTe 布局的范畴论基础

Categorical Foundations for CuTe Layouts（中文译注版）

Jack Carlisle Jay Shah Reuben Stern Paul VanKoughnett

Colfax Research

2026 年 1 月（原文） 中文译注版 2026-09

摘要

NVIDIA 的 CUTLASS 库提供了一套健壮且富有表达力的方法，用于在 GPU 上描述与操纵多维张量数据。这些方法在概念上植根于 CuTe 布局（layout）这一抽象概念，以及关于布局的一整套丰富代数，其中包括复合（composition）、逻辑乘积（logical product）与逻辑切分（logical division）等运算。本文为理解这套布局代数提出一个范畴论框架，其核心是一类自然出现的*可处理布局*（tractable layouts）。为此，我们定义了两个范畴 **Tuple** 与 **Nest**，它们的态射可以生成布局。我们在这些范畴的态射上定义了一组运算，并证明这些运算与相应的布局运算相容。此外，我们对由该构造产生的布局给出了完整的刻画。最后，我们给出了上述范畴构造的一个 Python 实现，以及验证其与 CUTLASS 行为对齐的测试。该实现位于代码仓库 `https://github.com/ColfaxResearch/layout-categories`。

译序

本译注版译自 Colfax Research 技术报告《Categorical Foundations for CuTe Layouts》(Jack Carlisle、Jay Shah、Reuben Stern、Paul VanKoughnett, arXiv:2601.05972, 2026 年 1 月) 的官方 LaTeX 源。该文从范畴论 (category theory) 视角对 CUTLASS/-CuTe 中的布局（layout）代数给出严格刻画，是理解 CuTe 布局运算（复合、逻辑乘积、逻辑切分等）数学基础的进阶材料。

翻译说明：

- 数学公式、tikz 交换图、Python 代码清单全部保持原文原样，不做任何改动；
- 定理/引理/定义/构造/注等环境名称已中文化，编号规则与原文完全一致（定理按小节编号，主定理以大写字母 A、B、C…编号）；
- 关键术语在每章首次出现时以「中文（English）」标注；
- 术语译名与《LeetCUDA》一书第 20–26 章 CuTe 篇保持一致。

版权声明： 原文版权归原文作者与 Colfax Research 所有；原文的 Python 参考实现见 https://github.com/ColfaxResearch/layout-categories。本中文译注版仅供个人学习交流使用，未经原文作者与译者同意，不得用于任何商业传播。

术语表

英文	中文	备注
CuTe 布局代数（与《LeetCUDA》ch20 一致）		
layout	布局	
flat layout	扁平布局	
nested tuple	嵌套元组	
tractable layout	可处理布局	全文核心概念
hierarchy	层级	
profile	轮廓	
composition	复合	
logical product	逻辑乘积	
logical division	逻辑切分	
coalesce	合并	
relative coalesce	相对合并	
complement	补	
inverse	逆	左逆/右逆
concatenate	拼接	
flatten / flat	扁平化 / 扁平	
compact	紧凑	
codomain	陪域	
range	值域	陪域中的像
domain	定义域	
shape	形状	
stride	步长	
size / cosize	大小 / 余大小	
depth / rank	深度 / 秩	
mode	模式	
slice	切片	
sublayout	子布局	

英文	中文	备注
范畴论		
category	范畴	
functor	函子	
morphism	态射	
object	对象	
identity morphism	恒等态射	
isomorphism	同构	
natural transformation	自然变换	
monad	单子	
commutative diagram	交换图	
categorical product	范畴积	注意区别于 CuTe 的「乘积」
coproduct	余积	
terminal / initial object	终对象 / 始对象	
subcategory	子范畴	
full subcategory	满子范畴	
equivalence of categories	范畴等价	
adjunction / adjoint	伴随	
poset	偏序集	
monoid	幺半群	
comma category	逗号范畴	
skeleton	骨架	
forgetful functor	遗忘函子	
generator	生成元	

目录

译序 1

术语表 2

第一章 引言 6
1.1 主要结果综述 . . . 9
1.2 结构安排 . . . 12
1.3 相关工作 . . . 12
1.4 实现 . . . 14
1.5 记号 . . . 20

第二章 布局及其代数 22
2.1 扁平布局 . . . 22
2.1.1 元组 . . . 22
2.1.2 基本定义 . . . 24
2.1.3 基本运算 . . . 32
2.1.4 扁平合并 . . . 44
2.1.5 紧凑扁平布局 . . . 49
2.1.6 补 . . . 52
2.1.7 进一步的运算 . . . 65
2.1.8 可处理扁平布局 . . . 67
2.2 嵌套元组 . . . 70
2.2.1 轮廓 . . . 70
2.2.2 基本定义 . . . 73
2.2.3 代入 . . . 76
2.2.4 加细 . . . 77
2.3 布局 . . . 80
2.3.1 基本定义 . . . 80
2.3.2 基本运算 . . . 84
2.3.3 合并 . . . 86

2.3.4 相对合并 89
2.3.5 紧凑布局 91
2.3.6 补 93
2.3.7 复合 95
2.3.8 逻辑切分 96
2.3.9 逻辑乘积 99
2.3.10 可处理布局 101

第三章 布局的范畴 102
3.1 范畴 **Tuple** 102
3.1.1 基本定义 102
3.1.2 从元组态射到扁平布局 106
3.1.3 例子 115
3.1.4 元组态射的实现 120
3.1.5 元组态射上的运算 123
3.2 范畴 **Nest** 145
3.2.1 基本定义 145
3.2.2 从嵌套元组态射到布局 147
3.2.3 例子 150
3.2.4 嵌套元组态射的实现 152
3.2.5 加细 153
3.2.6 嵌套元组态射上的运算 162

第四章 计算 172
4.1 可处理布局的复合 172
4.1.1 相互加细 174
4.1.2 从相互加细到可复合态射 178
4.1.3 复合算法 180
4.1.4 例子 181
4.1.5 更一般的复合 187
4.1.6 复合可许性 188
4.2 逻辑切分与逻辑乘积 190
4.2.1 逻辑切分的例子 191
4.2.2 逻辑乘积的例子 192

附录 A 范畴论入门 194
A.1 什么是范畴? 194
A.2 什么是函子? 198

第一章　引言

在现代计算中，尤其是 GPU 编程中，性能在很大程度上取决于多维数据在内存中如何存储与访问。我们关心的大多数数据——例如图像、视频以及机器学习中的张量——本质上是多维的，而计算机的内存从根本上说是一维的。这意味着，当我们想要加载、存储或以其他方式操纵数据时，需要把数据的多维逻辑坐标映射到一维的物理坐标。这种映射被称为**布局**（layout），它对于正确而高效地读写内存至关重要。此外，在 GPU 的 SIMT 执行模型下，布局还被用来描述与操纵数据上线程的划分，这对于确保优化的内存访问模式、以及正确调用面向张量核心（tensor core）的专用硬件指令十分重要。

作为一个启发性例子，设我们要把 4×8 矩阵

$$A = \begin{bmatrix} 12.47 & 87.21 & 34.08 & 56.93 & 45.65 & 9.17 & 73.02 & 21.39 \\ 64.88 & 30.41 & 1.72 & 88.04 & 92.55 & 17.06 & 50.91 & 68.77 \\ 3.33 & 77.19 & 61.58 & 29.46 & 15.82 & 80.75 & 44.62 & 39.28 \\ 91.40 & 26.12 & 6.97 & 53.03 & 58.66 & 33.79 & 11.20 & 70.55 \end{bmatrix}$$

存入内存。为此，我们需要为 A 的每个元素（entry）指定一个内存地址。做法是：为 A 的第 $(0,0)$ 个元素选定某个地址，并为 A 的其余每个元素指定一个**偏移**（offset）。一种常见的选择是**行主序**（row-major）布局

$$L^{\mathsf{row}} = (4,8):(8,1) = \begin{array}{|c|c|c|c|c|c|c|c|} \hline 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ \hline 8 & 9 & 10 & 11 & 12 & 13 & 14 & 15 \\ \hline 16 & 17 & 18 & 19 & 20 & 21 & 22 & 23 \\ \hline 24 & 25 & 26 & 27 & 28 & 29 & 30 & 31 \\ \hline \end{array}$$

记号 $L^{\mathsf{row}} = (4,8):(8,1)$ 表示矩阵第 (i,j) 个元素的偏移为

$$(i,j) \cdot (8,1) = 8i + j.$$

另一种常见的选择是**列主序**（column-major）布局

$L^{\mathsf{col}} = (4,8):(1,4) =$

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

同样，记号 $L^{\mathsf{col}} = (4,8):(1,4)$ 表示矩阵第 (i,j) 个元素的偏移为

$$(i,j)\cdot(1,4) = i + 4j.$$

这些布局极其有用，但并不能满足所有目的。例如，在高性能计算中，人们常常通过如下步骤来计算矩阵乘积 AB：

1. 把操作数矩阵 A 与 B 划分为若干块（tile），
2. 计算各块之间的矩阵乘积，以及
3. 合并这些部分结果以得到完整结果 AB。

譬如，我们可以把 4×8 矩阵 A 划分为 2×2 的块，如下所示。

$$A = \begin{bmatrix} \begin{bmatrix} 12.47 & 87.21 \\ 64.88 & 30.41 \end{bmatrix} & \begin{bmatrix} 34.08 & 56.93 \\ 1.72 & 88.04 \end{bmatrix} & \begin{bmatrix} 45.65 & 9.17 \\ 92.55 & 17.06 \end{bmatrix} & \begin{bmatrix} 73.02 & 21.39 \\ 50.91 & 68.77 \end{bmatrix} \\ \begin{bmatrix} 3.33 & 77.19 \\ 91.40 & 26.12 \end{bmatrix} & \begin{bmatrix} 61.58 & 29.46 \\ 6.97 & 53.03 \end{bmatrix} & \begin{bmatrix} 15.82 & 80.75 \\ 58.66 & 33.79 \end{bmatrix} & \begin{bmatrix} 44.62 & 39.28 \\ 11.20 & 70.55 \end{bmatrix} \end{bmatrix}$$

现在假设我们想切出 A 的各个块，并设 A 在内存中按列主序格式存放。为此，可以按如下方式手工计算偏移：对第 (i,j) 个块，索引其左上角元素的偏移由 $2i+8j$ 给出。另一方面，为了更好地组织这一计算，我们可以使用块的**交错**（interleaved）布局

$L^{\mathsf{tiled}} =$

0	2	8	10	16	18	24	26
1	3	9	11	17	19	25	27
4	6	12	14	20	22	28	30
5	7	13	15	21	23	29	31

其中列由 A 的块给出，行由块形状内的坐标给出。这里我们用**余字典序**（colexicographic ordering）来线性地枚举块以及块内的坐标，因此布局 L^{tiled} 的顶层形状为 $(4,8)$。

然而，请注意，L^{tiled} 所呈现的交错模式意味着，对任何步长 a, b，它都不能表示为布局 $(4,8):(a,b)$。作为替代，我们可以把形状 $(4,8)$ 的各模式分解因式，并定义

$$L^{\mathsf{tiled}} = ((2,2),(2,4)):((1,4),(2,8)).$$

这样，前面的偏移计算 $2i+8j$ 就体现为在坐标 $(0,(i,j))$ 上对 L^{tiled} 求值，而块布局本身则由第一个模式给出。于是，在为 A 赋予布局 L^{tiled} 而得到 A^{tiled} 之后，我们就可以把 A 的第 (i,j) 个块取为切片

$$A_{i,j} = A^{\mathsf{tiled}}(\_\,,(i,j)).$$

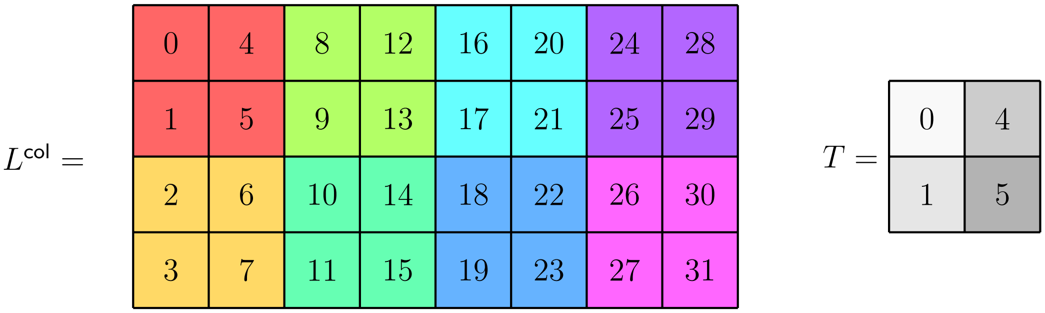
CUTLASS 中发展的一个关键思想是：像 L^{tiled} 这样有用但更复杂的辅助布局，可以通过某些基本运算从更简单的布局系统地推导出来。就 L^{tiled} 而言，所涉及的运算称为**逻辑切分**（logical division）。若记

$$T = (2,2):(1,4) = \begin{array}{|c|c|}\hline 0 & 4 \\ \hline 1 & 5 \\ \hline \end{array}$$

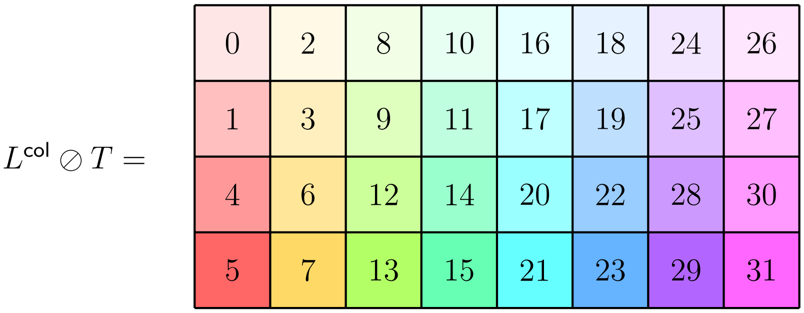
为块布局，则 L^{tiled} 就是逻辑切分

$$L^{\mathsf{tiled}} = L^{\mathsf{col}} \oslash T$$

如下图所示。



$$L^{\mathsf{col}} = \begin{array}{|c|c|c|c|c|c|c|c|}\hline 0 & 4 & 8 & 12 & 16 & 20 & 24 & 28 \\ \hline 1 & 5 & 9 & 13 & 17 & 21 & 25 & 29 \\ \hline 2 & 6 & 10 & 14 & 18 & 22 & 26 & 30 \\ \hline 3 & 7 & 11 & 15 & 19 & 23 & 27 & 31 \\ \hline \end{array} \qquad T = \begin{array}{|c|c|}\hline 0 & 4 \\ \hline 1 & 5 \\ \hline \end{array}$$



$$L^{\mathsf{col}} \oslash T = \begin{array}{|c|c|c|c|c|c|c|c|}\hline 0 & 2 & 8 & 10 & 16 & 18 & 24 & 26 \\ \hline 1 & 3 & 9 & 11 & 17 & 19 & 25 & 27 \\ \hline 4 & 6 & 12 & 14 & 20 & 22 & 28 & 30 \\ \hline 5 & 7 & 13 & 15 & 21 & 23 & 29 & 31 \\ \hline \end{array}$$

除逻辑切分之外，其他基本的布局运算还包括**逻辑乘积**（logical product）、**补**（complement），以及最重要的**复合**（composition）。这些布局运算是 CUTLASS 的支柱，深入理解它们的行为有助于编写正确且高性能的代码。然而，这些运算的定义与构造相当精妙。例如，布局 A 与 B 的复合 $B \circ A$ 只有在 A 与 B 满足某些整除性约束时才是良定义（well-defined）的，CUTLASS 会在底层检查这些约束。特别地，两个布局何时可复合、以及应如何解释它们的复合，并不总是显然的。

1.1　主要结果综述

本文的主要思想是：通过把注意力限制在**可处理布局**（tractable layout）上——其元素满足一个简单的整除性条件（见定义 2.3.10.1）——我们可以为布局的操纵发展出一套直观而强大的数学框架。可处理布局几乎涵盖了实践中会遇到的所有布局，例如

- 行主序与列主序布局，它们无处不在；
- 紧凑（compact）布局，它们把数据存储在连续的内存地址中；
- 投影（projection）布局，它们广播数据的多个副本；以及
- 膨胀（dilation）布局，它们支持带填充的加载与存储。

若 L 是可处理布局，则我们可以用**图表**（diagram）来表示 L。例如，布局 L^{row}、L^{col} 与 L^{tiled} 由下列图表表示。

$$\begin{tikzcd}
 & & 8 \arrow[rd, mapsto] & 4 \\
(4,8):(8,1) & \rightsquigarrow & 4 \arrow[ru, mapsto] & 8
\end{tikzcd}$$

$$\begin{tikzcd}
 & & 8 \arrow[r, mapsto] & 8 \\
(4,8):(1,4) & \rightsquigarrow & 4 \arrow[r, mapsto] & 4
\end{tikzcd}$$

$$\begin{tikzcd}
 & & & 4 \arrow[r, mapsto] & 4 \\
 & & 8 \arrow[ru, dash] \arrow[r, dash] & 2 \arrow[rd, mapsto] & 2 \\
((2,2),(2,4)):((1,4),(2,8)) & \rightsquigarrow & & 2 \arrow[ru, mapsto] & 2 \\
 & & 4 \arrow[ru, dash] \arrow[r, dash] & 2 \arrow[r, mapsto] & 2
\end{tikzcd}$$

这些图表可以解释为某个**范畴**（category）中的态射（morphism）。这使我们得以借助**范畴论**（category theory）的力量来描述布局及其运算。[1]

更确切地说，我们定义一个范畴 **Nest**，其对象（object）是正整数的嵌套元组（nested tuple），其态射 $f : S \to T$ 对应于如上那种图表（详见定义 3.1.1.13 与定义 3.2.1.1）。若 L 是**非退化**（non-degenerate）的可处理布局（见定义 2.3.1.24），则存在一个本质唯一的 **Nest**-态射 f 编码 L，正如下面的对应定理所展示的那样。

主定理 A. *(参见 3.2.2.15) 存在一一对应*

$$\left\{\begin{array}{c}\text{非退化}\\ \text{可处理布局}\end{array}\right\} \longleftrightarrow \left\{\begin{array}{c}\text{非退化}\\ \textsf{Nest}\text{-态射}\\ \text{(标准形式)}\end{array}\right\}$$

诸如复合、逻辑切分与逻辑乘积这样的布局运算，都可以在范畴 **Nest** 中得到自然的解释。若

$$S \xrightarrow{f} T \xrightarrow{g} U$$

是 **Nest**-态射，则我们可以通过把相应的图表粘合在一起来构成复合

$$S \xrightarrow{g\circ f} U$$

例如，

$$\begin{tikzcd}
 & & 2 & & & 2 \\
2 \arrow[rd, mapsto] & 2 \arrow[ru, mapsto] & 2 & \rightsquigarrow & 2 \arrow[r, mapsto] & 2 \\
2 \arrow[ru, mapsto] & 2 \arrow[ru, mapsto] & 5 & & 2 \arrow[ruu, mapsto] & 5 \\
f & g & & & g\circ f &
\end{tikzcd}$$

我们证明 **Nest** 中的复合与布局复合相容。

主定理 B. *(参见 3.2.6.21) 若 f 与 g 是非退化的、可复合的 **Nest**-态射，则*

$$L_{g\circ f} = L_g \circ L_f.$$

我们可以通过折叠相邻的箭头来合并（coalesce）**Nest**-态射 f。例如，

$$\begin{tikzcd}
 & 10 & & & \\
10 \arrow[ru, mapsto] & 10 & & & \\
10 \arrow[ru, mapsto] & 2 & & & 100 \\
2 \arrow[r, mapsto] & 2 & \rightsquigarrow & 100 \arrow[ru, mapsto] & 2 \\
2 \arrow[r, mapsto] & 2 & & 4 \arrow[r, mapsto] & 4 \\
f & & & \mathsf{coal}(f) &
\end{tikzcd}$$

我们证明该运算与布局的合并运算相容。

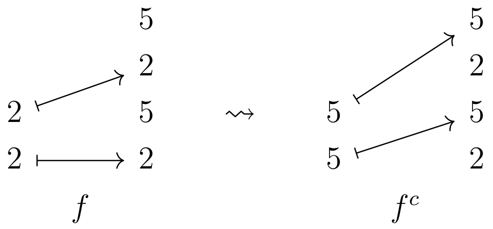
---

[1] 对于不熟悉这一学科的读者，我们在附录 A 中提供了范畴论入门。

主定理 C. *(参见 3.2.6.13)* 若 f 是 **Nest**-态射，则

$$L_{\mathsf{coal}(f)} = \mathsf{coal}(L_f).$$

Nest-态射 f 的补是未被 f 命中的元素所构成的包含映射（inclusion）。例如，


5
2
2
5
2
2
⇝
5
2
5
5
5
2
f
f^c

我们证明 **Nest** 中的补与布局的补相容。

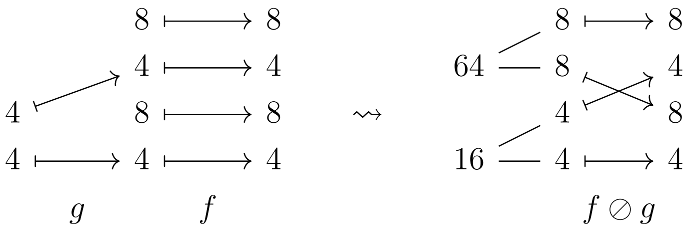
主定理 D. *(参见 3.2.6.20)* 若 $f : S \to T$ 是单射的 **Nest**-态射且 $N = \mathsf{size}(T)$，则

$$\mathsf{coal}(L_{f^c}) = \mathsf{comp}(L_f, N).$$

我们定义 **Nest**-态射的整除性，以及当 g 整除 f 时的逻辑切分运算

$$f, g \mapsto f \oslash g$$

例如，


8
8
4
4
4
8
8
4
4
4
g
f
⇝
8
8
64
8
4
4
8
16
4
4
f ⊘ g

我们证明 **Nest** 中的逻辑切分与布局的逻辑切分相容。

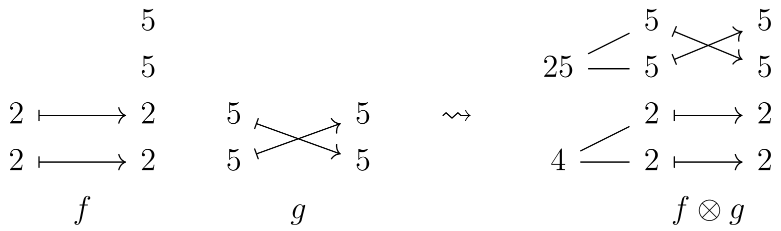
主定理 E. *(参见 3.2.6.26)* 若 f 与 g 是非退化 **Nest**-态射且 g 整除 f，则

$$\mathsf{coal}(L_{f \oslash g}) = \mathsf{coal}(L_f \oslash L_g).$$

我们定义 **Nest**-态射的乘积可许性（product admissibility），以及当 f 与 g 乘积可许时的逻辑乘积运算

$$f, g \mapsto f \otimes g$$

例如，


5
5
2
2
2
2
f
5
5
5
5
g
⇝
5
5
25
5
5
2
2
4
2
2
f ⊗ g

我们证明 **Nest** 中的逻辑乘积与布局的逻辑乘积相容。

主定理 F. *(参见 3.2.6.31) 若 f 与 g 是非退化* **Nest***-态射且二者乘积可许，则*

$$L_{f\otimes g} = L_f \otimes L_g.$$

在第 四 章中，我们展示如何用我们的新框架来计算复合、逻辑切分与逻辑乘积等重要布局运算。特别地，我们给出一个计算可处理布局 A 与 B 的复合 $B \circ A$ 的算法（算法 4.1.3）。略去细节，我们算法的基本思想是：若要计算复合 $B \circ A$，可以用适当选取的 **Nest**-态射 f 与 g 分别表示 A 与 B，把这两个态射复合成 $g \circ f$，再取其编码的布局，从而得到

$$B \circ A = L_{g\circ f}.$$

我们用大量例子演示这一算法。

1.2 结构安排

本文的组织结构如下。

在 1.4 节中，我们给出布局的 `cute` 实现的相关细节。我们以模块 `tract` 的形式提供了范畴 **Nest** 的一个 Python 实现，并演示 `tract` 与 `cute` 的相容性。我们的 Python 实现可在我们的 git 仓库 `https://github.com/ColfaxResearch/layout-categories` 找到。

第 二 章是关于布局及其代数的全面参考。该章给出布局及其所支持运算的严格定义，并建立这些运算的基本性质。该章富含例子，对一线程序员也可能颇有助益。

在第 三 章中，我们提出一个处理可处理布局的新数学框架。特别地，我们把布局及其代数与**范畴**和 **operad** 的理论联系起来。该章内容具有独立的数学趣味，同时也有实用价值：它为可视化布局以及计算布局的各种运算提供了一个新框架。

在第 四 章中，我们利用第 三 章发展的框架，给出一个计算可处理布局 A 与 B 的复合的算法。我们用大量例子演示该复合算法。

1.3 相关工作

虽然本文在性质上是理论性的，但其动机来自 GPU 编程中的实际应用，最著名的当属 CUTLASS。我们强调，这里发展的理论是独立于实现的：它不依赖于在实践中用来实现布局的具体编程语言或运行时系统。尽管如此，在使用具体实现时仍会出现一些实际的考量。例如，CUTLASS 会区分编译期常量（静态变量）与运行时值（动态变量）。这一信息使编译器能够在代码生成期间进行优化。这类与实现相关的细节虽然对性能很重要，但超出了我们数学框架的讨论范围。进一步的讨论见 CuTe 文档 [5]。

我们为布局发展的数学框架与计算机科学和数学的若干领域都有关联。我们简要回顾 GPU 编程及相邻领域的相关工作，以便为我们的贡献提供更完整的背景。

- **CUTLASS 的应用。**CUTLASS 的最先进应用包括 FlashAttention [7, 21]、EVT [4] 与 SonicMoE [10]。对于希望更深入地理解 CUTLASS 与 CuTe 实际应用的读者，我们推荐 NVIDIA [3, 24, 25] 与 Colfax Research [18, 20, 17, 19] 关于用这些库进行 GPU 编程的系列综合教程。

- **数据布局优化** 数据布局优化技术通过仔细考量张量在内存中的存储方式，来改善缓存局部性与内存访问模式 [30, 8, 15, 11]，[22]。选择高效的内存存储与访问模式对 GPU 性能至关重要，因为内存带宽往往正是瓶颈所在。

- **现代布局系统** CuTe [5, 6, 16] 与 Triton Linear Layouts [14, 32] 等布局系统已成为张量计算中管理内存存储与访问的行业标准。Triton 线性布局基于 $\mathbb{F}_2$-线性代数，并从 $\mathbb{F}_2$-线性算子的复合继承复合结构。它们还与布局 swizzle 自然相容，而后者一般无法表示为 CuTe 布局。另一方面，这些布局不如 CuTe 布局富有表达力，因为它们的大小与余大小必须等于 2 的幂，而且无法表达诸如乘以非 2 的幂整数之类的变换。最近的研究表明，这两种布局系统都可以用整数集合关系来表达 [1]。这为统一处理 CuTe 布局、Triton 线性布局以及更一般的布局（例如具有非矩形形状的布局）提供了共同基础。

- **多面体编译** 多面体模型（polyhedral model）[28]，[29]，[26] 为分析与变换具有仿射边界和数组访问的循环嵌套提供了数学框架。该模型的主要抽象，是把迭代空间表示为某个多面体中整点的集合。这一形式化方法允许进行复杂的循环变换，在优化局部性与并行性的同时保持程序语义。Pluto [2]、Polly [9] 与 Tensor Comprehensions [27] 等工具利用多面体技术自动生成优化代码。

- **张量缩并/分解** 张量缩并（tensor contraction）[23, 31, 12] 把矩阵乘法推广到更高秩的张量，在机器学习与科学计算中无处不在。张量缩并的高效实现依赖于缩并次序与中间张量布局的最优选择。

1.4 实现

本节演示如何在 NVIDIA 的 CuTe DSL（我们将其记作 `cute`）中使用布局。我们在 git 仓库 `https://github.com/ColfaxResearch/layout-categories` 中以 `Python` 模块 `tract` 的形式给出了我们范畴框架的一个实现。这里我们展示 `cute` 与 `tract` 的相容性。

1. **构造元组与嵌套元组：** 我们在 `Python` 中如下构造元组（tuple）与嵌套元组。

```
S = (2,2,2)
T = ((2,2),(5,5))
U = ((2,2),4,(9,(3,3)))
```

   注意，若要构造长度为 1 的元组，必须在元组的元素后面加上一个逗号。例如，

```
S = (10,)
T = (10)
```

   返回

```
S = (10,)
T = 10
```

2. **构造布局与态射：** 我们在 `cute` 中如下构造布局

$$L = S : D$$

   ：

```
L = cute.make_layout(shape=S, stride=D)
```

   例如，

```
A = cute.make_layout(shape=((4,4),4), stride=((16,1),4))
B = cute.make_layout(shape=(8,64), stride=(64,1))
C = cute.make_layout(shape=100, stride=2)
```

   返回

```
A = ((4,4),4):((16,1),4)
B = (8,64):(64,1)
C = 100:2
```

   我们在 `tract` 中如下构造嵌套元组态射

$$S \xrightarrow[\alpha]{f} T$$

   ：

```
f = tract.make_morphism(domain=S, codomain=T, map_=alpha)
```

例如，

```
f = tract.make_morphism(domain=(4,4), codomain=(4,2,4), map_=(1,3))
g = tract.make_morphism(domain=(2,2,2,2), codomain=(2,2,2,2), map_=(1,0,4,2))
h = tract.make_morphism(domain=(16,(4,4),(4,4)), codomain=(16,4,4), map_
    =(1,2,0,3,0))
```

返回

```
f = (4,4)--(1,3)-->(4,2,4)
g = (2,2,2,2)--(1,0,4,2)-->(2,2,2,2)
h = (16,(4,4),(4,4))--(1,2,0,3,0)-->(16,4,4)
```

注意，在 `tract` 中指定映射时，我们使用符号 `0` 而不是 $*$。

3. **在可处理布局与态射之间转换：**若 L 是布局，我们可以用如下方式检查 L 是否可处理：

```
tract.is_tractable(L)
```

例如，

```
A = cute.make_layout(shape=(2,2,2), stride=(1,2,4))
B = cute.make_layout(shape=(2,2,2), stride=(1,7,4))
A_is_tractable = tract.is_tractable(A)
B_is_tractable = tract.is_tractable(B)
```

返回

```
A = (2,2,2):(1,2,4)
B = (2,2,2):(1,7,4)
A_is_tractable = True
B_is_tractable = False
```

若 L 是可处理布局，则我们可以如下构造标准表示 f_L：

```
tract.compute_morphism(L)
```

例如，

```
L = cute.make_layout(shape=(2,2,2), stride=(1,2,4))
f_L = tract.compute_morphism(L)
```

返回

```
L = (2,2,2):(1,2,4)
f_L = (2,2,2)--(1,2,3)-->(2,2,2)
```

若 f 是嵌套元组态射，我们可以如下构造由 f 编码的布局 L_f：

```
tract.compute_layout(f)
```

例如，

```
f = tract.make_morphism(domain=((5,5),8), codomain=(5,8,5), map_=(1,3,2))
L_f = tract.compute_layout(f)
```

返回

```
f = ((5,5),8)--(1,3,2)-->(5,8,5)
L_f = ((5,5),8):((1,40),5)
```

4. **复合**：当有定义时，该运算从一对布局 A 与 B 产生布局 $B \circ A$。精确定义见定义 2.3.7.1。我们可以在 `cute` 中如下计算复合 $B \circ A$：

```
cute.composition(B,A)
```

例如，运行

```
A = cute.make_layout(shape=((4,4),4), stride=((16,1),4))
B = cute.make_layout(shape=(8,64), stride=(64,1))
B_o_A = cute.composition(B,A)
```

返回

```
A = ((4,4),4):((16,1),4)
B = (8,64):(64,1)
B_o_A = ((4,4),(2,2)):((2,64),(256,1))
```

若 f 与 g 是可复合的嵌套元组态射，我们可以在 `tract` 中如下计算复合 $g \circ f$：

```
tract.compose(f,g)
```

例如，

```
f = tract.make_morphism(domain=((2,2),(2,2)), codomain=((2,2,2),(2,2,2)), map_
    =(3,2,6,5))
g = tract.make_morphism(domain=((2,2,2),(2,2,2)), codomain=(2,2,2,2), map_
    =(1,0,2,0,3,4))
g_o_f = tract.compose(f,g)
```

返回

```
f = ((2,2),(2,2))--(3,2,6,5)-->((2,2,2),(2,2,2))
g = ((2,2,2),(2,2,2))--(1,0,2,0,3,4)-->(2,2,2,2)
g_o_f = ((2,2),(2,2))--(2,0,4,3)-->(2,2,2,2)
```

5. **合并**：该运算从布局 A 产生布局 coal(A)。详见定义 2.3.3.1。我们可以在 `cute` 中如下计算 coal(A)：

```
cute.coalesce(A)
```

例如，

```
A = cute.make_layout(shape = ((2,2),(2,2),(5,5)), stride = ((1,2),(16,32),(64,640)))
coal_A = cute.coalesce(A)
```

返回

```
A = ((2,2),(2,2),(5,5)):((1,2),(16,32),(64,640))
coal_A = (4,20,5):(1,16,640)
```

另外还有一个相对合并（relative coalesce）运算 $A \mapsto \text{coal}(A, S)$，它额外接收一个嵌套元组 S 作为输入，且 S 被 A 的形状所加细（refine）。详见定义 2.3.4.7。我们可以在 `cute` 中如下计算 coal(A, S)：

```
A = cute.make_layout(shape = ((2,2),(3,3),(5,5)), stride = ((1,2),(4,12),(36,180)))
S = ((2,2),9,25)
coal_A_over_S = cute.coalesce(A,target_profile=S)
```

返回

```
A = ((2,2),(3,3),(5,5)):((1,2),(4,12),(36,180))
S = ((2,2),9,25)
coal_A_over_S = ((2,2),9,25):((1,2),4,36)
```

若 f 是嵌套元组态射，我们可以构造 coal(f)。详见定义 3.2.6.11。我们在 `tract` 中如下计算 coal(f)：

```
tract.coalesce(f)
```

例如，

```
f = tract.make_morphism(domain=(2,2,10,10), codomain = (2,2,2,10,10), map_=(1,2,4,5))
coal_f = tract.coalesce(f)
```

返回

```
f = (2,2,10,10)--(1,2,4,5)-->(2,2,2,10,10)
coal_f = (4,100)--(1,3)-->(4,2,100)
```

6. **补**：当有定义时，该运算从布局 A 与正整数 N 产生布局 $\mathsf{comp}(A,N)$。详见定义 2.3.6.5。我们可以在 `cute` 中如下计算 $\mathsf{comp}(A,N)$：

```
cute.complement(A,N)
```

例如，

```
A = cute.make_layout(shape = ((2,2),(2,2)), stride = ((8,2),(64,256)))
comp_A = cute.complement(A,4096)
```

返回

```
A = ((2,2),(2,2)):((8,2),(64,256))
comp_A = (2,2,4,2,8):(1,4,16,128,512)
```

若 f 是嵌套元组态射，则我们可以构造 f 的补 f^c。详见定义 3.2.6.17。我们在 `tract` 中如下计算 f^c：

```
tract.complement(f)
```

例如，

```
f = tract.make_morphism(domain=(2,2), codomain=(2,5,2,5), map_=(1,3))
comp_f = tract.complement(f)
```

返回

```
f = (2,2)--(1,3)-->(2,5,2,5)
comp_A = (5,5)--(2,4)-->(2,5,2,5)
```

7. **逻辑切分**：当有定义时，该运算从一对布局 A 与 B 产生布局 $A \oslash B$。详见定义 2.3.8.1。我们在 `cute` 中如下计算 $A \oslash B$：

```
cute.logical_divide(A,B)
```

例如，

```
A = cute.make_layout((64,32), stride = (32,1))
B = cute.make_layout((4,4), stride = (1,64))
quotient = cute.logical_divide(A,B)
```

返回

```
A = (64,32):(32,1)
B = (4,4):(1,64)
quotient = ((4,4),(16,8)):((32,1),(128,4))
```

若 f 与 g 是嵌套元组态射且 g 整除 f，则我们可以构造逻辑切分 $f \oslash g$。详见定义 3.2.6.23。我们在 `tract` 中如下计算 $f \oslash g$：

```
tract.logical_divide(f,g)
```

例如，

```
f = tract.make_morphism(domain=(4,8,4,8), codomain=(4,8,4,8), map_=(1,2,3,4))
g = tract.make_morphism(domain=(4,4), codomain=(4,8,4,8), map_=(1,3))
quotient = tract.logical_divide(f,g)
```

返回

```
f = (4,8,4,8)--(1,2,3,4)-->(4,8,4,8)
g = (4,4)--(1,3)-->(4,8,4,8)
quotient = ((4,4),(8,8))--(1,3,2,4)-->(4,8,4,8)
```

8. **逻辑乘积**：当有定义时，该运算从一对布局 A 与 B 产生布局 $A \otimes B$。详见定义 2.3.9.1。我们在 `cute` 中如下计算 $A \otimes B$：

```
cute.logical_product(A,B)
```

例如，运行

```
A = cute.make_layout((3,10,10), stride = (200,1,20))
B = cute.make_layout((2,2), stride = (1,2))
product = cute.logical_product(A,B)
```

返回

```
A = (3,10,10):(200,1,20)
B = (2,2):(1,2)
product = ((3,10,10),(2,2)):((200,1,20),(10,600))
```

若 f 与 g 是嵌套元组态射且二者乘积可许，则我们可以构造逻辑乘积 $f \otimes g$。详见定义 3.2.6.28。我们在 `tract` 中如下计算 $f \otimes g$：

```
tract.logical_product(f,g)
```

例如，

```
f = tract.make_morphism(domain=(2,2), codomain=(2,2,5,5), map_=(1,2))
g = tract.make_morphism(domain=(5,5), codomain=(5,5), map_=(2,1))
product = tract.logical_product(f,g)
```

返回

```
f = (2,2)--(1,2)-->(2,2,5,5)
g = (5,5)--(2,1)-->(5,5)
product = ((2,2),(5,5))--(1,2,4,3)-->(2,2,5,5)
```

1.5　记号

$\mathbb{Z} = \{\dots, -1, 0, 1, 2, \dots\}$
$\mathbb{N} = \{0, 1, 2, \dots\}$
$\mathbb{Z}_{>0} = \{1, 2, \dots\}$
$\mathbb{F}_2 = \{0, 1\}$，即2 元有限域。
$[0, n) = \{0, \dots, n-1\}$，且$[0, 0) = \varnothing$.
$\langle n\rangle = \{1, 2, \dots, n\}$，且$\langle 0\rangle = \varnothing$.
$\langle n\rangle_* = \{*, 1, 2, \dots, n\}$
$\delta_i^m = (0, \dots, 1, \dots, 0)$，即长度为$m$ 的元组，
其第i 个元素为1，其余元素全为0.
$\Sigma_n = \langle n\rangle$ 上的对称群（symmetric group）。
$X^\sigma = (x_{\sigma(1)}, \dots, x_{\sigma(m)})$，其中$X = (x_1, \dots, x_m)$
是一个元组，$\sigma \in \Sigma_m$ 是一个置换（permutation）。
$X \star Y = X$ 与Y 的扁平拼接。
$X^\flat =$ 嵌套元组X 的扁平化。
$\mathsf{prof}(X) =$ 嵌套元组X 的轮廓（profile）。
$(X_1, \dots, X_k) = X_1, \dots, X_k$ 的（嵌套）拼接。
$(X_1, \dots, X_k)_Q =$ 以轮廓Q 对$X_1, \dots, X_k$ 所做的Q-代入。
$\mathsf{Tuple}(V) =$ 元素取自集合V 的全体元组。
$\mathsf{Nest}(V) =$ 元素取自集合V 的全体嵌套元组。
$\mathsf{Profile} =$ 全体轮廓构成的集合。
$\mathsf{FlatLayout} =$ 全体扁平布局（flat layout）构成的集合。

$\mathsf{Layout} =$ 全体布局构成的集合。
$B \circ A = A$ 与B 的复合。
$A \oslash B = A$ 除以B 的逻辑切分。
$A \otimes B = A$ 与B 的逻辑乘积。
$\mathbf{Set} =$ 集合范畴。
$\mathbf{FinSet} =$ 有限集范畴。
$\mathbf{Fin} =$ 由诸$\langle n\rangle(n \geq 0)$ 张成的$\mathbf{FinSet}$ 的满子范畴（full subcategory）。
$\mathbf{FinSet}_* =$ 基点有限集（pointed finite set）范畴。
$\mathbf{Fin}_* =$ 由诸$\langle n\rangle_*(n \geq 0)$ 张成的$\mathbf{FinSet}_*$ 的满子范畴。
$\mathbf{Tuple} =$ 元组与元组态射构成的范畴。
$\mathbf{Nest} =$ 嵌套元组与嵌套元组态射构成的范畴。
$\mathbf{Ref} =$ 嵌套元组与加细构成的范畴。
$\mathbf{Cat} =$（小）范畴与函子（functor）构成的范畴。

第二章　布局及其代数

本章的目标是为布局（layout）建立一个全面且数学上严谨的理论。我们首先在 2.1 节中建立扁平布局的理论，随后在 2.2 节中介绍嵌套元组（nested tuple）的必要背景知识，以便在 2.3 节中以最一般的形式讨论布局。

2.1　扁平布局

本节考察**扁平布局**（flat layout）——布局的一类重要子类，其中形状（shape）与步长（stride）都是元组（tuple），而不是更一般的嵌套元组。为使讨论形式化，我们先固定与元组相关的记号。

2.1.1　元组

定义 2.1.1.1. 若 V 是一个集合，则取值于 V 的元组是一个有限有序列表

$$X = (x_1, \ldots, x_m)$$

其中的元素 $x_i \in V$ 对每个 $1 \le i \le m$ 取定。这样的元组 $X = (x_1, \ldots, x_m)$ 的长度（length）为

$$\mathsf{len}(X) = m.$$

我们把所有取值于 V 的元组构成的整体记作 $\mathsf{Tuple}(V)$。我们尤其关心 $V = \mathbb{Z}$ 的情形，此时称 $X \in \mathsf{Tuple}(\mathbb{Z})$ 为整数元组。若 X 是整数元组，则 X 的大小（size）是乘积

$$\mathsf{size}(X) = x_1 \cdots x_m.$$

例 2.1.1.2. 下面给出若干元组的例子，并附上它们的长度与大小：

$$\begin{aligned}
X &= (3, 128, 128), & \mathsf{len}(X) &= 3, & \mathsf{size}(X) &= 49152 \\
X &= (512), & \mathsf{len}(X) &= 1, & \mathsf{size}(X) &= 512 \\
X &= (), & \mathsf{len}(X) &= 0, & \mathsf{size}(X) &= 1
\end{aligned}$$

定义 2.1.1.3. 若 $X = (x_1, \ldots, x_m)$ 与 $Y = (y_1, \ldots, y_n)$ 是元组，则记

$$X \star Y = (x_1, \ldots, x_m, y_1, \ldots, y_n)$$

为 X 与 Y 的拼接（concatenation）。

例 2.1.1.4. 若 $X = (64, 32)$ 且 $Y = (8, 8, 8)$，则

$$X \star Y = (64, 32, 8, 8, 8).$$

注 2.1.1.5. 若 V 是集合，则所有取值于 V 的元组构成的整体

$$\mathsf{Tuple}(V) = \coprod_{m \geq 0} V^{\times m}$$

是 V 上的自由结合幺半群（monoid）。其幺半群乘积就是拼接，单位元是空元组 $()$。

定义 2.1.1.6. 若 X 与 X' 是元组，则称 X' 整除（divides）X，如果存在元组 X'' 使得

$$X' \star X'' = X.$$

例 2.1.1.7. 若 $X' = (81, 9)$ 且 $X = (81, 9, 64, 8)$，则 X' 整除 X，因为元组 $X'' = (64, 8)$ 满足

$$X' \star X'' = X.$$

定义 2.1.1.8. 若 $X = (x_1, \ldots, x_m)$ 是元组，$\sigma \in \Sigma_m$ 是一个置换（permutation），则记

$$X^\sigma = (x_{\sigma(1)}, \ldots, x_{\sigma(m)})$$

为 X 被 σ 置换的结果。这给出了 Σ_m 在 $\mathbb{Z}^{\times m}$ 上的一个右作用。

例 2.1.1.9. 若 $X = (8, 16, 32, 64)$ 且 $\sigma = (1\ 2)(3\ 4)$，则

$$X^\sigma = (16, 8, 64, 32).$$

记号 2.1.1.10. 若 n 是正整数，则记

$$[0, n) = \{0, 1, \ldots, n-1\},$$

又若 $S = (s_1, \ldots, s_m)$ 是正整数元组，则记

$$[0, S) = [0, s_1) \times \cdots \times [0, s_m)$$

为所有满足 $0 \leq x_i < s_i$ 的元组 $(x_1, \ldots, x_m)$ 构成的集合。

例 2.1.1.11. 若 $S = (3, 2)$，则

$$[0, S) = \{(0,0), (1,0), (2,0), (0,1), (1,1), (2,1)\}$$

2.1.2　基本定义

记号既已固定，我们便可以定义扁平布局了。

定义 2.1.2.1. *扁平布局是一个二元组*

$$\begin{aligned} L &= S : D \\ &= (s_1, \ldots, s_m) : (d_1, \ldots, d_m) \end{aligned}$$

它由一个正整数元组

$$\begin{aligned} \mathsf{shape}(L) &= S \\ &= (s_1, \ldots, s_m) \end{aligned}$$

（称为 L 的形状）与一个非负整数元组

$$\begin{aligned} \mathsf{stride}(L) &= D \\ &= (d_1, \ldots, d_m) \end{aligned}$$

（称为 L 的步长）组成。

注 2.1.2.2. 若 L 是扁平布局，则由定义，$\mathsf{shape}(L)$ 与 $\mathsf{stride}(L)$ 具有相同的长度。

注 2.1.2.3. 扁平布局是定义 2.3.1.1 中更一般的布局的一个例子，因此我们有时也把扁平布局 L 简称为布局。

例 2.1.2.4. 下面是扁平布局的一些例子：

$$\begin{aligned} L_1 &= (2, 2, 2) : (1, 2, 4), \\ L_2 &= (128) : (5), \\ L_3 &= (16, 12, 512, 512) : (0, 0, 1, 512), \\ L_4 &= (6, 1, 12, 2, 2) : (2, 0, 12, 144, 1), \\ L_5 &= () : (). \end{aligned}$$

例 2.1.2.5. 我们可以把布局 $L = (8) : (5)$ 画成

$L =$

0	5	10	15	20	25	30	35

而布局 $L = (3, 5) : (2, 10)$ 可以画成

$L =$

0	10	20	30	40
2	12	22	32	42
4	14	24	34	44

我们将在注 2.1.2.17 中精确说明这些图在何种意义上表示相应的布局。

也许扁平布局最重要的例子是列主序（column-major）与行主序（row-major）布局，我们在下面给出定义。

定义 2.1.2.6. 设

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局。若对每个 $1 \leq i \leq m$ 都有

$$d_i = s_1 \cdots s_{i-1}$$

则称 L 是列主序的。若对每个 $1 \leq i \leq m$ 都有

$$d_i = s_{i+1} \cdots s_m.$$

则称 L 是行主序的。

例 2.1.2.7. 布局

$$L = (3,4) : (1,3) = \begin{array}{|c|c|c|c|}\hline 0 & 3 & 6 & 9 \\ \hline 1 & 4 & 7 & 10 \\ \hline 2 & 5 & 8 & 11 \\ \hline \end{array}$$

是列主序的，而布局

$$L = (3,4) : (4,1) = \begin{array}{|c|c|c|c|}\hline 0 & 1 & 2 & 3 \\ \hline 4 & 5 & 6 & 7 \\ \hline 8 & 9 & 10 & 11 \\ \hline \end{array}$$

是行主序的。这些图清楚地展示了该术语的由来：若 L 是秩（rank）为 2 的列主序布局，则 L 的各列是连续的；若 L 是秩为 2 的行主序布局，则 L 的各行是连续的。

例 2.1.2.8. 布局

$$L_1 = (2,2,2,2,2) : (1,2,4,8,16)$$
$$L_2 = (3,128,128) : (1,3,384)$$
$$L_3 = (64) : (1)$$

是列主序的，而布局

$$\begin{aligned} L_4 &= (2,2,2,2,2) : (16,8,4,2,1) \\ L_5 &= (3,128,128) : (16384,128,1) \\ L_6 &= (64) : (1) \end{aligned}$$

是行主序的。

看过若干例子之后，我们来定义扁平布局的一些重要属性。

定义 2.1.2.9. 设 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$ 是扁平布局。

- L 的秩为
$$\mathsf{rank}(L) = m.$$
- L 的大小为
$$\mathsf{size}(L) = \prod_{i=1}^{m} s_i.$$
- L 的余大小（cosize）为
$$\mathsf{cosize}(L) = 1 + \sum_{i=1}^{m} (s_i - 1) \cdot d_i.$$
- 对任意 $1 \leq i \leq \mathsf{rank}(L)$，$L$ 的第 i 个模式（mode）是数对
$$\mathsf{mode}_i(L) = s_i : d_i.$$

例 2.1.2.10. 布局

$$L = (64, 32) : (1, 128)$$

具有秩 $\mathsf{rank}(L) = 2$、大小 $\mathsf{size}(L) = 2048$ 与余大小 $\mathsf{cosize}(L) = 4032$。$L$ 的各模式为

$$\begin{aligned} \mathsf{mode}_1(L) &= 64 : 1 \\ \mathsf{mode}_2(L) &= 32 : 128. \end{aligned}$$

例 2.1.2.11. 布局

$$L = (3, 8, 8, 8) : (1, 3, 24, 192).$$

具有 $\mathsf{rank}(L) = 4$、$\mathsf{size}(L) = 1536$ 与 $\mathsf{cosize}(L) = 1536$。布局 L 有四个模式，例如 $\mathsf{mode}_3(L) = 8 : 24$。

例 2.1.2.12. 布局

$$L = (2,2,2,2,2) : (160,80,40,20,10).$$

具有 $\mathsf{rank}(L) = 5$、$\mathsf{size}(L) = 32$ 与 $\mathsf{cosize}(L) = 311$。布局 L 有 5 个模式，例如 $\mathsf{mode}_5(L) = 2 : 10$。

若 L 是扁平布局,则 L 编码了一个坐标函数(coordinate function)φ_L。L 的坐标函数是一个从多维到一维的变换，由与 $\mathsf{stride}(L)$ 做点积给出。回顾：若 $S = (s_1, \dots, s_m)$ 是正整数元组，则

$$[0, S) = [0, s_1) \times \cdots \times [0, s_m)$$

是所有满足 $0 \le x_i < s_i$ 的元组 $(x_1, \dots, x_m)$ 构成的集合。特别地，若 $S = ()$ 是空元组，则 $[0, S) = \{()\}$。

构造 2.1.2.13 (坐标函数)**.** 若

$$L = (s_1, \dots, s_m) : (d_1, \dots, d_m)$$

是扁平布局，则 L 的坐标函数是函数

$$[0, \mathsf{shape}(L)) \xrightarrow{\varphi_L} \mathbb{Z}$$

它由

$$\begin{aligned} \varphi_L(x_1, \dots, x_m) &= (x_1, \dots, x_m) \cdot (d_1, \dots, d_m) \\ &= x_1 d_1 + \cdots + x_m d_m. \end{aligned}$$

给出。坐标函数 φ_L 可经由包含映射 $[0, \mathsf{cosize}(L)) \subset \mathbb{Z}$ 分解，我们把分解得到的映射记作

$$[0, \mathsf{shape}(L)) \xrightarrow{\varphi_L^{\mathsf{cosize}(L)}} [0, \mathsf{cosize}(L)) \subset \mathbb{Z}$$

更一般地，对任意 $N \ge \mathsf{cosize}(L)$，我们把 φ_L 经由 $[0, N) \subset \mathbb{Z}$ 的分解记作 φ_L^N，并且在术语上稍作滥用，把这样的映射 φ_L^N 也称为 L 的坐标函数。

例 2.1.2.14. 若 $L = (2,3) : (1,5)$，则坐标函数

$$\varphi_L : [0,2) \times [0,3) \to \mathbb{Z}$$

由

$$\begin{aligned} \varphi_L(0,0) &= (0,0) \cdot (1,5) = 0, \\ \varphi_L(1,0) &= (1,0) \cdot (1,5) = 1, \\ \varphi_L(0,1) &= (0,1) \cdot (1,5) = 5, \\ \varphi_L(1,1) &= (1,1) \cdot (1,5) = 6, \\ \varphi_L(0,2) &= (0,2) \cdot (1,5) = 10, \\ \varphi_L(1,2) &= (1,2) \cdot (1,5) = 11. \end{aligned}$$

给出。

例 2.1.2.15. 若 $L=(2,2):(64,2)$，则坐标函数

$$\varphi_L:[0,2)\times[0,2)\to\mathbb{Z}$$

由

$$\begin{aligned}\varphi_L(0,0)&=(0,0)\cdot(64,2)=0,\\ \varphi_L(1,0)&=(1,0)\cdot(64,2)=64,\\ \varphi_L(0,1)&=(0,1)\cdot(64,2)=2,\\ \varphi_L(1,1)&=(1,1)\cdot(64,2)=66.\end{aligned}$$

给出。

例 2.1.2.16. 若 $E=():()$ 是空布局，则 E 的坐标函数是映射

$$\varphi_E:\{()\}\to\mathbb{Z}$$

它由

$$\varphi(())=0.$$

给出。

注 2.1.2.17. 现在，举例来说，我们可以精确描述下图

0	10	20	30	40
2	12	22	32	42
4	14	24	34	44

表示布局 $L=(3,5):(2,10)$ 的确切含义：网格中第 (i,j) 个格子以 L 的坐标函数的值

$$\varphi_L(i,j)=(i,j)\cdot(2,10)=2i+10j$$

标注。

在实践中，扁平布局 L 最重要的不变量是它的*布局函数*（layout function）Φ_L，它由坐标函数

$$\varphi_L:[0,S)\to\mathbb{Z}$$

与*余字典序同构*（colexicographic isomorphism）

$$\mathsf{colex}_S:[0,S)\to[0,\mathsf{size}(S)).$$

的逆复合而得。

定义 2.1.2.18. 设 $S = (s_1, \dots, s_m)$ 是正整数元组，并回顾

$$[0, S) = [0, s_1) \times \cdots \times [0, s_m).$$

余字典序同构是映射

$$\begin{tikzcd} {[0, S)} \arrow[r, "\mathsf{colex}_S"] & {[0, \mathsf{size}(S))} \\ (x_1, \dots, x_m) \arrow[r, mapsto] & \sum_{i=1}^m s_1 \cdots s_{i-1} x_i. \end{tikzcd}$$

当元组 S 从上下文中明确时，我们有时简写 $\mathsf{colex} = \mathsf{colex}_S$。余字典序同构的逆是映射

$$\begin{tikzcd} {[0, \mathsf{size}(S))} \arrow[r, "\mathsf{colex}_S^{-1}"] & {[0, S)} \end{tikzcd}$$

它由

$$\mathsf{colex}_S^{-1}(x) = (x_1, \dots, x_m)$$

给出，其中

$$x_i = \left\lfloor \frac{x}{s_1 \cdots s_{i-1}} \right\rfloor \mod s_i.$$

注意，若 $S = ()$ 是空元组，则

$$\mathsf{colex}_{()} : \{()\} \to \{0\}$$

与

$$\mathsf{colex}_{()}^{-1} : \{0\} \to \{()\}$$

都是典范的同构。

构造 2.1.2.19 (布局函数)**.** 若

$$L = (s_1, \dots, s_m) : (d_1, \dots, d_m),$$

是扁平布局，则 L 的布局函数是复合

$$\begin{tikzcd} {[0, \mathsf{size}(L))} \arrow[rr, "\Phi_L"] \arrow[dr, "\mathsf{colex}_{\mathsf{shape}(L)}^{-1}"'] & & \mathbb{Z}. \\ & {[0, \mathsf{shape}(L))} \arrow[ur, "\varphi_L"'] & \end{tikzcd}$$

显式地，Φ_L 由

$$\Phi_L(x) = x_1 d_1 + \cdots + x_m d_m$$

给出，其中

$$x_i = \left\lfloor \frac{x}{s_1 \cdots s_{i-1}} \right\rfloor \mod s_i.$$

布局函数 Φ_L 可经由包含映射 $[0, \mathsf{cosize}(L)) \subset \mathbb{Z}$ 分解，我们把分解得到的映射记作

$$[0, \mathsf{size}(L)) \xrightarrow{\Phi_L^{\mathsf{cosize}(L)}} [0, \mathsf{cosize}(L)) \subset \mathbb{Z}$$

更一般地，对任意 $N \geq \mathsf{cosize}(L)$，我们把 Φ_L 经由 $[0, N) \subset \mathbb{Z}$ 的分解记作 Φ_L^N，并且在术语上稍作滥用，把这样的映射 φ_L^N 也称为 L 的布局函数。

例 2.1.2.20. 若 $L = (2, 3) : (1, 5)$，则布局函数

$$\Phi_L : [0, 6) \to \mathbb{Z}$$

由

$$\begin{aligned}
\Phi_L(0) &= (0, 0) \cdot (1, 5) = 0, \\
\Phi_L(1) &= (1, 0) \cdot (1, 5) = 1, \\
\Phi_L(2) &= (0, 1) \cdot (1, 5) = 5, \\
\Phi_L(3) &= (1, 1) \cdot (1, 5) = 6, \\
\Phi_L(4) &= (0, 2) \cdot (1, 5) = 10, \\
\Phi_L(5) &= (1, 2) \cdot (1, 5) = 11.
\end{aligned}$$

给出。

例 2.1.2.21. 若 $L = (2, 2) : (64, 2)$，则布局函数

$$\Phi_L : [0, 4) \to \mathbb{Z}$$

由

$$\begin{aligned}
\Phi_L(0) &= (0, 0) \cdot (64, 2) = 0, \\
\Phi_L(1) &= (1, 0) \cdot (64, 2) = 64, \\
\Phi_L(2) &= (0, 1) \cdot (64, 2) = 2, \\
\Phi_L(3) &= (1, 1) \cdot (64, 2) = 66.
\end{aligned}$$

给出。

例 2.1.2.22. 若 $L = (4, 2, 2) : (3, 3, 100)$，则例如，$L$ 的布局函数满足

$$\begin{aligned}
\Phi_L(7) &= (3, 1, 0) \cdot (3, 3, 100) = 12, \\
\Phi_L(9) &= (1, 0, 1) \cdot (3, 3, 100) = 103.
\end{aligned}$$

例 2.1.2.23. 若 $E = () : ()$ 是空布局，则

$$\Phi_E : \{0\} \to \mathbb{Z}$$

由

$$\Phi_E(0) = 0.$$

给出。

例 2.1.2.24. 若 L 是任意扁平布局，则 L 的布局函数 Φ_L 满足

$$\Phi_L(0) = 0.$$

注 2.1.2.25. 若 $S = (s_1, \ldots, s_m)$ 是正整数元组，则余字典序同构

$$[0, S) \xrightarrow{\mathsf{colex}_S} [0, \mathsf{size}(S))$$

等于列主序布局

$$L = (s_1, s_2, \ldots, s_m) : (1, s_1, \ldots, s_1 \cdots s_{m-1}).$$

的坐标函数 $\varphi_L^{\mathsf{cosize}(L)}$。这意味着，若扁平布局 L 是列主序的，则

$$\begin{aligned}\Phi_L^{\mathsf{cosize}(L)} &= \varphi_L^{\mathsf{cosize}(L)} \circ \mathsf{colex}_{\mathsf{shape}(L)}^{-1} \\ &= \varphi_L^{\mathsf{cosize}(L)} \circ \left(\varphi_L^{\mathsf{cosize}(L)}\right)^{-1} \\ &= \mathsf{id}_{[0,\mathsf{size}(L))}\end{aligned}$$

是 $[0, \mathsf{size}(L))$ 上的恒等映射。

注 2.1.2.26. 存在互不相同的布局 $A \neq B$ 使得 $\Phi_A = \Phi_B$。例如，布局

$$\begin{aligned}A &= (7, 7) : (1, 7) \\ B &= (49) : (1)\end{aligned}$$

并不相等，却有 $\Phi_A = \Phi_B$。稍后我们将精确刻画两个扁平布局 A 与 B 何时具有相同的布局函数（见命题 2.1.4.18）。

在进入布局运算的讨论之前，我们需要定义非退化（non-degenerate）的概念。

定义 2.1.2.27. 设

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局。若对任意 $1 \leq i \leq m$ 都有

$$s_i = 1 \quad \Rightarrow \quad d_i = 0.$$

则称 L 是非退化的。

例 2.1.2.28. 布局

$$
\begin{aligned}
L_1 &= (4,1):(1,0)\\
L_2 &= (8,1,8,1):(2,0,16,0)
\end{aligned}
$$

是非退化的，而布局

$$
\begin{aligned}
L_3 &= (4,1):(1,4)\\
L_4 &= (8,1,8,1):(2,16,16,256)
\end{aligned}
$$

是退化的。

观察 2.1.2.29. 假设布局 L 非退化在实质上不损失一般性。更精确地说，若

$$
\begin{aligned}
L &= (s_1,\ldots,s_m):(d_1,\ldots,d_m)\\
L' &= (s_1,\ldots,s_m):(d'_1,\ldots,d'_m)
\end{aligned}
$$

是形状相同的两个扁平布局，且只要 $s_i > 1$ 就有 $d_i = d'_i$，那么 $\varphi_L = \varphi_{L'}$，且 $\Phi_L = \Phi_{L'}$。特别地，只要 $s_i = 1$，我们就可以自由地取 $d_i = 0$，而不会改变 L 的坐标函数或布局函数。

2.1.3　基本运算

在建立了扁平布局（flat layout）的基本词汇之后，我们转而讨论它们所支持的运算。本节定义若干基本运算，构造诸如合并（coalesce）、补（complement）与复合（composition）之类更复杂的运算时将会用到它们。

2.1.3.1　限制

设 L 是扁平布局，将 L 限制（restriction）到其模式（mode）的某个子集上往往很有用。回顾一下，对于非负整数 m，我们记

$$\langle m\rangle = \{1,\ldots,m\}.$$

定义 2.1.3.1. 设

$$L = (s_1,\ldots,s_m):(d_1,\ldots,d_m)$$

是扁平布局，并设

$$I = \{i_1 < \cdots < i_k\} \subset \langle m\rangle$$

是一个子集。我们定义 L 到 I 的限制为扁平布局

$$L\mid_I = (s_{i_1},\ldots,s_{i_k}):(d_{i_1},\ldots,d_{i_k}).$$

例 2.1.3.2. 若

$$L = (3,6):(10,5) = \begin{array}{|c|c|c|c|c|c|}\hline 0 & 5 & 10 & 15 & 20 & 25 \\ \hline 10 & 15 & 20 & 25 & 30 & 35 \\ \hline 20 & 25 & 30 & 35 & 40 & 45 \\ \hline \end{array}$$

且 $I = \{2\}$，则

$$L\mid_I = (6):(5) = \begin{array}{|c|c|c|c|c|c|}\hline 0 & 5 & 10 & 15 & 20 & 25 \\ \hline \end{array}$$

例 2.1.3.3. 若

$$L = (3,8,8,8):(1,3,24,192)$$

且 $I = \{1,2,3\}$，则

$$L\mid_I = (3,8,8):(1,3,24).$$

例 2.1.3.4. 若

$$L = (s_1,\dots,s_m):(d_1,\dots,d_m)$$

是扁平布局且 $I = \langle m\rangle$，则

$$L\mid_I = L.$$

例 2.1.3.5. 若

$$L = (s_1,\dots,s_m):(d_1,\dots,d_m)$$

是扁平布局且 $I = \varnothing$ 为空集，则

$$L\mid_I = ():()$$

是空布局（empty layout）。

2.1.3.2　压缩

若 L 是扁平布局，则压缩（squeeze）运算 $L \mapsto \mathsf{squeeze}(L)$ 会移除 L 中所有满足 $s_i = 1$ 的模式 $s_i : d_i$。

构造 2.1.3.6. 设

$$L = (s_1,\dots,s_m):(d_1,\dots,d_m)$$

是扁平布局，并令

$$I = \{i \in \langle m\rangle \mid s_i > 1\}$$

为对应形状（shape）元素（entry）不等于 1 的指标组成的集合。我们定义

$$\mathsf{squeeze}(L) = L\mid_I .$$

例 2.1.3.7. 若

$$L = (64, 64, 1) : (1, 64, 0),$$

则

$$\mathsf{squeeze}(L) = (64, 64) : (1, 64).$$

例 2.1.3.8. 若

$$L = (64, 64, 1, 32, 1) : (2048, 32, 0, 1, 0)$$

则

$$\mathsf{squeeze}(L) = (64, 64, 32) : (2048, 32, 1).$$

例 2.1.3.9. 若 L 是扁平布局，则

$$\mathsf{squeeze}(L) = L$$

当且仅当 $\mathsf{shape}(L)$ 不含等于 1 的元素。

例 2.1.3.10. 若 L 是扁平布局，则

$$\mathsf{squeeze}(L) = () : ()$$

是空布局当且仅当 $\mathsf{shape}(L)$ 的所有元素都等于 1。

这一构造（construction）的一个本质性质是：$L \mapsto \mathsf{squeeze}(L)$ 保持 L 的布局函数（layout function）不变。

引理 2.1.3.11. *若 L 是扁平布局，则*

1. $\mathsf{size}(\mathsf{squeeze}(L)) = \mathsf{size}(L)$,
2. $\mathsf{cosize}(\mathsf{squeeze}(L)) = \mathsf{cosize}(L)$，*且*
3. $\Phi_{\mathsf{squeeze}(L)} = \Phi_L$。

证明. 令

$$I = \{i_1 < \cdots < i_k\} \subset \langle m \rangle$$

表示满足 $s_{i_j} > 1$ 的指标组成的集合，于是

$$\mathsf{squeeze}(L) = (s_{i_1}, \ldots, s_{i_k}) : (d_{i_1}, \ldots, d_{i_k}).$$

对第一条断言，我们计算

$$\mathsf{size}(\mathsf{squeeze}(L)) = \prod_{j=1}^{k} s_{i_j} = \left(\prod_{j=1}^{k} s_{i_j}\right) \cdot \left(\prod_{\langle m \rangle \setminus I} 1\right) = \prod_{i=1}^{m} s_i = \mathsf{size}(L).$$

对第二条断言，我们计算

$$\begin{aligned}\mathsf{cosize}(\mathsf{squeeze}(L)) = 1 + \sum_{j=1}^{k}(s_{i_j} - 1)\cdot d_{i_j} &= 1 + \sum_{j=1}^{k}(s_{i_j} - 1)\cdot d_{i_j} + \left(\sum_{\langle m\rangle\setminus I} 0\right)\\ &= 1 + \sum_{i=1}^{m}(s_i - 1)\cdot d_i\\ &= \mathsf{cosize}(L).\end{aligned}$$

对第三条断言，只需证明从扁平布局中移除形如 $1 : d_i$ 的模式不改变布局函数。设 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$，并设某个 $s_i = 1$。令

$$L' = (s'_1, \ldots, s'_{m-1}) : (d'_1, \ldots, d'_{m-1})$$

表示从 L 中移除第 i 个模式后得到的扁平布局，于是

$$s'_j = \begin{cases} s_j & j < i\\ s_{j+1} & i \le j < m, \end{cases} \qquad \text{and} \qquad d'_j = \begin{cases} d_j & j < i\\ d_{j+1} & i \le j < m. \end{cases}$$

L 的布局函数由

$$\Phi_L(x) = x_1 d_1 + \cdots + x_m d_m$$

给出，其中 $x_j = \left\lfloor \dfrac{x}{s_1 \cdots s_{j-1}} \right\rfloor \mod s_j$；而 L' 的布局函数由

$$\Phi_{L'}(x) = x'_1 d'_1 + \cdots + x'_{m-1} d'_{m-1}$$

给出，其中 $x'_j = \left\lfloor \dfrac{x}{s'_1 \cdots s'_{j-1}} \right\rfloor \mod s'_j$。我们观察到

$$x'_j = \begin{cases} x_j & j < i\\ x_{j+1} & i \le j < m, \end{cases}$$

又由于 $x_i \in [0, 1)$ 必然等于 0，我们有

$$\begin{aligned}\Phi_L(x) &= x_1 d_1 + \cdots + x_m d_m\\ &= x_1 d_1 + \cdots + x_{i-1} d_{i-1} + x_{i+1} d_{i+1} + \cdots + x_m d_m\\ &= x'_1 d'_1 + \cdots + x'_{m-1} d'_{m-1}\\ &= \Phi_{L'}(x).\end{aligned}$$

□

2.1.3.3　过滤零

若 L 是扁平布局，则过滤零（filter zeros）运算 $L \mapsto \mathsf{filter}(L)$ 会移除所有满足 $d_i = 0$ 的模式 $s_i : d_i$。

定义 2.1.3.12. 设

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局，并令

$$I = \{i \in \langle m \rangle \mid d_i > 0\}$$

为对应步长（stride）元素不等于 0 的指标组成的集合。我们定义

$$\mathsf{filter}(L) = L\,|_I\,.$$

例 2.1.3.13. 若

$$L = (64, 8, 8, 128) : (8, 1, 0, 512)$$

则

$$\mathsf{filter}(L) = (64, 8, 128) : (8, 1, 512).$$

例 2.1.3.14. 若

$$L = (3, 2) : (12, 0) = \begin{array}{|c|c|}\hline 0 & 0 \\ \hline 12 & 12 \\ \hline 24 & 24 \\ \hline \end{array}$$

则

$$\mathsf{filter}(L) = (3) : (12) = \begin{array}{|c|}\hline 0 \\ \hline 12 \\ \hline 24 \\ \hline \end{array}$$

例 2.1.3.15. 若

$$L = (3, 8, 8, 8) : (16, 0, 0, 0)$$

则

$$\mathsf{filter}(L) = (3) : (16).$$

例 2.1.3.16. 若 L 是扁平布局，则

$$\mathsf{filter}(L) = L$$

当且仅当 $\mathsf{stride}(L)$ 的所有元素均非零。

例 2.1.3.17. 若 L 是扁平布局，则

$$\mathsf{filter}(L) = () : ()$$

是空布局当且仅当 $\mathsf{stride}(L)$ 的所有元素都等于 0。

2.1.3.4　置换

回顾一下，若 $X = (x_1, \ldots, x_m)$ 是长度为 m 的元组（tuple），且 $\sigma \in \Sigma_m$ 是一个置换（permutation），则我们记

$$X^\sigma = (x_{\sigma(1)}, \ldots, x_{\sigma(m)}).$$

表示 X 经 σ 置换后的结果。

定义 2.1.3.18. 若 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$ 是秩（rank）为 m 的扁平布局，且 $\sigma \in \Sigma_m$ 是一个置换，我们定义

$$\begin{aligned} L^\sigma &= \mathsf{shape}(L)^\sigma : \mathsf{stride}(L)^\sigma \\ &= (s_{\sigma(1)}, \ldots, s_{\sigma(m)}) : (d_{\sigma(1)}, \ldots, d_{\sigma(m)}). \end{aligned}$$

例 2.1.3.19. 若

$$L = (4,2) : (12,2) = \begin{array}{|c|c|} \hline 0 & 2 \\ \hline 12 & 14 \\ \hline 24 & 26 \\ \hline 36 & 38 \\ \hline \end{array}$$

且 $\sigma = (1\ 2) \in \Sigma_2$ 是对换（transposition），则

$$L^\sigma = (2,4) : (2,12) = \begin{array}{|c|c|c|c|} \hline 0 & 12 & 24 & 36 \\ \hline 2 & 14 & 26 & 38 \\ \hline \end{array}$$

是转置后的布局。

例 2.1.3.20. 若

$$L = (15,12,10) : (240,1,24)$$

且 $\sigma = (1\ 2) \in \Sigma_3$，则

$$L^\sigma = (12,15,10) : (1,240,24).$$

例 2.1.3.21. 若

$$L = (2, 2, 2, 2, 2) : (1, 2, 4, 8, 16)$$

且 $\sigma = (1\ 5)(3\ 2\ 4) \in \Sigma_5$，则

$$L^\sigma = (2, 2, 2, 2, 2) : (16, 8, 2, 4, 1).$$

例 2.1.3.22. 若

$$L = (s, \ldots, s) : (d, \ldots, d)$$

是所有模式都相同的扁平布局，则对任意 $\sigma \in \Sigma_m$，有

$$L^\sigma = L.$$

2.1.3.5　排序

若 L 是扁平布局，按以下意义置换 L 使其模式递增往往很有用。

定义 2.1.3.23. 我们对整数对 $s : d$ 定义线性序（linear ordering）：

$$s : d \preceq s' : d' \quad \Leftrightarrow \quad \begin{array}{c} d < d', \text{ or} \\ d = d' \text{ and } s \leq s'. \end{array}$$

例 2.1.3.24. 我们有

$$5 : 8 \preceq 4 : 12 \preceq 5 : 12.$$

定义 2.1.3.25. 设 L 是扁平布局。若对任意 $1 \leq i < \mathsf{rank}(L)$ 均有

$$\mathsf{mode}_i(L) \preceq \mathsf{mode}_{i+1}(L),$$

则称 L 是已排序（sorted）的。

例 2.1.3.26. 布局

$$\begin{aligned} L_1 &= (128, 64, 2, 2) : (1, 128, 8192, 16384) \\ L_2 &= (2, 2, 2) : (1, 1, 1) \end{aligned}$$

是已排序的，而布局

$$\begin{aligned} L_3 &= (2, 4, 8, 16) : (64, 1, 2, 4) \\ L_4 &= (5, 32, 16) : (1, 5, 5) \end{aligned}$$

不是已排序的。

例 2.1.3.27. 空布局 $E = () : ()$ 是已排序的。

例 2.1.3.28. 若

$$L = (s_1, \ldots, s_m) : (0, \ldots, 0)$$

是所有 stride(L) 元素均为 0 的扁平布局，则 L 已排序当且仅当

$$s_1 \leq s_2 \leq \cdots \leq s_m.$$

扁平布局 L 是否已排序，与 L 的布局函数 Φ_L 的性态密切相关，如下述引理所述。

引理 2.1.3.29. 设 L 是扁平布局。若 Φ_L 非减（*non-decreasing*），则 L 已排序。

证明. 我们证明逆否命题（contrapositive）。设 L 未排序。我们将证明在 Φ_L 的定义域（domain）中存在某个 $x \leq y$，使得 $\Phi_L(x) > \Phi_L(y)$。若存在某个 $1 \leq i < m$ 使得 $d_i > d_{i+1}$，则可取

$$x = \prod_{j<i} s_j, \quad \text{and} \quad y = \prod_{j<i+1} s_j,$$

此时 $x < y$，但

$$\begin{aligned}
\Phi_L(x) &= (0, \ldots, 1, 0, \ldots, 0) \cdot (d_1, \ldots, d_i, d_{i+1}, \ldots, d_m) \\
&= d_i \\
&> d_{i+1} \\
&= (0, \ldots, 0, 1, \ldots, 0) \cdot (d_1, \ldots, d_m) \\
&= \Phi_L(y).
\end{aligned}$$

另一方面，若存在某个 $1 \leq i < m$ 使得 $d_i = d_{i+1}$ 且 $s_i > s_{i+1}$，则可设

$$x = (s_i - 1)\left(\prod_{j<i} s_j\right), \quad \text{and} \quad y = (s_{i+1} - 1)\left(\prod_{j<i+1} s_j\right),$$

此时 $x < y$，但

$$\begin{aligned}
\Phi_L(x) &= (0, \ldots, s_i - 1, 0, \ldots, 0) \cdot (d_1, \ldots, d_i, d_{i+1}, \ldots, d_m) \\
&= (s_i - 1)d_i \\
&> (s_{i+1} - 1)d_i \\
&= (s_{i+1} - 1)d_{i+1} \\
&= (0, \ldots, 0, s_{i+1} - 1, \ldots, 0) \cdot (d_1, \ldots, d_m) \\
&= \Phi_L(y).
\end{aligned}$$

由此得出 Φ_L 并非非减。 □

注 2.1.3.30. 上一引理的逆命题不成立。例如（example），扁平布局

$$L = (3, 5, 7) : (1, 1, 1)$$

是已排序的，但

$$\Phi_L(7) = (0, 2, 0) \cdot (1, 1, 1) = 2$$

严格大于

$$\Phi_L(16) = (0, 0, 1) \cdot (1, 1, 1) = 1.$$

若 L 是扁平布局，则我们可以置换 L 的模式以获得已排序的布局 $\mathsf{sort}(L)$。

构造 2.1.3.31. 设

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局。在 $\langle m \rangle$ 上定义线性序 $\preceq$：$i \preceq j$ 当且仅当

1. $\mathsf{mode}_i(L) \preceq \mathsf{mode}_j(L)$，且
2. 若 $\mathsf{mode}_i(L) = \mathsf{mode}_j(L)$，则 $i \leq j$。

令 $\sigma \in \Sigma_m$ 为与 $\langle m \rangle$ 上线性序 $\preceq$ 相关联的置换。我们定义 $\mathsf{sort}(L)$ 为 L 经 σ 置换的结果：

$$\mathsf{sort}(L) = L^\sigma.$$

例 2.1.3.32. 若

$$L = (2, 4, 8, 16) : (64, 1, 2, 4)$$

则

$$\mathsf{sort}(L) = (4, 8, 16, 2) : (1, 2, 4, 64).$$

例 2.1.3.33. 若

$$L = (5, 32, 16) : (1, 5, 5)$$

则

$$\mathsf{sort}(L) = (5, 16, 32) : (1, 5, 5).$$

例 2.1.3.34. 若 L 已排序，则 $\mathsf{sort}(L) = L$。特别地，这蕴含 $\mathsf{sort}(-)$ 是幂等（idempotent）运算：

$$\mathsf{sort}(\mathsf{sort}(L)) = \mathsf{sort}(L).$$

观察 2.1.3.35. 若 L 是扁平布局，则通常 $\Phi_{\mathsf{sort}(L)} \neq \Phi_L$。然而，布局函数 Φ_L 与 $\Phi_{\mathsf{sort}(L)}$ 总具有相同的像（image）。为说明这一点，记

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m), \text{ and}$$
$$\mathsf{sort}(L) = (s_{\sigma(1)}, \ldots, s_{\sigma(m)}) : (d_{\sigma(1)}, \ldots, d_{\sigma(m)})$$

其中 $\sigma \in \Sigma_m$ 是某个置换。若整数 n 位于 Φ_L 的像中，则存在元组 $(x_1, \ldots, x_m) \in \prod_{i=1}^m [0, s_i)$ 使得

$$x_1 d_1 + \cdots + x_m d_m = n$$

此时元组 $(x_{\sigma(1)}, \ldots, x_{\sigma(m)}) \in \prod_{i=1}^m [0, s_{\sigma(i)})$ 满足

$$x_{\sigma(1)} d_{\sigma(1)} + \cdots + x_{\sigma(m)} d_{\sigma(m)} = n.$$

这就证明了 $\mathsf{Image}(\Phi_{\mathsf{sort}}(L)) \subseteq \mathsf{Image}(\Phi_L)$，反向包含同理可证。

2.1.3.6　拼接

回顾一下，若 $X = (x_1, \ldots, x_m)$ 与 $Y = (y_1, \ldots, y_n)$ 是元组，则 X 与 Y 的拼接（concatenation）是元组

$$X \star Y = (x_1, \ldots, x_m, y_1, \ldots, y_n).$$

这一定义（definition）可自然推广到扁平布局的拼接。

定义 2.1.3.36. 设

$$L_1 = S_1 : D_1$$
$$L_2 = S_2 : D_2$$

是扁平布局。则 L_1 与 L_2 的拼接是扁平布局

$$L_1 \star L_2 = S_1 \star S_2 : D_1 \star D_2.$$

扁平布局的拼接满足结合律，因此更一般地，若 $L_1, \ldots, L_k$ 是扁平布局，我们可以构成拼接

$$L_1 \star \cdots \star L_k.$$

例 2.1.3.37. 若 $L_1 = (7, 2) : (2, 1)$ 且 $L_2 = (3, 3, 3) : (0, 10, 30)$，则

$$L_1 \star L_2 = (7, 2, 3, 3, 3) : (2, 1, 0, 10, 30).$$

例 2.1.3.38. 若 $E = () : ()$ 是空布局，则对任意扁平布局 L 有

$$L \star E = L = E \star L.$$

观察 2.1.3.39. 设

$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局。若记

$$L_i = (s_i) : (d_i),$$

则我们可以把 L 写成拼接

$$L = L_1 \star \cdots \star L_m.$$

若 $L_1,\dots,L_k$ 是扁平布局，则拼接 $L_1\star\cdots\star L_k$ 的布局函数由 $L_1,\dots,L_k$ 的布局函数按如下方式确定。

命题 2.1.3.40. 设 $L_1,\dots,L_k$ 分别是形状为 $S_1,\dots,S_k$、大小（*size*）为 $N_1,\dots,N_k$ 的扁平布局。则 $L_1\star\cdots\star L_k$ 的坐标函数（*coordinate function*）

$$[0,S_1\star\cdots\star S_k)\xrightarrow{\varphi_{L_1\star\cdots\star L_k}}\mathbb{Z}$$

等于复合

$$[0,S_1\star\cdots\star S_k)\xrightarrow{\cong}[0,S_1)\times\cdots\times[0,S_k)\xrightarrow{\varphi_{L_1}+\cdots+\varphi_{L_k}}\mathbb{Z},$$

$$X_1\star\cdots\star X_k\longleftrightarrow(X_1,\dots,X_k)$$

而布局函数

$$[0,N_1\cdots N_k)\xrightarrow{\Phi_{L_1\star\cdots\star L_k}}\mathbb{Z}$$

等于复合

$$[0,N_1\cdots N_k)\xrightarrow{\mathsf{colex}^{-1}_{(N_1,\dots,N_k)}}[0,N_1)\times\cdots\times[0,N_k)\xrightarrow{\Phi_{L_1}+\cdots+\Phi_{L_k}}\mathbb{Z}.$$

证明. 对每个 $1\le i\le k$，记 $L_i=S_i:D_i$。第一条断言成立，因为若

$$X\in[0,S_1\star\cdots\star S_k)$$

在典范同构 $[0,S_1\star\cdots\star S_k)\cong[0,S_1)\times\cdots\times[0,S_k)$ 下对应于

$$X_1\star\cdots\star X_k\in[0,S_1)\times\cdots\times[0,S_k)$$

则

$$\begin{aligned}\varphi_{L_1\star\cdots\star L_k}(X)&=X\cdot(D_1\star\cdots\star D_k)\\&=(X_1\star\cdots\star X_k)\cdot(D_1\star\cdots\star D_k)\\&=(X_1\cdot D_1)+\cdots+(X_k\cdot D_k)\\&=\varphi_{L_1}(X_1)+\cdots+\varphi_{L_k}(X_k).\end{aligned}$$

对第二条断言，我们论证下图

$$\begin{tikzcd}
{[0,N_1)\times\cdots\times[0,N_1)} \arrow[r, "\mathsf{colex}^{-1}_{S_1}\times\cdots\times\mathsf{colex}^{-1}_{S_k}"] & {[0,S_1)\times\cdots\times[0,S_1)} \arrow[dr, "\varphi_{L_1}+\cdots+\varphi_{L_k}"] & \\
{[0,N_1\cdots N_k)} \arrow[u, "\mathsf{colex}^{-1}_{(N_1,\dots,N_k)}"] \arrow[r, "\mathsf{colex}^{-1}_{S_1\star\cdots\star S_k}"'] & {[0,S_1\star\cdots\star S_k)} \arrow[u, "\cong"] \arrow[r, "\varphi_{L_1\star\cdots\star L_k}"'] & \mathbb{Z}
\end{tikzcd}$$

交换。左侧的正方形交换，因为余字典序（colexicographic）同构满足结合律；右侧的三角形由第一条断言知其交换。 □

我们可以如下描述拼接布局的重要属性。

命题 2.1.3.41. 设 $L_1, \ldots, L_k$ 是扁平布局。则

1. $L_1 \star \cdots \star L_k$ 的秩为

$$\mathsf{rank}(L_1 \star \cdots \star L_k) = \sum_{i=1}^{k} \mathsf{rank}(L_i),$$

2. $L_1 \star \cdots \star L_k$ 的大小为

$$\mathsf{size}(L_1 \star \cdots \star L_k) = \prod_{i=1}^{k} \mathsf{size}(L_i),$$

3. $L_1 \star \cdots \star L_k$ 的余大小（*cosize*）为

$$\mathsf{cosize}(L_1 \star \cdots \star L_k) = 1 - k + \sum_{i=1}^{k} \mathsf{cosize}(L_i).$$

证明. 对每个 $1 \leq i \leq k$，记 $L_i = S_i : D_i$。对 1，我们计算

$$\mathsf{rank}(L_1 \star \cdots \star L_k) = \mathsf{len}(S_1 \star \cdots \star S_k) = \sum_{i=1}^{k} \mathsf{len}(S_i) = \sum_{i=1}^{k} \mathsf{rank}(L_i).$$

对 2，我们计算

$$\mathsf{size}(L_1 \star \cdots \star L_k) = \mathsf{size}(S_1 \star \cdots \star S_k) = \prod_{i=1}^{k} \mathsf{size}(S_i) = \prod_{i=1}^{k} \mathsf{size}(L_i).$$

对 3，我们计算

$$\begin{aligned}
\mathsf{cosize}(L_1 \star \cdots \star L_k) &= 1 + \mathsf{max}(\Phi_{L_1 \star \cdots \star L_k}) \\
&= 1 + \sum_{i=1}^{k} \mathsf{max}(\Phi_{L_i}) \\
&= 1 - k + (1 + \mathsf{max}(\Phi_{L_1})) + \cdots + (1 + \mathsf{max}(\Phi_{L_1})) \\
&= 1 - k + \mathsf{cosize}(L_1) + \cdots + \mathsf{cosize}(L_k).
\end{aligned}$$

其中我们用到了命题 2.1.3.40 对 $\Phi_{L_1 \star \cdots \star L_k}$ 的等同。 □

2.1.4　扁平合并

我们已经看到，扁平布局 L 的布局函数 Φ_L 是一个重要的不变量（invariant）。在许多情形下，我们只关心布局函数 Φ_L，因而可以自由地使用任何布局函数为 Φ_L 的布局。扁平合并（flat coalesce）运算

$$L \mapsto \mathsf{coal}^\flat(L)$$

为我们提供了布局函数为 Φ_L 的最简扁平布局（见命题 2.1.4.19）。

我们先定义已合并扁平布局的概念。

定义 2.1.4.1. 设 $L = (s_1, \dots, s_m) : (d_1, \dots, d_m)$ 是扁平布局。若

1. 对任意 $1 \le i \le m$，有 $s_i \neq 1$，且
2. 对任意 $1 \le i < m$，有 $s_i d_i \neq d_{i+1}$，

则称 L 是已合并（coalesced）的。

例 2.1.4.2. 扁平布局

$$L = (3, 5, 2) : (7, 21, 4)$$

不是已合并的，因为 $3 \cdot 7 = 21$。

例 2.1.4.3. 扁平布局

$$L = (2, 7, 6) : (1, 3, 10)$$

是已合并的。

例 2.1.4.4. 空布局 $E = () : ()$ 是已合并的。

例 2.1.4.5. 若 $L = (s) : (d)$ 且 $s \neq 1$，则 L 是已合并的。

例 2.1.4.6. 若 $L = (s_1, \dots, s_m) : (d_1, \dots, d_m)$ 是 $\mathsf{rank}(L) > 1$ 的列主序（column-major）布局，则 L 不是已合并的，因为对任意 $1 \le i < m$，有

$$s_i d_i = s_i (s_1 \cdots s_{i-1}) = s_1 \cdots s_i = d_{i+1}.$$

例 2.1.4.7. 若 $L = (s_1, \dots, s_m) : (d_1, \dots, d_m)$ 是行主序（row-major）布局且对所有 $1 \le i \le m$ 有 $s_i > 1$，则 L 是已合并的：若 $1 \le i < m$，则

$$s_i d_i = s_i s_{i+1} \cdots s_m > s_{i+2} \cdots s_m = d_{i+1}.$$

例 2.1.4.8. 形如

$$L = (s_1, \dots, s_m) : (0, \dots, 0)$$

的扁平布局是已合并的当且仅当 $m \le 1$。

若 L 是扁平布局，则我们可以通过移除满足 $s_i = 1$ 的模式、并合并满足 $s_i d_i = d_{i+1}$ 的模式，得到一个与 L 具有相同布局函数的已合并布局 $\mathsf{coal}^\flat(L)$。更确切地说，我们给出如下构造（construction）。

构造 2.1.4.9. 设 L 是扁平布局，记

$$\mathsf{squeeze}(L) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

令 $\sim$ 为 $\langle m \rangle$ 上由如下规则生成的等价关系（equivalence relation）：当

$$s_i d_i = d_{i+1}.$$

时，规定 $i \sim i+1$。商（quotient）$\langle m \rangle / \sim$ 按如下方式定序：若 $i \leq i'$，则 $[i] \leq [i']$；于是我们可将 $\langle m \rangle / \sim$ 与 $\langle \bar{m} \rangle$ 等同，其中 $\bar{m}$ 是 $\langle m \rangle / \sim$ 的大小。若 $i \in \langle \bar{m} \rangle$ 对应等价类（equivalence class）

$$I = \{i', i'+1, \ldots, i'+k\} \in \langle m \rangle / \sim,$$

则我们定义整数 $\bar{s}_i$ 与 $\bar{d}_i$ 为

$$\bar{s}_i = s_{i'} s_{i'+1} \cdots s_{i'+k}$$

与

$$\bar{d}_i = d_{i'},$$

并定义

$$\mathsf{coal}^\flat(L) = (\bar{s}_1, \ldots, \bar{s}_{\bar{m}}) : (\bar{d}_1, \ldots, \bar{d}_{\bar{m}}).$$

观察 2.1.4.10. 细察该定义（definition），我们可以等价地把 $\mathsf{coal}^\flat(L)$ 定义为：从

$$\mathsf{squeeze}(L) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

出发，反复执行操作

$$s_i, s_{i+1} : d_i, s_i d_i \quad \rightsquigarrow \quad s_i s_{i+1} : d_i$$

直至结果已合并，所得到的扁平布局。

例 2.1.4.11. 若 $L = (2, 2, 2, 2, 2) : (8, 16, 1024, 2048, 4096)$，则

$$\mathsf{coal}^\flat(L) = (4, 8) : (8, 1024).$$

例 2.1.4.12. 若 $L = (3, 4, 1, 5) : (1, 8, 3, 32)$，则

$$\mathsf{coal}^\flat(L) = (3, 20) : (1, 8).$$

例 2.1.4.13. 若 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$ 是列主序的，且并非所有 s_i 都等于 1，则

$$\mathsf{coal}^\flat(L) = (s_1 \cdots s_m) : (1).$$

例 2.1.4.14. 若 L 是行主序的，则

$$\mathsf{coal}^\flat(L) = \mathsf{squeeze}(L).$$

我们来论证运算 $L \mapsto \mathsf{coal}^\flat(L)$ 的结果确为已合并布局。

引理 2.1.4.15. *若 L 是扁平布局，则 $\mathsf{coal}^\flat(L)$ 是已合并的。*

证明. 沿用构造 2.1.4.9 的记号（notation），令

$$\mathsf{squeeze}(L) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

并令

$$\mathsf{coal}^\flat(L) = (\bar{s}_1, \ldots, \bar{s}_{\bar{m}}) : (\bar{d}_1, \ldots, \bar{d}_{\bar{m}}).$$

我们要证明 $\mathsf{coal}^\flat(L)$ 是已合并的。设 $1 \leq i \leq \bar{m}$。则 i 对应一个（非空）等价类 $I \in \langle m \rangle / \sim$，且

$$\bar{s}_i = \prod_{i' \in I} s_{i'}$$

是一些整数 $s_{i'} > 1$ 的乘积，故 $\bar{s}_i > 1$。

设 $1 \leq i < \bar{m}$。我们断言 $\bar{s}_i \bar{d}_i \neq \bar{d}_{i+1}$。设 i 对应等价类

$$\{i', i'+1, \ldots, i'+k\} \in \langle m \rangle / \sim,$$

并设 $i+1$ 对应等价类

$$\{i'+k+1, i'+k+2, \ldots, i'+k+\ell\} \in \langle m \rangle / \sim .$$

则利用对 $0 \leq t < k$ 成立的等式 $s_{i'+t} d_{i'+t} = d_{i'+t+1}$，我们可以写出

$$\begin{aligned}
\bar{s}_i \bar{d}_i = \bar{d}_i \bar{s}_i &= d_{i'} s_{i'} s_{i'+1} \cdots s_{i'+k} \\
&= d_{i'+1} s_{i'+1} \cdots s_{i'+k} \\
&\ \vdots \\
&= d_{i'+k} s_{i'+k} \\
&= s_{i'+k} d_{i'+k}
\end{aligned}$$

又由于 $i'+k$ 与 $i'+k+1$ 不属于同一等价类，我们有

$$\bar{s}_i \bar{d}_i = s_{i'+k} d_{i'+k} \neq d_{i'+k+1} = \bar{d}_{i+1}.$$

□

例 2.1.4.16. 若 L 已合并，则 $\mathsf{coal}^\flat(L) = L$。特别地，这蕴含 $\mathsf{coal}^\flat(-)$ 是幂等运算：

$$\mathsf{coal}^\flat(\mathsf{coal}^\flat(L)) = \mathsf{coal}^\flat(L).$$

接下来，我们论证对扁平布局做合并不改变其布局函数。

引理 2.1.4.17. *若 L 是扁平布局，则 $\Phi_{\mathsf{coal}^\flat(L)} = \Phi_L$。*

证明. 由观察 2.1.4.10，只需证明把 $s_i, s_{i+1} : d_i, s_i d_i$ 的一处出现替换为 $s_i s_{i+1} : d_i$ 不改变布局函数。设

$$L = (s_1, \dots, s_m) : (d_1, \dots, d_m)$$

是扁平布局，且存在某个 $1 \le i < m$ 使得 $d_{i+1} = s_i d_i$。令

$$L' = (s'_1, \dots, s'_{m-1}) : (d'_1, \dots, d'_{m-1})$$

表示由 L 合并其第 i 个与第 $(i+1)$ 个模式后得到的扁平布局。更确切地说，我们有

$$s'_j = \begin{cases} s_j & j < i \\ s_i s_{i+1} & j = i \\ s_{j+1} & i < j < m, \end{cases} \qquad \text{and} \qquad d'_j = \begin{cases} d_j & j \le i \\ d_{j+1} & i < j < m. \end{cases}$$

L 的布局函数由

$$\Phi_L(x) = x_1 d_1 + \cdots + x_m d_m$$

给出，其中 $x_j = \left\lfloor \dfrac{x}{s_1 \cdots s_{j-1}} \right\rfloor \mod s_j$；而 L' 的布局函数由

$$\Phi_{L'}(x) = x'_1 d'_1 + \cdots + x'_{m-1} d'_{m-1}$$

给出，其中 $x'_j = \left\lfloor \dfrac{x}{s'_1 \cdots s'_{j-1}} \right\rfloor \mod s'_j$。我们观察到

$$x'_j = \begin{cases} x_j & j < i \\ x_i + x_{i+1} s_i & j = i \\ x_{j+1} & i < j < m, \end{cases}$$

于是

$$\begin{aligned} \Phi_L(x) &= x_1 d_1 + \cdots + x_m d_m \\ &= x_1 d_1 + \cdots + x_i d_i + x_{i+1} s_i d_i + \cdots + x_m d_m \\ &= x_1 d_1 + \cdots + (x_i + x_{i+1} s_i) d_i + \cdots + x_m d_m \\ &= x'_1 d'_1 + \cdots + x'_{m-1} d'_{m-1} \\ &= \Phi_{L'}(x). \end{aligned}$$

□

我们可以利用合并运算来刻画两个扁平布局何时具有相同的布局函数。

命题 2.1.4.18. 设 A 与 B 是扁平布局。则

$$\Phi_A = \Phi_B \quad \Leftrightarrow \quad \mathsf{coal}^\flat(A) = \mathsf{coal}^\flat(B).$$

证明. 若 $\mathsf{coal}^\flat(A) = \mathsf{coal}^\flat(B)$，则由引理 2.1.4.17，有

$$\Phi_A = \Phi_{\mathsf{coal}^\flat(A)} = \Phi_{\mathsf{coal}^\flat(B)} = \Phi_B.$$

反之，设 $\mathsf{coal}^\flat(A) \neq \mathsf{coal}^\flat(B)$。我们论证 $\Phi_A \neq \Phi_B$。记

$$\begin{aligned}\mathsf{coal}^\flat(A) &= (s_1, \ldots, s_m) : (d_1, \ldots, d_m),\\ \mathsf{coal}^\flat(B) &= (t_1, \ldots, t_n) : (e_1, \ldots, e_m).\end{aligned}$$

若 m, n 中一个非零而另一个为 0，则显然 $\Phi_A \neq \Phi_B$，故可设 $m, n \geq 1$。令 i 为使得 $(s_i, d_i) \neq (t_i, e_i)$ 的最小整数。于是特别地，对任意 $j < i$ 有 $s_1 \cdots s_j = t_1 \cdots t_j$。分两种情形考虑:

- (情形 1)：设 $d_i \neq e_i$。令 $N = s_1 \cdots s_{i-1} = t_1 \cdots t_{i-1}$。则

  $$\Phi_{\mathsf{coal}^\flat(A)}(N) = d_i \neq e_i = \Phi_{\mathsf{coal}^\flat(B)}(N)$$

  故 $\Phi_{\mathsf{coal}^\flat(A)} \neq \Phi_{\mathsf{coal}^\flat(B)}$，从而 $\Phi_A \neq \Phi_B$。

- (情形 2)：设 $d_i = e_i$，从而 $s_i \neq t_i$。不失一般性，可设 $s_i < t_i$。令 $N = s_1 \cdots s_i = (t_1 \cdots t_{i-1}) s_i$。则

  $$\Phi_{\mathsf{coal}^\flat(A)}(N) = d_{i+1}$$

  而

  $$\begin{aligned}\Phi_{\mathsf{coal}^\flat(B)}(N) &= s_i e_i\\ &= s_i d_i,\end{aligned}$$

  又由于 $\mathsf{coal}^\flat(A)$ 已合并，有 $d_{i+1} \neq s_i d_i$。我们推出 $\Phi_{\mathsf{coal}^\flat(A)} \neq \Phi_{\mathsf{coal}^\flat(B)}$，从而 $\Phi_A \neq \Phi_B$。

□

上一命题给出了 $\mathsf{coal}^\flat(L)$ 的如下抽象刻画。

命题 2.1.4.19. 若 L 是扁平布局，则 $\mathsf{coal}^\flat(L)$ 是唯一的秩最小且布局函数为 Φ_L 的扁平布局。

证明. 设 L' 是满足 $\Phi_{L'} = \Phi_L$ 的扁平布局。则由命题 2.1.4.18，有

$$\mathsf{coal}^{\flat}(L) = \mathsf{coal}^{\flat}(L'),$$

进而

$$\mathsf{rank}(\mathsf{coal}^{\flat}(L)) = \mathsf{rank}(\mathsf{coal}^{\flat}(L')) \leq \mathsf{rank}(L'),$$

其中等号成立当且仅当

$$L' = \mathsf{coal}^{\flat}(L') = \mathsf{coal}^{\flat}(L).$$

□

2.1.5　紧凑扁平布局

在讨论布局的补（complement）之前，我们必须先定义一类重要的布局（layout），称为紧凑（compact）扁平布局（flat layout）。这类扁平布局的布局函数（layout function）是双射（bijective）的。用刻画布局的标准网格图（grid diagram）来说，扁平布局 L 是紧凑的，如果每个整数 $0 \leq i < \mathsf{size}(L)$ 恰好出现一次。例如，布局

$$A = (3,6):(1,3) = \begin{array}{|c|c|c|c|c|c|}\hline 0 & 3 & 6 & 9 & 12 & 15 \\ \hline 1 & 4 & 7 & 10 & 13 & 16 \\ \hline 2 & 5 & 8 & 11 & 14 & 17 \\ \hline \end{array}$$

是紧凑的，而布局

$$B = (3,6):(2,6) = \begin{array}{|c|c|c|c|c|c|}\hline 0 & 6 & 12 & 18 & 24 & 30 \\ \hline 2 & 8 & 14 & 20 & 26 & 32 \\ \hline 4 & 10 & 16 & 22 & 28 & 34 \\ \hline \end{array}$$

与

$$C = (3,6):(1,2) = \begin{array}{|c|c|c|c|c|c|}\hline 0 & 2 & 4 & 6 & 8 & 10 \\ \hline 1 & 3 & 5 & 7 & 9 & 11 \\ \hline 2 & 4 & 6 & 8 & 10 & 12 \\ \hline \end{array}$$

不是紧凑的。更精确地说，我们有如下定义。

定义 2.1.5.1. 设 L 是扁平布局。若

$$[0,\mathsf{size}(L))\xrightarrow{\Phi_L^{\mathsf{cosize}(L)}}[0,\mathsf{cosize}(L))$$

是同构（isomorphism），则称 L 是紧凑的。

例 2.1.5.2. 扁平布局

$$L=(2,2,2,2):(1,2,4,8)$$

是紧凑的。更一般地，若 L 是列主序（column-major）的，则 L 是紧凑的。

例 2.1.5.3. 扁平布局

$$L=(3,64,32):(2048,32,1)$$

是紧凑的。更一般地，若 L 是行主序（row-major）的，则 L 是紧凑的。

例 2.1.5.4. 空布局（empty layout）

$$E=():()$$

是紧凑的。

例 2.1.5.5. 设

$$L=(s_1,\dots,s_m):(d_1,\dots,d_m)$$

是扁平布局。若 L 的某个模式（mode）满足 $s_i>1$ 且 $d_i=0$，则 L 不是紧凑的。

我们可以如下给出紧凑布局的一个显式刻画。

命题 2.1.5.6. 设 L 是扁平布局，并记

$$\mathsf{squeeze}(L)=(s_1,\dots,s_m):(d_1,\dots,d_m).$$

那么 L 紧凑当且仅当存在置换（*permutation*）$\sigma\in\Sigma_m$，使得

$$d_{\sigma(i)}=s_{\sigma(1)}\cdots s_{\sigma(i-1)}$$

对所有 $1\le i\le m$ 成立。换言之，L 紧凑当且仅当存在置换 $\sigma\in\Sigma_m$ 使得 $\mathsf{squeeze}(L)^\sigma$ 是列主序的。

证明. 设 L 是扁平布局，并记

$$\mathsf{squeeze}(L)=(s_1,\dots,s_m):(d_1,\dots,d_m).$$

首先设 L 紧凑，于是存在置换 $\sigma \in \Sigma_m$ 使得对每个 $1 \leq i \leq m$ 都有 $d_{\sigma(i)} = s_{\sigma(1)} \cdots s_{\sigma(i-1)}$。若记 $S^\sigma = (s_{\sigma(1)}, \ldots, s_{\sigma(m)})$，则可以把 $\Phi_L^{\mathsf{cosize}(L)}$ 写成复合

$$\begin{tikzcd}[0, \mathsf{size}(L)) \arrow[r, "\mathsf{colex}_S^{-1}"] & {[0, S)} \arrow[r, "\cong"] & {[0, S^\sigma)} \arrow[r, "\mathsf{colex}_{S^\sigma}"] & {[0, \mathsf{cosize}(L))}\end{tikzcd}$$

$$(x_1, \ldots, x_m) \longmapsto (x_{\sigma(1)}, \ldots, x_{\sigma(m)})$$

由于其中每个映射都是同构，其复合 $\Phi_L^{\mathsf{cosize}(L)}$ 也是同构。

反之，设 $\Phi_L^{\mathsf{cosize}(L)}$ 是同构。首先，我们指出诸步长（stride）$d_1, \ldots, d_m$ 必定两两不同：设 $d_i = d_j$，并令 δ_i^m 与 δ_j^m 表示第 i 个（相应地，第 j 个）元素为 1、其余元素均为 0 的元组（tuple）。这些元组满足

$$\delta_i^m \cdot (d_1, \ldots, d_m) = d_i = d_j = \delta_j^m \cdot (d_1, \ldots, d_m),$$

而由于 $\Phi_L^{\mathsf{cosize}(L)}$ 是单射（injective）的，必有 $i = j$。给定步长 $d_1, \ldots, d_m$ 两两不同，取 $\sigma \in \Sigma_m$ 为满足

$$d_{\sigma(1)} < d_{\sigma(2)} < \cdots < d_{\sigma(m)}.$$

的置换。我们对 $i \geq 1$ 作归纳，论证 $d_{\sigma(i)} = s_{\sigma(1)} \cdots s_{\sigma(i-1)}$。对基例 $i = 1$，我们注意到 1 在 $\Phi_L^{\mathsf{cosize}(L)}$ 的像（image）中，而 $\Phi_L^{\mathsf{cosize}(L)}$ 的最小非零值是 $d_{\sigma(1)}$，因此 $d_{\sigma(1)} = 1$。设 $i > 1$，并设对所有 $j < i$ 命题已证。考虑步长 $d_{\sigma(i)}$。我们知道不存在形如 $(x_1, \ldots, x_{i-1}, 0, \ldots, 0)^\sigma$ 的元组使得

$$(x_1, \ldots, x_{i-1}, 0, \ldots, 0)^\sigma \cdot (d_1, \ldots, d_m) = s_{\sigma(1)} \cdots s_{\sigma(i-1)},$$

因为这类表达式的最大可能值为

$$\sum_{j=1}^{i-1} (s_{\sigma(j)} - 1)(s_{\sigma(1)} \cdots s_{\sigma(j-1)}) = s_{\sigma(1)} \cdots s_{\sigma(i-1)} - 1.$$

由于 $\Phi_L^{\mathsf{cosize}}(L)$ 是满射（surjective）的，且 $d_{\sigma(i)} < d_{\sigma(i+1)} < \cdots < d_{\sigma(m)}$，可知 $\Phi_L^{\mathsf{cosize}(L)}$ 接下来能取到的值就是 $d_{\sigma(i)}$，故必有 $d_{\sigma(i)} = s_{\sigma(1)} \cdots s_{\sigma(i-1)}$，即证。 □

本节最后，我们给出一组扁平布局 L 紧凑的等价条件。

命题 2.1.5.7. 设 L 是扁平布局。则下列条件等价。

1. L 是紧凑的。
2. $\mathsf{coal}^\flat(L)$ 是紧凑的。
3. $\mathsf{squeeze}(L)$ 是紧凑的。

4. $\mathsf{sort}(L)$ 是紧凑的。

证明．条件 1、2、3 的等价性来自

$$\Phi_L = \Phi_{\mathsf{coal}^\flat(L)} = \Phi_{\mathsf{squeeze}(L)}.$$

还需证明 L 紧凑当且仅当 $\mathsf{sort}(L)$ 紧凑。利用

$$\mathsf{squeeze}(\mathsf{sort}(L)) = \mathsf{sort}(\mathsf{squeeze}(L)),$$

我们有

$$\begin{aligned} \mathsf{sort}(L) \text{ 是紧凑的。} \quad &\Leftrightarrow \quad \mathsf{squeeze}(\mathsf{sort}(L)) \text{ 是紧凑的。} \\ &\Leftrightarrow \quad \mathsf{sort}(\mathsf{squeeze}(L)) \text{ 是紧凑的。} \end{aligned}$$

现在对某个置换 $\tau \in \Sigma_m$ 有 $\mathsf{sort}(\mathsf{squeeze}(L)) = \mathsf{squeeze}(L)^\tau$，因此存在置换 σ 使 $\mathsf{squeeze}(L)^\sigma$ 是列主序的，当且仅当存在置换 $\sigma' \in \Sigma_m$ 使 $\mathsf{sort}(\mathsf{squeeze}(L))$ 是列主序的，即取 $\sigma' = \tau^{-1}\sigma$。于是

$$\begin{aligned} \mathsf{sort}(\mathsf{squeeze}(L)) \text{ 是紧凑的。} \quad &\Leftrightarrow \quad \mathsf{squeeze}(L) \text{ 是紧凑的。} \\ &\Leftrightarrow \quad L \text{ 是紧凑的。} \end{aligned}$$

□

2.1.6　补

本节定义扁平布局互补的概念。回顾定义 2.1.5.1：扁平布局 L 称为紧凑的，如果布局函数

$$\Phi_L^{\mathsf{cosize}(L)} : [0, \mathsf{size}(L)) \to [0, \mathsf{cosize}(L))$$

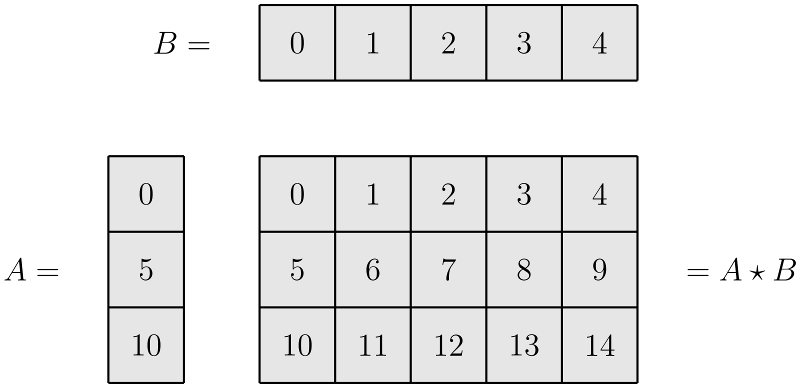
是同构。

定义 2.1.6.1. 设 A 与 B 是扁平布局。若拼接（concatenation）所得的布局 $A \star B$ 是紧凑的，则称 B 是 A 的补（complement），记作 $A \perp B$。

例 2.1.6.2. 若 $A = (3) : (5)$ 且 $B = (5) : (1)$，则 $A \perp B$，因为

$$A \star B = (3, 5) : (5, 1)$$

是紧凑的。



例 2.1.6.3. 若 $A = (4, 2, 10) : (1400, 2, 20)$ 且 $B = (2, 5, 7, 2) : (1, 4, 200, 5600)$，则 $A \perp B$，因为

$$A \star B = (4, 2, 10, 2, 5, 7, 2) : (1400, 2, 20, 1, 4, 200, 5600)$$

是紧凑的。

例 2.1.6.4. 若 A 是扁平布局且 $E = () : ()$ 是空布局，则 $A \perp A$ 当且仅当 A 是紧凑的，因为

$$A \star E = A.$$

例 2.1.6.5. 若 A 与 B 是扁平布局，则

$$A \perp B \quad \Leftrightarrow \quad B \perp A.$$

例 2.1.6.6. 若 A 是扁平布局，则 $A \perp A$ 当且仅当 $\mathsf{size}(A) = 1$。

观察 2.1.6.7. 要让 A 拥有补，必要条件是 Φ_A 为单射。然而，确实存在这样的扁平布局 A：Φ_A 是单射，但 A 没有补。例如，考虑布局

$$A = (2, 2) : (1, 3).$$

A 的布局函数是单射，因为

$$\Phi_A(0) = 0,\ \Phi_A(1) = 1,\ \Phi_A(2) = 3\text{，以及}\Phi_A(3) = 4,$$

但 A 没有补：设

$$B = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是任意其它扁平布局。若不存在元组

$$(x_1, x_2, y_1, \ldots, y_m) \in [0, 2) \times [0, 2) \times [0, s_1) \times \cdots \times [0, s_m)$$

使得 $\varphi_{A \star B}(x_1, x_2, y_1, \ldots, y_m) = 2$，则 $A \star B$ 不是紧凑的。反之，设存在这样的元组 $(x_1, x_2, y_1, \ldots, y_m)$，则 $\varphi_B(y_1, \ldots, y_m) \in \{0, 1, 2\}$。

- （情形 1）：若 $\varphi_B(y_1,\ldots,y_m)=0$，则
$$\varphi_{A\star B}(0,0,0,\ldots,0)=0=\varphi_{A\star B}(0,0,y_1,\ldots,y_m).$$
- （情形 2）：若 $\varphi_B(y_1,\ldots,y_m)=1$，则
$$\varphi_{A\star B}(1,0,0,\ldots,0)=1=\varphi_{A\star B}(0,0,y_1,\ldots,y_m).$$
- （情形 3）：若 $\varphi_B(y_1,\ldots,y_m)=2$，则
$$\varphi_{A\star B}(0,1,0,\ldots,0)=3=\varphi_{A\star B}(1,0,y_1,\ldots,y_m).$$

无论哪种情形，我们都推出 $\varphi_{A\star B}$ 不是单射，因而 $\Phi_{A\star B}$ 也不是。这意味着 $A\star B$ 不是紧凑的，故 B 不是 A 的补。

观察 2.1.6.8. 补并不唯一。例如，若

$$A=(8,8):(2,32),$$

则布局

$$\begin{aligned}B_1&=(2,2):(1,16)\\B_2&=(2,2):(16,1)\\B_3&=(5,2,2,1):(256,1,16,0)\end{aligned}$$

中的每一个都是 A 的补。取而代之，我们有一个（可能为空的）集合

$$\mathsf{complements}^\flat(A)=\{\text{扁平布局}B\mid B\ \text{是}A\ \text{的补}\}.$$

它由所有与 A 互补的布局组成。

给出 B 是 A 的补的一组等价条件将很有用（见命题 2.1.6.10）。为此，我们需要下面这个技术性引理，它描述了拼接与运算 $\mathsf{squeeze}(-)$（压缩）、$\mathsf{sort}(-)$（排序）以及 $\mathsf{coal}^\flat(-)$（合并）之间的相互作用。

引理 2.1.6.9. *设 A 与 B 是扁平布局。则*

1. $\mathsf{squeeze}(A\star B)=\mathsf{squeeze}(A)\star\mathsf{squeeze}(B)$，
2. $\mathsf{sort}(A\star B)=\mathsf{sort}(L\star\mathsf{sort}(B))$，*以及*
3. $\mathsf{coal}^\flat(A\star B)=\mathsf{coal}^\flat(A\star\mathsf{coal}^\flat(B))$。

证明. 记

$$
\begin{aligned}
A &= (s_1, \ldots, s_m) : (d_1, \ldots, d_m) \\
B &= (t_1, \ldots, t_n) : (e_1, \ldots, e_n).
\end{aligned}
$$

令 $\{i_1 < \cdots < i_{m'}\} \subset \langle m \rangle$ 表示满足 $s_{i_k} \neq 1$ 的指标，$\{j_1, \ldots, j_{n'}\} \subset \langle n \rangle$ 表示满足 $t_{j_\ell} \neq 1$ 的指标，则

$$
\begin{aligned}
\mathsf{squeeze}(A \star B) &= (s_{i_1}, \ldots, s_{i_{m'}}, t_{j_1}, \ldots, t_{j_{n'}}) : (d_{i_1}, \ldots, d_{i_{m'}}, e_{j_1}, \ldots, e_{j_{n'}}) \\
&= \mathsf{squeeze}(A) \star \mathsf{squeeze}(B).
\end{aligned}
$$

这就证明了 1。对于 2，我们注意到对任意扁平布局 L 与任意置换 $\sigma \in \Sigma_{\mathsf{len}(L)}$，都有 $\mathsf{sort}(L) = \mathsf{sort}(L^\sigma)$。由

$$
A \star \mathsf{sort}(B) = (A \star B)^\sigma
$$

即得结论，其中 σ 是形如 $\sigma = \mathsf{id} \times \sigma' \in \Sigma_m \times \Sigma_n \subset \Sigma_{m+n}$ 的块置换。对于 3，只需证明 $A \star B$ 与 $A \star \mathsf{coal}^\flat(B)$ 有相同的布局函数。这由命题 2.1.3.40 立得。 □

命题 2.1.6.10. *设 A 与 B 是扁平布局。则下列条件等价。*

1. $A \perp B$。
2. $B \perp A$。
3. $A \perp \mathsf{squeeze}(B)$。
4. $A \perp \mathsf{coal}^\flat(B)$。
5. $A \perp \mathsf{sort}(B)$。

证明. 我们利用命题 2.1.5.7 与引理 2.1.6.9 来证明这些条件的等价性。首先，我们注意到 $\mathsf{sort}(A \star B) = \mathsf{sort}(B \star A)$，由此可得条件 1 与 2 等价。其次，由引理 2.1.6.9，若 $\mathsf{op}(-)$ 是 $\mathsf{squeeze}(-)$、$\mathsf{sort}(-)$ 或 $\mathsf{coal}^\flat(-)$ 中的任一运算，则

$$
\mathsf{op}(A \star B) = \mathsf{op}(A \star \mathsf{op}(B)),
$$

于是

$$
\begin{aligned}
A \perp B \quad &\Leftrightarrow \quad A \star B \text{ 是紧凑的。} \\
&\Leftrightarrow \quad \mathsf{op}(A \star B) \text{ 是紧凑的。} \\
&\Leftrightarrow \quad \mathsf{op}(A \star \mathsf{op}(B)) \text{ 是紧凑的。} \\
&\Leftrightarrow \quad \mathsf{op}(B) \text{ 是} A \text{ 的补.}
\end{aligned}
$$

□

我们想刻画扁平布局何时拥有补。为此，给出如下定义。

定义 2.1.6.11. 设 A 是扁平布局，并记

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

若对所有 $1 \leq i < m$，整数 $s_i d_i$ 都整除（divides）d_{i+1}，则称 A 是可求补的（complementable）。

例 2.1.6.12. 扁平布局

$$A_1 = (4, 1, 1, 4, 4) : (64, 0, 0, 1, 8)$$

是可求补的，而扁平布局

$$A_2 = (4, 4, 4) : (64, 1, 1)$$

不是可求补的。

例 2.1.6.13. 扁平布局

$$A_1 = (10, 2) : (4, 80)$$

是可求补的，而扁平布局

$$A_2 = (10, 2) : (80, 4)$$

不是可求补的。

例 2.1.6.14. 若 A 是紧凑的，则由命题 2.1.5.6，A 是可求补的。

例 2.1.6.15. 设

$$A = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是扁平布局。若存在某个 $1 \leq i \leq m$ 使得 $s_i \neq 1$ 且 $d_i = 0$，则 A 不是可求补的。

若 A 可求补，则可以如下构造 A 的一个补。

构造 2.1.6.16. 设 A 是扁平布局，并记

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

若 A 可求补，则定义扁平布局 $\mathsf{comp}^\flat(A)$ 为

$$\mathsf{comp}^\flat(A) = \mathsf{coal}^\flat(C)$$

其中

$$C = \left(d_1, \frac{d_2}{s_1 d_1}, \frac{d_3}{s_2 d_2}, \ldots, \frac{d_m}{s_{m-1} d_{m-1}}\right) : \left(1, s_1 d_1, s_2 d_2, \ldots, s_{m-1} d_{m-1}\right).$$

例 2.1.6.17. 若 $A=(8,8):(1,8)$，则

$$\mathsf{comp}^\flat(A)=():()$$

是空布局。更一般地，若 A 是紧凑的，则 $\mathsf{comp}^\flat(A)=():()$ 是空布局。

例 2.1.6.18. 若 $A=(2,2):(2,8)$，则

$$\mathsf{comp}^\flat(A)=(2,2):(1,4).$$

例 2.1.6.19. 若 $A=(3,3,8):(16,96,1)$，则

$$\mathsf{comp}^\flat(A)=(2,2):(8,48).$$

下面说明 $\mathsf{comp}^\flat(A)$ 确实是 A 的一个补。

引理 2.1.6.20. *设 A 是扁平布局。若 A 可求补，则*

$$A\perp\mathsf{comp}^\flat(A).$$

证明. 记

$$\mathsf{sort}(\mathsf{squeeze}(A))=(s_1,\dots,s_m):(d_1,\dots,d_m),$$

于是 $\mathsf{comp}^\flat(A)=\mathsf{coal}^\flat(C)$，其中

$$C=\left(d_1,\frac{d_2}{s_1d_1},\frac{d_3}{s_2d_2},\dots,\frac{d_m}{s_{m-1}d_{m-1}}\right):\left(1,s_1d_1,s_2d_2,\dots,s_{m-1}d_{m-1}\right).$$

由命题 2.1.6.10，只需证明 C 是 $\mathsf{sort}(\mathsf{squeeze}(A))$ 的补。事实如此，因为拼接

$$\mathsf{sort}(\mathsf{squeeze}(A))\star C$$

等于

$$\left(s_1,\dots,s_m,d_1,\frac{d_2}{s_1d_1},\dots,\frac{d_m}{s_{m-1}d_{m-1}}\right):(d_1,\dots,d_m,1,s_1d_1,\dots,s_{m-1}d_{m-1}),$$

而它的排序等于

$$\left(d_1,s_1,\frac{d_2}{s_1d_1},\dots,\frac{d_m}{s_{m-1}d_{m-1}},s_m\right):(1,d_1,s_1d_1,\dots,s_{m-1}d_{m-1},d_m)$$

后者是列主序的。 □

我们已经证明：若 A 可求补，则 A 拥有补。接下来证明其逆也成立。

命题 2.1.6.21. *设 A 是扁平布局。则 A 拥有补 B 当且仅当 A 可求补。*

证明. 若 A 可求补，则由引理 2.1.6.20，布局 $B = \mathsf{comp}^\flat(A)$ 是 A 的补。反之，设 A 有补 B，并考虑扁平布局

$$\begin{aligned} L &= \mathsf{sort}\big(\mathsf{squeeze}(A) \star \mathsf{squeeze}(B)\big) \\ &= (s_1, \ldots, s_m) : (d_1, \ldots, d_n). \end{aligned}$$

由于 $\Phi_L(0) = 0$ 且 Φ_L 是单射，可知 $d_1 \neq 0$。我们将论证 $d_i = s_1 \cdots s_{i-1}$，即 L 是列主序的。由于

$$\Phi_L^{\mathsf{cosize}(L)} : [0, \mathsf{size}(L)) \to [0, \mathsf{cosize}(L))$$

是双射，可知 1 在 Φ_L 的像中，从而 $d_1 = 1$。设 $1 < i \leq m$，并设对所有 $j < i$ 已证 $d_j = s_1 \cdots s_{j-1}$。考虑步长 d_i。我们知道不存在 $(x_1, \ldots, x_{i-1}, 0, \ldots, 0)$ 使得 $(x_1, \ldots, x_{i-1}, 0, \ldots, 0) \cdot (d_1, \ldots, d_m) = s_1 \cdots s_{i-1}$，因为这类表达式的最大可能值为

$$\sum_{j=1}^{i-1} (s_j - 1)(s_1 \cdots s_{j-1}) = s_1 \cdots s_i - 1.$$

由于 Φ_L 是满射，且 $d_i \leq d_{i+1} \leq \cdots \leq d_m$，可知 Φ_L 接下来能取到的值就是 d_i，故必有 $d_i = s_1 \cdots s_{i-1}$，即证。

回到主要目标，考虑布局

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s'_1, \ldots, s'_{m'}) : (d'_1, \ldots, d'_{m'}).$$

于是存在 $j_1 < \cdots < j_{m'}$，使得对每个 $1 \leq i \leq m'$ 有 $s'_i = s_{j_i}$ 且 $d'_i = d_{j_i}$。若 $1 \leq i < m'$，则

$$s'_i d'_i = s_{j_i} d_{j_i} = s_{j_i} s_1 \cdots s_{j_i - 1}$$

整除

$$d'_{i+1} = s_1 \cdots s_{j_{i+1}-1},$$

因此我们断定 A 是可求补的。 □

我们的下一个目标是给出扁平布局 A 的补 $\mathsf{comp}^\flat(A)$ 的一个抽象刻画。为此，我们需要下面的引理。

引理 2.1.6.22. 设 A 是扁平布局。若 A 既可求补又已排序（*sorted*），则布局函数

$$\Phi_A : [0, \mathsf{size}(A)) \to \mathbb{Z}$$

是递增的。

证明. 记

$$A = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

若 $1 \le k \le m$，我们断言

$$d_1(s_1-1)+d_2(s_2-1)+\cdots+d_{k-1}(s_{k-1}-1) \le d_k.$$

当 $k=1$ 时该式空真（vacuously）成立，而对 k 作归纳，有

$$\begin{aligned} d_1(s_1-1)+\cdots+d_{k-2}(s_{k-2}-1)+d_{k-1}(s_{k-1}-1) &\le d_{k-1}+d_{k-1}(s_k-1) \\ &= d_{k-1}s_{k-1} \\ &\le d_k. \end{aligned}$$

现设 $x, y \in [0, \mathsf{size}(A))$ 且 $x \le y$。在余字典序（colexicographic）同构下，这些整数对应于元组

$$(x_1,\ldots,x_m),(y_1,\ldots,y_m) \in [0,s_1)\times\cdots\times[0,s_m)$$

由于 $x \le y$，可知存在最大的 $1 \le k \le m$ 使得 $x_k < y_k$，且对所有 $k < \ell \le m$ 有 $x_\ell = y_\ell$。于是可以计算

$$\begin{aligned} \Phi_A(x) &= d_1x_1+\cdots+d_{k-1}x_{k-1}+d_kx_k+d_{k+1}x_{k+1}+\cdots+d_mx_m \\ &= d_1x_1+\cdots+d_{k-1}x_{k-1}+d_kx_k+d_{k+1}y_{k+1}+\cdots+d_my_m \\ &\le d_1(s_1-1)+\cdots+d_{k-1}(s_{k-1}-1)+d_kx_k+d_{k+1}y_{k+1}+\cdots+d_my_m \\ &\le d_k+d_kx_k+d_{k+1}y_{k+1}+\cdots+d_my_m \\ &= d_k(x_k+1)+d_{k+1}y_{k+1}+\cdots+d_my_m \\ &\le d_ky_k+d_{k+1}y_{k+1}+\cdots+d_my_m \\ &\le d_1y_1+\ldots d_my_m \\ &= \Phi_A(y). \end{aligned}$$

□

命题 2.1.6.23. *设 A 与 B 是扁平布局。若*

1. *$A \perp B$，*
2. *$\mathsf{size}(B) = \mathsf{size}(\mathsf{comp}^\flat(A))$，*
3. *B 已合并（coalesced），且*
4. *B 已排序，*

则 $B = \mathsf{comp}^\flat(A)$。

证明. 条件 1 与 2 蕴含 Φ_B 与 $\Phi_{\mathsf{comp}^\flat(A)}$ 有相同的像。由于 B 与 $\mathsf{comp}^\flat(A)$ 都已排序，由引理 2.1.6.22 可知 Φ_B 与 $\Phi_{\mathsf{comp}^\flat(A)}$ 都是递增的。综合这两点，即得 $\Phi_B = \Phi_{\mathsf{comp}^\flat(A)}$。再由命题 2.1.4.18 与条件 3 可得

$$B = \mathsf{coal}^\flat(B) = \mathsf{coal}^\flat(\mathsf{comp}^\flat(A)) = \mathsf{comp}^\flat(A).$$

□

定义 2.1.6.24. 设 A 与 B 是扁平布局，N 是正整数。若 B 是 A 的补，且

$$\mathsf{size}(A) \cdot \mathsf{size}(B) = N.$$

则称 B 是 A 的 N 补（N-complement）。

定义 2.1.6.25. 设 A 是扁平布局，并记

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

若满足

1. 对所有 $1 \leq i < m$，整数 $s_i d_i$ 整除 d_{i+1}，且
2. 整数 $s_m d_m$ 整除 N，

则称 A 是 N-可求补的（N-complementable）。

观察 2.1.6.26. 若 A 可求补，且 $s_m : d_m$ 是布局 $\mathsf{sort}(\mathsf{squeeze}(A))$ 中的最后一个模式，则 A 是 N-可求补的当且仅当 N 是 $s_m d_m$ 的正整数倍。

观察 2.1.6.27. N-补并不唯一。例如，若 $A = (2, 2) : (1, 50)$ 且 $N = 100$，则布局 $B_1 = (25) : (2)$ 与 $B_2 = (5, 5) : (2, 10)$ 都是 A 的 N-补。更一般地，若 B 是 A 的 N-补，则 $\mathsf{coal}^\flat(B)$ 也是 A 的 N-补。

注 2.1.6.28. 设 A 是扁平布局，B_1 与 B_2 是 A 的 N-补。则布局函数 Φ_{B_1} 与 Φ_{B_2} 未必相等，但它们必然有相同的像。例如，若 $A = (4) : (63)$ 且 $N = 252$，则 $B_1 = (7, 9) : (1, 7)$ 与 $B_2 = (9, 7) : (7, 1)$ 都是 A 的 N-补，而 $\Phi_{B_1} \neq \Phi_{B_2}$，因为

$$\Phi_{B_1}(1) = 1 \neq 7 = \Phi_{B_2}(1).$$

更一般地，若 B 是 A 的 N-补，则 $\mathsf{sort}(B)$ 也是 A 的 N-补。

构造 2.1.6.29. 设 A 是扁平布局，N 是正整数，且 A 是 N-可求补的。若记

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m),$$

则定义扁平布局 $\mathsf{comp}^\flat(A, N)$ 为

$$\mathsf{comp}^\flat(A, N) = \mathsf{coal}^\flat(C)$$

其中

$$C = \left(d_1, \frac{d_2}{s_1 d_1}, \frac{d_3}{s_2 d_2}, \ldots, \frac{N}{s_m d_m}\right) : \left(1, s_1 d_1, s_2 d_2, \ldots, s_m d_m\right).$$

例 2.1.6.30. 若 $A = (3, 10) : (80, 4)$ 且 $N = 2400$，则

$$\mathsf{comp}^\flat(A, N) = (4, 2, 10) : (1, 40, 240).$$

引理 2.1.6.31. 设 A 是扁平布局，N 是正整数，且 A 是 N-可求补的。则 $\mathsf{comp}^\flat(A,N)$ 是 A 的 N-补。

证明. 记

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s_1,\dots,s_m):(d_1,\dots,d_m),$$

于是 $\mathsf{comp}^\flat(A,N)=\mathsf{coal}^\flat(C)$，其中

$$C=\left(d_1,\frac{d_2}{s_1d_1},\frac{d_3}{s_2d_2},\dots,\frac{N}{s_md_m}\right):(1,s_1d_1,s_2d_2,\dots,s_md_m)\,.$$

首先，我们计算

$$\begin{aligned}\mathsf{size}(A)\cdot\mathsf{size}(B) &= \left(\prod_{i=1}^m s_i\right)\cdot\left(d_1\cdot\left(\prod_{i=2}^m\frac{d_i}{s_{i-1}d_{i-1}}\right)\cdot\frac{N}{s_md_m}\right)\\ &=\frac{\left(\prod_{i=1}^m s_i\right)\left(\prod_{i=1}^m d_i\right)}{\left(\prod_{i=1}^m s_id_i\right)}\cdot N\\ &=N.\end{aligned}$$

我们需要验证 $A\star B$ 是紧凑的。等价地，需要验证 $\Phi^N_{A\star B}$ 是同构。由引理 2.1.5.6，只需证明

$$\mathsf{squeeze}(A)\star\mathsf{squeeze}(B)$$

是紧凑的。事实如此，因为该布局等于

$$\left(s_1,\dots,s_m,d_1,\frac{d_2}{s_1d_1},\dots,\frac{N}{s_md_m}\right):(d_1,\dots,d_m,1,s_1d_1,\dots,s_md_m)$$

而其排序

$$\mathsf{sort}(\mathsf{squeeze}(A)\star\mathsf{squeeze}(B))$$

等于

$$\left(d_1,s_1,\frac{d_2}{s_1d_1},\dots,\frac{d_m}{s_{m-1}d_{m-1}},s_m,\frac{N}{s_md_m}\right):(1,d_1,s_1d_1,\dots,s_{m-1}d_{m-1},d_m,s_md_m)$$

后者是列主序的。 □

命题 2.1.6.32. 设 A 是扁平布局，N 是正整数。则存在 A 的 N-补 B 当且仅当 A 是 N-可求补的。

证明. 若 A 是 N-可求补的，则由引理 2.1.6.31，布局 $B = \mathsf{comp}^\flat(L, N)$ 是 A 的 N-补。

另一方面，设存在 A 的 N-补 B。考虑扁平布局

$$\begin{aligned} L &:= \mathsf{sort}\big(\mathsf{squeeze}(A) \star \mathsf{squeeze}(B)\big) \\ &= (s_1, \ldots, s_m) : (d_1, \ldots, d_n). \end{aligned}$$

由于 $\Phi_L(0) = 0$ 且 Φ_L 是单射，可知 $d_1 \neq 0$。我们将论证 $d_i = s_1 \cdots s_{i-1}$，即 L 是列主序的。由于

$$\Phi_L^N : [0, N) \to [0, N)$$

是双射，可知 1 在 Φ_L 的像中，从而 $d_1 = 1$。设 $1 < i \leq m$，并设对所有 $j < i$ 已证 $d_j = s_1 \cdots s_{j-1}$。考虑步长 d_i。我们知道不存在 $(x_1, \ldots, x_{i-1}, 0, \ldots, 0)$ 使得 $(x_1, \ldots, x_{i-1}, 0, \ldots, 0) \cdot (d_1, \ldots, d_m) = s_1 \cdots s_{i-1}$，因为这类表达式的最大可能值为

$$\sum_{j=1}^{i-1} (s_j - 1)(s_1 \cdots s_{j-1}) = s_1 \cdots s_i - 1.$$

由于 Φ_L 是满射，且 $d_i \leq d_{i+1} \leq \cdots \leq d_m$，可知 Φ_L 接下来能取到的值就是 d_i，故必有 $d_i = s_1 \cdots s_{i-1}$，即证。

回到主要目标，考虑布局

$$\mathsf{sort}(\mathsf{squeeze}(A)) = (s'_1, \ldots, s'_{m'}) : (d'_1, \ldots, d'_{m'}).$$

于是存在 $j_1 < \cdots < j_{m'}$，使得对每个 $1 \leq i \leq m'$ 有 $s'_i = s_{j_i}$ 且 $d'_i = d_{j_i}$。若 $1 \leq i < m'$，则

$$s'_i d'_i = s_{j_i} d_{j_i} = s_{j_i} s_1 \cdots s_{j_i - 1}$$

整除

$$d'_{i+1} = s_1 \cdots s_{j_{i+1} - 1}.$$

若 $i = m'$，则

$$s'_{m'} d'_{m'} = s_{j_{m'}} d_{j_{m'}} = s_{j_{m'}} s_1 \cdots s_{j_{m'} - 1}$$

整除

$$N = s_1 \cdots s_m.$$

于是我们断定 A 是 N-可求补的。 □

命题 2.1.6.33. *设 N 是正整数，A 是 N-可求补的扁平布局。若扁平布局 B 满足*

1. *B 是 L 的 N-补，*
2. *B 已合并，且*
3. *B 已排序，*

则 $B = \mathsf{comp}^\flat(A, N)$。

证明. 条件 1 与 2 蕴含 Φ_B 与 $\Phi_{\mathsf{comp}^\flat(A,N)}$ 有相同的像。由于 B 与 $\mathsf{comp}^\flat(A, N)$ 都已排序，由引理 2.1.6.22 可知 Φ_B 与 $\Phi_{\mathsf{comp}^\flat(A,N)}$ 都是递增的。综合这两点，即得 $\Phi_B = \Phi_{\mathsf{comp}^\flat(A,N)}$。再由命题 2.1.4.18 与条件 3 可得

$$B = \mathsf{coal}^\flat(B) = \mathsf{coal}^\flat(\mathsf{comp}^\flat(A, N)) = \mathsf{comp}^\flat(A, N).$$

□

引理 2.1.6.34. 设 A 是扁平布局。若 $N_1 \leq N_2$ 是正整数，且 A 是 N_1-可求补的、A 是 N_2-可求补的，则

$$\Phi_{\mathsf{comp}^\flat(A,N_2)}\ |_{[0,N_1)} = \Phi_{\mathsf{comp}^\flat(A,N_1)}.$$

证明. 记

$$\begin{aligned} \mathsf{sort}(\mathsf{squeeze}(A)) &= (s_1, \ldots, s_m) : (d_1, \ldots, d_m), \\ C &= \left(d_1, \frac{d_2}{s_1 d_1}, \frac{d_3}{s_2 d_2}, \ldots, \frac{d_m}{s_{m-1} d_{m-1}}\right) : \left(1, s_1 d_1, s_2 d_2, \ldots, s_{m-1} d_{m-1}\right) \end{aligned}$$

并记

$$\begin{aligned} E_1 &= \left(\frac{N_1}{s_m d_m}\right) : (s_m d_m), \\ E_2 &= \left(\frac{N_2}{s_m d_m}\right) : (s_m d_m), \\ C_1 &= C \star E_1, \\ C_2 &= C \star E_2, \end{aligned}$$

于是

$$\begin{aligned} \mathsf{comp}^\flat(A) &= \mathsf{coal}^\flat(C) \\ \mathsf{comp}^\flat(A, N_1) &= \mathsf{coal}^\flat(C_1) \\ \mathsf{comp}^\flat(A, N_2) &= \mathsf{coal}^\flat(C_2). \end{aligned}$$

我们有一个交换图（commutative diagram）

$$\begin{tikzcd}
{[0, \mathsf{size}(C_1))} \arrow[r, "\mathrm{colex}^{-1}_{(\mathsf{size}(C), N_1)}"] \arrow[d, "\subseteq"] & {[0, \mathsf{size}(C)) \times [0, N_1)} \arrow[r, "\Phi_C \times s_m d_m"] \arrow[d, "\mathsf{id} \times \subseteq"] & \mathbb{Z} \times \mathbb{Z} \arrow[r, "+"] \arrow[d, "\mathsf{id}"] & \mathbb{Z} \arrow[d, "\mathsf{id}"] \\
{[0, \mathsf{size}(C_2))} \arrow[r, "\mathrm{colex}^{-1}_{(\mathsf{size}(C), N_2)}"] & {[0, \mathsf{size}(C)) \times [0, N_2)} \arrow[r, "\Phi_C \times s_m d_m"] & \mathbb{Z} \times \mathbb{Z} \arrow[r, "+"] & \mathbb{Z}
\end{tikzcd}$$

其中，由命题 2.1.3.40，上行的复合是 $C_1 = C \star E_1$ 的布局函数，下行的复合是 $C_2 = C \star E_2$ 的布局函数。这告诉我们 Φ_{C_2} 在 $[0, \mathsf{size}(C_2))$ 上的限制就是 Φ_{C_1}，再由

$$\begin{aligned} \Phi_{\mathsf{comp}^\flat(A,N_1)} &= \Phi_{C_1} \\ \Phi_{\mathsf{comp}^\flat(A,N_2)} &= \Phi_{C_2}. \end{aligned}$$

即得结论。 □

2.1.7　进一步的运算

本节定义扁平布局上的另外几个运算，即*复合*（composition）、*扁平切分*（flat division）与*扁平乘积*（flat product）。它们是（嵌套）布局上更自然的运算经扁平化后的变体。我们不常使用这些运算，但为完整起见仍将其收入。

2.1.7.1　复合

若 A 与 B 是扁平布局，则复合 $B \circ A$ 是一个扁平布局，其布局函数是 A 与 B 的布局函数的复合。更精确地说，我们有如下定义。

定义 2.1.7.1. 设 A 与 B 是扁平布局。若

1. C 是非退化（non-degenerate）的，
2. $\mathsf{shape}(A) = \mathsf{shape}(R)$，
3. $\Phi_R = \Phi_B \circ \Phi_A^{\mathsf{size}(B)}$，

则称扁平布局 C 是 A 与 B 的*复合*，记作 $C = B \circ A$。

注 2.1.7.2. 注意，定义中的条件 2 保证 Φ_R 与 Φ_A 有相同的定义域（domain），条件 3 则蕴含 $\mathsf{cosize}(A) \leq \mathsf{size}(B)$。

例 2.1.7.3. 若 $A = (2, 3) : (5, 6)$ 且 $B = (80) : (10)$，则

$$B \circ A = (2, 3) : (50, 60).$$

更一般地，若

$$A = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是非退化扁平布局，且

$$B = (t) : (e)$$

是满足 $t \geq \mathsf{cosize}(A)$ 的秩（rank）1 扁平布局，则 A 与 B 可复合（composable），且

$$B \circ A = (s_1, \ldots, s_m) : (td_1, \ldots, td_m).$$

例 2.1.7.4. 若 $A = (128, 128) : (0, 0)$ 且 $B = (64, 32) : (1, 64)$，则

$$B \circ A = (128, 128) : (0, 0).$$

更一般地，若 A 是每个步长元素均为零的扁平布局，B 是任意扁平布局，则 A 与 B 可复合，且 $B \circ A = A$。

例 2.1.7.5. 若 $A = (64,32):(2,256)$ 且 $B = (2048,2048):(1,2048)$，则

$$B \circ A = (64,32):(2,256).$$

更一般地，若 A 是任意扁平布局，B 是满足 $\mathsf{cosize}(A) \le \mathsf{size}(B)$ 的列主序扁平布局，则 $B \circ A = A$。

例 2.1.7.6. 若 $A = (4):(2)$ 且 $B = (2,2,6):(12,6,1)$，则不存在满足 $R = B \circ A$ 的扁平布局 R。

注 2.1.7.7. 若 B' 与 B 有相同的布局函数，则 $B \circ A = B' \circ A$。

注 2.1.7.8. 扁平布局是更一般的概念*布局*（layout，定义 2.3.1.1）的特例。事实上，存在这样的情形（例如例 2.1.7.6）：不存在满足 $C = B \circ A$ 的扁平布局 C，却存在满足 $C = B \circ A$ 的（嵌套）布局 C（见例 2.3.7.6）。因此，我们把关于复合的进一步讨论与分析推迟到给出完全一般意义上的布局定义之后。

2.1.7.2　扁平切分

若 A 与 B 是扁平布局，则 A 除以 B 的扁平切分是布局更自然的*逻辑切分*（logical division）经扁平化后的版本。细节见第 2.3.8 节。

定义 2.1.7.9. 设 A 与 B 是扁平布局，且 B 是 $\mathsf{size}(A)$-可求补的，其中

$$B^c = \mathsf{comp}^\flat(B, \mathsf{size}(A)).$$

我们定义 A 除以 B 的*扁平切分*为扁平布局

$$A \oslash^\flat B = A \circ (B \star B^c).$$

例 2.1.7.10. 若 $A = (2,2,2,2):(1,4,2,8)$ 且 $B = (2,2):(4,2)$，则

$$A \oslash^\flat B = (2,2,2,2):(4,2,1,8).$$

例 2.1.7.11. 若 $A = (3,5,9,6):(54,0,6,1)$ 且 $B = (6,3):(135,1)$，则

$$A \oslash^\flat B = (6,3,5,9):(1,54,0,6).$$

例 2.1.7.12. 若 A 是任意扁平布局且 $B = ():()$ 是空布局，则

$$A \oslash^\flat B = A.$$

2.1.7.3　扁平乘积

若 A 与 B 是扁平布局，则 A 与 B 的扁平乘积 $A \otimes^\flat B$ 是布局更自然的逻辑乘积（logical product）经扁平化后的版本。细节见第 2.3.9 节。

定义 2.1.7.13. 设 A 与 B 是扁平布局，且 A 是 $\mathsf{size}(A) \cdot \mathsf{cosize}(B)$-可求补的，其中

$$A^c = \mathsf{comp}^\flat(A, \mathsf{size}(A) \cdot \mathsf{cosize}(B)).$$

我们定义 A 与 B 的扁平乘积为

$$A \otimes^\flat B = A \star (A^c \circ B).$$

例 2.1.7.14. 若 $A = (2,2,2) : (1,2,4)$ 且 $B = (2,2,2) : (1,2,4)$，则

$$A \otimes^\flat B = (2,2,2,2,2,2) : (1,2,4,8,16,32).$$

例 2.1.7.15. 若 $A = (2,2,2) : (1,2,4)$ 且 $B = (3,5) : (5,1)$，则

$$A \otimes^\flat B = (2,2,2,3,5) : (1,2,4,40,8).$$

例 2.1.7.16. 若 A 是任意扁平布局且 $B = () : ()$ 是空布局，则

$$A \otimes^\flat B = A.$$

2.1.8　可处理扁平布局

本节中我们定义一类性质特别良好的扁平布局（flat layout），称为可处理（tractable）扁平布局。可处理扁平布局包含了最受关注的一批重要例子，例如行主序（row-major）、列主序（column-major）、紧凑（compact）以及可求补（complementable）布局（layout）。后文将会看到，可处理扁平布局恰好是来自某个范畴（category）**Tuple** 的布局。

定义 2.1.8.1. 设 L 是扁平布局，并记

$$\mathsf{sort}(L) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

我们称 L 是可处理的，如果对每个 $1 \leq i < m$，都有

1. $d_i = 0$，或
2. $s_i d_i$ 整除 d_{i+1}。

例 2.1.8.2. 扁平布局

$$L = (12) : (17)$$

是可处理的。更一般地，任何秩（rank）为 1 的扁平布局都是可处理的。

例 2.1.8.3. 扁平布局

$$L = (2,4,32) : (1,2,8)$$

是可处理的。更一般地，任何列主序布局

$$L = (s_1, \ldots, s_m) : (1, s_1, \ldots, s_1 \cdots s_{m-1})$$

都是可处理的。

例 2.1.8.4. 扁平布局

$$L = (2,4,32) : (128,32,1)$$

是可处理的。更一般地，任何行主序布局

$$L = (s_1, \ldots, s_m) : (s_2 \cdots s_m, \ldots, s_m, 1)$$

都是可处理的。

例 2.1.8.5. 扁平布局

$$L = (3,3,1,3,3,1,3) : (81,1,0,9,3,0,27)$$

是可处理的。更一般地，任何紧凑扁平布局都是可处理的。

例 2.1.8.6. 扁平布局

$$L = (3,7,7) : (0,15,0)$$

是可处理的。更一般地，任何恰有一个非零步长（stride）的扁平布局都是可处理的。

例 2.1.8.7. 扁平布局

$$L = (2,2,2,2) : (1,2048,16,64)$$

是可处理的。更一般地，任何可求补扁平布局都是可处理的。

例 2.1.8.8. 设 L 是扁平布局。若 L 可处理且 $I \subset \langle m \rangle$ 是任意子集，则限制（restriction）$L\mid_I$ 是可处理的。特别地，若 L 可处理，则 $\mathsf{squeeze}(L)$ 与 $\mathsf{filter}(L)$ 都是可处理的。

例 2.1.8.9. 扁平布局

$$L = (4,8) : (3,3)$$

不是可处理的。特别地，这说明可处理扁平布局 L_1 与 L_2 的拼接（concatenation）$L_1 \star L_2$ 未必可处理。

观察 2.1.8.10. 若 L 是可处理扁平布局且 $\mathsf{stride}(L)$ 的元素（entry）均不等于 0，则 L 可求补。特别地，若 L 可处理，则 $\mathsf{filter}(L)$ 可求补。

本节最后，我们列举扁平布局 L 可处理的一族等价条件。

命题 2.1.8.11. 设 L 是扁平布局。则下列条件等价。

1. L 可处理。
2. $\mathsf{sort}(L)$ 可处理。
3. $\mathsf{filter}(L)$ 可处理。
4. $\mathsf{filter}(L)$ 可求补。

证明. 设 L 是扁平布局。

- $(1 \Leftrightarrow 2)$：这由
$$\mathsf{sort}(\mathsf{sort}(L)) = \mathsf{sort}(L).$$
这一事实得出。
- $(1 \Leftrightarrow 3)$：这由
$$\mathsf{sort}(\mathsf{filter}(L)) = \mathsf{filter}(\mathsf{sort}(L)).$$
这一事实得出。
- $(3 \Leftrightarrow 4)$：这由如下事实得出：若
$$L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$
是一个每个步长元素 d_i 都非零的扁平布局，则可处理性的定义与可求补性的定义一致。

$\square$

2.2　嵌套元组

本节引入嵌套元组（nested tuple），它是为在最一般意义下定义布局所需的元组（tuple）的推广。

2.2.1　轮廓

嵌套元组 S 由它的*扁平化*(flattening)——一个普通元组——与它的*轮廓*(profile)——描述 S 上括号模式的对象——所决定。我们如下精确定义轮廓。

定义 2.2.1.1. 轮廓 P 是以下二者之一

1. $P = *$，或
2. 对某个 $r \geq 0$，由轮廓 $P_1, \ldots, P_r$ 组成的元组 $P = (P_1, \ldots, P_r)$。

我们记 Profile 为轮廓的集合。

例 2.2.1.2. 以下是一些轮廓的例子。

$$\begin{aligned}
P_1 &= (*, *)\\
P_2 &= (*, (*, *))\\
P_3 &= ((*, *), (*, *))\\
P_4 &= ((*, *, *), (*, ()))\\
P_5 &= ()\\
P_6 &= *
\end{aligned}$$

我们来定义轮廓的一些重要属性。

定义 2.2.1.3. 设 P 是轮廓。

- X 的秩为

$$\mathsf{rank}(P) = \begin{cases} 1 & P = * \\ r & P = (P_1, \ldots, P_r) \text{ 是一个由轮廓组成的元组。} \end{cases}.$$

- P 的长度（length）为

$$\mathsf{len}(P) = \begin{cases} 1 & P = * \\ \sum_{i=1}^{r} \mathsf{len}(P_i) & P = (P_1, \ldots, P_r) \text{ 是一个由轮廓组成的元组。} \end{cases}$$

- P 的深度（depth）为

$$\mathsf{depth}(P) = \begin{cases} 0 & P = * \\ 1 + \mathsf{max}_{1\le i\le r}(\mathsf{depth}(P_i)) & P = (P_1, \ldots, P_r) \text{ 为轮廓元组。} \end{cases}$$

例 2.2.1.4. 以下是一些轮廓的例子，连同它们的秩、长度与深度：

$$\begin{array}{llll} P = * & \mathsf{rank}(P) = 1, & \mathsf{len}(P) = 1, & \mathsf{depth}(P) = 0 \\ P = (*, *, *) & \mathsf{rank}(P) = 3, & \mathsf{len}(P) = 3, & \mathsf{depth}(P) = 1 \\ P = (((*, *), *, *), *, *), & \mathsf{rank}(P) = 3, & \mathsf{len}(P) = 6, & \mathsf{depth}(P) = 3 \\ P = (((), ()), (*, (*, *))), & \mathsf{rank}(P) = 2, & \mathsf{len}(P) = 3, & \mathsf{depth}(P) = 3 \end{array}$$

定义 2.2.1.5. 设 P 是满足 $\mathsf{rank}(P) = r$ 的轮廓。若 $1 \le i \le r$，则 P 的第 i 个模式（mode）为

$$\mathsf{mode}_i(P) = \begin{cases} P & \mathsf{depth}(P) = 0 \text{ (从而} i = r = 1), \\ P_i & P = (P_1, \ldots, P_r) \text{ 的深度} \ge 1. \end{cases}$$

例 2.2.1.6. 若 $P = ((*, *), (()), ((*, (*, *))))$，则 P 的各模式为

$$\begin{aligned} \mathsf{mode}_1(P) &= (*, *) \\ \mathsf{mode}_2(P) &= (()) \\ \mathsf{mode}_3(P) &= (*, (*, *)). \end{aligned}$$

下面的记号会很有用。

记号 2.2.1.7. 设 P 是深度 > 0 的轮廓。对任意 $1 \le j \le \mathsf{rank}(P)$，我们记

$$\begin{aligned} \mathsf{len}_j(X) &= \mathsf{len}(\mathsf{mode}_j(P)), \\ \mathsf{len}_{<j}(P) &= \sum_{i=1}^{j-1} \mathsf{len}_i(X), \\ \mathsf{len}_{\le j}(X) &= \mathsf{len}_{<j}(P) + \mathsf{len}_j(P) \end{aligned}$$

轮廓所支持的最重要运算是代入（substitution）：若 Q 是长度为 m 的轮廓，且 $P_1, \ldots, P_m$ 是轮廓，则通过对每个 $1 \le i \le m$ 把 Q 的第 i 个元素替换为轮廓 P_i，我们可以得到一个新的轮廓 $(P_1, \ldots, P_m)_Q$。更精确地说，我们有如下定义。

定义 2.2.1.8. 设 Q 是长度为 m 的轮廓，并设 $P_1, \ldots, P_m$ 是轮廓。则 $P_1, \ldots, P_m$ 的 Q-代入是如下定义的轮廓

$$(P_1, \ldots, P_m)_Q$$

记 $\mathsf{depth}(Q) = d$ 且 $\mathsf{rank}(Q) = r$。

- 若 $d=0$，则 $m=1$，我们定义

$$(P_1)_Q = P_1.$$

- 再设 $d>0$，且我们已对所有深度 $<d$ 的轮廓 Q' 定义了 Q'-代入。我们可以写出

  $$Q = (Q_1, \dots, Q_r)$$

  其中每个模式 $Q_i = \mathsf{mode}_i(Q)$ 的深度均 $<d$。若对每个 $1 \le i \le r$，令

  $$\ell_i = \mathsf{len}(P_1) + \cdots + \mathsf{len}(P_{i-1}),$$

  则我们定义

  $$(P_1, \dots, P_r)_Q = ((P_1, \dots, P_{\ell_2})_{Q_1}, \dots, (P_{\ell_r+1}, \dots, P_{\ell_{r+1}})_{Q_r}).$$

例 2.2.1.9. 若 $Q=(*,*)$ 且 $P_1=(*,*)$、$P_2=(*,*,*)$，则

$$(P_1, P_2)_Q = ((*,*),(*,*,*)).$$

更一般地，若 $Q=(*,\dots,*)$ 是满足 $\mathsf{depth}(Q)=1$ 且 $\mathsf{len}(Q)=\mathsf{rank}(Q)=r$ 的轮廓，则

$$(P_1, \dots, P_r)_Q = (P_1, \dots, P_r)$$

就是普通拼接。

旁注 2.2.1.10. Q-代入有一种 operad 式的解释。轮廓的集合 Profile 具有（非对称）operad 的结构：集合

$$\mathsf{Profile}(n) = \{P \in \mathsf{Profile} \mid \mathsf{len}(P) = n\}$$

构成 Profile 的 n 元运算的全体，且若 $n = m_1 + \cdots + m_r$，则结构映射

$$\begin{aligned} \mathsf{Profile}(m_1) \times \cdots \times \mathsf{Profile}(m_r) \times \mathsf{Profile}(n) &\longrightarrow \mathsf{Profile}(m_1 + \cdots + m_r) \\ (P_1, \dots, P_r), Q &\longmapsto (P_1, \dots, P_r)_Q \end{aligned}$$

由 Q-代入给出。还可以在这个非对称 operad 上构造余自由对称 operad（cofree symmetric operad），这相当于给各 n 元运算集合赋予平凡的对称群作用。

2.2.2　基本定义

定义了轮廓及其基本性质之后，我们现在可以定义嵌套元组了。

定义 2.2.2.1. 若 V 是集合，则以 V 中元素为元素的*嵌套元组* X 是由以下两部分组成的对 $(X^\flat, P)$：

1. 一个元素取自 V 的元组 $X^\flat = (x_1, \ldots, x_m)$，称为 X 的*扁平化*；以及
2. 一个长度为 m 的轮廓 $\mathsf{prof}(X) = P$，称为 X 的*轮廓*。

我们记 $\mathsf{Nest}(V)$ 为以集合 V 中元素为元素的所有嵌套元组构成的集合。

例 2.2.2.2. 以下是一些嵌套元组的例子，连同它们的扁平化与轮廓。

$$\begin{array}{lll}
X = (2,(2,2)) & X^\flat = (2,2,2) & \mathsf{prof}(X) = (*,(*,*)) \\
X = 25 & X^\flat = (25) & \mathsf{prof}(X) = * \\
X = (((2,2,2),8),64) & X^\flat = (2,2,2,8,26) & \mathsf{prof}(X) = (((*,*,*),*),*) \\
X = ((),(32,()),(4,8)) & X^\flat = (32,4,8) & \mathsf{prof}(X) = ((),(*,()),(*,*))
\end{array}$$

记号 2.2.2.3. 我们有时写

$$X = (x_1, \ldots, x_m)_P$$

来表示满足 $X^\flat = (x_1, \ldots, x_m)$ 且轮廓为 $\mathsf{prof}(X) = P$ 的嵌套元组。

观察 2.2.2.4. 若 V 是任意集合，则按定义，我们有一个拉回方块（pullback square）

$$\begin{tikzcd}
\mathsf{Nest}(V) \arrow[r, "\mathsf{prof}(-)"] \arrow[d, "(-)^\flat"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathsf{Profile} \arrow[d, "\mathsf{len}(-)"] \\
\mathsf{Tuple}(V) \arrow[r, "\mathsf{len}(-)"'] & \mathbb{N}.
\end{tikzcd}$$

注 2.2.2.5. 鉴于轮廓的递归定义，我们也可以等价地把以 V 中元素为元素的嵌套元组定义为以下二者之一：

1. V 中的一个元素；或
2. 一个由以 V 中元素为元素的嵌套元组组成的元组。

我们来定义嵌套元组的一些重要属性。嵌套元组 X 的每个这样的属性都继承自它的扁平化 $X^\flat$ 或它的轮廓 $\mathsf{prof}(X)$。

定义 2.2.2.6. 设 X 是以 V 中元素为元素的嵌套元组。

- X 的秩为

$$\mathsf{rank}(X) = \mathsf{rank}(P)$$

- X 的长度为
$$\mathsf{len}(X) = \mathsf{len}(P) = \mathsf{len}(X^\flat)$$
- X 的深度为
$$\mathsf{depth}(X) = \mathsf{depth}(P)$$
- 若 $V = \mathbb{Z}$，则 X 的大小（size）为
$$\mathsf{size}(X) = \mathsf{size}(X^\flat).$$

例 2.2.2.7. 以下是一些整数嵌套元组的例子，连同它们的秩、长度、深度与大小：

$$\begin{array}{llll}
X = 27 & \mathsf{rank}(X) = 1, \mathsf{len}(X) = 1, & \mathsf{depth}(X) = 0, & \mathsf{size}(X) = 27 \\
X = (2, 10, 5) & \mathsf{rank}(X) = 3, \mathsf{len}(X) = 3, & \mathsf{depth}(X) = 1, & \mathsf{size}(X) = 100 \\
X = (((3, 4), 2, 2), 8, 9), & \mathsf{rank}(X) = 3, \mathsf{len}(X) = 6, & \mathsf{depth}(X) = 3, & \mathsf{size}(X) = 3096 \\
X = (((), ()), (2, (5, 5))), & \mathsf{rank}(X) = 2, \mathsf{len}(X) = 3, & \mathsf{depth}(X) = 3, & \mathsf{size}(X) = 50
\end{array}$$

例 2.2.2.8. 深度为 0 的整数嵌套元组就是一个整数。

例 2.2.2.9. 深度为 1 的整数嵌套元组就是一个整数元组。若 X 是这样的嵌套元组，则 $\mathsf{rank}(X) = \mathsf{len}(X)$。

定义 2.2.2.10. 设 $X = (x_1, \ldots, x_m)_P$ 是满足 $\mathsf{rank}(X) = r$ 的嵌套元组。若 $1 \leq i \leq r$，则 X 的第 i 个模式定义为嵌套元组

$$\mathsf{mode}_i(X) = (x_{\mathsf{len}_{<i}(P)+1}, \ldots, x_{\mathsf{len}_{\leq i}(P)})_{\mathsf{mode}_i(P)}.$$

例 2.2.2.11. 若
$$X = ((3), 4, ((10, 10), 12)),$$
则 X 的各模式为

$$\begin{aligned}
\mathsf{mode}_1(X) &= (3) \\
\mathsf{mode}_2(X) &= 4 \\
\mathsf{mode}_3(X) &= ((10, 10), 12)
\end{aligned}$$

例 2.2.2.12. 若 $X = (32, 5, 6, 64)$，则 X 的各模式为

$$\begin{aligned}
\mathsf{mode}_1(X) &= 32 \\
\mathsf{mode}_2(X) &= 5 \\
\mathsf{mode}_3(X) &= 6 \\
\mathsf{mode}_4(X) &= 64
\end{aligned}$$

引入以下记号会带来方便。

记号 2.2.2.13. 设 X 是满足 $\mathsf{depth}(X) > 0$ 的整数嵌套元组。对任意 $1 \leq j \leq \mathsf{rank}(X)$，我们记

$$\begin{aligned}
\mathsf{len}_j(X) &= \mathsf{len}(\mathsf{mode}_j(X)),\\
\mathsf{len}_{<j}(X) &= \sum_{i=1}^{j-1} \mathsf{len}_i(X),\\
\mathsf{len}_{\leq j}(X) &= \mathsf{len}_{<j}(X) + \mathsf{len}_j(X)
\end{aligned}$$

类似地，我们记

$$\begin{aligned}
\mathsf{size}_j(X) &= \mathsf{size}(\mathsf{mode}_j(X)),\\
\mathsf{size}_{<j}(X) &= \prod_{i=1}^{j-1} \mathsf{size}_j(X), \text{ and}\\
\mathsf{size}_{\leq j}(X) &= \mathsf{size}_{<j}(X) \cdot \mathsf{size}_j(X).
\end{aligned}$$

定义 2.2.2.14. 若 $X = (x_1, \ldots, x_m)_P$ 是嵌套元组且 $1 \leq i \leq m$，则 X 的第 i 个元素为

$$\mathsf{entry}_i(X) = \mathsf{entry}_i(X^\flat) = x_i.$$

例 2.2.2.15. 若

$$X = ((3), 4, ((10, 10), 12)),$$

则 X 的各元素为

$$\begin{aligned}
\mathsf{entry}_1(X) &= 3\\
\mathsf{entry}_2(X) &= 4\\
\mathsf{entry}_3(X) &= 10\\
\mathsf{entry}_4(X) &= 10\\
\mathsf{entry}_5(X) &= 12.
\end{aligned}$$

例 2.2.2.16. 若 $X = (32, 5, 6, 64)$，则 X 的各元素为

$$\begin{aligned}
\mathsf{entry}_1(X) &= 32\\
\mathsf{entry}_2(X) &= 5\\
\mathsf{entry}_3(X) &= 6\\
\mathsf{entry}_4(X) &= 4.
\end{aligned}$$

例 2.2.2.17. 若 X 是深度为 1 的嵌套元组，则对所有 $1 \leq i \leq \mathsf{rank}(X) = \mathsf{len}(X)$，都有 $\mathsf{mode}_i(X) = \mathsf{entry}_i(X)$。

观察 2.2.2.18. 若 X 是整数嵌套元组，则 X 的元素是整数，而 X 的模式本身也是整数嵌套元组。

最后，我们引入嵌套元组的同余（congruence）概念，它刻画嵌套元组何时具有相同的轮廓。

定义 2.2.2.19. 若 X_1 与 X_2 是嵌套元组，我们称 X_1 与 X_2 同余，如果

$$\mathsf{prof}(X_1) = \mathsf{prof}(X_2).$$

例 2.2.2.20. 以下是一些嵌套元组 X_1 与 X_2 的例子，以及它们是否同余

$X_1 = 27$	$X_2 = 100$	同余
$X_1 = (2,2)$	$X_2 = (8,64)$	同余
$X_1 = ((4,8),(4,8))$	$X_2 = ((1,1),(5,10))$	同余
$X_1 = ((64,(8,8)),(25,(5,5)))$	$X_2 = ((2,(3,5)),(7,(11,13)))$	同余
$X_1 = 27$	$X_2 = (100)$	不同余
$X_1 = (2,2)$	$X_2 = (8,64,128)$	不同余
$X_1 = ((4,8),(4,8))$	$X_2 = (((1,1),(5,10)))$	不同余

2.2.3　代入

回顾：若 Q 是长度为 r 的轮廓，且 $P_1,\ldots,P_r$ 是轮廓，则我们定义过轮廓

$$(P_1,\ldots,P_r)_Q$$

称为 $P_1,\ldots,P_r$ 的 Q-代入。这个轮廓由 Q 把第 i 个元素替换为轮廓 P_i 而得到。我们可以把它如下推广为嵌套元组上的运算。

定义 2.2.3.1. 设 $X_1,\ldots,X_m$ 是轮廓分别为 $P_1,\ldots,P_m$ 的嵌套元组，并设 Q 是长度为 m 的轮廓。我们定义 $X_1,\ldots,X_m$ 的 Q-代入

$$(X_1,\ldots,X_m)_Q$$

为满足扁平化

$$(X_1,\ldots,X_m)^\flat_Q = X_1^\flat \star \cdots \star X_m^\flat$$

与轮廓

$$(P_1,\ldots,P_m)_Q.$$

的嵌套元组。更一般地，若 $X_1,\ldots,X_m$ 是嵌套元组且 Y 是长度为 m 的嵌套元组，我们定义

$$(X_1,\ldots,X_m)_Y = (X_1,\ldots,X_m)_{\mathsf{prof}(Y)}.$$

例 2.2.3.2. 若 $(X_1, X_2, X_3) = (64, 16, 4)$ 且 $Q = (*, (*, *))$，则

$$(X_1, X_2, X_3)_Q = (64, (32, 4))$$

例 2.2.3.3. 若 $(X_1, X_2, X_3, X_4) = ((2, 2), (3, 3), (5, 5), (7, 7))$ 且 $Q = ((*, *), (*, *))$，则

$$(X_1, X_2, X_3, X_4)_Q = (((2, 2), (3, 3)), ((5, 5), (7, 7))).$$

例 2.2.3.4. 若 $X = (12)$ 且 $Q = *$，则

$$(X)_Q = 12.$$

例 2.2.3.5. 若 $X_1 = 2$、$X_2 = 2$、$X_3 = (5, 5)$，且 $Q = (*, *, *)$，则

$$(X_1, X_2, X_3)_Q = (2, 2, (5, 5)) = (X_1, X_2, X_3).$$

更一般地，若 $X_1, \ldots, X_m$ 是任意嵌套元组且 $P = (*, \ldots, *)$，则

$$(X_1, \ldots, X_m)_Q = (X_1, \ldots, X_k)$$

就是 $X_1, \ldots, X_m$ 的拼接。

旁注 2.2.3.6. 嵌套元组的代入有一种 *operad* 式的解释。整数嵌套元组的集合 $\mathsf{Nest}(\mathbb{Z})$ 是 operad $\mathsf{Profile}$ 上的一个代数（algebra），其结构映射由 Q-代入给出：

$$\begin{tikzcd}
\mathsf{Nest}(\mathbb{Z}) \times \cdots \times \mathsf{Nest}(\mathbb{Z}) \times \mathsf{Profile}(n) \arrow[r] & \mathsf{Nest}(\mathbb{Z}) \\
(X_1, \ldots, X_m), Q \arrow[r, mapsto] & (X_1, \ldots, X_m)_Q.
\end{tikzcd}$$

2.2.4　加细

本节引入嵌套元组上的一种重要关系，称为*加细*（refinement）。直观上，若 X' 与 X 是整数嵌套元组，我们说 X' 加细 X，如果 X' 可以由 X 把每个元素替换为某个大小相同的嵌套元组而得到。更精确地说，我们有如下定义。

定义 2.2.4.1. 若 X' 与 X 是嵌套元组，则我们称 X' *加细* X，如果下列之一成立

1. $X = \mathsf{size}(X')$；或
2. (a) $\mathsf{depth}(X'), \mathsf{depth}(X) > 0$,
   (b) $\mathsf{rank}(X') = \mathsf{rank}(X)$，且
   (c) 对每个 $1 \leq i \leq \mathsf{rank}(X)$，$\mathsf{mode}_i(X')$ 加细 $\mathsf{mode}_i(X)$。

记号 2.2.4.2. 我们记

$$X' \twoheadrightarrow X$$

以表示 X' 加细 X。

例 2.2.4.3. 以下是一些嵌套元组加细的例子。

$$\begin{aligned}(2,(2,2)) &\twoheadrightarrow 8\\((2,2),(3,3),(5,5)) &\twoheadrightarrow (4,9,25)\\(64) &\twoheadrightarrow 64\\(8,((2,2,2),((1,4),(2,2)))) &\twoheadrightarrow (8,(8,8))\end{aligned}$$

观察 2.2.4.4. 嵌套元组的加细是自反、传递且反对称的，因此加细给出了正整数嵌套元组全体上的一个偏序（partial ordering）。

若 X' 加细 X，则我们可以把 X' 看作由 X 把每个元素 x_i 替换为某个大小为 x_i 的嵌套元组 X'_i 而得到。我们把嵌套元组 X'_i 称为 X' 相对于 X 的第 i 个模式。更精确地说，我们有如下定义。

构造 2.2.4.5. 设 X 是长度为 m 的整数嵌套元组，并设 X' 加细 X。对任意 $1 \le i \le m$，我们定义嵌套元组

$$X'_i = \mathsf{mode}_i(X', X),$$

称为 X' 相对于 X 的第 i 个模式，其公式为

$$\mathsf{mode}_i(X', X) = \begin{cases} X' & \mathsf{depth}(X) = 0 \text{ (从而} i = \ell = 1) \\ \mathsf{mode}_{i-N}(\mathsf{mode}_j(X'), \mathsf{mode}_j(X)) & \begin{array}{c} j \text{ 是满足如下条件的最大整数:} \\ N := \mathsf{len}_{<j}(X) < i. \end{array} \end{cases}$$

例 2.2.4.6. 若 $X = ((4,9),(25,36))$，且 $X' = (((2,2),(3,3)),(25,(6,(2,3))))$，则 X' 加细 X，且 X' 相对于 X 的各模式为

$$\begin{aligned}\mathsf{mode}_1(X', X) &= (2,2)\\\mathsf{mode}_2(X', X) &= (3,3)\\\mathsf{mode}_3(X', X) &= 25\\\mathsf{mode}_4(X', X) &= (6,(2,3)).\end{aligned}$$

例 2.2.4.7. 若 X 是任意嵌套元组，则 X 加细 X，且对任意 $1 \le i \le \mathsf{len}(X)$，有

$$\mathsf{mode}_i(X, X) = \mathsf{entry}_i(X).$$

例 2.2.4.8. 若 $X = X^\flat$ 是元组，且 X' 加细 X，则对任意 $1 \le i \le \mathsf{len}(X)$，有

$$\mathsf{mode}_i(X', X) = \mathsf{mode}_i(X').$$

例 2.2.4.9. 若 X' 是满足 $\mathsf{size}(X') = N$ 的嵌套元组，则 X' 加细 N，且 X' 相对于 N 的唯一模式为

$$\mathsf{mode}_1(X', N) = X'.$$

记号 2.2.4.10. 若 $X' \twoheadrightarrow X$ 是一个加细且 $1 \leq i \leq \mathsf{len}(X)$，则我们记

$$\begin{aligned}
\mathsf{len}_i(X', X) &= \mathsf{len}(\mathsf{mode}_i(X', X)) \\
\mathsf{len}_{<i}(X', X) &= \sum_{j<i} \mathsf{len}_j(X', X) \\
\mathsf{len}_{\leq i}(X', X) &= \sum_{j\leq i} \mathsf{len}_j(X', X)
\end{aligned}$$

定义 2.2.4.11. 设 X' 加细 X，并记 $X'_i = \mathsf{mode}_i(X', X)$。则 X' 相对于 X 的*扁平化*是嵌套元组

$$\mathsf{flat}(X', X) = (X'_1, \ldots, X'_m).$$

例 2.2.4.12. 若 $X' = (((2,2),(3,3)),((5,5),(7,7)))$ 且 $X = ((4,9),(25,49))$，则

$$\mathsf{flat}(X', X) = ((2,2),(3,3),(5,5),(7,7)).$$

例 2.2.4.13. 若 X 是任意嵌套元组，则 X 相对于 X 的扁平化为

$$\mathsf{flat}(X, X) = X^\flat.$$

例 2.2.4.14. 若 $X = X^\flat$ 是元组，且 X' 加细 X，则 X' 相对于 X 的扁平化为

$$\mathsf{flat}(X', X) = X'.$$

例 2.2.4.15. 若 X' 是满足 $\mathsf{size}(X') = N$ 的嵌套元组，则 X' 加细 N，且 X' 相对于 N 的扁平化为

$$\mathsf{flat}(X', N) = (N).$$

观察 2.2.4.16. 若 X' 加细 X，则 $\mathsf{flat}(X', X)$ 加细 $X^\flat$。

2.3　布局

在建立了嵌套元组 (nested tuple) 的必要背景之后，我们将注意力转向布局 (layout)。布局是扁平布局（flat layout）的推广：其中允许形状与步长取嵌套元组，而不再是（扁平）元组。

2.3.1　基本定义

定义 2.3.1.1. 布局是一个数对

$$L = S : D$$

它由一个正整数的嵌套元组

$$\mathsf{shape}(L) = S$$

（称为 L 的形状（shape））与一个非负整数的嵌套元组

$$\mathsf{stride}(L) = D$$

（称为 L 的步长（stride））组成，且要求 S 与 D 同余（congruent）。

定义 2.3.1.2. 若 $L = S : D$ 是布局，则 L 的秩 (rank)、长度 (length)、深度 (depth)、大小（size）与轮廓（profile）分别定义为 S 的秩、长度、深度、大小与轮廓。

例 2.3.1.3. 布局 $L = (3, (3, 2)) : (3, (1, 10))$ 可图示如下。

$L =$

0	1	2	10	11	12
3	4	5	13	14	15
6	7	8	16	17	18

例 2.3.1.4. 布局 $L = ((2, 2), (2, 2)) : ((1, 4), (2, 8))$ 可图示如下。

$L =$

0	2	8	10
1	3	9	11
4	6	12	14
5	7	13	15

例 2.3.1.5. 布局

$$L = 10 : 4$$

满足 $\mathsf{rank}(L) = 1$，$\mathsf{len}(L) = 1$，$\mathsf{depth}(L) = 0$，$\mathsf{size}(L) = 10$，且 $\mathsf{prof}(L) = *$。

例 2.3.1.6. 布局

$$L = (7, (2, 10, 4), (3, 7)) : (1, (7, 14, 140), (560, 1680))$$

满足 $\mathsf{rank}(L) = 3$，$\mathsf{len}(L) = 6$，$\mathsf{depth}(L) = 2$，$\mathsf{size}(L) = 11760$，且 $\mathsf{prof}(L) = (*, (*, *, *), (*, *))$.

例 2.3.1.7. 布局

$$L = ((2, 2, 2, (2, 2))) : ((1, 0, 8, (0, 16)))$$

满足 $\mathsf{rank}(L) = 1$，$\mathsf{len}(L) = 5$，$\mathsf{depth}(L) = 3$，$\mathsf{size}(L) = 32$，且 $\mathsf{prof}(L) = ((*, *, *, (*, *)))$。

例 2.3.1.8. 数对

$$S : D = (2, (2, 2)) : (1, 2, 4)$$

不是布局，因为 S 与 D 不同余。

定义 2.3.1.9. 若 $L = S : D$ 是布局，则对任意 $1 \le i \le \mathsf{rank}(L)$，定义 L 的第 i 个模式（mode）为布局

$$\mathsf{mode}_i(L) = \mathsf{mode}_i(S) : \mathsf{mode}_i(D),$$

且对任意 $1 \le i \le \mathsf{len}(L)$，定义 L 的第 i 个元素（entry）为布局

$$\mathsf{entry}_i(L) = \mathsf{entry}_i(S) : \mathsf{entry}_i(D).$$

例 2.3.1.10. 若 $L = \big((2, 2), 9\big) : \big((3, 6), 12\big)$，则 L 的各模式为

$$\begin{aligned} \mathsf{mode}_1(L) &= (2, 2) : (3, 6) \\ \mathsf{mode}_2(L) &= 9 : 12 \end{aligned}$$

而 L 的各元素为

$$\begin{aligned} \mathsf{entry}_1(L) &= 2 : 3 \\ \mathsf{entry}_2(L) &= 2 : 6 \\ \mathsf{entry}_3(L) &= 9 : 12. \end{aligned}$$

注 2.3.1.11. 若 L 是布局，则 L 的各模式也是布局，而 L 的各元素是深度为 0 的布局。

注 2.3.1.12. 扁平布局 L 恰是深度为 1 的布局。反过来，若 L 是布局，我们可以按如下方式得到一个扁平布局 $L^\flat$。

定义 2.3.1.13. 若 $L = S : D$ 是布局，定义 L 的*扁平化*（flattening）为扁平布局

$$L^\flat = S^\flat : D^\flat.$$

例 2.3.1.14. 布局 $L = 10 : 4$ 的扁平化是 $L^\flat = (10) : (4)$。

例 2.3.1.15. 布局

$$L = \big((2,2,2,(2,2))\big) : \big((1,0,8,(0,16))\big)$$

的扁平化是

$$L^\flat = (2,2,2,2,2) : (1,0,8,0,16).$$

注 2.3.1.16. 若 L 是布局，则 $\mathsf{len}(L) = \mathsf{rank}(L^\flat)$，且对任意 $1 \le i \le \mathsf{len}(L)$，有

$$\mathsf{entry}_i(L) = \mathsf{mode}_i(L^\flat).$$

我们可以利用上述扁平化构造，把扁平布局的许多概念推广到嵌套布局。例如：

构造 2.3.1.17 (布局函数)**.** 若 L 是嵌套布局，我们把 L 的布局函数（layout function）Φ_L 定义为

$$\Phi_L = \Phi_{L^\flat},$$

其中 $\Phi_{L^\flat}$ 是构造 2.1.2.19 中的布局函数。类似地，若 N 满足 $\mathsf{Image}(\Phi_L) \subset [0,N)$，则把

$$\Phi_L^N = \Phi_{L^\flat}^N$$

定义为 Φ_L 经由包含映射（inclusion）$[0,N) \subset \mathbb{Z}$ 的分解。

例 2.3.1.18. 若 $L = ((2,2),2) : ((3,0),10)$，则 L 的布局函数

$$\Phi_L : [0,8) \to \mathbb{Z}$$

由

$$\begin{tikzcd} 0 \arrow[d, "\Phi_L"'] & 1 \arrow[d] & 2 \arrow[d] & 3 \arrow[d] & 4 \arrow[d] & 5 \arrow[d] & 6 \arrow[d] & 7 \arrow[d] \\ 0 & 3 & 0 & 3 & 10 & 13 & 10 & 13 \end{tikzcd}$$

给出。

给定布局 L，我们可以得到扁平布局 $L^\flat$ 以及轮廓 $P = \mathsf{prof}(L)$。反过来，若给定扁平布局 L 以及与 L 等长的轮廓 P，则可以按下述方式构造一个以 L 为扁平化、以 P 为轮廓的布局。

构造 2.3.1.19. 若 L 是扁平布局，P 是满足 $\mathsf{len}(P) = \mathsf{len}(L)$ 的轮廓，则可以定义

$$L = L_P$$

为形状为

$$\mathsf{shape}(L) = \mathsf{shape}(L)_P$$

且步长为

$$\mathsf{stride}(L) = \mathsf{stride}(L)_P$$

的布局，其中 $(-)_P$ 是定义 2.2.1.8 中的 P-代入（P-substitution）运算。

例 2.3.1.20. 若 $L = (8, 8, 8) : (1, 64, 8)$ 且 $P = (*, (*, *))$，则

$$L_P = (8, (8, 8)), (1, (64, 8)).$$

例 2.3.1.21. 若 $L = (128) : (2)$ 且 $P = *$，则

$$L_P = 128 : 2.$$

命题 2.3.1.22. *若 L' 是扁平布局，P 是满足 $\mathsf{len}(L') = \mathsf{len}(P)$ 的轮廓，则存在唯一的布局 L，其扁平化为 $L^\flat = L'$、轮廓为 $\mathsf{prof}(L) = P$，即 $L = L'_P$。*

证明. 这由嵌套元组的定义即得，因为嵌套元组由其扁平化与轮廓唯一确定。 □

观察 2.3.1.23. 上一命题告诉我们，我们有如下拉回（pullback）方块

$$\begin{tikzcd}
\mathsf{Layout} \arrow[r, "\mathsf{prof}(-)"] \arrow[d, "(-)^\flat"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathsf{Profile} \arrow[d, "\mathsf{len}(-)"] \\
\mathsf{FlatLayout} \arrow[r, "\mathsf{len}(-)"'] & \mathbb{N}
\end{tikzcd}$$

我们可以如下把非退化（non-degenerate）的概念推广到嵌套情形。

定义 2.3.1.24. 设 L 是布局。若对所有 $1 \le i \le \mathsf{len}(L)$，下列条件成立：

$$\mathsf{entry}_i(\mathsf{shape}(L)) \quad \Rightarrow \quad \mathsf{entry}_i(\mathsf{stride}(L))$$

则称 L 是非退化的。

例 2.3.1.25. 布局

$$L_1 = ((2, 2), 1) : ((1, 2), 0)$$
$$L_2 = ((8, 8), (1, 16)) : ((2, 32), (0, 128))$$

是非退化的，而布局

$$L_3 = ((2, 2), 1) : ((1, 2), 4)$$
$$L_4 = ((8, 8), (1, 16)) : ((2, 32), (1024, 128))$$

是退化（degenerate）的。

2.3.2　基本运算

在建立了布局的基本词汇之后，我们转而讨论布局所支持的运算。本节定义一些基本运算，构造更复杂的运算——如合并（coalesce）、补（complement）、复合（composition）、逻辑切分（logical division）与逻辑乘积（logical product）——时将会用到它们。

2.3.2.1　扁平化

若 L 是布局，则把 L 的形状与步长扁平化，即可得到扁平布局 $L^\flat$。

定义 2.3.2.1. 若 $L = S : D$ 是布局，定义 L 的*扁平化*为扁平布局

$$L^\flat = S^\flat : D^\flat.$$

例 2.3.2.2. 布局

$$L = ((2,2,2,(2,2))) : ((1,0,8,(0,16)))$$

的扁平化是

$$L^\flat = (2,2,2,2,2) : (1,0,8,0,16).$$

例 2.3.2.3. 布局 $L = 10 : 4$ 的扁平化是 $L^\flat = (10) : (4)$。

例 2.3.2.4. 设 L 是布局。则 $\mathsf{depth}(L) = 1$ 当且仅当 $L = L^\flat$。

2.3.2.2　拼接

拼接两个布局，就是拼接它们的形状并拼接它们的步长。

定义 2.3.2.5. 若 $L = S : D$ 与 $L' = S' : D'$ 是布局，则 L 与 L' 的*拼接*（concatenation）是布局 (L, L')

$$(L, L') = (S, S') : (D, D').$$

更一般地，若 $L_1, \ldots, L_k$ 是任意有限多个布局，其中 $L_i = S_i : D_i$，则 $L_1, \ldots, L_k$ 的拼接是布局

$$(L_1, \ldots, L_k) = (S_1, \ldots, S_k) : (D_1, \ldots, D_k).$$

注 2.3.2.6. 嵌套元组（因而布局）的拼接不满足结合律。例如，取 $L_1 = 3 : 4$、$L_2 = 2 : 2$ 与 $L_3 = 5 : 1$。则

$$\big(L_1, (L_2, L_3)\big) = \big(3, (2,5)\big) : \big(4, (2,1)\big) \neq \big((3,2), 5\big) : \big((4,2), 1\big) = \big((L_1, L_2), L_3\big).$$

而且，这两个布局都不等于「三重」拼接 $(L_1, L_2, L_3) = (3,2,5) : (4,2,1)$。不过，我们看到这些布局都有相同的扁平化，因此都有相同的布局函数。

例 2.3.2.7. 若 $L = (3,7,2):(1,3,6)$ 且 $L' = (2,(2,(4,3))):(5,3,(2,2))$，则

$$(L, L') = ((3,7,2),(2,(2,(4,3)))) : ((1,3,6),(5,(3,(2,2))))$$

注 2.3.2.8. 拼接会增加布局的深度。更精确地说，我们有

$$\mathsf{depth}(L, L') = 1 + \max(\mathsf{depth}(L), \mathsf{depth}(L')).$$

注 2.3.2.9. 当 L 与 L' 都是扁平布局时，定义 2.3.2.5 的拼接与定义 2.1.3.36 的扁平布局拼接并不一致。两个运算之间的关系由公式

$$L \star L' = (L, L')^\flat.$$

给出。

注 2.3.2.10. 若 L 是满足 $\mathsf{depth}(L) > 0$ 且 $\mathsf{rank}(L) = r$ 的任意布局，则可以把

$$L = (\mathsf{mode}_1(L), \ldots, \mathsf{mode}_r(L))$$

写成其各模式的拼接。

例 2.3.2.11. 若

$$L = ((5,(7,7)),2,(4,5)) : ((1,(35,5)),0,(1,8))$$

则 $L = (L_1, L_2, L_3)$，其中

$$\begin{aligned} L_1 &= \big(5,(7,7)\big) : \big(1,(35,5)\big), \\ L_2 &= 2 : 0, \text{ and} \\ L_3 &= (4,5) : (1,8). \end{aligned}$$

2.3.2.3　代入

回顾一下，若 $X_1, \ldots, X_k$ 是嵌套元组，P 是满足 $\mathsf{len}(P) = k$ 的轮廓，则可以构成 P-代入

$$(X_1, \ldots, X_k)_P$$

它是把 P 的第 i 个元素替换为嵌套元组 X_i 得到的。我们可以如下把这一构造从嵌套元组推广到布局。

定义 2.3.2.12. 设 $L = S : D$ 是布局，P 是满足 $\mathsf{len}(P) = \mathsf{rank}(L)$ 的轮廓。定义

$$L_P = S_P : D_P$$

其中 S_P 与 D_P 是 S 与 D（的各模式）的 P-代入。

例 2.3.2.13. 若 $P = (*, (*, *))$ 且 $L = (8, 8, 8) : (1, 8, 64)$，则

$$L_P = (8, (8, 8)) : (1, (8, 64)).$$

例 2.3.2.14. 若 $P = (*, (*, *)))$ 且

$$L = ((2, 2), (3, 3), (5, 5)) : ((2, 1), (12, 4), (180, 36)),$$

则

$$L_P = ((2, 2), ((3, 3), (5, 5))) : ((2, 1), ((12, 4), (180, 36))).$$

例 2.3.2.15. 若 $L = (16) : (1)$ 且 $P = *$，则

$$L_P = 16 : 1.$$

2.3.3　合并

回顾一下，若 L 是扁平布局，则 $\mathsf{coal}^\flat(L)$ 是唯一的、以 Φ_L 为布局函数且秩极小的扁平布局。在任意（嵌套）布局的情形下也可以做出类似的构造。我们先定义已合并布局的概念。

定义 2.3.3.1. 设 L 是布局。若下列条件之一成立，则称 L 是已合并（coalesced）的。

1. $L = 1 : 0$，
2. $\mathsf{depth}(L) = 0$ 且 $\mathsf{shape}(L) > 1$，或
3. $\mathsf{depth}(L) = 1$、$\mathsf{rank}(L) > 1$，且 L 在定义 2.1.4.1 的意义下已合并。

例 2.3.3.2. 布局

$$L = (2, (2, 2)) : (1, (16, 512))$$

不是已合并的，因为 $\mathsf{depth}(L) > 1$。

例 2.3.3.3. 布局

$$L = (64) : (2)$$

不是已合并的，而布局

$$L' = 64 : 2$$

是已合并的。

例 2.3.3.4. 布局

$$L = 1 : 8$$

不是已合并的，而布局

$$L' = 1 : 0$$

是已合并的。

例 2.3.3.5. 空布局（empty layout）

$$E = () : ()$$

不是已合并的。

观察 2.3.3.6. 回顾一下，布局 L 非退化的条件是

$$\mathsf{entry}_i(\mathsf{shape}(L)) = 1 \quad \Rightarrow \quad \mathsf{entry}_i(\mathsf{stride}(L)) = 0.$$

若 L 已合并，则 L 非退化。

若 L 是任意布局，我们可以如下得到一个已合并布局 $\mathsf{coal}(L)$。

构造 2.3.3.7. 设 L 是布局，并记

$$\mathsf{coal}^\flat(L^\flat) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

1. 若 $m > 1$，定义
$$\mathsf{coal}(L) = \mathsf{coal}^\flat(L^\flat)$$
2. 若 $m = 1$，定义
$$\mathsf{coal}(L) = s_1 : d_1$$
3. 若 $m = 0$，定义
$$\mathsf{coal}(L) = 1 : 0.$$

例 2.3.3.8. 若 $E = () : ()$ 是空布局，则

$$\mathsf{coal}(E) = 1 : 0.$$

例 2.3.3.9. 若 $L = (1, 1) : (2, 4)$，则

$$\mathsf{coal}(L) = 1 : 0.$$

例 2.3.3.10. 若 $L = (512) : (4)$，则

$$\mathsf{coal}(L) = 512 : 4.$$

例 2.3.3.11. 若 $L = (2, 2, 2) : (1, 2, 4)$，则

$$\mathsf{coal}(L) = 8 : 1.$$

例 2.3.3.12. 若 $L = ((2, 2, 2), (5, 5)) : ((1, 2, 4), (10, 50))$，则

$$\mathsf{coal}(L) = (8, 25) : (1, 10).$$

注 2.3.3.13. 若 L 是布局，则 $\mathsf{coal}(L)$ 的深度为 0 或 1。

命题 2.3.3.14. 若 A 与 B 是布局，则

$$\Phi_A = \Phi_B \quad \Leftrightarrow \quad \mathsf{coal}(A) = \mathsf{coal}(B).$$

证明. 利用命题 2.1.4.18，我们有

$$\begin{aligned} \Phi_A = \Phi_B \quad &\Leftrightarrow \quad \Phi_{A^\flat} = \Phi_{B^\flat} \\ &\Leftrightarrow \quad \mathsf{coal}^\flat(A^\flat) = \mathsf{coal}^\flat(B^\flat) \\ &\Leftrightarrow \quad \mathsf{coal}(A) = \mathsf{coal}(B). \end{aligned}$$

□

定义 2.3.3.15. 若 L 是布局，定义 L 的复杂度（complexity）为整数

$$\mathsf{complexity}(L) = \mathsf{len}(L) + \mathsf{depth}(L).$$

命题 2.3.3.16. 若 L 是布局且 $\mathsf{size}(L) > 1$，则 $\mathsf{coal}(L)$ 是唯一的以 Φ_L 为布局函数的复杂度极小布局。

证明. 设 L' 是与 L 有相同布局函数的布局，并设 $\mathsf{coal}(L') \neq 1:0$。则

$$\mathsf{len}(L') \geq \mathsf{len}(\mathsf{coal}(L')) = \mathsf{len}(\mathsf{coal}(L)).$$

需分两种情形讨论。

- （情形 1）：设 $\mathsf{len}(L') > 1$。则 $\mathsf{depth}(L') \geq 1 \geq \mathsf{depth}(\mathsf{coal}(L))$。综合这些不等式，我们观察到
$$\mathsf{complexity}(L') \geq \mathsf{complexity}(\mathsf{coal}(L)),$$
且等号成立当且仅当 $L' = \mathsf{coal}(L') = \mathsf{coal}(L)$。
- （情形 2）：设 $\mathsf{len}(L') = 1$。则对某些整数 $s > 1$ 与 $d \geq 0$，有 $L' = (s) : (d)$ 或 $L' = s : d$。无论哪种情形，都有 $\mathsf{coal}(L') = s : d$，且
$$\mathsf{complexity}(L') \geq \mathsf{complexity}(\mathsf{coal}(L)),$$
其中等号成立当且仅当 $L' = s : d = \mathsf{coal}(L)$。

□

注 2.3.3.17. 我们必须排除 $\mathsf{size}(L) = 1$ 的情形，唯一的原因在于：若 $\mathsf{size}(L) = 1$，则 $1:0$ 与空布局 $():()$ 是两个不同的布局，它们都具有极小复杂度，且与 L 有相同的布局函数（即平凡布局函数 $0 \mapsto 0$）。

2.3.4　相对合并

合并有一个重要的不变量（invariant），称为相对合并（relative coalesce），记作 $\mathsf{coal}(L, \bar{S})$。该运算额外接收一个嵌套元组 $\bar{S}$ 作为输入，要求 $\bar{S}$ 被 $\mathsf{shape}(L)$ 加细（refine）。此时，相对合并运算在保证所得形状仍加细 $\bar{S}$ 的前提下，尽可能地化简布局 L。

定义 2.3.4.1. 设 $L = S : D$ 是布局，$\bar{S}$ 是被 S 加细的、长度为 m 的嵌套元组。回顾一下，对任意 $1 \le i \le m$，可以考虑 S 相对于 $\bar{S}$ 的第 i 个模式，记作

$$\mathsf{mode}_i(S, \bar{S}).$$

由于 S 与 D 同余，存在与 $\mathsf{mode}_i(S, \bar{S})$ 对应的嵌套元组

$$\mathsf{mode}_i(D, \bar{S})$$

我们定义 L 相对于 $\bar{S}$ 的第 i 个模式为布局

$$\mathsf{mode}_i(L, \bar{S}) = \mathsf{mode}_i(S, \bar{S}) : \mathsf{mode}_i(D, \bar{S}).$$

例 2.3.4.2. 若 $\bar{S} = (4, (9, 25))$ 且

$$L = ((2,2),((3,3),(5,(1,5)))) : ((1,2),((6,18),(90,(0,450))))$$

则

$$\begin{aligned}
\mathsf{mode}_1(L, \bar{S}) &= (2,2) : (1,2)\\
\mathsf{mode}_2(L, \bar{S}) &= (3,3) : (6,18)\\
\mathsf{mode}_3(L, \bar{S}) &= (5,(1,5)) : (90,(0,450)).
\end{aligned}$$

观察 2.3.4.3. 设 $L = S : D$ 是布局，$\bar{S}$ 是被 S 加细的、长度为 m 且轮廓为 P 的嵌套元组。若对任意 $1 \le i \le m$，记

$$L_i = \mathsf{mode}_i(L, \bar{S}),$$

则

$$L = (L_1, \ldots, L_m)_P$$

是其各相对模式的 P-代入。

定义 2.3.4.4. 设 $L = S : D$ 是布局，$\bar{S}$ 是被 S 加细的、长度为 m 且轮廓为 P 的嵌套元组。若每个相对模式

$$\mathsf{mode}_i(L, \bar{S})$$

都已合并，则称 L 是相对于 $\bar{S}$ 已合并的（coalesced over $\bar{S}$）。

观察 2.3.4.5. 在定义 2.3.4.4 的情形下，若 L 相对于 $\bar{S}$ 已合并，则 L 非退化。

例 2.3.4.6. 若 L 是布局，则 L 相对于 $\mathsf{shape}(L)$ 已合并当且仅当 L 非退化，即

$$\mathsf{entry}_i(\mathsf{shape}(L)) = 1 \quad \Rightarrow \quad \mathsf{entry}_i(\mathsf{stride}(L)) = 0.$$

定义 2.3.4.7 (相对合并). 设 $L = S : D$ 是布局，$\bar{S}$ 是被 S 加细的、长度为 m 且轮廓为 P 的嵌套元组。定义

$$\mathsf{coal}(L, \bar{S}) = (\mathsf{coal}(L_1), \dots, \mathsf{coal}(L_m))_P.$$

注 2.3.4.8. 在定义 2.3.4.7 的情形下，$\mathsf{coal}(L, \bar{S})$ 的形状加细 $\bar{S}$。

引理 2.3.4.9. *若 $L = S : D$ 是布局且 S 加细 $\bar{S}$，则*

$$\Phi_{\mathsf{coal}(L,\bar{S})} = \Phi_L.$$

证明. 如上，设

$$L_i = \mathsf{mode}_i(L, S)$$

表示 L 相对于 S 的第 i 个模式，并记 $\bar{L}_i = \mathsf{coal}(L_i)$。则

$$\begin{aligned}
\Phi_{\mathsf{coal}(L,\bar{S})} &= \Phi_{(\bar{L}_1,\dots,\bar{L}_m)_{\bar{S}}} \\
&= \Phi_{(\bar{L}_1,\dots,\bar{L}_m)} \\
&= \Phi_{\mathsf{coal}((\bar{L}_1,\dots,\bar{L}_m))} \\
&= \Phi_{\mathsf{coal}((L_1,\dots,L_m))} \\
&= \Phi_{(L_1,\dots,L_m)} \\
&= \Phi_{(L_1,\dots,L_m)_{\bar{S}}} \\
&= \Phi_L.
\end{aligned}$$

□

命题 2.3.4.10. *设 A 与 B 是布局，$\bar{S}$ 是长度为 m 的嵌套元组，且 $\mathsf{shape}(A)$ 加细 $\bar{S}$、$\mathsf{shape}(B)$ 加细 $\bar{S}$。则*

$$\Phi_A = \Phi_B \quad \Leftrightarrow \quad \mathsf{coal}(A, \bar{S}) = \mathsf{coal}(B, \bar{S})$$

证明. 若 $\mathsf{coal}(A, \bar{S}) = \mathsf{coal}(B, \bar{S})$，则利用引理 2.3.4.9，我们有

$$\Phi_A = \Phi_{\mathsf{coal}(A,\bar{S})} = \Phi_{\mathsf{coal}(B,\bar{S})} = \Phi_B.$$

反之，设 $\Phi_A = \Phi_B$。我们论证 $\mathsf{coal}(A, \bar{S}) = \mathsf{coal}(B, \bar{S})$。记 $P = \mathsf{prof}(\bar{S})$，且对任意 $1 \le i \le m$，记

$$\begin{aligned}
A_i &= \mathsf{mode}_i(A, \bar{S}) \\
B_i &= \mathsf{mode}_i(B, \bar{S}).
\end{aligned}$$

由于

$$\mathsf{coal}(A, \bar{S}) = (\mathsf{coal}(A_1), \ldots, \mathsf{coal}(A_m))_P$$

与

$$\mathsf{coal}(B, \bar{S}) = (\mathsf{coal}(B_1), \ldots, \mathsf{coal}(B_m))_P$$

只需证明对所有 $1 \leq i \leq m$ 都有 $\mathsf{coal}(A_i) = \mathsf{coal}(B_i)$。由余字典序同构的结合律，$A$ 的布局函数 Φ_A 可以写成

$$[0, \mathsf{size}(A)) \xrightarrow{\mathsf{colex}^{-1}} \textstyle\prod_{j=1}^{m} [0, \mathsf{size}(A_j)) \xrightarrow{\prod \Phi_{A_j}} \textstyle\prod_{j=1}^{m} \mathbb{Z} \xrightarrow{+} \mathbb{Z}$$

而 B 的布局函数 Φ_B 可以写成

$$[0, \mathsf{size}(B)) \xrightarrow{\mathsf{colex}^{-1}} \textstyle\prod_{j=1}^{m} [0, \mathsf{size}(B_j)) \xrightarrow{\prod \Phi_{B_j}} \textstyle\prod_{j=1}^{m} \mathbb{Z} \xrightarrow{+} \mathbb{Z}$$

对固定的 $1 \leq i \leq m$，考虑子集

$$[0, \mathsf{size}(A_i)) \subset \prod_{j=1}^{m} [0, \mathsf{size}(A_j))$$

及其像

$$\mathsf{colex}([0, \mathsf{size}(A_i))) \subset [0, \mathsf{size}(A)).$$

由于对所有 $1 \leq j \leq m$ 有 $\mathsf{size}(A_j) = \mathsf{size}(B_j)$，它等同于像

$$\mathsf{colex}([0, \mathsf{size}(B_j))) \subset [0, \mathsf{size}(B)) = [0, \mathsf{size}(B)).$$

Φ_A 在该子集上的限制为 Φ_{A_i}，而 B 在该子集上的限制为 Φ_{B_i}，由此可得 $\Phi_{A_i} = \Phi_{B_i}$，再由命题 2.3.3.14，有 $\mathsf{coal}(A_i) = \mathsf{coal}(B_i)$。于是我们推出

$$\mathsf{coal}(A, \bar{S}) = \mathsf{coal}(B, \bar{S}),$$

这正是所要证明的。 □

2.3.5　紧凑布局

我们可以很容易地把紧凑布局的概念推广到嵌套情形。同样地，用刻画布局的标准网格图（grid diagram）来说，布局 L 是紧凑（compact）的，如果每个整数 $0 \leq i < \mathsf{size}(L)$ 恰好出现一次。更精确地说，我们有如下定义。

定义 2.3.5.1. 设 L 是布局。若布局函数

$$\Phi_L^{\mathsf{cosize}(L)} : [0, \mathsf{size}(L)) \to [0, \mathsf{cosize}(L))$$

是同构（isomorphism），则称 L 是紧凑的。

例 2.3.5.2. 布局

$$A = ((2,2),(2,2)) : ((1,4),(2,8)) =$$

0	2	8	10
1	3	9	11
4	6	12	14
5	7	13	15

是紧凑的，而布局

$$B = ((2,2),(2,2)) : ((1,4),(2,32)) =$$

0	2	32	34
1	3	33	35
4	6	36	38
5	7	37	39

与

$$C = ((2,2),(2,2)) : ((1,4),(2,0)) =$$

0	2	0	2
1	3	1	3
4	6	4	6
5	7	5	7

不是紧凑的。

例 2.3.5.3. 下列布局是紧凑的：

$$\begin{aligned} L_1 &= (2,(2,2)) : (8,(1,4)) \\ L_2 &= ((8,1),(8,32)) : ((2,0),(16,128)) \\ L_3 &= 64 : 1 \end{aligned}$$

例 2.3.5.4. 布局

$$L = (2,(2,2)) : (4,(8,16))$$

不是紧凑的，因为整数 $1 \in [0,29) = [0,\mathsf{cosize}(L))$ 不在 Φ_L 的像中。更一般地，若 $\mathsf{size}(L) \neq \mathsf{cosize}(L)$，则 L 不是紧凑的。

本节最后，我们列出布局 L 紧凑的若干等价条件。

命题 2.3.5.5. *设 L 是布局。则下列条件等价。*

1. *L 紧凑。*
2. *$L^\flat$ 紧凑。*
3. *$\mathsf{coal}(L)$ 紧凑。*

证明. 这些条件的等价性由

$$\Phi_L = \Phi_{L^\flat} = \Phi_{\mathsf{coal}(L)}.$$

这一事实得出。 □

2.3.6　补

我们可以如下把补的概念轻易推广到嵌套情形。

定义 2.3.6.1. 设 A 与 B 是布局。若拼接所得布局 (A, B) 紧凑，则称 B 是 A 的补（complement），记作 $A \perp B$。

引理 2.3.6.2. *设 A 与 B 是布局。则*

$$A \perp B \quad \Leftrightarrow \quad A^\flat \perp B^\flat.$$

证明. 这由 $(A, B)^\flat = A^\flat \star B^\flat$ 这一观察得出。 □

定义 2.3.6.3. 设 A 是布局。若 $A^\flat$ 可求补，则称 A 是可求补的（complementable）。

引理 2.3.6.4. *设 A 是布局。则存在 A 的补 B 当且仅当 A 可求补。*

证明. 若 A 可求补，则 $A^\flat$ 可求补，故存在扁平布局 B 使扁平拼接 $A^\flat \star B$ 紧凑。于是拼接 (A, B) 也紧凑，故 A 拥有补。反之，设存在布局 B 使 (A, B) 紧凑。则 $B^\flat$ 是 $A^\flat$ 的补，由命题 2.1.6.21，$A^\flat$ 可求补，从而由定义，A 也可求补。 □

定义 2.3.6.5. 设 A 是布局。若 A 可求补，则定义

$$\mathsf{comp}(A) = \mathsf{coal}(\mathsf{comp}^\flat(A^\flat)),$$

如构造 2.1.6.16 所述。若 N 是正整数且 A 是 N-可求补的，则定义

$$\mathsf{comp}(A, N) = \mathsf{coal}(\mathsf{comp}^\flat(A^\flat, N))$$

如构造 2.1.6.29 所述。

注 2.3.6.6. 设 A 是可求补布局。则我们几乎总有 $\mathsf{comp}(A) = \mathsf{comp}^\flat(A^\flat)$。更精确地说，若 $\mathsf{comp}^\flat(A^\flat)$ 的长度 > 1，则

$$\mathsf{comp}(A) = \mathsf{comp}^\flat(A^\flat),$$

若 $\mathsf{comp}^\flat(A^\flat) = (s) : (d)$ 的长度为 1，则

$$\mathsf{comp}(A) = s : d,$$

而若 $\mathsf{comp}^\flat(A^\flat) = () : ()$，则

$$\mathsf{comp}(A) = 1 : 0.$$

定义 2.3.6.7. 设 A 是布局，N 是正整数。若 $A \perp B$，且

$$\mathsf{size}(A) \cdot \mathsf{size}(B) = N.$$

则称布局 B 是 A 的 N 补（N-complement）。

定义 2.3.6.8. 设 A 是布局，N 是正整数。若扁平布局 $A^\flat$ 如定义 2.1.6.24 那样是 N-可求补的，则称 A 是 N-可求补的。

命题 2.3.6.9. *设 A 是布局。则存在 A 的 N-补当且仅当 A 是 N-可求补的。*

证明. 若 B 是 A 的 N-补，则 $B^\flat$ 是 $A^\flat$ 的 N-补，于是由命题 2.1.6.32，$A^\flat$ 是 N-可求补的，从而 A 也是。反之，若 A 是 N-可求补的，则 $\mathsf{comp}(A, N)$ 是 A 的 N-补。 □

例 2.3.6.10. 若 $A = ((4, 2), (2, 2)) : ((3, 24), (192, 96))$ 且 $N = 768$，则

$$\mathsf{comp}(A, N) = (3, 2, 2, 2) : (1, 12, 48, 384).$$

例 2.3.6.11. 若 $A = ((16, 4), 64) : ((1, 16), 64)$ 且 $N = 4096$，则

$$\begin{aligned}\mathsf{comp}(A, N) &= \mathsf{coal}(() : ()) \\ &= 1 : 0.\end{aligned}$$

例 2.3.6.12. 若 $A = ((16, 4), 64) : ((1, 16), 64)$ 且 $N = 8192$，则

$$\begin{aligned}\mathsf{comp}(A, N) &= \mathsf{coal}((2) : (4096)) \\ &= 2 : 4096.\end{aligned}$$

例 2.3.6.13. 若 $A = ((16, 4), 64) : ((8, 1), 128)$ 且 $N = 16384$，则

$$\begin{aligned}\mathsf{comp}(A, N) &= \mathsf{coal}((2, 2) : (4, 8192)) \\ &= (2, 2) : (4, 8192).\end{aligned}$$

2.3.7　复合

本节讨论布局上最重要的运算，即复合（composition）。若 A 与 B 是布局，则 A 与 B 的复合是一个布局 $B \circ A$，其布局函数是 A 与 B 的布局函数的复合。更精确地说，我们有如下定义。

定义 2.3.7.1 (布局的复合)**.** 设 A 与 B 是布局。A 与 B 的复合是满足下列性质的唯一布局 $B \circ A$。

1. $\mathsf{shape}(B \circ A)$ 加细 $\mathsf{shape}(A)$，
2. $B \circ A$ 相对于 $\mathsf{shape}(A)$ 已合并，且
3. $\Phi_{B \circ A} = \Phi_B \circ \Phi_A^{\mathsf{size}(B)}$。

注 2.3.7.2. 要使 $B \circ A$ 存在，必须

$$\mathsf{Image}(\Phi_A) \subseteq [0, \mathsf{size}(B)).$$

注 2.3.7.3. 布局复合的定义中隐含了一个断言：满足这三个条件的布局至多只有一个。其依据是命题 2.3.4.10。我们可以把满足条件 1. 与 3.（但未必满足 2.）的布局 C 定义为 A 与 B 的弱复合（weak composite），此时

$$B \circ A = \mathsf{coal}(C, \mathsf{shape}(A))$$

我们将在后面看到，在尝试计算布局的复合时，先计算 A 与 B 的任意一个弱复合 C，再相对于 $\mathsf{shape}(A)$ 合并以得到真正的复合 $B \circ A$，是很有用的。

注 2.3.7.4. 注意，由观察 2.3.4.5，复合定义中的条件 2. 蕴含 $B \circ A$ 非退化。

例 2.3.7.5. 若 $A = (3, 5) : (10, 2)$ 且 $B = (100) : (7)$，则

$$B \circ A = (3, 5) : (70, 14).$$

例 2.3.7.6. 若 $A = (4) : (2)$ 且 $B = (2, 2, 6) : (12, 6, 1)$，则 A 与 B 的复合是

$$B \circ A = ((2, 2)) : ((6, 1)).$$

注 2.3.7.7. 例 2.3.7.6 说明了一个事实：扁平布局 A 与 B 的复合未必是扁平的。

例 2.3.7.8. 若 $A = ((2, 4), 8) : ((4, 8), 8)$ 且 $B = (4, 4, 4, 4) : (2, 4, 8, 16)$，则

$$B \circ A = ((2, (2, 2)), (2, 4)) : ((4, (8, 8)), (8, 8)).$$

例 2.3.7.9. 若 $A = ((3, (2, 2)), 24) : ((3, (9, 18)), 72)$ 且 $B = (9, 8, 3, 8) : (24, 3, 1, 384)$，则

$$B \circ A = ((3, (2, 2)), (3, 8)) : ((72, (3, 6)), (1, 384))$$

接下来，我们建立一些对计算布局复合有用的性质。

命题 2.3.7.10. *设 A 是布局，B 与 $\tilde{B}$ 是满足*

- $\mathsf{size}(B) \le \mathsf{size}(\tilde{B})$，且
- $\Phi_{\tilde{B}} \mid_{\mathsf{size}(B)} = \Phi_B$

的布局。若 A 与 B 可复合（composable），则

$$B \circ A = \tilde{B} \circ A.$$

证明. 设 A 与 B 可复合。则 $\mathsf{cosize}(A) \le \mathsf{size}(B)$，而 $B \circ A$ 是 A 与 $\tilde{B}$ 的复合这一事实由等式

$$\begin{aligned}\Phi_{\tilde{B}} \circ \Phi_A^{\mathsf{size}(\tilde{B})} &= (\Phi_{\tilde{B}}) \mid_{\mathsf{size}(B)} \circ \Phi_A^{\mathsf{size}(B)} \\ &= \Phi_B \circ \Phi_A^{\mathsf{size}(B)}.\end{aligned}$$

得出。 □

推论 2.3.7.11. *若 A 与 B 是布局，则 A 与 B 可复合当且仅当 A 与 $\mathsf{coal}(B)$ 可复合，且*

$$B \circ A = \mathsf{coal}(B) \circ A.$$

至此我们已建立了布局复合的基本性质，接下来将注意力转向复合的两个最重要特例，即*逻辑切分*与*逻辑乘积*。

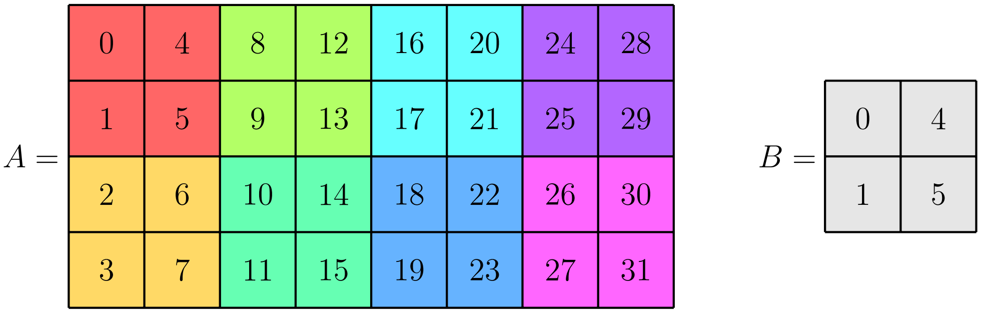
2.3.8　逻辑切分

本节定义布局（layout）的**逻辑切分**（logical division）。作为一个动机示例，考虑布局

$$A = (4, 8) : (1, 4) =$$

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

出于各种目的，我们可能希望对布局 A 进行分块（tile）。例如，下面给出了用几种不同布局 B 对 A 所作的分块（tiling）。

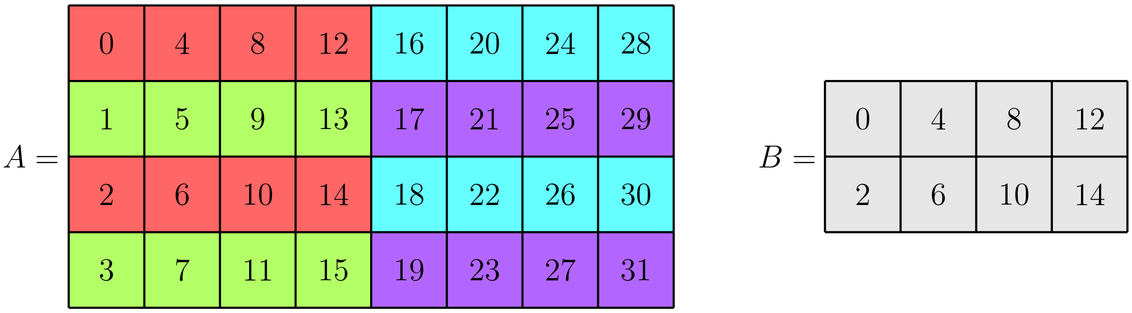


$A =$

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

$B =$

0	4
1	5

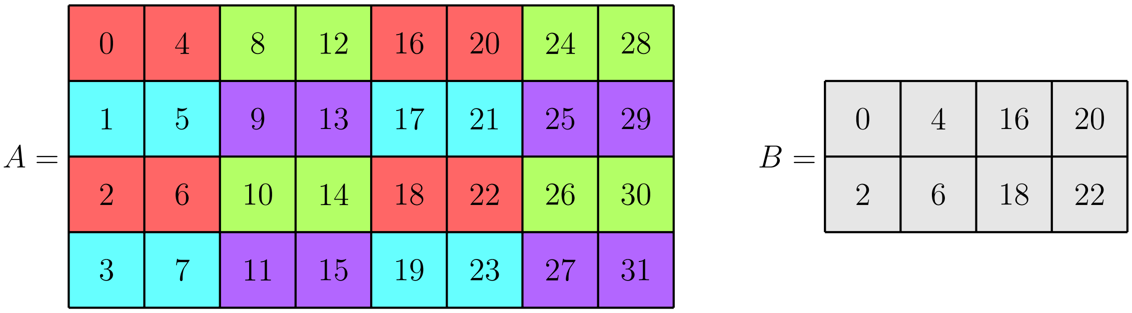


$A =$

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

$B =$

0	4	8	12
2	6	10	14



$A =$

0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

$B =$

0	4	16	20
2	6	18	22

在处理这类经过分块的布局时，我们希望能够用形如 (`tile_coordinate`, `tile`) 的坐标对布局进行索引，其中 `tile` 指明我们正在操作的是哪一个块（tile），而 `tile_coordinate` 指明所给块内部的一个坐标。例如，若 A 与 B 的秩（rank）均为 2，我们便希望把 $((i,j),(k,\ell))$ 写作 A 的第 (k,ℓ) 个块中第 (i,j) 个元素（entry）的索引。逻辑切分 $A \oslash B$ 正是为我们提供这一能力的布局。

定义 2.3.8.1. 设 A 与 B 为布局，并设

$$B^c = \mathsf{comp}(B, \mathsf{size}(A))$$

为 B 关于 $\mathsf{size}(A)$ 的补（complement）。则 A 除以 B 的逻辑切分是布局

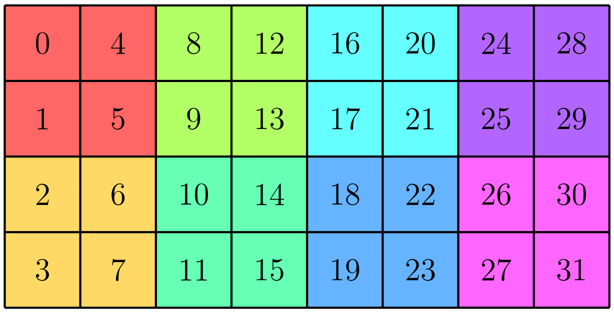
$$\begin{aligned} A \oslash B &= A \circ (B, B^c) \\ &= (A \circ B, A \circ B^c). \end{aligned}$$

例 2.3.8.2. 若 $A = (4,8):(1,4)$ 且 $B = (2,2):(1,4)$，则

$$A \oslash B = ((2,2),(2,4)):((1,4),(2,8)),$$

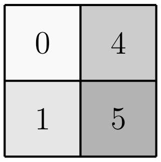
如下图所示。

$A =$



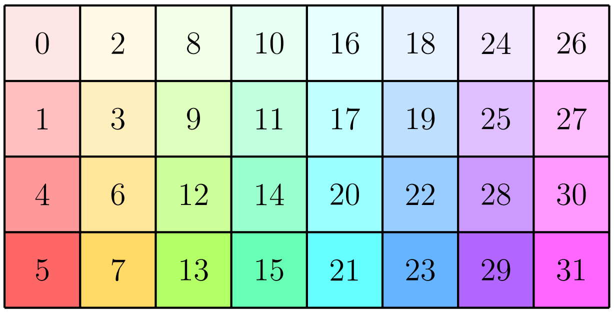
0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

$B =$



0	4
1	5

$A \oslash B =$



0	2	8	10	16	18	24	26
1	3	9	11	17	19	25	27
4	6	12	14	20	22	28	30
5	7	13	15	21	23	29	31

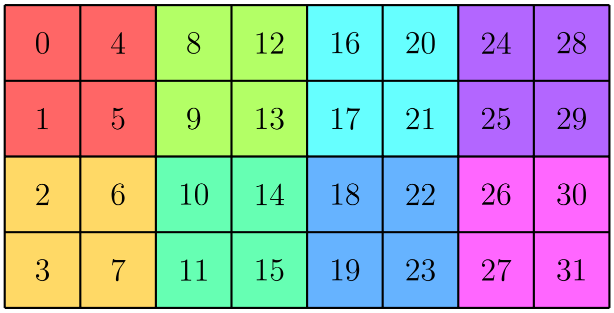
注 2.3.8.3. $A \oslash B$ 中每个元素的颜色（color）表明它属于哪个块，而 $A \oslash B$ 中每个元素的不透明度（opacity）表明它代表该块中的哪个元素。这正是 $A \oslash B$ 的每一列颜色相同、每一行不透明度相同的原因。

例 2.3.8.4. 若 $A = (4,8):(1,4)$ 且 $B = (2,2):(4,1)$，则

$$A \oslash B = ((2,2),(2,4)):((4,1),(2,8)),$$

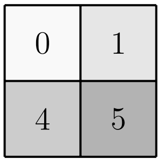
如下图所示。

$A =$



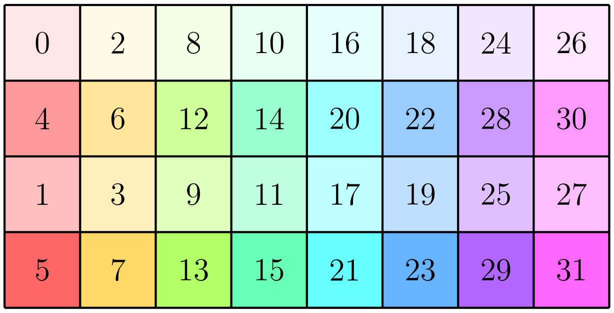
0	4	8	12	16	20	24	28
1	5	9	13	17	21	25	29
2	6	10	14	18	22	26	30
3	7	11	15	19	23	27	31

$B =$



0	1
4	5

$$A \oslash B =$$



0	2	8	10	16	18	24	26
4	6	12	14	20	22	28	30
1	3	9	11	17	19	25	27
5	7	13	15	21	23	29	31

注 2.3.8.5. 注意前两个例子之间的差别。两个例子中对 A 的分块完全相同，但每个块内部的布局各不相同。在第一个例子中，各块具有列主序（column-major）布局，而在第二个例子中，各块具有行主序（row-major）布局。这导致进行逻辑切分时得到不同的布局。

例 2.3.8.6. 若 $A=(4,8):(1,4)$ 且 $B=(2,4):(2,4)$，则

$$A \oslash B=((2,4),(2,2)):((2,4),(1,16)).$$

例 2.3.8.7. 若 $A=(4,6):(1,40)$ 且 $B=6:4$，则

$$A \oslash B=(6,4):(40,1).$$

例 2.3.8.8. 若 $A=(4,6,2,4,2,5):(36,1,18,0,0,144)$ 且 $B=(4,10):(1,192)$，则

$$A \oslash B=(((4,(2,5)),(6,2,4)):((36,(0,144)),(1,18,0))$$

例 2.3.8.9. 若 $A=(8,(4,4))$ 且 $B=(2,(8,16))$，则

$$A \oslash B=((2,2),(2,(4,4))):((4,8),(2,(8,16))).$$

2.3.9　逻辑乘积

本节定义布局的**逻辑乘积**（logical product）。

定义 2.3.9.1. 设 A 与 B 为布局，并设

$$A^c=\mathsf{comp}\big(A,\mathsf{size}(A)\cdot\mathsf{cosize}(B)\big)$$

为 A 关于 $\mathsf{size}(A)\cdot\mathsf{cosize}(B)$ 的补。则 A 与 B 的逻辑乘积是布局

$$A\otimes B=(A,A^c\circ B).$$

观察 2.3.9.2. 由命题 2.3.7.10 与命题 2.3.7.11 可知，若对任意合法的 $N \geq \mathsf{size}(A) \cdot \mathsf{cosize}(B)$ 令

$$\widetilde{A^c} = \mathsf{comp}(A, N)$$

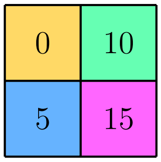
则有

$$A^c \circ B = \widetilde{A^c} \circ B.$$

这意味着在计算 $A \otimes B$ 时，可以把 A^c 取为 A 的任意足够大的（已排序的）补。

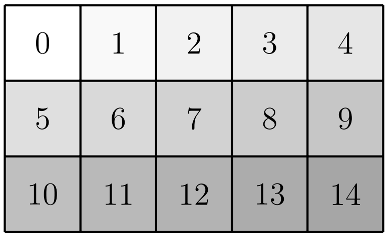
例 2.3.9.3. 若 $A = (2,2) : (5,10)$ 与 $B = (3,5) : (5,1)$ 为布局

$A =$



0	10
5	15

$B =$

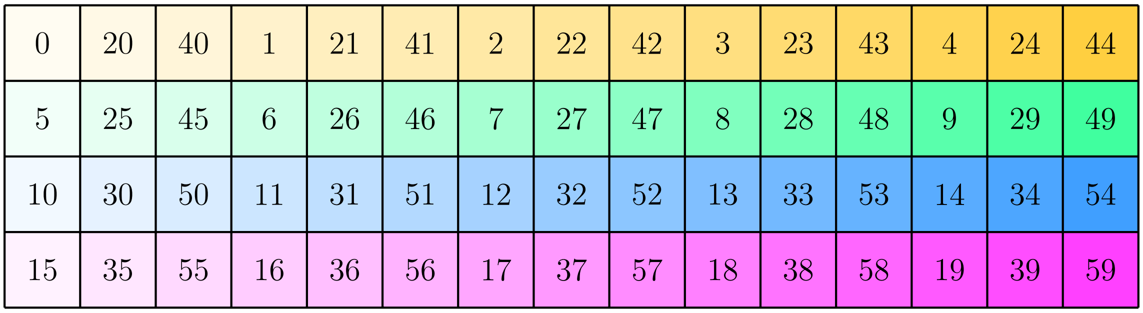


0	1	2	3	4
5	6	7	8	9
10	11	12	13	14

则 $A \otimes B$ 为布局

$$A \otimes B = ((2,2),(3,5)) : ((5,10),(20,1))$$

如下图所示。



0	20	40	1	21	41	2	22	42	3	23	43	4	24	44
5	25	45	6	26	46	7	27	47	8	28	48	9	29	49
10	30	50	11	31	51	12	32	52	13	33	53	14	34	54
15	35	55	16	36	56	17	37	57	18	38	58	19	39	59

例 2.3.9.4. 若 $A = (3,3) : (6,1)$ 且 $B = (10,12) : (24,2)$，则

$$A \otimes B = \big((3,3),(10,12)\big) : \big((6,1),(216,18)\big).$$

例 2.3.9.5. 若 $A = (2,10) : (1680,4)$ 且 $B = (4,9) : (2,56)$，则

$$A \otimes B = ((2,10),((2,2),(3,3))) : ((1680,4),((2,40),(560,3360))).$$

例 2.3.9.6. 若 $A = (4,(2,2)) : (9,(1,3))$ 且 $B = ((2,4),8) : ((1,4),2)$，则

$$A \otimes B = ((4,(2,2)),((2,4),8)) : ((9,(1,3)),((36,144),72)).$$

2.3.10　可处理布局

本节定义一类性质特别良好的布局，称为可处理（tractable）布局。我们将看到，可处理布局恰好是来自某个范畴（category）**Nest** 的布局。

定义 2.3.10.1. 我们称布局 L 是可处理的，如果扁平布局（flat layout）$L^\flat$ 依定义 2.1.8.1 的意义是可处理的。确切地说，L 是可处理的，如果扁平布局

$$\mathsf{sort}(L^\flat) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

满足：对每个 $1 \leq i < m$，有

1. $d_i = 0$，或
2. $s_i d_i$ 整除（divides）d_{i+1}。

例 2.3.10.2. 布局

$$L = (((12))) : (((17)))$$

是可处理的。更一般地，任何长度为 1 的布局 L 都是可处理的。

例 2.3.10.3. 布局

$$L = ((2,4),32) : ((1,2),8)$$

是可处理的。更一般地，任何列主序布局都是可处理的。

例 2.3.10.4. 布局

$$L = (2,(4,32)) : (128,(32,1))$$

是可处理的。更一般地，任何行主序布局 L 都是可处理的。

例 2.3.10.5. 布局

$$L = ((3,3),(1,3),(3,1,3)) : ((81,1),(0,8),(3,0,27))$$

是可处理的。更一般地，任何紧凑（compact）布局都是可处理的。

例 2.3.10.6. 布局

$$L = ((3,7,7)) : ((0,15,0))$$

是可处理的。更一般地，任何恰有一个非零步长（stride）元素的布局都是可处理的。

例 2.3.10.7. 布局

$$L = (2,(2,(2,2))) : (1,(2048,(16,64)))$$

是可处理的。更一般地，任何可求补（complementable）布局都是可处理的。

例 2.3.10.8. 布局

$$L = ((8,8),(5,5)) : ((8,1),(10,2))$$

不是可处理的。特别地，这表明可处理布局 L_1 与 L_2 的拼接（concatenation）(L_1, L_2) 未必是可处理的。

第三章　布局的范畴

在深入探讨了布局的代数之后，我们现在把注意力转向本书的数学核心：把布局（layout）实现为适当定义的范畴（category）中的态射（morphism）。在此过程中，我们将发展一套布局图的图像演算，它使布局运算的计算更为直接。

3.1　范畴 Tuple

本节中，我们定义一个范畴 **Tuple**，其对象（object）是正整数元组（tuple），其态射我们称为元组态射（tuple morphism）。每个元组态射 $f : S \to T$ 编码一个扁平布局（flat layout）L_f。元组态射的复合（composition）与布局复合相容：若 f 与 g 是可复合的元组态射，则

$$L_{g\circ f} = L_g \circ L_f.$$

我们定义一个实现函子（realization functor）（构造 3.1.4.4）

$$|\cdot| : \mathbf{Tuple} \to \mathbf{FinSet}$$

它通过公式

$$|f| = \Phi_{L_f}^{\mathsf{size}(T)}.$$

恢复 L_f 的布局函数（layout function）。我们还将发展一套「元组态射代数」，其中包括排序（sort）（节 3.1.5.3）、合并（coalesce）（节 3.1.5.4）、求补（complement）（节 3.1.5.6）、拼接（concatenate）（节 3.1.5.5）、扁平切分（flat division）（节 3.1.5.7）与扁平乘积（flat products）（节 3.1.5.8）等运算，它们与扁平布局上的相应运算相容。

3.1.1　基本定义

定义 3.1.1.1. 令 $\mathbf{Fin}_*$ 表示这样一个范畴：其对象是基点有限集（pointed finite set）

$$\langle m\rangle_* = \{*, 1, 2, \ldots, m\}$$

其中 $m \geq 0$；其态射 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 是满足 $\alpha(*) = *$ 的函数。我们称这类态射为基点映射（pointed map），或简称映射。

旁注 3.1.1.2. **Fin**$_*$ 是由有限基点集构成的范畴 **FinSet**$_*$ 的一个骨架（skeleton）。

记号 3.1.1.3. 若基点映射 $\alpha : \langle m \rangle_* \to \langle n \rangle_*$ 的陪域（codomain）已知，我们有时把 α 写成

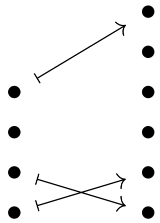
$$\alpha = (\alpha(1), \ldots, \alpha(m))$$

即一个长度为 m、元素（entry）取自 $\langle n \rangle_*$ 的元组。

例 3.1.1.4. 在 **Fin**$_*$ 中有一个态射 $\alpha : \langle 4 \rangle_* \to \langle 6 \rangle_*$，由

$$\alpha = (2, 1, *, 6),$$

给出，我们可以用下图来可视化它。



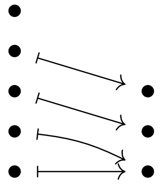
α

注意，与元素 3 对应的圆点不带有箭头，这反映了它被送到 $*$ 这一事实。

例 3.1.1.5. 在 **Fin**$_*$ 中有一个态射 $\beta : \langle 5 \rangle_* \to \langle 3 \rangle_*$，由

$$\beta = (*, 1, 2, 3, *)\ ,$$

给出，我们可以用下图来可视化它。



β

例 3.1.1.6. 对任意 $m \geq 0$，**Fin**$_*$ 中存在唯一形如 $\pi : \langle m \rangle_* \to \langle 0 \rangle_*$ 的态射，即

$$\pi = (*, \ldots, *).$$

例 3.1.1.7. 对任意 $n \geq 0$，**Fin**$_*$ 中存在唯一形如 $\delta : \langle 0 \rangle_* \to \langle m \rangle_*$ 的态射，即

$$\delta = ().$$

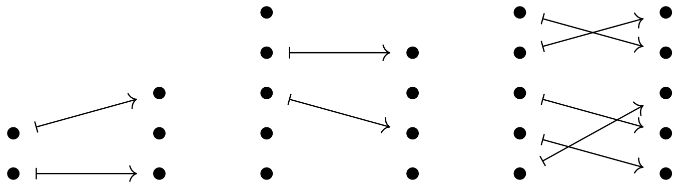
旁注 3.1.1.8. 范畴 Fin_* 是交换 operad 的算子范畴（category of operators），因此我们有时记

$$\mathsf{Fin}_* = \mathsf{Comm}^{\otimes}.$$

我们尤其关注 Fin_* 中的可处理（tractable）态射，其定义如下。

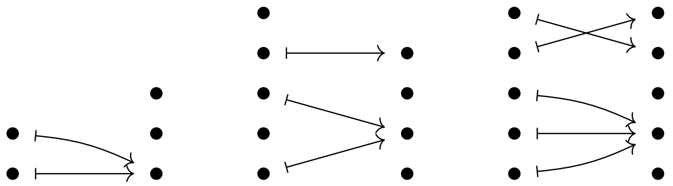
定义 3.1.1.9. 我们称基点映射 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 是可处理的，如果对任意 $j \in \langle n\rangle \subset \langle n\rangle_*$，原像（preimage）$\alpha^{-1}(j)$ 为空或由单个元素组成。

例 3.1.1.10. 映射



α_1　α_2　α_3

是可处理的，而映射



β_1　β_2　β_3

不是可处理的

注 3.1.1.11. 若我们把 Fin_* 中的态射 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 表示为元组，即

$$\alpha = (\alpha(1), \ldots, \alpha(m))$$

则 α 可处理当且仅当 α 中没有正整数出现多于一次。

旁注 3.1.1.12. 由可处理基点映射确定的宽子范畴（wide subcategory）

$$\mathsf{E}_0^{\otimes} \subset \mathsf{Comm}^{\otimes} = \mathsf{Fin}_*$$

是 E_0 operad 的算子范畴。

定义 3.1.1.13. 令 Tuple 表示这样一个范畴：其对象是正整数元组

$$S = (s_1, \ldots, s_m)$$

其中的态射

$$f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$$

由一个可处理基点映射 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 指定，且满足以下性质：

- 若 $1 \leq i \leq m$ 且 $\alpha(i) \neq *$，则 $s_i = t_{\alpha(i)}$ 。

我们称这样的态射 f 位于 α 之上（lies over），并把 f 称为元组态射。

记号 3.1.1.14. 若 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是位于 α 之上的元组态射，我们有时把 f 画成

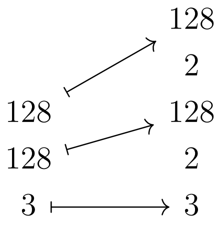
$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n).$$

我们所发展的布局图像演算，基于 **Tuple** 中态射的自然可视化，如下例所示。

例 3.1.1.15. 元组态射

$$(3, 128, 128) \xrightarrow[(1,3,5)]{f} (3, 2, 128, 2, 128)$$

可以用下图来可视化。



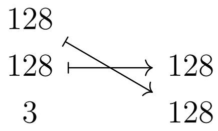
128
2
128
128
128
2
3
3

f

例 3.1.1.16. 元组态射

$$(3, 128, 128) \xrightarrow[(*,2,1)]{g} (128, 128)$$

可以用下图来可视化。



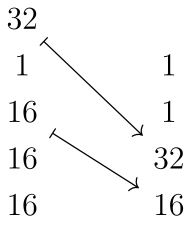
128
128
128
3
128

g

例 3.1.1.17. 元组态射

$$(16, 16, 16, 1, 32) \xrightarrow[(*,*,1,*,2)]{h} (16, 32, 1, 1)$$

可以用下图来可视化。



32
1
1
16
1
16
32
16
16

h

观察 3.1.1.18. 我们可以如下把范畴 **Tuple** 与一些熟知的 operad 联系起来。令 $\mathbb{Z}^{\mathsf{div}}_{>0}$ 表示正整数依整除关系构成的偏序集(poset),并把它视为一个对称幺半范畴(symmetric monoidal category)，其乘积由整数的乘法给出。令 $(\mathbb{Z}^{\mathsf{div}}_{>0})^{\otimes}$ 表示 $\mathbb{Z}^{\mathsf{div}}_{>0}$ 的算子范畴。于是存在显然的函子（functor）

$$\mathbf{Tuple} \to (\mathbb{Z}^{\mathsf{div}}_{>0})^{\otimes},$$

与

$$\mathbf{Tuple} \to \mathbf{E}_0^{\otimes},$$

使得图

$$\begin{tikzcd}
\mathbf{Tuple} \arrow[r] \arrow[d] & (\mathbb{Z}^{\mathsf{div}}_{>0})^{\otimes} \arrow[d] \\
\mathbf{E}_0^{\otimes} \arrow[r] & \mathbf{Comm}^{\otimes}
\end{tikzcd}$$

交换。这把 **Tuple** 展现为拉回（pullback）operad

$$\mathbf{Tuple} \subset \mathbf{E}_0^{\otimes} \times_{\mathbf{Comm}^{\otimes}} (\mathbb{Z}^{\mathsf{div}}_{>0})^{\otimes}$$

中由满足

$$\alpha(i) \neq 1 \quad \Rightarrow \quad s_i = t_{\alpha(i)}.$$

的态射

$$(s_1, \dots, s_m) \xrightarrow[\alpha]{f} (t_1, \dots, t_n)$$

构成的宽子范畴。

3.1.2　从元组态射到扁平布局

使用范畴 **Tuple** 的动因在于：每个元组态射 f 都编码一个扁平布局 L_f。而且，每个可处理布局 L 都产生一个元组态射 f_L。我们在 命题 3.1.2.10 中证明：这两个构造在某种意义上互为逆，且可处理布局恰是那些由元组态射编码的布局。

构造 3.1.2.1. 设

$$(s_1, \dots, s_m) \xrightarrow[\alpha]{f} (t_1, \dots, t_n)$$

是一个元组态射。我们定义 L_f 为这样一个扁平布局：其形状（shape）

$$\mathsf{shape}(L_f) = (s_1, \dots, s_m)$$

是 f 的定义域（domain），其步长（stride）

$$\mathsf{stride}(L_f) = (d_1, \dots, d_m)$$

由公式

$$d_i = \begin{cases} 0 & \alpha(i) = * \\ \prod_{j<\alpha(i)} t_j & \alpha(i) \neq *. \end{cases}$$

定义。我们把 L_f 称为 f 所编码的布局（layout encoded by f），或 f 所关联的布局（layout associated to f）。

例 3.1.2.2. 例 3.1.1.15 中的元组态射

$$\begin{tikzcd}
 & 128 \\
 & 2 \\
128 \arrow[ruu, mapsto] & 128 \\
128 \arrow[ru, mapsto] & 2 \\
3 \arrow[r, mapsto] & 3
\end{tikzcd}$$

$$f$$

编码布局

$$L_f = (3, 128, 128) : (1, 6, 1536).$$

注意，按 构造 3.1.2.1 中的公式计算步长，相当于沿着从某个形状元素指向其目标元素的箭头，把该目标元素下方的所有元素相乘（空积取为 1）。

例 3.1.2.3. 例 3.1.1.16 中的元组态射

$$(3, 128, 128) \xrightarrow[(*,2,1)]{g} (128, 128)$$

编码布局

$$L_g = (3, 128, 128) : (0, 128, 1).$$

例 3.1.2.4. 例 3.1.1.17 中的元组态射

$$(16, 16, 16, 1, 32) \xrightarrow[(*,*,1,*,2)]{h} (16, 32, 1, 1)$$

编码布局

$$L_h = (16, 16, 16, 1, 32) : (0, 0, 1, 0, 16).$$

我们已经看到如何计算由元组态射 f 编码的扁平布局 L_f。另一方面，若 L 是可处理的，则我们可以沿相反方向进行，构造一个编码 L 的元组态射 f。回顾定义 2.1.8.1：扁平布局 L 称为可处理的，如果

$$\mathsf{sort}(L) = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

满足以下性质：

$$若 1 \leq i < m，则 d_i = 0，或 s_i d_i \text{ 整除} d_{i+1}.$$

构造 3.1.2.5. 设 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$ 可处理，并设

$$\mathsf{sort}(L) = (s'_1, \ldots, s'_m) : (d'_1, \ldots, d'_m),$$

于是存在某个置换（permutation）$\sigma \in \Sigma_m$ 使得 $\mathsf{sort}(L) = L^\sigma$。换言之，对每个 $1 \le i \le m$，有 $s'_i = s_{\sigma(i)}$ 且 $d'_i = d_{\sigma(i)}$。若每个 d'_i 都非零，则令 $k = 0$；否则，令 k 为使 $d'_k = 0$ 的最大整数。令 $\ell = 2(m-k)$，并令

$$(t'_1, \ldots, t'_\ell) = \left(d'_{k+1}, s'_{k+1}, \frac{d'_{k+2}}{s'_{k+1}d'_{k+1}}, s'_{k+2}, \frac{d'_{k+3}}{s'_{k+2}d'_{k+2}}, \ldots, \frac{d'_m}{s'_{m-1}d'_{m-1}}, s'_m\right).$$

我们把

$$f'_L : (s_1, \ldots, s_m) \to (t'_1, \ldots, t'_\ell)$$

定义为位于映射 $\alpha : \langle m\rangle_* \to \langle \ell\rangle_*$ 之上的元组态射，α 由

$$\alpha'(i) = \begin{cases} * & \sigma^{-1}(i) \le k \\ 2(\sigma^{-1}(i) - k) & k+1 \le \sigma^{-1}(i) \le m. \end{cases}$$

给出。令 $J = \{j_1 < \cdots < j_n\} \subset \langle \ell\rangle$ 表示由满足「j_i 为偶数或 $t_{j_i} \neq 1$」的指标构成的集合。令

$$(t_1, \ldots, t_n) = (t'_{j_1}, \ldots, t'_{j_n}),$$

并令 $\iota : \langle n\rangle_* \to \langle \ell\rangle_*$ 为包含映射（inclusion）$i \mapsto j_i$。于是按构造，映射 α' 可分解为 $\alpha' = \iota \circ \alpha$，我们定义 L 的标准表示（standard representation）为元组态射

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f_L} (t_1, \ldots, t_n).$$

例 3.1.2.6. 若

$$L = (2, 2) : (3, 30),$$

则 L 可处理，且 L 的标准表示是元组态射

$$\begin{tikzcd}[row sep=tiny]
 & 2 \\
 & 5 \\
2 \arrow[ruu, mapsto] & 2 \\
2 \arrow[ru, mapsto] & 3
\end{tikzcd}$$

$$f_L$$

注意，非正式地说，按 构造 3.1.2.5 计算 f_L 相当于：

- 把陪域初始化为 $()$，

- 按递增顺序遍历 L 的非零步长，
- 若 d_j 是当前步长，而 d_i 是上一个访问过的步长，则追加
    - (s_j)（若 $s_i d_i = d_j$），或
    - $\left(\frac{d_j}{s_i d_i}, s_j\right)$（若 $s_i d_i < d_j$），

  以及
- 把 s_j 映到 s_j。

例 3.1.2.7. 若

$$L = (128, 128) : (128, 1),$$

则 L 可处理，且 L 的标准表示是元组态射

$$\begin{tikzcd}[row sep=tiny]
128 \arrow[rd, mapsto] & 128 \\
128 \arrow[ru, mapsto] & 128
\end{tikzcd}$$

$$f_L$$

例 3.1.2.8. 若

$$L = (2, 2, 2, 2) : (24, 0, 3, 480),$$

则 L 可处理，且 L 的标准表示是元组态射

$$\begin{tikzcd}[row sep=tiny]
 & 2 \\
 & 10 \\
2 \arrow[ruu, mapsto] & 2 \\
2 \arrow[rd, mapsto] & 4 \\
2 \arrow[ruu, mapsto] & 2 \\
2 & 3
\end{tikzcd}$$

$$f_L$$

我们来论证 构造 3.1.2.5 中的元组态射 f_L 确实编码布局 L。

引理 3.1.2.9. 设 L 是可处理扁平布局，$f = f_L$ 是 L 的标准表示。则 f 编码的布局为

$$L_f = L.$$

证明. 设 $L = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$ 可处理，并令

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n)$$

为 L 的标准表示。显然

$$\mathsf{shape}(L_f) = (s_1, \ldots, s_m) = \mathsf{shape}(L).$$

我们需要验证 $\mathsf{stride}(L_f) = \mathsf{stride}(L)$。换言之，需要验证对任意 $1 \le i \le m$，有

$$d_i = \begin{cases} 0 & \alpha(i) = * \\ \prod_{j<\alpha(i)} t_j & \alpha(i) \neq *. \end{cases}$$

我们沿用 构造 3.1.2.5 的记号。若 $\alpha(i) = *$，则 $\alpha'(i) = *$，于是 $\sigma^{-1}(i) \le k$。由此可得

$$d_i = d'_{\sigma^{-1}(i)} = 0.$$

反之设 $\alpha(i) \neq *$。则 $\alpha'(i) \neq *$，于是 $k+1 \le \sigma^{-1}(i) \le m$。我们计算

$$\begin{aligned} \prod_{j<\alpha(i)} t_j = \prod_{\substack{j'<\alpha'(i) \\ t'_{j'}\neq 1}} t'_{j'} = \prod_{j'<\alpha'(i)} t'_{j'} = \prod_{j'<2(\sigma^{-1}(i)-k)} t'_{j'} &= d'_{k+1} \cdot \left(\prod_{v=1}^{\sigma^{-1}(i)-(k+1)} s'_{k+v} \frac{d'_{k+v+1}}{s'_{k+v} d'_{k+v}} \right) \\ &= d'_{\sigma^{-1}(i)} \\ &= d_i. \end{aligned}$$

□

我们已经证明：若 L 是可处理扁平布局，则存在编码 L 的元组态射 f。接下来证明逆命题，它蕴含可处理扁平布局恰是那些由元组态射编码的布局。

命题 3.1.2.10. *设 L 是扁平布局。则存在编码 L 的元组态射 f，当且仅当 L 可处理。*

证明. 首先，设 L 是扁平布局，且 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是满足 $L_f = L$ 的元组态射。我们要证 L_f 可处理。令

$$\mathsf{sort}(L) = (s'_1, \ldots, s'_m) : (d_1, \ldots, d_m)$$

为 L 的排序，并设 $1 \le i < m$。我们将论证 $d_i = 0$，或 $s'_i d_i$ 整除（divides）d_{i+1}。若 $d_i = 0$，则已得证。设 $d_i \neq 0$。则对某个满足 $s'_i = t_k$ 的 $1 \le k \le n$，有

$$d_i = \prod_{j<k} t_j$$

由于 $d_{i+1} \ge d_i$，可知 $d_{i+1} \neq 0$，故 d_{i+1} 形如

$$d_{i+1} = \prod_{j<\ell} t_j$$

其中 $1 \le \ell \le n$。分两种情形讨论：

- （情形 1）若 $\ell > k$，则

$$d_{i+1} = \prod_{j<\ell} t_j = \left(\prod_{j\le k} t_j\right)\left(\prod_{k<j<\ell} t_j\right) = s_i' d_i \left(\prod_{k<j<\ell} t_j\right),$$

  故 $s_i' d_i$ 整除 d_{i+1}。

- （情形 2）若 $\ell \le k$，则由

$$\prod_{j<\ell} t_j = d_{i+1} \ge d_i = \prod_{j<k} t_j,$$

  必有

$$t_\ell = \cdots = t_{k-1} = 1,$$

  以及

$$d_{i+1} = d_i.$$

  特别地，有 $s_{i+1}' = t_\ell = 1$。但由于 $\mathsf{sort}(L_f)$ 是已排序（sorted）的且 $d_{i+1} = d_i$，有 $s_i' \le s_{i+1}' = 1$，故 $s_i' = 1$。我们推得

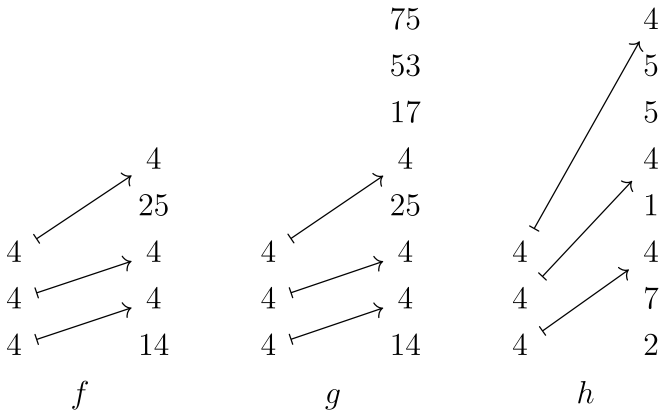
$$s_i' d_i = d_{i+1},$$

  特别地，$s_i' d_i$ 整除 d_{i+1}。

我们断定 L 可处理。

其次，设 L 可处理。此时可取 $f = f_L$ 为 L 的标准表示（见构造 3.1.2.5），于是由引理 3.1.2.9，有 $L = L_f$。 □

注 3.1.2.11. 值得注意的是，有许多不同的元组态射给出同一个布局。例如，下面展示的每个元组态射


4
4
4
4
25
4
4
14
f
4
4
4
75
53
17
4
25
4
4
14
g
4
4
4
4
5
5
4
1
4
7
2
h

都编码布局

$$L_f = L_g = L_h = (4,4,4) : (14, 56, 5600).$$

其中 f 最简单：在 f 的像（image）上方没有多余的元素（不同于 g），且未被 f 命中的元素是紧缩在一起的（不同于 h）。为把 f 在这些态射中的简单性精确化，我们引入标准形式（standard form）的概念。

定义 3.1.2.12. 设

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n)$$

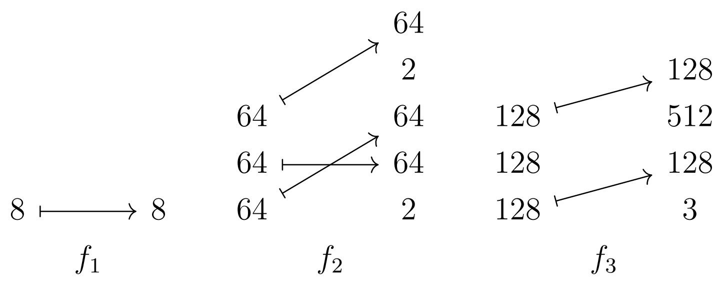
是一个元组态射。若下列条件成立，则称 f 具有标准形式：

1. 若 $n > 1$，则 $n \in \mathsf{Image}(\alpha)$。
2. 若 $1 \leq j < n$，则

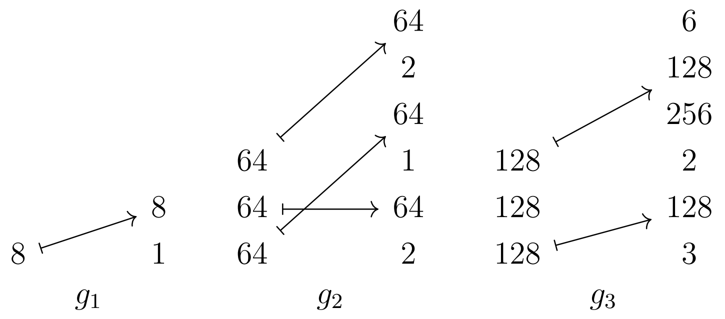
$$j \notin \mathsf{Image}(\alpha) \quad \Rightarrow \quad \begin{array}{c} t_j \neq 1, \text{ and} \\ j+1 \in \mathsf{Image}(\alpha) \end{array}$$

例 3.1.2.13. 注记 3.1.2.11 中的元组态射 f 具有标准形式，而 g 与 h 不具有。

例 3.1.2.14. 元组态射


8
8
64
64
64
64
2
64
64
2
128
128
128
128
512
128
3
f_1
f_2
f_3

具有标准形式，而元组态射


8
8
1
64
64
64
64
2
64
1
64
2
128
128
128
6
128
256
2
128
3
g_1
g_2
g_3

不具有。

例 3.1.2.15. 若 L 是可处理布局，则按构造，L 的标准表示 f_L 具有标准形式。

若我们把范围限制到具有标准形式的元组态射，则它与可处理布局之间几乎是一一对应的。然而，有一种需要排除的麻烦情形，如下例所阐明的。

例 3.1.2.16. 考虑下面展示的元组态射 f 与 g。

$$\begin{tikzcd}
1 \arrow[r, mapsto] & 1 & 1 \arrow[dr, mapsto] & 1 \\
1 \arrow[r, mapsto] & 1 & 1 \arrow[ur, mapsto] & 1 \\
8 \arrow[r, mapsto] & 8 & 8 \arrow[r, mapsto] & 8 \\
f & & g &
\end{tikzcd}$$

f 与 g 都具有标准形式，且

$$L_f = (8,1,1):(1,8,8) = L_g.$$

本例说明：形如 $s_i = 1$ 且 $\alpha(i) \neq *$ 的元素的存在，可能导致具有标准形式的表示元组态射不唯一。在布局这一侧，这对应于形状元素 $s_i = 1$ 而步长 $d_i \neq 0$ 的情形。为了排除这类病态例子，我们引入非退化（non-degeneracy）的概念。

定义 3.1.2.17. 设

$$(s_1,\dots,s_m) \xrightarrow[\alpha]{f} (t_1,\dots,t_n)$$

是一个元组态射，且

$$L = (s_1,\dots,s_m):(d_1,\dots,d_m)$$

是一个扁平布局。

1. 我们称 f 是非退化的，如果

$$s_i = 1 \quad \Rightarrow \quad \alpha(i) = *.$$

2. 我们称 L 是非退化的，如果

$$s_i = 1 \quad \Rightarrow \quad d_i = 0.$$

观察 3.1.2.18. 若 f 是非退化元组态射，则 f 编码的布局 L_f 非退化。反之，若 L 是非退化扁平布局，则 L 的标准表示 f_L 非退化。

观察 3.1.2.19. 限制到非退化扁平布局并不真正损失一般性。若 L 是任意扁平布局，则 $\mathsf{filter}(L)$ 是与 L 具有相同坐标函数（coordinate function）与布局函数的非退化扁平布局。

具有标准形式的非退化元组态射的一条本质性质是：它们由其所编码的布局刻画。精确表述如下。

引理 3.1.2.20. 设 f 与 g 是具有标准形式的非退化元组态射。若 $L_f = L_g$，则 $f = g$。

证明. 设

$$(s_1,\ldots,s_m) \xrightarrow[\alpha]{f} (t_1,\ldots,t_n)$$

与

$$(s_1,\ldots,s_m) \xrightarrow[\beta]{g} (u_1,\ldots,u_p)$$

是具有标准形式的非退化元组态射，且

$$L_f = (s_1,\ldots,s_m) : (d_1,\ldots,d_m) = L_g.$$

我们要证 $f = g$。首先论证 $(t_1,\ldots,t_n) = (u_1,\ldots,u_p)$。令

$$X = \{t_1 \cdots t_j \mid 1 \le j \le n\}$$
$$Y = \{u_1 \cdots u_k \mid 1 \le k \le p\}$$

分别表示 $(t_1,\ldots,t_n)$ 与 $(u_1,\ldots,u_p)$ 的前缀积（prefix product）之集。我们断言 $X = Y$，因为这两个集合都等于

$$Z = \{d_i, s_i d_i \mid 1 \le i \le m \text{ and } d_i \ne 0\}.$$

来论证 $X = Z$。设 $1 \le j \le n$。若存在某个 $i \in \langle m \rangle$ 使 $\alpha(i) = j$，则 $t_1 \cdots t_j = s_i d_i$。另一方面，若 j 不在 α 的像中，则由于 f 具有标准形式，存在某个 $i \in \langle m \rangle$ 使 $\alpha(i) = j+1$，此时 $t_1 \cdots t_j = d_i$。这证明了 $X \subseteq Z$。反之，若 $1 \le i \le m$ 且 $d_i \ne 0$，则 $d_i = t_1 \cdots t_{\alpha(i)-1}$ 且 $s_i d_i = t_1 \cdots t_{\alpha(i)}$，这证明 $Z \subseteq X$。于是 $X = Z$。同理可证 $Y = Z$。

由于 f 与 g 都是非退化且具有标准形式的，可知每个 t_j 与每个 u_k 都大于 1，从而

$$t_1 < t_1 t_2 < \cdots < t_1 \cdots t_n,$$
$$u_1 < u_1 u_2 < \cdots < u_1 \cdots u_p,$$

又因 $X = Y$，可得 $n = p$，且对每个 $1 \le j \le n$ 有 $t_1 \cdots t_j = u_1 \cdots u_j$。我们推得 $(t_1,\ldots,t_n) = (u_1,\ldots,u_p)$。

接下来需要论证 $\alpha = \beta$。反设存在某个 $i \in \langle m \rangle$ 使 $\alpha(i) \ne \beta(i)$。分两种情形讨论。

- 若 $\alpha(i) = * \ne \beta(i)$，则
  $$0 = d_i = t_1 \cdots t_{\beta(i)-1},$$
  矛盾。情形 $\alpha(i) \ne * = \beta(i)$ 类似。

- 若 $\alpha(i) \ne * \ne \beta(i)$，则不失一般性可设 $\alpha(i) < \beta(j)$，此时
  $$d_i = t_1 \cdots t_{\alpha(i)-1} < t_1 \cdots t_{\beta(i)-1} = d_i,$$
  矛盾。

我们推得 $\alpha = \beta$，故 $f = g$。 □

现在我们可以证明对应定理了，它把具有标准形式的非退化元组态射与非退化可处理扁平布局等同起来。

定理 3.1.2.21. 构造 *3.1.2.1* 与 *3.1.2.5* 给出的映射

$$f \longmapsto L_f$$

$$\left\{\begin{array}{c}\text{非退化}\\ \text{具有标准形式的}\\ \text{元组态射}\end{array}\right\} \longleftrightarrow \left\{\begin{array}{c}\text{非退化}\\ \text{可处理扁平}\\ \text{布局}\end{array}\right\}$$

$$f_L \longleftarrow\!\shortmid\; L$$

在具有标准形式的非退化元组态射与非退化可处理扁平布局之间建立了一一对应。

证明. 我们要证：当限制到所述形式的元组态射与布局时，构造 $f \mapsto L_f$ 与 $L \mapsto f_L$ 互为逆。若 L 是非退化可处理扁平布局，则由引理 3.1.2.9，有 $L_{f_L} = L$。其次设 f 是具有标准形式的非退化元组态射，且 $L = L_f$ 是 f 编码的布局。由于 f 与 f_{L_f} 都是具有标准形式的非退化元组态射，且它们编码的布局相等，由引理 3.1.2.20 得 $f = f_{L_f}$。 □

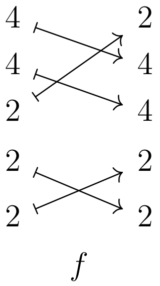
3.1.3　例子

本节引入若干重要的元组态射族，并描述由它们产生的扁平布局。

例 3.1.3.1 (恒等态射). 我们称元组态射 f 为恒等态射（identity morphism），如果对某个元组 S 有 $f = \mathrm{id}_S$。若 $f = \mathrm{id}_S$ 是恒等态射，则 L_f 是形状为 S 的列主序（column-major）布局。例如，下面给出一个恒等态射 f 及其关联布局 L_f 的例子。

$$\begin{array}{ccc} 4 & \longmapsto & 4 \\ 4 & \longmapsto & 4 \\ 2 & \longmapsto & 2 \\ 2 & \longmapsto & 2 \\ 2 & \longmapsto & 2 \\ & f & \end{array} \qquad \rightsquigarrow \qquad L_f = (2,2,2,4,4):(1,2,4,8,32)$$

例 3.1.3.2 (同构)**.** 元组态射 $f : S \to T$ 称为同构（isomorphism），如果存在元组态射 $g : T \to S$ 使得 $g \circ f = \mathsf{id}_S$ 且 $f \circ g = \mathsf{id}_T$。若 f 是同构，则其关联布局 L_f 是紧凑（compact）的。例如，下面给出一个同构 f 及其关联布局 L_f。



$$\rightsquigarrow \qquad L_f = (2, 2, 2, 4, 4) : (2, 1, 64, 4, 16)$$

.

观察 3.1.3.3. 注意，若元组态射

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n)$$

是同构，则 $\alpha : \langle m \rangle_* \to \langle m \rangle_*$ 是双射，从而 $\alpha\,|_{\langle m \rangle} \in \Sigma_m$ 是一个置换。反之，若 $\sigma \in \Sigma_m$ 是置换，且 $(s_1, \ldots, s_m)$ 是正整数元组，则我们可以构造同构

$$(s_1, \ldots, s_m) \xrightarrow[\sigma_*]{f} (s_{\sigma(1)}, \ldots, s_{\sigma(m)}).$$

由此断定：以 $(s_1, \ldots, s_m)$ 为定义域的元组同构 f 与 Σ_m 中的置换之间存在一一对应。

例 3.1.3.4 (投影)**.** 设 $S = (s_1, \ldots, s_m)$ 是一个形状，并设

$$\{i_1 < \cdots < i_r\} \subset \langle m \rangle$$

是某个子集。令

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{p} (s_{i_1}, \ldots, s_{i_r})$$

为位于映射 α 之上的元组态射，其中

$$\alpha(x) = \begin{cases} j & x = i_j \\ * & \text{否则。} \end{cases}$$

我们称 p 为 $(s_1, \ldots, s_m)$ 到 $(s_{i_1}, \ldots, s_{i_r})$ 上的投影（projection）。p 编码的布局为

$$L_p = (s_1, \ldots, s_m) : (d_1, \ldots, d_m),$$

其中

$$d_i = \begin{cases} s_{i_1} \cdots s_{i_{j-1}} & i = i_j\text{，对某个}1 \leq j \leq r \\ 0 & \text{其他情形。} \end{cases}$$

例如，下面给出 $(64,64,3,8)$ 到 $(64,3)$ 的投影 p 及其关联布局。

$$\begin{tikzcd}
8 & & & \\
3 \arrow[dr, mapsto] & & & \\
64 & 3 & \rightsquigarrow & L_p = (64,64,3,8):(1,0,64,0) \\
64 \arrow[r, mapsto] & 64 & & \\
\quad p & & &
\end{tikzcd}$$

例 3.1.3.5 (膨胀)**.** 设 $S=(s_1,\ldots,s_m)$ 是一个形状，并设 $c_1,\ldots,c_m$ 是正整数。元组态射

$$(s_1,\ldots,s_m) \xrightarrow[(*,2,*,4,\ldots,*,2m)]{f} (c_1,s_1,\ldots,c_m,s_m)$$

称为 $(s_1,\ldots,s_m)$ 经 $(c_1,\ldots,c_m)$ 的膨胀（dilation）。与该态射关联的布局 L_f 为 $L_f=(s_1,\ldots,s_m):(d_1,\ldots,d_m)$，其中

$$d_i = \prod_{j<i} c_j s_j.$$

例如，下面给出 $(512,512)$ 经 $(2,4)$ 的膨胀 f 及其关联布局。

$$\begin{tikzcd}
 & 512 & & \\
 & 4 & & \\
512 \arrow[uur, mapsto] & 512 & \rightsquigarrow & L_f = (512,512):(2,4096) \\
512 \arrow[ur, mapsto] & 2 & & \\
\quad f & & &
\end{tikzcd}$$

例 3.1.3.6 (扩张)**.** 设 $S=(s_1,\ldots,s_m)$ 是正整数元组，并设 $1\leq i\leq m'$，使得 $S'=(s_1,\ldots,s_{m'})$ 整除 S。则元组态射

$$(s_1,\ldots,s_{m'}) \xrightarrow[(1,2,\ldots,m')]{e} (s_1,\ldots,s_{m'},\ldots,s_m)$$

称为 S' 到 S 的扩张（expansion）。e 编码的布局是形状为 $(s_1,\ldots,s_{m'})$ 的列主序布局。例如，下面给出 $S'=(4,4)$ 到 $S=(4,4,8,8)$ 的扩张。

$$\begin{tikzcd}
 & 8 & & \\
 & 8 & & \\
4 \arrow[r, mapsto] & 4 & \rightsquigarrow & L_e = (4,4):(1,4) \\
4 \arrow[r, mapsto] & 4 & & \\
\quad e & & &
\end{tikzcd}$$

扩张的一个重要性质是：若 $f : S \to T$ 是任意元组态射，且 $e : T \to T'$ 是扩张，则

$$L_{e \circ f} = L_f.$$

换言之，把 f 与扩张后复合，不会改变 f 编码的布局。

例 3.1.3.7 (限制)**.** 设

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n)$$

是一个元组态射，并设

$$I = \{i_1 < \cdots < i_r\} \subset \langle m \rangle$$

是一个指标子集。则元组态射

$$(s_{i_1}, \ldots, s_{i_r}) \xrightarrow[\alpha \circ \iota]{f|_I} (t_1, \ldots, t_n)$$

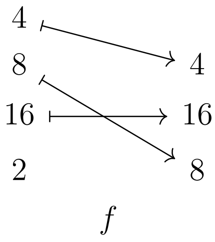
称为 f 在 I 上的限制（restriction）。若 f 编码的布局是

$$L_f = (s_1, \ldots, s_m) : (d_1, \ldots, d_m),$$

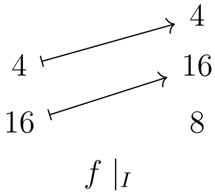
则 $f\,|_I$ 编码的布局是

$$L_{f|_I} = (s_{i_1}, \ldots, s_{i_r}) : (d_{i_1}, \ldots, d_{i_r}).$$

例如，下面给出某个元组态射 f 的限制 $f\,|_I$，其中 $I = \{2, 4\}$。



$$\rightsquigarrow \qquad L_f = (2, 16, 8, 4) : (0, 8, 1, 128)$$



$$\rightsquigarrow \qquad L_{f|_I} = (16, 4) : (8, 128)$$

例 3.1.3.8 (元素包含)**.** 前一个构造的一个重要特例如下。若 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_m)$ 是元组态射，且 $1 \leq i \leq m$，则 f 的第 i 个元素 f_i 是

$$(s_i) \xrightarrow[(i)]{f_i} (t_1, \ldots, t_n)$$

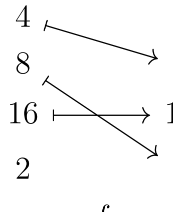
若 f 编码的布局是

$$L_f = (s_1, \dots, s_m) : (d_1, \dots, d_m),$$

则 f_i 编码的布局是

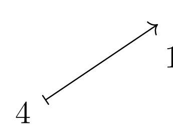
$$L_{f_i} = (s_i) : (d_i).$$

例如，下面给出一个元组态射 f 及其第四个元素 f_4。



f

$$\rightsquigarrow \qquad L_f = (2, 16, 8, 4) : (0, 8, 1, 128)$$



f_4

$$\rightsquigarrow \qquad L_{f_4} = (4) : (128)$$

注 3.1.3.9. 给定 **Fin**$_*$ 中的 $\langle n\rangle_*$，对每个 $i \in \langle n\rangle$ 存在态射 $\varphi_i : \langle 1\rangle_* \to \langle n\rangle_*$，它把 $* \mapsto *$、$1 \mapsto i$。对位于 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 之上的元组态射 $f : (s_1, \dots, s_m) \to (t_1, \dots, t_n)$，其第 i 个元素位于复合 $\alpha \circ \varphi_i : \langle 1\rangle_* \to \langle n\rangle_*$ 之上。

例 3.1.3.10 (因子分解). 设

$$(s_1, \dots, s_m) \xrightarrow[\alpha]{f} (t_1, \dots, t_n)$$

是一个元组态射，并设

$$J = \{j_1 < \cdots < j_\ell\} \subset \langle n\rangle$$

是满足 $\mathsf{Image}(\alpha) \subseteq J \cup \{*\}$ 的子集。若记 $\iota : \langle \ell\rangle_* \to \langle n\rangle_*$ 为映射 $k \mapsto j_k$，则 α 可分解为 $\alpha = \iota \circ \bar{\alpha}$，其中映射 $\bar{\alpha} : \langle m\rangle_* \to \langle \ell\rangle_*$ 唯一；我们定义 f 经由 J 的因子分解（factorization）为元组态射

$$(s_1, \dots, s_m) \xrightarrow[\bar{\alpha}]{f|^J} (t_{j_1}, \dots, t_{j_\ell}).$$

若 f 编码的布局是

$$L_f = (s_1, \dots, s_m) : (d_1, \dots, d_m),$$

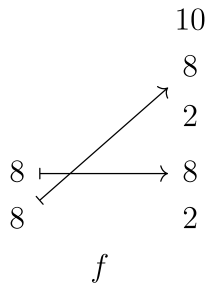
则 $f\,|^J$ 编码的布局是

$$L_{f|^J} = (s_1, \dots, s_m) : (d'_1, \dots, d'_m),$$

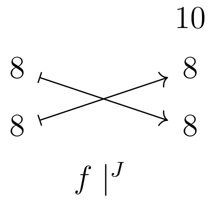
其中

$$d_i' = \frac{d_i}{\left(\displaystyle\prod_{k<\alpha(i) \text{ and } k\notin J} t_j\right)}.$$

例如，下面给出某个元组态射 f 的因子分解 $f\,|^J$，其中 $J = \{2, 4, 5\}$。



$$\rightsquigarrow \qquad L_f = (8,8) : (32,2)$$



$$\rightsquigarrow \qquad L_{f|^J} = (8,8) : (8,1)$$

注 3.1.3.11. 因子分解有一个范畴论解释。沿用例 3.1.3.10 的记号，可以观察到一个位于 ι 之上的元组态射 $i : (t_{j_1}, \ldots, t_{j_\ell}) \to (t_1, \ldots, t_n)$，而 $f\,|^J$ 是 f 沿 i 的拉回：

$$\begin{tikzcd}
(s_1, \ldots, s_m) \arrow[r, "f|^J"] \arrow[d, "\mathsf{id}"'] \arrow[dr, phantom, "\lrcorner", very near start] & (t_{j_1}, \ldots, t_{j_\ell}) \arrow[d, "i"] \\
(s_1, \ldots, s_m) \arrow[r, "f"'] & (t_1, \ldots, t_n)
\end{tikzcd}$$

3.1.4　元组态射的实现

如我们所见，元组态射 $f : S \to T$ 编码一个扁平布局 L_f。本节将构造一个实现函子

$$|\cdot| : \textbf{Tuple} \to \textbf{FinSet}.$$

使这一编码显式化。实现函子 $|\cdot|$ 把元组态射 f 送到 L_f 的布局函数 $|f|$。为构造实现函子 $|\cdot|$，我们先构造一个辅助函子

$$F : \textbf{Tuple} \to \textbf{FinSet}$$

它将在我们的构造中使用。

构造 3.1.4.1. 我们定义函子

$$F : \textbf{Tuple} \to \textbf{FinSet}$$

如下。

- 对 **Tuple** 中的对象 $S = (s_1, \ldots, s_m)$，定义
$$FS = [0, S) = \prod_{i=1}^{m} [0, s_i).$$
- 对 **Tuple** 中位于 α 之上的态射 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$，定义 Ff 为映射
$$[0, S) \xrightarrow{Ff} [0, T)$$
它由
$$(Ff)(x_1, \ldots, x_m) = (y_1, \ldots, y_n)$$
给出，其中
$$y_j = \begin{cases} x_i & \text{存在} 1 \le i \le m \text{ 使得} \alpha(i) = j, \\ 0 & \text{否则。} \end{cases}$$

容易验证 $F(g \circ f) = Fg \circ Ff$ 且 $F\mathrm{id}_S = \mathrm{id}_{FS}$，故 F 确为一个函子。

例 3.1.4.2. 设 $f : (4, 4) \to (4, 4, 4)$ 是位于 $\alpha = (1, 3)$ 之上的元组态射。则

$$Ff : [0, (4, 4)) \to [0, (4, 4, 4))$$

由

$$(Ff)(x_1, x_2) = (x_1, 0, x_2).$$

给出。

例 3.1.4.3. 设 $g : (3, 256, 256, 512) \to (3, 256, 256)$ 是位于 $\beta = (*, 3, 2, *)$ 之上的元组态射。则

$$Fg : [0, (3, 256, 256, 512)) \to [0, (3, 256, 256))$$

由

$$(Fg)(x_1, x_2, x_3, x_4) = (0, x_3, x_2).$$

给出。

构造 3.1.4.4. 我们定义函子

$$|\cdot| : \textbf{Tuple} \to \textbf{FinSet}$$

如下。

- 对 **Tuple** 中的对象 $S=(s_1,\dots,s_m)$，定义
$$|S|=[0,\mathsf{size}(S))=\{0,1,\dots,\mathsf{size}(S)-1\}.$$
- 对元组态射 $f:S\to T$，定义
$$|f|=\mathsf{colex}_T\circ Ff\circ\mathsf{colex}_S^{-1}$$
（回顾 定义 2.1.2.18）。

若 $f:S\to T$ 与 $g:T\to U$ 是可复合的元组态射，则

$$\begin{aligned}|g\circ f|&=\mathsf{colex}_U\circ F(g\circ f)\circ\mathsf{colex}_S^{-1}\\&=\mathsf{colex}_U\circ Fg\circ Ff\circ\mathsf{colex}_S^{-1}\\&=\mathsf{colex}_U\circ Fg\circ\mathsf{colex}_T^{-1}\circ\mathsf{colex}_T\circ Ff\circ\mathsf{colex}_S^{-1}\\&=|g|\circ|f|\end{aligned}$$

而若 $f=\mathsf{id}_S$ 是恒等态射，则

$$\begin{aligned}|\mathsf{id}_S|&=\mathsf{colex}_S\circ F\mathsf{id}_S\circ\mathsf{colex}_S^{-1}\\&=\mathsf{colex}_S\circ\mathsf{id}_S\circ\mathsf{colex}_S^{-1}\\&=\mathsf{colex}_S\circ\mathsf{colex}_S^{-1}\\&=\mathsf{id}_{|S|}\ ,\end{aligned}$$

故 $|\cdot|$ 确实给出一个函子。接下来我们注意到：对 **Tuple** 中的态射 f，映射 $|f|$ 是 L_f 的布局函数。由此容易推出 **Tuple** 中态射的复合与扁平布局的复合相容（见推论 3.1.4.6）。

引理 3.1.4.5. *若 $f:S\to T$ 是元组态射，则 f 的实现 $|f|$ 是 L_f 的布局函数：*

$$|f|=\Phi_{L_f}^{\mathsf{size}(T)}$$

证明. 记 $S=(s_1,\dots,s_m)$，$T=(t_1,\dots,t_n)$，并令

$$L_f=(s_1,\dots,s_m):(d_1,\dots,d_m)$$

为与 f 关联的布局，其步长 d_i 由公式

$$d_i=\begin{cases}0 & \alpha(i)=*\\ \prod_{j<\alpha(i)}t_j & \text{否则。}\end{cases}$$

定义。用 $\mathsf{colex}_S:\prod_{i=1}^m[0,s_i)\to[0,\mathsf{size}(S))$ 预复合后，只需证明：对任意 $(x_1,\dots,x_m)\in\prod_{i=1}^m[0,s_i)$，有

$$(\mathsf{colex}_T\circ Ff)(x_1,\dots,x_m)=(x_1,\dots,x_m)\cdot(d_1,\dots,d_m).$$

对一般输入 $(x_1, \ldots, x_m) \in \prod_{i=1}^m [0, s_i)$，有

$$(Ff)(x_1, \ldots, x_m) = (y_1, \ldots, y_n)$$

其中 y_j 在 $\alpha(i) = j$ 时等于 x_i，否则等于 0。由此可得

$$\begin{aligned}(\mathsf{colex}_T \circ Ff)(x_1, \ldots, x_m) &= (y_1, \cdots, y_n) \cdot (1, t_1, \ldots, t_1 \cdots t_{n-1}) \\ &= \sum_{j=1}^{n} y_j \cdot t_1 \cdots t_{j-1} \\ &= \sum_{i=1}^{m} x_i d_i \\ &= (x_1, \ldots, x_m) \cdot (d_1, \ldots, d_m),\end{aligned}$$

这正是所要证明的。□

推论 3.1.4.6. *若 f 与 g 是可复合的非退化元组态射，则*

$$L_{g \circ f} = L_g \circ L_f$$

证明. 设 $f : S \to T$ 与 $g : T \to U$ 是 **Tuple** 中分别位于 α 与 β 之上的态射。记 $S = (s_1, \ldots, s_m)$，$T = (t_1, \ldots, t_n)$。我们需要验证：

1. $\mathsf{shape}(L_{g \circ f})$ 加细（*refines*）$\mathsf{shape}(L_f)$：由于 L_f 与 $L_{g \circ f}$ 的形状都等于 S，该条成立。

2. $L_{g \circ f}$ 在 $\mathsf{shape}(L_f)$ 上已合并（*coalesced*）：由于元组态射 $g \circ f$ 非退化，故布局 $L_{g \circ f}$ 亦非退化，该条成立。

3. $\Phi_{L_{g \circ f}} = \Phi_{L_g} \circ \Phi_{L_f}^{\mathsf{size}(L_g)}$：利用引理 3.1.4.5，有

$$\begin{aligned}\Phi_{L_{g \circ f}}^{\mathsf{size}(U)} &= |g \circ f| \\ &= |g| \circ |f| \\ &= \Phi_{L_g}^{\mathsf{size}(U)} \circ \Phi_{L_f}^{\mathsf{size}(T)}.\end{aligned}$$

   再以包含映射 $[0, \mathsf{size}(U)) \subset \mathbb{Z}$ 作后复合，并注意到 $\mathsf{size}(T) = \mathsf{size}(L_g)$，即得结论。

□

3.1.5　元组态射上的运算

我们的下一个目标是发展一套「元组态射代数」，其中包括合并（coalesce）、求补（complement）、复合（composition）、扁平切分（flat division）以及扁平乘积（flat products）等运算。我们将证明，这些运算中的每一个都与扁平布局（flat layout）上的相应运算相容。

3.1.5.1　和

元组态射 f 与 g 的和（sum）$f \oplus g$ 通过拼接 f 与 g 的定义域（domain）与陪域（codomain）得到。为了精确定义这个运算，我们先在 Fin_* 中的态射上定义一个相应的运算。

定义 3.1.5.1. 设 $\alpha : \langle m \rangle_* \to \langle n \rangle_*$ 与 $\beta : \langle p \rangle_* \to \langle q \rangle_*$ 是 Fin_* 中的态射。我们把 α 与 β 的和定义为态射

$$\alpha \oplus \beta : \langle m + p \rangle_* \to \langle n + q \rangle_*$$

它由

$$(\alpha \oplus \beta)(x) = \begin{cases} \alpha(x) & 1 \leq x \leq m \\ n + \beta(x - m) & m + 1 \leq x \leq m + p \\ * & x = *. \end{cases}$$

给出。该运算满足结合律，因此对 Fin_* 中任意有限组态射 $\alpha_1, \ldots, \alpha_k$，可以考虑和 $\alpha_1 \oplus \cdots \oplus \alpha_k$。

注 3.1.5.2. 若 α 与 β 是可处理（tractable）的基点映射（pointed map），则 $\alpha \oplus \beta$ 是可处理的。

现在我们可以定义 Tuple 中态射的和。

定义 3.1.5.3. 设 $f : S \to T$ 与 $g : U \to V$ 是分别位于 α 与 β 之上（lies over）的元组态射。我们把 f 与 g 的和定义为元组态射

$$f \oplus g : S \star U \to T \star V$$

它位于 $\alpha \oplus \beta$ 之上。该运算满足结合律，因此对 Tuple 中任意有限组态射 $f_1, \ldots, f_k$，可以考虑和 $f_1 \oplus \cdots \oplus f_k$。

例 3.1.5.4. 下面给出元组态射 f 与 g 的和 $f \oplus g$ 的一个例子。

$$\begin{tikzcd}
 & & & & 4 & 4 \\
 & & & & 4 \arrow[ur, mapsto] & 2 \\
32 \arrow[r, mapsto] & 32 & 4 & 4 & 32 \arrow[r, mapsto] & 32 \\
16 \arrow[r, mapsto] & 16 & 4 \arrow[ur, mapsto] & 2 & 16 \arrow[r, mapsto] & 16 \\
f & & g & & f \oplus g &
\end{tikzcd}$$

例 3.1.5.5. 下面给出元组态射 f 与 g 的和 $f \oplus g$ 的另一个例子。

$$\begin{tikzcd}
 & & & & 64 \arrow[r, mapsto] & 64 \\
 & & & & 512 \arrow[dr, mapsto] & 256 \\
 & & 64 \arrow[r, mapsto] & 64 & 256 \arrow[ur, mapsto] & 512 \\
128 \arrow[dr, mapsto] & 128 & 512 \arrow[dr, mapsto] & 256 & 128 \arrow[dr, mapsto] & 128 \\
128 \arrow[ur, mapsto] & 128 & 256 \arrow[ur, mapsto] & 512 & 128 \arrow[ur, mapsto] & 128 \\
f & & g & & f \oplus g &
\end{tikzcd}$$

注 3.1.5.6. 元组态射的和有一个范畴论解释：若 $f : S \to T$ 与 $g : U \to V$ 是元组态射，则

$$f \oplus g : S \star U \to T \star V$$

是箭头范畴（arrow category）$\mathsf{Ar}(\textbf{Tuple})$ 中 f 与 g 的余积（coproduct）。

3.1.5.2　压缩

我们常常希望从元组中去掉所有等于整数 1 的分量。这一任务由压缩（squeeze）函子（functor）完成。

定义 3.1.5.7. 我们定义函子

$$\textbf{Tuple} \xrightarrow{\mathsf{squeeze}(-)} \textbf{Tuple}$$

如下。若 $S = (s_1, \ldots, s_m)$ 是 **Tuple** 中的对象（object），我们定义

$$\mathsf{squeeze}(S) = (s_{i_1}, \ldots, s_{i_k})$$

其中 $\{i_1 < \cdots < i_k\} \subset \langle m \rangle$ 是满足 $s_{i_j} \neq 1$ 的那些指标。若 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是元组态射，我们定义

$$\mathsf{squeeze}(f) : \mathsf{squeeze}(S) \to \mathsf{squeeze}(T)$$

为元组态射

$$\mathsf{squeeze}(f) = (f\mid_I)\mid^J$$

其中 $f\mid_I$ 是 f 到

$$I = \{i \in \langle m \rangle \mid s_i \neq 1\}$$

的限制（restriction），见定义 3.1.3.7；而 $(f\mid_I)\mid^J$ 是 $f\mid_I$ 经由

$$J = \{j \in \langle n \rangle \mid t_j \neq 1\},$$

的因子分解（factorization），见定义 3.1.3.10。

例 3.1.5.8. 下面给出一个态射 f 及相应态射 $\mathsf{squeeze}(f)$ 的例子。

$$\begin{tikzcd}
 & 256 & & & \\
256 \arrow[ur, mapsto] & 128 & & & 256 \\
128 \arrow[ur, mapsto] & 1 & & & 128 \\
1 & 32 & & 256 \arrow[uur, mapsto] & 32 \\
8 \arrow[dr, mapsto] & 32 & \rightsquigarrow & 128 \arrow[uur, mapsto] & 32 \\
1 & 8 & & 8 \arrow[r, mapsto] & 8 \\
f & & & \mathsf{squeeze}(f) &
\end{tikzcd}$$

例 3.1.5.9. 若 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是元组态射，则

$$f = \mathsf{squeeze}(f) \quad \Leftrightarrow \quad \text{没有} s_i\text{、}t_j \text{ 等于} 1.$$

命题 3.1.5.10. 若 f 是元组态射，则

$$L_{\mathsf{squeeze}(f)} = \mathsf{squeeze}(L_f).$$

证明. 设 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_m)$ 是元组态射，并令

$$L_f = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

为 f 所关联的布局。令 $I = \{i_1 < \cdots < i_{m'}\} \subset \langle m \rangle$ 表示满足 $s_{i_k} \neq 1$ 的指标子集。于是

$$\begin{aligned} L_{f|_I} &= (s_{i_1}, \ldots, s_{i_k}) : (d_{i_1}, \ldots, d_{i_k}) \\ &= \mathsf{squeeze}(L_f). \end{aligned}$$

令 $J = \{j_1 < \cdots < j_{n'}\} \subset \langle n \rangle$ 表示满足 $t_{j_k} \neq 1$ 的指标子集，于是 $\mathsf{squeeze}(f) = (f\ |_I)\ |^J$。令 β 表示 $\mathsf{squeeze}(f)$ 所位于的映射。则

$$L_{\mathsf{squeeze}(f)} = L_{(f|_I)|^J} = (s_{i_1}, \ldots, s_{i_k}) : (d'_{i_1}, \ldots, d'_{i_k})$$

其中

$$\begin{aligned} d'_{i_k} &= \frac{d_{i_k}}{\left(\displaystyle\prod_{\ell < \beta(k) \text{ and } \ell \notin J} t_\ell \right)} \\ &= d_{i_k} \end{aligned}$$

因为对任意 $\ell \notin J$ 有 $t_\ell = 1$。我们得到

$$\begin{aligned} L_{\mathsf{squeeze}(f)} &= (s_{i_1}, \ldots, s_{i_k}) : (d_{i_1}, \ldots, d_{i_k}) \\ &= \mathsf{squeeze}(L_f). \end{aligned}$$

□

观察 3.1.5.11. 若 f 是元组态射，则

$$\mathsf{squeeze}(\mathsf{squeeze}(f)) = \mathsf{squeeze}(f),$$

因此

$$\textbf{Tuple} \xrightarrow{\mathsf{squeeze}(-)} \textbf{Tuple}$$

是一个幂等（idempotent）函子。

3.1.5.3　排序

排序（sort）运算 $f \mapsto \mathsf{sort}(f)$ 通过置换（permutation）f 的定义域，使所得态射在下述意义上是已排序（sorted）的。

定义 3.1.5.12. 我们称元组态射

$$(s_1, \ldots, s_m) \xrightarrow[\alpha]{f} (t_1, \ldots, t_n)$$

为已排序的，如果对任意 $1 \le i, j \le m$，以下条件成立。

1. 若 $\alpha(i) = * \neq \alpha(j)$，则 $i < j$。
2. 若 $\alpha(i) = * = \alpha(j)$，则
$$i \le j \quad \Rightarrow \quad s_i \le s_j.$$
3. 若 $\alpha(i) \neq * \neq \alpha(j)$，则
$$i \le j \quad \Rightarrow \quad \alpha(i) \le \alpha(j).$$

例 3.1.5.13. 下面展示的态射 f_1、f_2 与 f_3

$$\begin{tikzcd}
 & \\
128 \arrow[dr, mapsto] & \\
512 \arrow[dr, mapsto] & 128 \\
3 & 512
\end{tikzcd}
\qquad
\begin{tikzcd}
 & 4 \\
4 \arrow[ur, mapsto] & 1 \\
1 & 8 \\
1 & 64
\end{tikzcd}
\qquad
\begin{tikzcd}
60 \arrow[r, mapsto] & 60 \\
20 \arrow[dr, mapsto] & 2 \\
32 & 20 \\
8 & 4
\end{tikzcd}$$

f_1　　f_2　　f_3

是已排序的，而下面展示的态射 g_1、g_2 与 g_3

$$\begin{tikzcd}
 & 512 \\
512 \arrow[ur, mapsto] & 2 \\
32 \arrow[dr, mapsto] & 32 \\
32 \arrow[ur, mapsto] & 32
\end{tikzcd}
\qquad
\begin{tikzcd}
1 & 4 \\
4 \arrow[ur, mapsto] & 8 \\
8 \arrow[ur, mapsto] & 64
\end{tikzcd}
\qquad
\begin{tikzcd}
2 \arrow[r, mapsto] & 2 \\
8 & 24 \\
24 & 16
\end{tikzcd}$$

g_1　　g_2　　g_3

不是已排序的。态射 g_1、g_2 与 g_3 分别违反条件 3、1 与 2。

命题 3.1.5.14. 若 f 是已排序的元组态射，则扁平布局 L_f 是已排序的。

证明. 设

$$(s_1, \dots, s_m) \xrightarrow[\alpha]{f} (t_1, \dots, t_n)$$

已排序，并考虑布局

$$L_f = (s_1, \dots, s_m) : (d_1, \dots, d_m).$$

设 $1 \leq i < m$。我们要证明 $d_i < d_{i+1}$，或 $d_i = d_{i+1}$ 且 $s_i \leq s_{i+1}$。分两种情形讨论。

- （情形 1）设 $\alpha(i) = *$，于是 $d_i = 0$。若 $\alpha(i+1) = *$，则 $d_{i+1} = 0$，且由于 f 已排序，有 $s_i \leq s_{i+1}$。若 $\alpha(i+1) \neq *$，则 $d_{i+1} \geq 1 > 0 = d_i$。
- （情形 2）设 $\alpha(i) \neq *$，此时 $\alpha(i+1) \neq *$ 且 $\alpha(i) < \alpha(i+1)$。则

$$d_i = \prod_{j<\alpha(i)} t_j \leq \prod_{j<\alpha(i+1)} = d_{i+1},$$

  其中等号仅当 $s_i = t_{\alpha(i)} = 1$ 时成立，而这蕴含 $s_i \leq s_{i+1}$。

我们得到 L_f 是已排序的。 □

接下来，我们定义 **Tuple** 上的 $\mathsf{sort}(-)$ 运算。若 f 是元组态射，则 $\mathsf{sort}(f)$ 将通过用一个适当的置换 g 预复合（precompose）f 而得到。

构造 3.1.5.15. 设

$$(s_1, \dots, s_m) \xrightarrow[\alpha]{f} (t_1, \dots, t_n)$$

是一个元组态射。我们如下定义一个置换 $\sigma \in \Sigma_m$。令

$$P = \{i \in \langle m \rangle \mid \alpha(i) = *\},$$
$$Q = \{i \in \langle m \rangle \mid \alpha(i) \neq *\},$$

于是 $\langle m \rangle$ 是 P 与 Q 的不相交并 (disjoint union)。我们定义 P 上的一个线性序 (linear ordering)：$i_1 \preceq_P i_2$，如果

1. $s_{i_1} < s_{i_2}$，或
2. $s_{i_1} = s_{i_2}$ 且 $i_1 \leq i_2$。

我们定义 Q 上的线性序：若 $\alpha(i_1) \leq \alpha(i_2)$，则 $j_1 \preceq_Q j_2$。我们定义 $\langle m \rangle$ 上的线性序：$i_1 \preceq i_2$，如果

1. $i_1 \in P$ 且 $i_2 \in Q$，
2. $i_1, i_2 \in P$ 且 $i_1 \preceq_P i_2$，或

3. $i_1, i_2 \in Q$ 且 $i_1 \preceq_Q i_2$。

令 σ 为与 $\langle m \rangle$ 的线性序 $\preceq$ 相关联的置换，并令 σ^{-1} 为其逆（inverse）。映射 $\sigma_*^{-1} : \langle m \rangle_* \to \langle m \rangle_*$ 由一个元组态射覆盖（covered by）

$$g : (s_{\sigma^{-1}(1)}, \ldots, s_{\sigma^{-1}(m)}) \to (s_1, \ldots, s_m),$$

我们定义 $\mathsf{sort}(f)$ 为复合

$$\mathsf{sort}(f) = f \circ g.$$

例 3.1.5.16. 例 3.1.5.13 中态射 g_1、g_2 与 g_3 的排序结果如下所示。

$$\begin{tikzcd}
 & 512 & & & 512 \\
512 \arrow[ur, mapsto] & 2 & & 512 \arrow[ur, mapsto] & 2 \\
32 \arrow[dr, mapsto] & 32 & \rightsquigarrow & 32 \arrow[r, mapsto] & 32 \\
32 \arrow[ur, mapsto] & 32 & & 32 \arrow[r, mapsto] & 32 \\
 & g_1 & & & \mathsf{sort}(g_1)
\end{tikzcd}$$

$$\begin{tikzcd}
1 & 4 & & 4 \arrow[r, mapsto] & 4 \\
4 \arrow[ur, mapsto] & 8 & \rightsquigarrow & 8 \arrow[r, mapsto] & 8 \\
8 \arrow[ur, mapsto] & 64 & & 1 & 64 \\
 & g_2 & & & \mathsf{sort}(g_2)
\end{tikzcd}$$

$$\begin{tikzcd}
2 \arrow[r, mapsto] & 2 & & 2 \arrow[r, mapsto] & 2 \\
8 & 24 & \rightsquigarrow & 24 & 24 \\
24 & 16 & & 8 & 16 \\
 & g_3 & & & \mathsf{sort}(g_3)
\end{tikzcd}$$

引理 3.1.5.17. 设 $f : S \to T$ 是元组态射。则 f 已排序当且仅当 $\mathsf{sort}(f) = f$。

证明. 我们对 $\mathsf{sort}(-)$ 的构造保证：对任意元组态射 f，$\mathsf{sort}(f)$ 都是已排序的。特别地，若 $f = \mathsf{sort}(f)$，则 f 已排序。反之，若 f 已排序，则构造 3.1.5.15 中的置换 $\sigma \in \Sigma_m$ 是恒等置换，于是 $g = \mathsf{id}_S$，从而

$$\mathsf{sort}(f) = f \circ \mathsf{id}_S = f.$$

□

命题 3.1.5.18. 若 f 是元组态射，则

$$L_{\mathsf{sort}(f)} = \mathsf{sort}(L_f).$$

证明. 沿用构造 3.1.5.15 的记号，我们有 $\mathsf{sort}(f) = f \circ g$，其中

$$g : (s_{\sigma^{-1}(1)}, \ldots, s_{\sigma^{-1}(m)}) \to (s_1, \ldots, s_m)$$

位于 $\sigma_*^{-1} : \langle m \rangle_* \to \langle m \rangle_*$ 之上。若 $L_f = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$，则

$$\begin{aligned} L_{\mathsf{sort}(f)} &= (s'_1, \ldots, s'_m) : (d'_1, \ldots, d'_m) \\ &= (s_{\sigma^{-1}(1)}, \ldots, s_{\sigma^{-1}(m)}) : (d_{\sigma^{-1}(1)}, \ldots, d_{\sigma^{-1}(m)}). \end{aligned}$$

由于 $L_{\mathsf{sort}(f)}$ 的各模式（mode）是 L_f 的各模式的一个置换，只需证明 $L_{\mathsf{sort}(f)}$ 是已排序的。设 $1 \leq i < m$。先设 $\sigma^{-1}(i) \in P$，于是 $d'_i = d_{\sigma^{-1}(i)} = 0$。若 $\sigma^{-1}(i+1) \in P$，则 $d'_{i+1} = d_{\sigma^{-1}(i+1)} = 0$。由 σ 的构造，有 $s'_i = s_{\sigma^{-1}(i)} \leq s_{\sigma^{-1}(i+1)} = s'_{i+1}$。若 $\sigma^{-1}(i+1) \in Q$，则 $d'_{i+1} = d'_{\sigma^{-1}(i+1)} > 0 = d'_i$。再设 $\sigma^{-1}(i) \in Q$。则由 σ 的构造，有 $\sigma^{-1}(i+1) \in Q$ 且 $\alpha(\sigma^{-1}(i)) < \alpha(\sigma^{-1}(i+1))$，并且

$$\begin{aligned} d'_i = d_{\sigma^{-1}(i)} &= \prod_{j < \alpha(\sigma^{-1}(i))} t_j \\ &\leq \prod_{j < \alpha(\sigma^{-1}(i+1))} t_j \\ &= d_{\sigma^{-1}(i+1)} \\ &= d'_{i+1}, \end{aligned}$$

其中等号成立当且仅当 $t_{\alpha(\sigma^{-1}(i))} = \cdots = t_{\alpha(\sigma^{-1}(i+1))-1} = 1$。特别地，我们有 $s_i = s_{\sigma^{-1}(i)} = t_{\alpha(\sigma^{-1}(i))} = 1$，于是 $s'_i \leq s'_{i+1}$。我们得到 $L_{\mathrm{sort(f)}}$ 是已排序的，因此 $L_{\mathsf{sort}(f)} = \mathsf{sort}(L_f)$。 □

注 3.1.5.19. 运算 $\mathsf{sort}(-)$ 不具函子性。例如，考虑元组态射

$$(2,3) \xrightarrow[(2,1)]{f} (3,2) \qquad \text{and} \qquad (10,25) \xrightarrow[(2,1)]{g} (25,10)$$

则 f 与 g 可复合，且 $g \circ f = \mathsf{id}_{(25,10)}$，但已排序的态射

$$(10,25) \xrightarrow[(1,2)]{\mathsf{sort}(f)} (10,25) \qquad \text{and} \qquad (25,10) \xrightarrow[(1,2)]{\mathsf{sort}(g)} (25,10)$$

不可复合。

3.1.5.4　合并

我们先引入已合并（coalesced）元组态射的概念。

定义 3.1.5.20. 设 $f : S \to T$ 是位于 α 之上的元组态射。我们称 f 是已合并的，如果

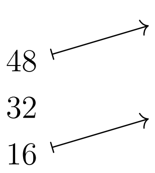
1. $S = \mathsf{squeeze}(S)$，且
2. 对任意 $1 \le i < \mathsf{len}(S)$，以下条件中恰有一个成立：
    - (a) $\alpha(i) = * \ne \alpha(i+1)$，
    - (b) $\alpha(i) \ne * = \alpha(i+1)$，
    - (c) $\alpha(i) > \alpha(i+1)$，或
    - (d) $\alpha(i) < \alpha(i+1)$，且存在 $\alpha(i) < j < \alpha(i+1)$ 使得 $t_j > 1$。

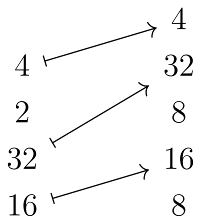
例 3.1.5.21. 若存在某个 $1 \le i < \mathsf{len}(S)$ 使得 $\alpha(i+1) = \alpha(i) + 1$，则 f 不是已合并的。

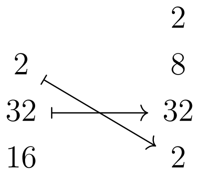
注 3.1.5.22. 若 $f : S \to T$ 是满足 $f = \mathsf{squeeze}(f)$ 的元组态射，则 f 已合并当且仅当对任意 $1 \le i < \mathsf{len}(S)$，以下条件之一成立：

1. $\alpha(i) = * \ne \alpha(i+1)$，
2. $\alpha(i) \ne * = \alpha(i+1)$，
3. $\alpha(i) > \alpha(i+1)$，或
4. $\alpha(i+1) \ne \alpha(i) + 1$。

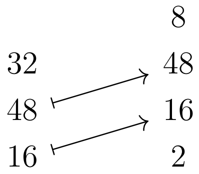
例 3.1.5.23. 态射

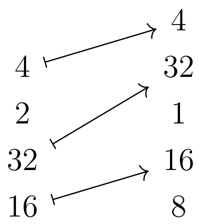

48
48
8
32
16
16
2
f_1

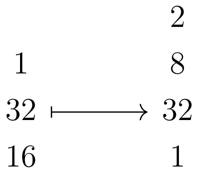

4
4
32
2
8
32
16
16
8
f_2


2
2
8
32
32
16
2
f_3

是已合并的，而态射


8
32
48
48
16
16
2
g_1


4
4
32
2
1
32
16
16
8
g_2


2
1
8
32
32
16
1
g_3

不是已合并的。

命题 3.1.5.24. 设 f 是元组态射。则 f 已合并当且仅当 L_f 已合并。

证明. 设 $f:(s_1,\ldots,s_m)\to(t_1,\ldots,t_n)$ 是元组态射，并令

$$L_f=(s_1,\ldots,s_m):(d_1,\ldots,d_m)$$

为 f 所编码的布局。

先设 f 是已合并的。则 $\mathsf{shape}(L_f)=\mathsf{domain}(f)$ 中没有元素（entry）等于 1。设 $1\le i<m$。我们要证明 s_id_i 不等于 d_{i+1}。若 $d_i=0$，则 $\alpha(i)=*$，且

$$\begin{aligned} s_id_i=d_{i+1} \quad &\Leftrightarrow \quad d_{i+1}=0 \\ &\Leftrightarrow \quad \alpha(i+1)=* \end{aligned}$$

但由 f 已合并的假设，有 $\alpha(i+1)\neq *$，因此 $s_id_i\neq d_{i+1}$。若 $d_i\neq 0$，则 $\alpha(i)\neq *$。若 $\alpha(i+1)=*$，则 $d_{i+1}=0$，故 $s_id_i\neq d_{i+1}$。若 $\alpha(i+1)<\alpha(i)$，则 $d_i\ge d_{i+1}$，又因 $s_i\neq 1$，有 $s_id_i>d_{i+1}$。最后，若 $\alpha(i)<\alpha(i+1)$，则

$$\begin{aligned} s_id_i=s_i\cdot\left(\prod_{j<\alpha(i)}t_j\right) &= \prod_{j\le\alpha(i)}t_j \\ &< \prod_{j<\alpha(i+1)}t_j \\ &= d_{i+1}. \end{aligned}$$

我们得到 L_f 是已合并的。

再设布局 L_f 是已合并的。则 $\mathsf{domain}(f)=\mathsf{shape}(L_f)$ 中没有元素等于 1。设 $1\le i<m$。若 $\alpha(i)=*$，则 $d_i=0$，且由于 L_f 已合并，必有 $d_{i+1}\neq s_id_i=0$，因此 $\alpha(i+1)\neq *$。设 $\alpha(i)\neq *$ 且 $\alpha(i)<\alpha(i+1)$。由于 L_f 已合并，有 $s_id_i\neq d_{i+1}$。但若记

$$s_id_i=\prod_{j\le\alpha(i)}t_j,$$

与

$$d_{i+1}=\prod_{j<\alpha(i+1)}t_j,$$

则这蕴含 $\prod_{\alpha(i)<j<\alpha(i+1)}t_j\neq 1$。特别地，存在某个 $\alpha(i)<j<\alpha(i+1)$ 使得 $t_j>1$。我们得到 f 是已合并的。 □

接下来，我们定义元组态射上的 $\mathsf{coal}(-)$ 运算。

构造 3.1.5.25. 设 f 是元组态射。我们如下定义一个态射 $\mathsf{coal}(f)$:

1. 首先，令 $g=\mathsf{squeeze}(f)$，并把 g 所位于的映射记作 $\beta:\langle m\rangle_*\to\langle n\rangle_*$。

2. 接着，我们在 $\langle m\rangle$ 上定义一个等价关系（equivalence relation）$\sim$：$i\sim i'$ 如果满足以下两条之一：
   (a) 对所有 $i\leq i''\leq i'$ 有 $\beta(i'')=*$，或
   (b) 对所有 $i\leq i''\leq i'$ 有 $\beta(i'')=\beta(i)+(i''-i)$。

   商（quotient）$\langle m\rangle/\sim$ 按「若 $i_1\leq i_2$ 则 $[i_1]\leq[i_2]$」定序，因此我们可以把这个商等同于 $\langle\bar{m}\rangle$，其中 $\bar{m}$ 是 $\langle m\rangle/\sim$ 的大小（size）。

3. 接着，我们在 $\langle n\rangle$ 上定义等价关系 $\sim$：$j\sim j'$ 如果存在 $i\in\langle m\rangle$ 使得对所有 $j\leq j''\leq j'$ 都有
   $$\beta(i+(j''-j))=\beta(i)+(j''-j)$$
   商 $\langle n\rangle/\sim$ 按「若 $j_1\leq j_2$ 则 $[j_1]\leq[j_2]$」定序，因此我们可以把这个商等同于 $\langle\bar{n}\rangle$，其中 $\bar{n}$ 是 $\langle n\rangle/\sim$ 的大小。

4. 接着，我们观察到映射 $\beta:\langle m\rangle_*\to\langle n\rangle_*$ 下降（descends to）为一个映射
   $$\bar{\beta}:\langle\bar{m}\rangle_*\to\langle\bar{n}\rangle_*$$
   它由 $\bar{\beta}([i])=[\beta(i)]$ 给出。

5. $\mathsf{coal}(f)$ 的定义域 $\bar{S}=(\bar{s}_1,\ldots,\bar{s}_{\bar{m}})$ 定义如下：若 $i\in\langle\bar{m}\rangle$ 对应于等价类（equivalence class）$I\in\langle m\rangle/\sim$，则令
   $$\bar{s}_i=\prod_{i'\in I}s_{i'}$$
   $\mathsf{coal}(f)$ 的陪域 $\bar{T}=(\bar{t}_1,\ldots,\bar{t}_{\bar{n}})$ 定义如下：若 $j\in\langle\bar{n}\rangle$ 对应于等价类 $J\in\langle n\rangle/\sim$，则令
   $$\bar{t}_j=\prod_{j'\in J}t_{j'}$$
   然后我们把
   $$\mathsf{coal}(f):\bar{S}\to\bar{T}$$
   定义为位于 $\bar{\beta}$ 之上的元组态射。

例 3.1.5.26. 下面给出一个元组态射 f 及已合并态射 $\mathsf{coal}(f)$ 的例子。

$$\begin{tikzcd}
7 \arrow[r, mapsto] & 7 & & & \\
5 & 2 & & & \\
5 & 3 & & & \\
3 \arrow[ur, mapsto] & 3 & & & 7 \\
3 \arrow[ur, mapsto] & 2 & & 7 \arrow[ur, mapsto] & 2 \\
2 \arrow[r, mapsto] & 2 & & 25 & 9 \\
2 \arrow[r, mapsto] & 2 & \rightsquigarrow & 9 \arrow[ur, mapsto] & 2 \\
2 \arrow[r, mapsto] & 2 & & 8 \arrow[r, mapsto] & 8 \\
f & & & \mathsf{coal}(f) &
\end{tikzcd}$$

例 3.1.5.27. 我们可以把例 3.1.5.8 中的态射 f 合并如下

$$\begin{tikzcd}
 & 256 & & & & & & \\
256 \arrow[ur, mapsto] & 128 & & & 256 & & & \\
128 \arrow[ur, mapsto] & 1 & & & 128 & & & 32768 \\
1 & 32 & & 256 \arrow[uur, mapsto] & 32 & & & 32 \\
8 \arrow[dr, mapsto] & 32 & \rightsquigarrow & 128 \arrow[uur, mapsto] & 32 & \rightsquigarrow & 32768 \arrow[uur, mapsto] & 32 \\
1 & 8 & & 8 \arrow[r, mapsto] & 8 & & 8 \arrow[r, mapsto] & 8 \\
f & & & \mathsf{squeeze}(f) & & & \mathsf{coal}(f) &
\end{tikzcd}$$

命题 3.1.5.28. *若 f 是元组态射，则*

1. $\mathsf{coal}(f)$ *是已合并的，且*
2. $L_{\mathsf{coal}(f)} = \mathsf{coal}(L_f)$.

证明. 首先，我们论证 $\mathsf{coal}(f)$ 是已合并的。这由我们的构造直接可得：施用 squeeze 消除了所有等于 1 的模式，而在我们的构造中取商则归并了所有满足 $\alpha(i+1) = \alpha(i)+1$ 的相邻模式。

接下来，我们证明 $L_{\mathsf{coal}(f)} = \mathsf{coal}(L_f)$。由命题 2.1.4.18 与命题 3.1.5.24，只需证明 $\Phi_{\mathsf{coal}(f)} = \Phi_f$。对 f 施用 $\mathsf{squeeze}(-)$ 当然不会影响所关联的布局函数（layout function），因此我们只需论证：在我们的构造中取商不会改变所关联布局的布局函数。这来自如下事实：我们的商可以分步构造，每一步合并满足 $\alpha(i) = * = \alpha(i+1)$ 或 $\alpha(i+1) = \alpha(i) + 1$ 的相邻模式。这两种情形分别对应于把形如 $s_i, s_{i+1} : 0, 0$ 的相邻模式替换为 $s_i s_{i+1} : 0$，以及把 $s_i, s_{i+1} : d_i, s_i d_i$ 替换为 $s_i s_{i+1} : d_i$。这两种操作都不改变布局的布局函数，于是我们得到 $\Phi_{L_{\mathsf{coal}(f)}} = \Phi_{\mathsf{coal}(L_f)}$，如所欲证。 □

3.1.5.5　拼接

接下来，我们将定义元组态射上的拼接（concatenate）运算。该运算可以作用于满足某个「不交性」（disjointness）条件的元组态射，我们将在下面具体说明。

定义 3.1.5.29. 设 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 与 $\beta : \langle p\rangle_* \to \langle n\rangle_*$ 是 $\textsf{\textbf{Fin}}_*$ 中陪域相同的态射。我们称 α 与 β 像不交（disjoint images），如果

$$\mathsf{Image}(\alpha) \cap \mathsf{Image}(\beta) = \{*\}.$$

构造 3.1.5.30. 若 $\alpha : \langle m\rangle_* \to \langle n\rangle_*$ 与 $\beta : \langle p\rangle_* \to \langle n\rangle_*$ 像不交，则我们有一个良定义（well-defined）的态射

$$\alpha \star \beta : \langle m+p\rangle_* \to \langle n\rangle_*$$

它由

$$(\alpha \star \beta)(i) = \begin{cases} * & i = * \\ \alpha(i) & 1 \leq i \leq m \\ \beta(i-m) & m+1 \leq i \leq m+p. \end{cases}$$

给出。该运算满足结合律，因此对 $\textsf{\textbf{Fin}}_*$ 中任意一组两两像不交的态射 $\alpha_1, \ldots, \alpha_k$，可以考虑 $\alpha_1 \star \cdots \star \alpha_k$。

注 3.1.5.31. 若 α 与 β 是可处理的基点映射且 α 与 β 像不交，则 $\alpha \star \beta$ 是可处理的。

定义 3.1.5.32. 设

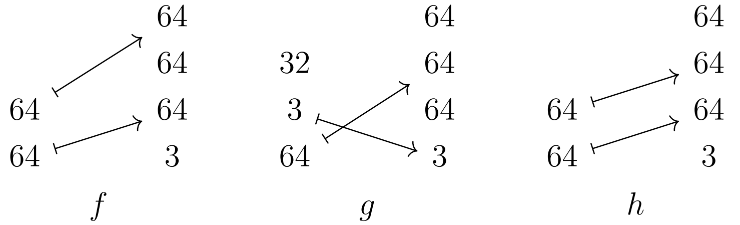
$$f : S \to T$$

与

$$g : U \to T$$

是分别位于 α 与 β 之上的元组态射。我们称 f 与 g 像不交，如果态射 α 与 β 像不交。

例 3.1.5.33. 考虑下面展示的元组态射 f、g 与 h。



则 f 与 g 像不交，而 h 与 g 并非像不交。

构造 3.1.5.34. 设

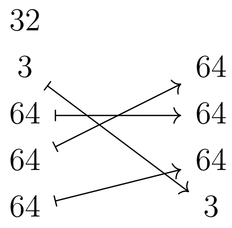
$$f : S \to T, \text{ and } g : U \to T$$

是分别位于 α 与 β 之上的元组态射，且 f 与 g 像不交。我们把 f 与 g 的拼接定义为位于 $\alpha \star \beta$ 之上的态射

$$f \star g : S \star U \to T$$

该运算满足结合律，因此对任意有限组两两像不交的态射 f_i，可以考虑 $f_1 \cdots f_k$。

例 3.1.5.35. 若 f 与 g 是例 3.1.5.33 中 **Tuple** 的态射，则 f 与 g 的拼接是下面展示的态射。



$f \star g$

例 3.1.5.36. 设 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是元组态射，且对任意 $1 \le i \le m$，令

$$f_i : (s_i) \to (t_1, \ldots, t_n)$$

表示 f 的第 i 个元素，如例 3.1.3.8 所述。于是我们可以把

$$f = f_1 \star \cdots \star f_m$$

写成它的诸元素的拼接。

引理 3.1.5.37. 设 $f_1 : S_1 \to T$ 与 $f_2 : S_2 \to T$ 是像不交的元组态射。若 $g : T \to U$ 是任意元组态射，则

$$g \circ (f_1 \star f_2) = (g \circ f_1) \star (g \circ f_2).$$

证明. 设 f_1、f_2 与 g 分别位于 $\alpha_1 : \langle m_1 \rangle_* \to \langle n \rangle$、$\alpha_2 : \langle m_2 \rangle_* \to \langle n \rangle$ 与 $\beta : \langle n \rangle \to \langle p \rangle$ 之上。所讨论的两个映射有相同的定义域与相同的陪域，因此只需证明

$$\beta \circ (\alpha_1 \star \alpha_2) = (\beta \circ \alpha_1) \star (\beta \circ \alpha_2).$$

我们计算

$$
\begin{aligned}
(\beta \circ (\alpha_1 \star \alpha_2))(i) &= \beta((\alpha_1 \star \alpha_2)(i)) \\
&= \begin{cases} \beta(*) & i = * \\ \beta(\alpha_1(i)) & 1 \leq i \leq m_1 \\ \beta(\alpha_2(i - m_1)) & m_1 + 1 \leq i \leq m_1 + m_2 \end{cases} \\
&= \begin{cases} * & i = * \\ (\beta \circ \alpha_1)(i) & 1 \leq i \leq m_1 \\ (\beta \circ \alpha_2)(i - m_1) & m_1 + 1 \leq i \leq m_1 + m_2 \end{cases} \\
&= ((\beta \circ \alpha_1) \star (\beta \circ \alpha_2))(i).
\end{aligned}
$$

□

命题 3.1.5.38. 设 $f_1, \ldots, f_k$ 是 **Tuple** 中陪域相同且两两像不交的态射。则布局 $L_{f_1}, \ldots, L_{f_k}$ 满足

$$L_{f_1 \star \cdots \star f_k} = L_{f_1} \star \cdots \star L_{f_k}.$$

证明. 首先，我们证明 $k = 2$ 的情形。设

$$f = (s_1, \ldots, s_m) \to (t_1, \ldots, t_n), \text{ and } g : (u_1, \ldots, u_p) \to (t_1, \ldots, t_n)$$

像不交，并记

$$L_f = (s_1, \ldots, s_m) : (d_1, \ldots, d_m), \text{ and } L_g = (u_1, \ldots, u_p) : (d'_1, \ldots, d'_p).$$

则布局 $L_{f \star g}$ 由

$$L_{f \star g} = (s_1, \ldots, s_m, u_1, \ldots, u_p) : (e_1, \ldots, e_{m+m'})$$

给出，其中

$$
\begin{aligned}
e_i &= \prod_{j < (\alpha \star \beta)(i)} t_j \\
&= \begin{cases} \displaystyle\prod_{j < \alpha(i)} t_j & 1 \leq i \leq m \\ \displaystyle\prod_{j < \beta(i-m)} t_j & m + 1 \leq i \leq m + m'. \end{cases} \\
&= \begin{cases} d_i & 1 \leq i \leq m \\ d'_{i-m} & m + 1 \leq i \leq m + m'. \end{cases}
\end{aligned}
$$

这就完成了 $k = 2$ 情形的证明。一般情形可由元组态射拼接的结合律与扁平布局拼接的结合律得到。 □

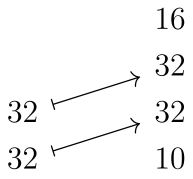
3.1.5.6　补

我们先定义元组态射的补的概念。

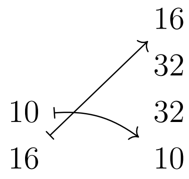
定义 3.1.5.39. 设 $f : S \to T$ 与 $g : U \to T$ 是元组态射。我们称 g 是 f 的一个补，如果

1. f 与 g 像不交，且
2. 拼接
$$f \star g : S \star U \xrightarrow{\cong} T$$
是一个同构（isomorphism）。

例 3.1.5.40. 若 f 与 g 是下面展示的态射



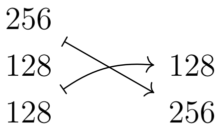
f



g

则 g 是 f 的一个补。

例 3.1.5.41. 若 f 是下面展示的态射



f

则 f 不容许（admit）有补。

接下来，我们证明互补的元组态射产生互补的扁平布局。

命题 3.1.5.42. 若 $f : S \to T$ 是元组态射且 g 是 f 的补，则 L_g 是 L_f 的一个 $\mathsf{size}(T)$-补。

证明. 记 $S = \mathsf{domain}(f)$、$U = \mathsf{domain}(g)$、$T = \mathsf{codomain}(f) = \mathsf{codomain}(g)$。首先，我们注意到

$$\begin{aligned}\mathsf{size}(L_f) \cdot \mathsf{size}(L_g) &= \mathsf{size}(L_f \star L_g) \\ &= \mathsf{size}(L_{f \star g}) \\ &= \mathsf{size}(S \star U) \\ &= \mathsf{size}(T).\end{aligned}$$

其次，我们注意到 $f \star g$ 是同构，因此

$$|f \star g| = \Phi_{L_{f\star g}}^{\mathsf{size}(T)}$$

也是同构，这里我们使用了引理 3.1.4.5 中对 $\Phi_{L_{f\star g}}^{\mathsf{size}(T)}$ 的等同。 □

命题 3.1.5.43. *若 f 是单射（injective）元组态射，则*

$$\mathsf{coal}^\flat(L_{f^c}) = \mathsf{comp}^\flat(L_f, \mathsf{size}(T)).$$

证明. 由命题 3.1.5.42，我们知道 L_{f^c} 是 L_f 的一个 $\mathsf{size}(T)$-补。由于 f^c 已排序，L_{f^c} 亦然，于是由命题 2.1.6.33 可得

$$\mathsf{coal}^\flat(L_{f^c}) = \mathsf{comp}^\flat(L_f, \mathsf{size}(T)),$$

因为这两个布局都是 L_f 的大小相同、扁平、已排序、已合并的补。 □

命题 3.1.5.44. *若 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是标准形式（standard form）的单射元组态射，则*

$$L_{f^c} = \mathsf{comp}^\flat(L_f).$$

证明. 记

$$L_f = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

为 f 所编码的布局。由命题 3.1.5.42，我们知道 L_{f^c} 是 L_f 的一个 $\mathsf{size}(T)$-补，其中

$$\begin{aligned} \mathsf{size}(T) &= t_1 \cdots t_n = (t_1 \cdots t_{n-1}) t_n \\ &= d_m s_m. \end{aligned}$$

按构造，f^c 已排序，因此 L_{f^c} 亦然。此外，由于 f 具有标准形式，可得 f^c 是已合并的。由命题 2.1.6.23，我们推出

$$L_{f^c} = \mathsf{comp}^\flat(L_f).$$

□

定义 3.1.5.45. 设 f 是位于 $\alpha : \langle m \rangle_* \to \langle n \rangle_*$ 之上的元组态射。我们称 f 是可求补（complementable）的，如果 α 是单射。

构造 3.1.5.46. 设 $f : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n)$ 是可求补的元组态射。令 $j_1 < \cdots < j_{n-m}$ 表示 $\langle n \rangle$ 中不在 α 的像（image）中的那些指标。我们把 f 的补定义为元组态射

$$f^c : (t_{j_1}, \ldots, t_{j_k}) \to (t_1, \ldots, t_n)$$

它位于映射 $\mathsf{complement}(\alpha) : \langle n - m \rangle_* \to \langle n \rangle_*$ 之上，该映射由 $k \mapsto j_k$ 给出。由构造可以观察到，f^c 是定义 3.1.5.39 意义下 f 的补

例 3.1.5.47. 下面给出一个态射 f 及其补 f^c 的例子。

$$\begin{tikzcd}
& 512 & & & 512 \\
& 512 & & & 512 \\
512 \arrow[ur, mapsto] & 256 & & 512 \arrow[uur, mapsto] & 256 \\
256 \arrow[ur, mapsto] & 10 & & 10 \arrow[r, mapsto] & 10 \\
& f & & f^c &
\end{tikzcd}$$

命题 3.1.5.48. 若 f 是元组态射且 g 是 f 的补，则

$$\mathsf{sort}(g) = f^c.$$

证明. 设 f 位于 $\alpha : \langle m \rangle_* \to \langle n \rangle_*$ 之上，$\mathsf{sort}(g)$ 位于 $\beta : \langle n - m \rangle_* \to \langle n \rangle_*$ 之上，f^c 位于 $\alpha^c : \langle n - m \rangle_* \to \langle n \rangle_*$ 之上。则 β 与 α^c 是有相同像的递增映射（increasing map），即

$$\mathsf{Image}(\beta) = \langle n \rangle \setminus \mathsf{Image}(\alpha) = \mathsf{Image}(\alpha^c),$$

因此 $\beta = \alpha^c$，从而 $\mathsf{sort}(g) = f^c$。 □

命题 3.1.5.49. 设 f 是元组态射。则 f 容许有补当且仅当 f 是定义 *3.1.5.45* 意义下可求补的。

证明. 若 f 位于一个非单射的映射 α 之上，则对任意使 f 与 f^* 像不交的态射 f^*，态射 $f \star f^*$ 位于一个非单射的映射之上，因此 $f \star f^*$ 不是同构。反之，若 f 位于一个单射映射之上，则构造 3.1.5.46 中的态射 f^c 是 f 的补。 □

命题 3.1.5.50. 若 f 是可求补的元组态射，则

$$\mathsf{sort}(f) = (f^c)^c.$$

证明. 这两个映射都是递增的、单射的，且有相同的像，因此它们相等。 □

3.1.5.7 扁平切分

本节中，我们在元组态射上定义一种切分（division）运算。

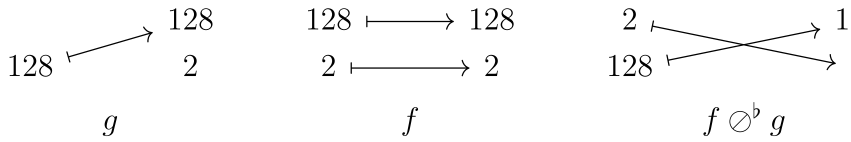
定义 3.1.5.51. 若 f 与 g 是元组态射，我们称 g 整除 f，如果 g 与 f 可复合。换言之，

$$\mathsf{codomain}(g) = \mathsf{domain}(f).$$

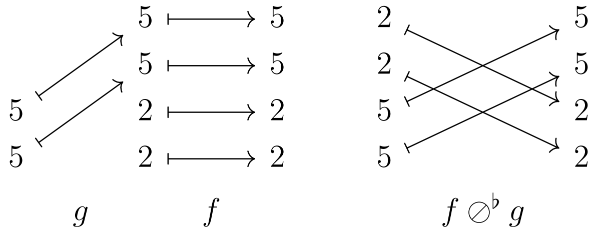
定义 3.1.5.52. 设 $g : S \to T$ 与 $f : T \to U$ 是元组态射。f 除以 g 的扁平切分是元组态射

$$f \oslash^\flat g = f \circ (g \star g^c).$$

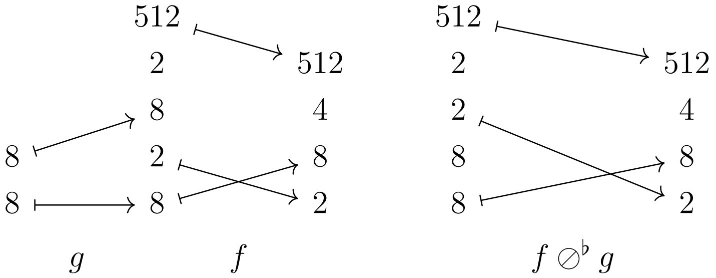
例 3.1.5.53. 下面给出元组态射 f 与 g 及其扁平商（flat quotient）$f \oslash^\flat g$ 的一个例子。


128
128
2
g
128
128
2
2
f
2
128
128
2
f ⊘♭ g

例 3.1.5.54. 下面给出元组态射 f 与 g 及其扁平商 $f \oslash^\flat g$ 的一个例子。


5
5
5
5
2
2
5
5
2
2
g
f
2
2
5
5
5
5
2
2
f ⊘♭ g

例 3.1.5.55. 下面给出元组态射 f 与 g 及其扁平商 $f \oslash^\flat g$ 的一个例子。


8
8
512
2
8
2
8
512
4
8
2
g
f
512
2
2
8
8
512
4
8
2
f ⊘♭ g

命题 3.1.5.56. *若 f 与 g 是非退化（non-degenerate）的可复合元组态射，则*

$$\mathsf{coal}^\flat(L_{f\oslash^\flat g}) = \mathsf{coal}^\flat(L_f \oslash^\flat L_g)$$

证明. 由命题 3.2.6.20，我们有

$$\mathsf{coal}^\flat(L_{g^c}) = \mathsf{comp}^\flat(L_g, \mathsf{size}(L_f)),$$

我们计算

$$\begin{aligned}
\mathsf{coal}^\flat(L_f \oslash^\flat L_g) &= \mathsf{coal}^\flat(L_f \circ (L_g \star \mathsf{comp}(L_g, \mathsf{size}(L_f)))) \\
&= \mathsf{coal}(L_f \circ (L_g \star L_{g^c})) \\
&= \mathsf{coal}(L_f \circ L_{g\star g^c}) \\
&= \mathsf{coal}(L_{f\circ(g\star g^c)}) \\
&= \mathsf{coal}(L_{f\oslash^\flat g}).
\end{aligned}$$

□

3.1.5.8 扁平乘积

本节中，我们在元组态射上定义一种乘积运算。

定义 3.1.5.57. 设 f 与 g 是元组态射。我们称 f 与 g 是乘积可许 (product admissible) 的，如果 $\mathsf{codomain}(g) = \mathsf{domain}(f^c)$。若 f 与 g 乘积可许，则我们把 f 与 g 的扁平乘积定义为

$$f \otimes^\flat g = f \star (f^c \circ g).$$

例 3.1.5.58. 若 f 与 g 是下面展示的元组态射

$$\begin{tikzcd}
 & & & 16 \\
 & & & 16 \\
16 \arrow[r, mapsto] & 16 & 8 \arrow[r, mapsto] & 8 \\
16 \arrow[r, mapsto] & 16 & 8 \arrow[r, mapsto] & 8 \\
g & & f &
\end{tikzcd}$$

则 f 与 g 乘积可许，且 $f \otimes^\flat g$ 是下面展示的元组态射。

$$\begin{tikzcd}
16 \arrow[r, mapsto] & 16 \\
16 \arrow[r, mapsto] & 16 \\
8 \arrow[r, mapsto] & 8 \\
8 \arrow[r, mapsto] & 8 \\
f \otimes^\flat g &
\end{tikzcd}$$

例 3.1.5.59. 若 f 与 g 是下面展示的元组态射

$$\begin{tikzcd}
 & & & 128 \\
 & & & 128 \\
 & 32 & 128 \arrow[uur, mapsto] & 32 \\
32 \arrow[ur, mapsto] & 32 & 128 \arrow[uur, mapsto] & 32 \\
g & & f &
\end{tikzcd}$$

则 f 与 g 乘积可许，且 $f \otimes^\flat g$ 是下面展示的元组态射。

$$\begin{tikzcd}
 & 128 \\
32 \arrow[dr, mapsto] & 128 \\
128 \arrow[uur, mapsto] & 32 \\
128 \arrow[uur, mapsto] & 32 \\
f \otimes^\flat g &
\end{tikzcd}$$

引理 3.1.5.60. *若 f 与 g 乘积可许且 g 是单射，则 $f \otimes^\flat g$ 是单射，且*

$$(f \otimes^\flat g)^c = f^c \circ g^c.$$

证明．元组态射 $(f \otimes^\flat g)^c$ 与 $f^c \circ g^c$ 都是单射、递增，且有相同的陪域，因此只需证明它们有相同的像。$(f \otimes^\flat g)^c = (f \star (f^c \circ g))^c$ 的像由那些既不在 f 的像中、也不在 $f^c \circ g$ 的像中的元素组成。f^c 的像由那些不在 f 的像中的元素组成，于是复合 $f^c \circ g^c$ 的像由那些既不在 f 的像中、也不在 $f^c \circ g$ 的像中的元素组成。 □

命题 3.1.5.61. *设 f 与 g 乘积可许，且 g 与 h 乘积可许。则*

1. *$f \otimes^\flat g$ 与 h 乘积可许，*
2. *f 与 $g \otimes^\flat h$ 乘积可许，且*
3. *$(f \otimes^\flat g) \otimes^\flat h = f \otimes^\flat (g \otimes^\flat h)$。*

证明．利用引理 3.1.5.37 与引理 3.1.5.60，我们计算

$$\begin{aligned} f \otimes^\flat (g \otimes^\flat h) &= f \star (f^c \circ (g \otimes^\flat h)) \\ &= f \star (f^c \circ (g \star (g^c \circ h))) \\ &= f \star ((f^c \circ g) \star (f^c \circ (g^c \circ h))) \\ &= f \star ((f^c \circ g) \star ((f^c \circ g^c) \circ h)) \\ &= f \star (f^c \circ g) \star ((f \otimes^\flat g)^c \circ h) \\ &= (f \otimes^\flat g) \star ((f \otimes^\flat g)^c \circ h) \\ &= (f \otimes^\flat g) \otimes^\flat h. \end{aligned}$$

□

命题 3.1.5.62. *设 f 与 g 是非退化的元组态射，且 f 与 g 乘积可许。则*

$$L_{f \otimes^\flat g} = L_f \otimes^\flat L_g.$$

证明．设 $f : S \to T$ 与 $g : U \to V$ 乘积可许，并令

$$L_f^* = \mathsf{comp}^\flat(L_f, \mathsf{size}(L_f) \cdot \mathsf{cosize}(L_g)).$$

由于 f 是单射且 g 的陪域是 f^c 的定义域，可得

$$\mathsf{size}(L_f) \cdot \mathsf{cosize}(L_g) \leq \mathsf{size}(S) \cdot \mathsf{size}(V) = \mathsf{size}(T).$$

利用这一事实，以及事实

$$\Phi_{\mathsf{comp}(L_f, \mathsf{size}(T))} = \Phi_{L_{f^c}},$$

我们有

$$\begin{aligned} L_f^* \circ L_g &= \mathsf{comp}(L_f, \mathsf{size}(T)) \circ L_g \\ &= L_{f^c} \circ L_g. \end{aligned}$$

利用这一事实，我们计算

$$\begin{aligned} L_f \otimes^\flat L_g &= L_f \star (L_f^* \circ L_g) \\ &= L_f \star (L_{f^c} \circ L_g) \\ &= L_f \star L_{f^c \circ g} \\ &= L_{f \star (f^c \circ g)} \\ &= L_{f \otimes^\flat g} \end{aligned}$$

□

3.2 范畴 Nest

在上一节中，我们引入了一个范畴（category）**Tuple**，其态射（morphism）编码扁平（flat）可处理布局（tractable layout）。本节中，我们引入一个范畴 **Nest**，其态射编码允许任意嵌套（nesting）的可处理布局。

3.2.1 基本定义

回顾一下，对于嵌套元组（nested tuple）S，我们把 S 的扁平化（flattening）记作 $S^\flat$。例如，若 $S=(64,(8,8))$，则 $S^\flat=(64,8,8)$。

定义 3.2.1.1. 令 **Nest** 表示这样的范畴：其对象（object）是正整数的嵌套元组，其中 **Nest** 里的态射

$$f : S \to T$$

由一个元组态射（tuple morphism）

$$f^\flat : S^\flat \to T^\flat.$$

指定。换言之，

$$\mathsf{Hom}_{\mathbf{Nest}}(S,T) = \mathsf{Hom}_{\mathbf{Tuple}}(S^\flat,T^\flat).$$

显式地，**Nest** 中的态射 $f : S \to T$ 由一个可处理的基点映射（pointed map）$\alpha : \langle \mathsf{len}(S)\rangle_* \to \langle \mathsf{len}(T)\rangle_*$ 指定，它满足以下性质：

- 若 $1 \le i \le \mathsf{len}(S)$ 且 $\alpha(i) \ne *$，则 $\mathsf{entry}_i(S) = \mathsf{entry}_{\alpha(i)}(T)$。

我们称这样的态射 f 位于 α 之上 (lies over)，并把 f 称为*嵌套元组态射* (nested tuple morphism)。

记号 3.2.1.2. 若 $f : S \to T$ 是位于 α 之上的嵌套元组态射，我们把 f 画成

$$S \xrightarrow[\alpha]{f} T$$

例 3.2.1.3. 以下是一些嵌套元组态射的例子。

$$(64,(8,8)) \xrightarrow[(1,2,3)]{f} (64,8,8)$$

$$((2,2),2) \xrightarrow[(*,5,2)]{g} (10,2,2,(3,2,3))$$

$$64 \xrightarrow[(2)]{h} ((64,64),512).$$

观察 3.2.1.4. 若 X 是集合，我们把 X 上的余离散（indiscrete）范畴记作 X^{ind}。这是这样的范畴：其对象是 X 的元素，且任意两个对象之间存在唯一的（同构）态射。于是按 **Nest** 的定义，我们有一个拉回方块（pullback square）

$$\begin{tikzcd}
\textbf{Nest} \arrow[r, "\mathsf{prof}(-)"] \arrow[d, "(-)^\flat"'] \arrow[dr, phantom, very near start, "\lrcorner"] & \mathsf{Profile}^{\mathsf{ind}} \arrow[d, "\mathsf{len}(-)"] \\
\textbf{Tuple} \arrow[r, "\mathsf{len}(-)"'] & \mathbb{N}^{\mathsf{ind}}
\end{tikzcd}$$

我们可以把它视为拉回方块 2.2.2.4 的范畴化（categorification）。

例 3.2.1.5. 设 S 是长度（length）为 m 的嵌套元组。若 $1 \leq i \leq m$，则存在一个嵌套元组态射

$$\mathsf{entry}_i(S) \to S$$

它位于由 $1 \mapsto i$ 给出的映射 $\langle 1\rangle_* \to \langle m\rangle_*$ 之上。例如，若 $S = (64, (8, 8))$ 且 $i = 1$，则我们有嵌套元组态射

$$64 \xrightarrow[(1)]{} (64, (8, 8)).$$

例 3.2.1.6. 设 S 是秩（rank）为 r 的嵌套元组。若 $1 \leq i \leq r$，则存在一个典范的嵌套元组态射

$$\mathsf{mode}_i(S) \to S$$

它位于由 $j \mapsto j + \mathsf{len}_{<i}(S)$ 给出的映射 $\langle \mathsf{len}_i(S)\rangle_* \to \langle \mathsf{len}(S)\rangle_*$ 之上。例如，若 $S = (64, (8, 8))$，则我们有嵌套元组态射

$$(8, 8) \xrightarrow[(2,3)]{} (64, (8, 8)).$$

观察 3.2.1.7. 有一些函子（functor）把范畴 **Nest** 与 **Tuple** 联系起来。首先，有一个包含（inclusion）函子

$$\textbf{Tuple} \xrightarrow{\subset} \textbf{Nest}$$

它把元组态射 $f : S \to T$ 视为嵌套元组态射。其次，有一个*扁平化函子*

$$\textbf{Nest} \xrightarrow{(-)^\flat} \textbf{Tuple}$$

它把嵌套元组态射 $f : S \to T$ 送到其底层的元组态射 $f^\flat : S^\flat \to T^\flat$。复合（composite）

$$\textbf{Tuple} \xrightarrow{\subset} \textbf{Nest} \xrightarrow{(-)^\flat} \textbf{Tuple}$$

是 **Tuple** 上的恒等函子，因此 **Tuple** 是 **Nest** 的收缩子范畴（retractive subcategory）。此外，这些函子构成范畴的一个伴随等价（adjoint equivalence）。

注 3.2.1.8. 人们可能希望考虑某个范畴 **C**，其态射编码可处理布局，但它不等价于 **Tuple**。作者们已经考虑过若干这样的例子，但把对它们的研究留待未来工作。

3.2.2　从嵌套元组态射到布局

范畴 **Nest** 的关键特征在于：若 $f : S \to T$ 是嵌套元组态射，则 f 编码一个布局 L_f。这个布局是通过给扁平布局 $L_{f^\flat}$ 配备 S 的嵌套轮廓（profile）而得到的。更精确地说，我们有如下构造。

构造 3.2.2.1. 设

$$f : S \to T$$

是嵌套元组态射，并设 $P = \mathsf{prof}(S)$。我们定义 L_f 为布局

$$L_f = (L_{f^\flat})_P$$

其中 $(-)_P$ 是定义 2.3.1.19 中的 P-代入（substitution）运算。我们把 L_f 称为由 f 编码的布局。

构造 3.2.2.2. 设

$$(s_1, \dots, s_m)_P \xrightarrow[\alpha]{f} (t_1, \dots, t_n)_Q$$

是嵌套元组态射。我们定义 L_f 为这样的布局：其形状（shape）

$$\mathsf{shape}(L_f) = (s_1, \dots, s_m)_P$$

是 f 的定义域（domain），其步长（stride）

$$\mathsf{stride}(L_f) = (d_1, \dots, d_m)_P$$

的各元素由公式

$$d_i = \begin{cases} 0 & \alpha(i) = * \\ \prod_{j<\alpha(i)} t_j & \alpha(i) \neq *. \end{cases}$$

定义。我们把 L_f 称为由 f 编码的布局。

例 3.2.2.3. 由

$$((8,8),(4,4)) \xrightarrow[(1,4,3,2)]{f} (8,4,4,8)$$

编码的布局是

$$L_f = ((8,8),(4,4)) : ((1,128),(32,8)).$$

例 3.2.2.4. 由

$$(128,(4,4,2)) \xrightarrow[(3,1,2,*)]{g} ((4,4),128)$$

编码的布局是

$$L_g = (128,(4,4,2)) : (16,(1,4,0)).$$

观察 3.2.2.5. 扁平化函子

$$\textsf{Nest} \xrightarrow{(-)^\flat} \textsf{Tuple}$$

与布局的扁平化相容：若 f 是嵌套元组态射，则

$$(L_f)^\flat = L_{f^\flat}.$$

若 L 是可处理布局，则我们可以如下构造一个编码 L 的嵌套元组态射。

构造 3.2.2.6. 设 L 是可处理布局。我们定义 L 的标准表示（standard representation）为嵌套元组态射

$$f_L : S \to T$$

其中 $(f_L)^\flat = f_{L^\flat}$ 是 $L^\flat$ 的标准表示，$S = \mathsf{shape}(L)$ 是 L 的形状，而 T 是 $f_{L^\flat}$ 的陪域（codomain）。

例 3.2.2.7. 若

$$L = (32, (2, 2)) : (192, (24, 3))$$

则 L 的标准表示是

$$(32, (2, 2)) \xrightarrow[(6,4,2)]{f_L} (3, 2, 4, 2, 4, 32).$$

引理 3.2.2.8. *若 L 是可处理布局，且 $f = f_L$ 是 L 的标准表示，则*

$$L_f = L.$$

证明. 我们有

$$(L_f)^\flat = L_{f^\flat} = L^\flat$$

以及

$$\mathsf{shape}(L_f) = \mathsf{shape}(L).$$

□

命题 3.2.2.9. *设 L 是布局。则存在编码 L 的嵌套元组态射 f，当且仅当 L 是可处理的。*

证明. 首先设对某个嵌套元组态射 f 有 $L = L_f$。则 $(L_f)^\flat = L_{f^\flat}$，且由命题 3.1.2.10，我们知道 $L^\flat$ 是可处理的，从而 L 也是。反之，若 L 可处理，则可取 $f = f_L$ 为 L 的标准表示，于是由引理 3.2.2.8，有 $L_f = L$。□

为了在可处理布局与某些嵌套元组态射之间建立一一对应，我们为嵌套元组态射引入标准形式（standard form）的概念。

定义 3.2.2.10. 设 $f : S \to T$ 是嵌套元组态射。若下列条件成立，则称 f 具有标准形式：

1. $f^\flat$ 具有标准形式（如定义 3.1.2.12 所述），且
2. T 是扁平的。

例 3.2.2.11. 嵌套元组态射

$$((2,2),(3,3)) \xrightarrow[(4,6,2,3]{f} (10,3,3,2,10,2)$$

具有标准形式。

例 3.2.2.12. 嵌套元组态射

$$((2,2),(3,3)) \xrightarrow[(4,6,2,3]{f} ((10,3,3),(2,10,2))$$

不具有标准形式，因为 g 的陪域不是扁平的。

正如扁平情形那样，为了在具有标准形式的嵌套元组态射与可处理布局之间得到一一对应，我们需要排除非退化（non-degenerate）的嵌套元组态射与非退化布局。为此，我们给出以下定义。

定义 3.2.2.13. 设

$$S \xrightarrow[\alpha]{f} T$$

是嵌套元组态射，并设

$$L = S : D$$

是布局。

1. 我们称 f 是非退化的，如果
$$\mathsf{entry}_i(S) = 1 \quad \Rightarrow \quad \alpha(i) = *.$$
2. 我们称 L 是非退化的，如果
$$\mathsf{entry}_i(S) = 1 \quad \Rightarrow \quad \mathsf{entry}_i(D) = 0.$$

注 3.2.2.14. 若 f 是嵌套元组态射，则 f 非退化当且仅当 $f^\flat$ 非退化。若 L 是布局，则 L 非退化当且仅当 $L^\flat$ 非退化。

命题 3.2.2.15. 构造 *3.2.2.2* 与 *3.2.2.6* 给出的映射

$$f \longmapsto L_f$$

$$\left\{\begin{array}{c}\text{非退化}\\ \text{嵌套元组态射}\\ \text{(具有标准形式)}\end{array}\right\} \longleftrightarrow \left\{\begin{array}{c}\text{非退化}\\ \text{可处理布局}\end{array}\right\}$$

$$f_L \longmapsfrom L$$

在具有标准形式的嵌套元组态射与可处理布局之间建立了一一对应。

证明. 我们已在命题 3.2.2.9 中证明：若 L 是可处理布局且 $f = f_L$ 是 L 的标准形式，则 $L_f = L$。其次设 f 具有标准形式，并令 $L = L_f$ 为 f 编码的布局。我们要证 f 等于 L 的标准表示 f_L。由命题 3.1.2.21，我们知道 $f^\flat$ 等于 $L^\flat$ 的标准表示 $f_{L^\flat}$；又由于

$$\mathsf{domain}(f) = \mathsf{shape}(L) = \mathsf{domain}(f_L),$$

以及

$$\mathsf{codomain}(f) = \mathsf{codomain}(f^\flat) = \mathsf{codomain}(f_{L^\flat}) = \mathsf{codomain}(f_L),$$

可推得 $f = f_L$。 □

3.2.3　例子

本节列出嵌套元组态射的一些重要族。

例 3.2.3.1 (重新加括号)**.** 设 S_1 与 S_2 是具有相同扁平化

$$S_1^\flat = S_2^\flat.$$

的嵌套元组，则存在一个重新加括号同构（reparenthesization isomorphism）

$$\mathsf{id}_{S_1}^{S_2} : S_1 \xrightarrow{\cong} S_2$$

它位于恒等映射（identity map）之上。这些态射是传递的，即

$$\mathsf{id}_{S_2}^{S_3} \circ \mathsf{id}_{S_1}^{S_2} = \mathsf{id}_{S_1}^{S_3},$$

并且与恒等态射相容，即

$$\mathsf{id}_S^S = \mathsf{id}_S.$$

若 $f = \mathsf{id}_{S_1}^{S_2}$ 是重新加括号同构，则 L_f 是形状为 S_1 的列主序（column-major）布局。

例 3.2.3.2 (扁平化)**.** 作为上一例的特殊情形，若 S 是任意嵌套元组，则我们有一个扁平化同构（flattening isomorphism）

$$\mathsf{id}_S^{S^\flat} : S \xrightarrow{\cong} S^\flat$$

以及一个去扁平化同构（unflattening isomorphism）

$$\mathsf{id}_{S^\flat}^{S} : S^\flat \xrightarrow{\cong} S$$

观察 3.2.3.3. 若 $f : S \to T$ 是一个嵌套元组，则 f 等于复合

$$\begin{tikzcd} S \arrow[r, "\mathsf{id}_S^{S^\flat}"] \arrow[rrr, bend right, "f"'] & S^\flat \arrow[r, "f^\flat"] & T^\flat \arrow[r, "\mathsf{id}_{T^\flat}^{T}"] & T. \end{tikzcd}$$

换言之，我们有一个典范因子分解（factorization）

$$f = \mathsf{id}_{T^\flat}^{T} \circ f^\flat \circ \mathsf{id}_S^{S^\flat}.$$

例 3.2.3.4 (元素)**.** 设

$$S \xrightarrow[\alpha]{f} T$$

是嵌套元组态射。设 $1 \leq i \leq \mathsf{len}(S)$，并记 $j = \alpha(i)$。我们把嵌套元组态射

$$\mathsf{entry}_i(S) \xrightarrow[(j)]{\mathsf{entry}_i(f)} T$$

称为 f 的第 i 个元素（entry）。由 $\mathsf{entry}_i(f)$ 编码的布局是

$$L_{\mathsf{entry}_i(f)} = \mathsf{entry}_i(L_f).$$

例 3.2.3.5 (元素包含)**.** 作为上一例的特殊情形，若 S 是嵌套元组且 $1 \leq i \leq \mathsf{len}(S)$，可取 $f = \mathsf{id}_S$，此时

$$\mathsf{entry}_i(\mathsf{id}_S) : \mathsf{entry}_i(S) \longrightarrow S$$

是 S 的第 i 个元素的包含映射（inclusion）。

例 3.2.3.6 (模式)**.** 设

$$S \xrightarrow[\alpha]{f} T$$

是嵌套元组态射。设 $1 \leq i \leq \mathsf{rank}(S)$，并记

$$\begin{aligned} N &= \mathsf{len}_{<i}(S) \\ \ell &= \mathsf{len}_i(S). \end{aligned}$$

我们把嵌套元组态射

$$\mathsf{mode}_i(S) \xrightarrow[(N+1,\ldots,N+\ell)]{\mathsf{mode}_i(f)} T$$

称为 S 的第 i 个模式（mode）。由 $\mathsf{mode}_i(L_f)$ 编码的布局是

$$L_{\mathsf{mode}_i(f)} = \mathsf{mode}_i(L_f).$$

例 3.2.3.7 (模式包含). 作为上一例的特殊情形，我们可取 $f = \mathsf{id}_S$，此时

$$\mathsf{mode}_i(\mathsf{id}_S) : \mathsf{mode}_i(S) \to S$$

是 S 的第 i 个模式的包含映射。我们有时把这个映射记作

$$\mathsf{incl}_i(S) = \mathsf{mode}_i(\mathsf{id}_S).$$

3.2.4　嵌套元组态射的实现

在扁平情形中，我们构造了一个实现函子（realization functor）

$$\textbf{Tuple} \xrightarrow{|\cdot|} \textbf{FinSet}$$

它把元组态射 f 送到 L_f 的布局函数（layout function）。通过用扁平化函子 $\textbf{Nest} \to \textbf{Tuple}$ 预复合（precompose），我们可以把它扩张为实现函子

$$\textbf{Nest} \xrightarrow{|\cdot|} \textbf{FinSet}$$

定义 3.2.4.1. 我们定义实现函子

$$\textbf{Nest} \xrightarrow{|\cdot|} \textbf{FinSet}$$

为复合

$$\textbf{Nest} \xrightarrow{(-)^\flat} \textbf{Tuple} \xrightarrow{|\cdot|} \textbf{FinSet}$$

引理 3.2.4.2. *若 $f : S \to T$ 是嵌套元组态射，则 f 的实现 $|f|$ 是 L_f 的布局函数：*

$$|f| = \Phi_{L_f}^{\mathsf{size}(T)}.$$

证明. 这由 3.1.4.5 立得，因为

$$|f| = |f^\flat| = \Phi_{L_{f^\flat}}^{\mathsf{size}(T)} = \Phi_{L_f}^{\mathsf{size}(T)}$$

□

3.2.5　加细

本节中，我们从范畴（category）论的视角重新审视嵌套元组（nested tuple）的加细（refinement）。回顾 2.2.4 节：若 S' 可由 S 把 S 的每个元素（entry）替换为某个大小（size）相同的嵌套元组而得到，则嵌套元组 S' *加细* S，记为

$$S' \longrightarrow\!\!\!\!\!\rightarrow S$$

例如，

$$(2,(2,2)) \twoheadrightarrow 8,$$

以及

$$((2,2),(3,3),(5,5)) \twoheadrightarrow (4,9,25).$$

若 $\mathsf{len}(S)=m$ 且 $\mathsf{prof}(S)=P$，则我们可以把

$$S' = (S'_1,\dots,S'_m)_P$$

写成诸相对模式（relative mode）

$$S'_i = \mathsf{mode}_i(S',S).$$

的 P-代入（substitution）。我们把普通拼接（concatenation）

$$(S'_1,\dots,S'_m) = \mathsf{flat}(S',S)$$

称为 S' 相对于 S 的扁平化（flattening）。

令 $\textbf{Ref}$ 表示正整数的嵌套元组依加细关系构成的偏序集范畴（poset category），于是 $\textbf{Ref}$ 中的态射（morphism）就是加细 $S' \twoheadrightarrow S$。若 S 是嵌套元组，令

$$\textbf{Ref}(S) = \{S' \mid S' \text{ 加细} S\}$$

表示由加细 S 的嵌套元组构成的偏序集（poset）。等价地，$\textbf{Ref}(S)$ 就是切片范畴（slice category）$\textbf{Ref}_{/S}$。

构造 3.2.5.1. [相对模式包含映射] 设 $S' \twoheadrightarrow S$ 是一个加细，并把

$$S'_i = \mathsf{mode}_i(S',S)$$

记为 S' 相对于 S 的各模式（mode）。S' 与 $(S'_1,\dots,S'_m)$ 有相同的扁平化，于是我们有一个重新加括号同构（reparenthesization isomorphism）

$$\mathsf{id}^{S'}_{(S'_1,\dots,S'_m)} : (S'_1,\dots,S'_m) \xrightarrow{\cong} S'$$

并且我们定义

$$\mathsf{incl}_i(S',S) : S'_i \to S'$$

为 $(S'_1,\ldots,S'_m)$ 的第 i 个模式包含映射与重新加括号同构 $(S'_1,\ldots,S'_m)\cong S'$ 复合（composite）而成的态射

$$S'_i \xrightarrow{\mathsf{incl}_i((S'_1,\ldots,S'_m))} (S'_1,\ldots,S'_m) \xrightarrow{\mathsf{id}^{S'}_{(S'_1,\ldots,S'_m)}} S'$$

例 3.2.5.2. 若 $S=(4,(9,25))$ 且 $S'=((2,2),((3,3),25))$，则 S' 加细 S，且 $\mathsf{incl}_2(S',S)$ 是嵌套元组态射（nested tuple morphism）

$$(3,3)\xrightarrow[(3,4)]{\mathsf{incl}_2(S',S)}((2,2),((3,3),25)).$$

构造 3.2.5.3. [相对模式] 设 $f':S'\to T'$ 是嵌套元组态射，且设 S' 加细 S。我们定义 f' 相对于 S 的第 i 个模式，记为

$$\mathsf{mode}_i(f',S)=f'\circ\mathsf{incl}_i(S',S):S'_i\to T'$$

它是复合

$$S'_i\xrightarrow{\mathsf{incl}_i(S',S)}S'\xrightarrow{f'}T'$$

特别地，我们有

$$\mathsf{mode}_i(\mathsf{id}_{S'},S)=\mathsf{incl}_i(S',S).$$

例 3.2.5.4. 设 $S=(4,(9,25))$ 且 $S'=((2,2),((3,3),25))$，于是 S' 加细 S。若 f' 是嵌套元组态射

$$((2,2),((3,3),25))\xrightarrow[(1,3,2,*,4)]{f'}(2,3,2,25).$$

则 $\mathsf{mode}_2(f',S)$ 是嵌套元组态射

$$(3,3)\xrightarrow[(2,*)]{\mathsf{mode}_2(f',S)}(2,3,2,25).$$

构造 3.2.5.5 (拉回)**.** 设 $f:S\to T$ 是位于 α 之上（lies over）的嵌套元组态射，并设 $T'\twoheadrightarrow T$ 是一个加细。令

$$T'_j=\mathsf{mode}_j(T',T)$$

表示 T' 相对于 T 的第 j 个模式，且对任意 $1\le i\le\mathsf{len}(S)$，规定

$$S'_i=\begin{cases}\mathsf{entry}_i(S) & \alpha(i)=*\\ T'_j & \alpha(i)=j.\end{cases}$$

我们定义 T' 沿 f 的拉回（pullback）为嵌套元组

$$S'=f^*T'=\mathsf{sub}(S,(S'_1,\ldots,S'_m)).$$

对任意 $1 \leq i \leq m$，令

$$f'_i : S'_i \to T'$$

在 $\alpha(i) = *$ 时为平凡映射（trivial map），在 $\alpha(i) = j$ 时为包含映射（inclusion）

$$\mathsf{incl}_j(T', T) : S'_i = T'_j \to T'$$

诸映射 $f'_1, \ldots, f'_m$ 的像（image）两两不相交，因此我们作拼接

$$(f'_1, \ldots, f'_m) : (S'_1, \ldots, S'_m) \to T'.$$

我们定义 $f' = T'^{*} f$ 为复合

$$\begin{tikzcd}
S' \arrow[r, "\mathsf{id}_{S'}^{(S'_1,\ldots,S'_m)}"] & (S'_1, \ldots, S'_m) \arrow[r, "(f'_1,\ldots,f'_m)"] & T'.
\end{tikzcd}$$

我们把 f' 称为 f 沿 T 的拉回，并把这样的拉回描绘为一个方块

$$\begin{tikzcd}
S' \arrow[r, "f'"] \arrow[d, two heads] \arrow[dr, phantom, very near start, "\lrcorner"] & T' \arrow[d, two heads] \\
S \arrow[r, "f"'] & T.
\end{tikzcd}$$

例 3.2.5.6. 设 $f : (64, 32) \to (4, 64, 4, 32)$ 位于 $\alpha = (2, 4)$ 之上。则我们有一个拉回方块（pullback square）

$$\begin{tikzcd}
((16,4),(16,2)) \arrow[r, "f'"] \arrow[d, two heads] \arrow[dr, phantom, very near start, "\lrcorner"] & ((2,2),(16,4),(2,2),(16,2)) \arrow[d, two heads] \\
(64,32) \arrow[r, "f"'] & (4,64,4,32)
\end{tikzcd}$$

其中 f' 位于 $\alpha' = (3, 4, 7, 8)$ 之上。

例 3.2.5.7. 设 S 是嵌套元组，其扁平化为

$$S^{\flat} = (s_1, \ldots, s_m),$$

并设 $S' \twoheadrightarrow S$ 是一个加细，其相对扁平化（relative flattening）为

$$(S'_1, \ldots, S'_m).$$

则 $S' \twoheadrightarrow S$ 沿去扁平化同构（unflattening isomorphism）

$$\mathsf{id}^{S}_{(s_1,\ldots,s_m)} : (s_1, \ldots, s_m) \to S$$

的拉回是重新加括号同构

$$\begin{tikzcd}
(S'_1,\dots,S'_m) \arrow[r, "\mathsf{id}^{S'}_{(S'_1,\dots,S'_m)}"] \arrow[d, two heads] \arrow[dr, phantom, "\lrcorner", very near start] & S' \arrow[d, two heads] \\
(s_1,\dots,s_m) \arrow[r, "\mathsf{id}^{S}_{(s_1,\dots,s_m)}"'] & S.
\end{tikzcd}$$

例 3.2.5.8. 设 $S' \twoheadrightarrow S$ 是一个加细，并考虑第 i 个元素包含映射

$$s_i \to S.$$

则 $S' \twoheadrightarrow S$ 沿 $s_i \to S$ 的拉回是第 i 个相对模式包含映射

$$\begin{tikzcd}
S'_i \arrow[r, "\mathsf{incl}_i(S',S)"] \arrow[d, two heads] \arrow[dr, phantom, "\lrcorner", very near start] & S' \arrow[d, two heads] \\
s_i \arrow[r] & S.
\end{tikzcd}$$

观察 3.2.5.9. 上面的拉回构造给出了一个反变函子（contravariant functor）

$$\mathbf{Nest}^{\mathrm{op}} \longrightarrow \mathbf{Cat}$$

$$\begin{tikzcd}
S \arrow[r, mapsto] \arrow[d, "f"'] & \mathbf{Ref}(S) & f^*T' & f^*T'' \arrow[l] \\
T \arrow[r, mapsto] & \mathbf{Ref}(T) \arrow[u, "f^*"'] & T' & T'' \arrow[l] \arrow[from=2-3, to=1-3, shift left=4ex, phantom]
\end{tikzcd}$$

拉回的关键性质是：f' 的布局函数（layout function）与 f 的布局函数相等。

引理 3.2.5.10. 设

$$\begin{tikzcd}
S' \arrow[r, "f'"] \arrow[d, two heads] \arrow[dr, phantom, "\lrcorner", very near start] & T' \arrow[d, two heads] \\
S \arrow[r, "f"'] & T
\end{tikzcd}$$

是一个拉回方块，其中 f 位于 α 之上。令

$$f'_i : S'_i \to T$$

表示 f' 相对于 S 的第 i 个模式，并令

$$(L_f)^\flat = (s_1,\dots,s_m) : (d_1,\dots,d_m).$$

则对任意 $1 \le i \le m$，我们有

$$\mathsf{coal}(L_{f'_i}) = s_i : d_i.$$

证明. 设 $1 \leq i \leq m$。若 $\alpha(i) = *$，则 f'_i 是平凡映射，于是

$$L_{f'_i} = s_i : 0 = s_i : d_i.$$

特别地，$\mathsf{coal}(L_{f'_i}) = s_i : 0 = s_i : d_i$。接下来设 $\alpha(i) = j \neq *$。由 f' 的构造，我们有

$$f'_i = \mathsf{incl}_j(T', T) : T'_j \to T'.$$

它位于由 $t \mapsto \mathsf{len}_{<j}(T', T) + t$ 给出的映射 α'_i 之上。对每个 $1 \leq t < \mathsf{len}(T'_j)$，都有 $\alpha'_i(t) = \alpha'_i(t+1)$，因此 $L_{f'_i}$ 是大小为 $\mathsf{size}(T'_j) = t_j = s_i$ 的列主序（column-major）布局。这意味着 $\mathsf{coal}(L_{f'_i})$ 是形如

$$\mathsf{coal}(L_{f'_i}) = s_i : e$$

的深度（depth）为 0 的布局，其中 $e \geq 0$ 是某个整数。我们断言 $e = d_i$。若记 $t'_{j'} = \mathsf{entry}_{j'}(T')$，则我们有

$$\begin{aligned} e = \mathsf{entry}_1(\mathsf{stride}(L_{f'_i})) &= \prod_{j' < \alpha'_i(1)} t'_{j'} \\ &= \prod_{j' \leq \mathsf{len}_{<j}(T', T)} t'_{j'} \\ &= \prod_{j' < j} \mathsf{size}(T'_{j'}) \\ &= \prod_{j' < j} t_{j'} \\ &= d_i. \end{aligned}$$

□

命题 3.2.5.11. 若

$$\begin{tikzcd} S' \arrow[r, "f'"] \arrow[d, two heads] \arrow[dr, phantom, "\lrcorner", very near start] & T' \arrow[d, two heads] \\ S \arrow[r, "f"'] & T \end{tikzcd}$$

是一个拉回方块，则 $\Phi_{L_f} = \Phi_{L_{f'}}$。

证明. 我们先固定记号。令 $m = \mathsf{len}(S)$，并令

$$\begin{aligned} S^\flat &= (s_1, \ldots, s_m), \\ S'_i &= \mathsf{mode}_i(S', S), \\ T'_j &= \mathsf{mode}_j(T', T), \\ (L_f)^\flat &= (s_1, \ldots, s_m) : (d_1, \ldots, d_m). \end{aligned}$$

考虑重新加括号同构

$$\mathsf{id}^{S'}_{(S'_1,\ldots,S'_m)} : (S'_1,\ldots,S'_m) \to S'$$

该映射与 f' 的复合是拼接 $(f'_1,\ldots,f'_m)$，其中 f'_i 在 $\alpha(i) = *$ 时为平凡映射，否则为相对模式包含映射

$$\mathsf{incl}_i(T',T) : S'_i = T'_j \to T'$$

利用引理 3.2.5.10 以及 $L_{f'} = L_{(f'_1,\ldots,f'_m)}$ 这一事实，我们计算出

$$\begin{aligned}
\mathsf{coal}(L_{f'}) &= \mathsf{coal}(L_{(f'_1,\ldots,f'_m)}) \\
&= \mathsf{coal}((L_{f'_1},\ldots,L_{f'_m})) \\
&= \mathsf{coal}((\mathsf{coal}(L_{f'_1}),\ldots,\mathsf{coal}(L_{f'_m}))) \\
&= \mathsf{coal}((s_1,\ldots,s_m) : (d_1,\ldots,d_m)) \\
&= \mathsf{coal}(L_f).
\end{aligned}$$

由命题 2.3.3.14，我们推出 $\Phi_{L_{f'}} = \Phi_{L_f}$。 □

构造 3.2.5.12 (前推)**.** 设 $f : S \to T$ 是位于 α 之上的嵌套元组态射，并设 $S' \twoheadrightarrow S$ 是一个加细。令

$$S'_i = \mathsf{mode}_i(S',S)$$

表示 S' 相对于 S 的第 i 个模式，且对任意 $1 \le j \le \mathsf{len}(T)$，规定

$$T'_j = \begin{cases} \mathsf{entry}_j(T) & j \notin \mathsf{Image}(\alpha) \\ S'_i & \alpha(i) = j. \end{cases}$$

我们定义 S' 沿 f 的前推（pushforward）为嵌套元组

$$T' = f_*S' = \mathsf{sub}(T,(T'_1,\ldots,T'_n)).$$

对任意 $1 \le i \le m$，令

$$f'_i : S'_i \to T'$$

在 $\alpha(i) = *$ 时为平凡映射，在 $\alpha(i) = j$ 时为相对模式包含映射

$$\mathsf{incl}_j(T',T) : S'_i = T'_j \to T'$$

诸态射 $f'_1,\ldots,f'_m$ 的像两两不相交，因此我们可以作拼接

$$(f'_1,\ldots,f'_m) : (S'_1,\ldots,S'_m) \to T'.$$

我们定义 $f' = S'_* f$ 为复合

$$S' \xrightarrow{\mathsf{id}_{S'}^{(S'_1,\ldots,S'_m)}} (S'_1,\ldots,S'_m) \xrightarrow{(f'_1,\ldots,f'_m)} T'.$$

我们把 f' 称为 f 沿 T 的前推。我们把这样的前推描绘为

$$\begin{tikzcd}
S' \arrow[r, "f'"] \arrow[d, two heads] & T' \arrow[d, two heads] \\
S \arrow[r, "f"'] \arrow[ur, phantom, "\llcorner", very near end] & T
\end{tikzcd}$$

例 3.2.5.13. 若 $f : (64, 32) \to (4, 64, 4, 32)$ 位于 $\alpha = (2, 4)$ 之上，则我们有一个前推方块（pushforward square）

$$\begin{tikzcd}
((16,4),(16,2)) \arrow[r, "f'"] \arrow[d, two heads] & (4,(16,4),4,(16,2)) \arrow[d, two heads] \\
(64,32) \arrow[r, "f"'] \arrow[ur, phantom, "\llcorner", very near end] & (4,64,4,32)
\end{tikzcd}$$

拉回的关键性质是：f' 的布局函数与 f 的布局函数相等。

引理 3.2.5.14. 设

$$\begin{tikzcd}
S' \arrow[r, "f'"] \arrow[d, two heads] & T' \arrow[d, two heads] \\
S \arrow[r, "f"'] \arrow[ur, phantom, "\llcorner", very near end] & T
\end{tikzcd}$$

是一个前推方块，其中 f 位于 α 之上。令

$$f'_i : S'_i \to T$$

表示 f' 相对于 S 的第 i 个模式，并令

$$(L_f)^\flat = (s_1, \ldots, s_m) : (d_1, \ldots, d_m).$$

则对任意 $1 \leq i \leq m$，我们有

$$\mathsf{coal}(L_{f'_i}) = s_i : d_i.$$

证明. 其证明与引理 3.2.5.10 的证明完全相同。 □

命题 3.2.5.15. 若

$$\begin{tikzcd}
S' \arrow[r, "f'"] \arrow[d, two heads] & T' \arrow[d, two heads] \\
S \arrow[r, "f"'] \arrow[ur, phantom, "\llcorner", very near end] & T
\end{tikzcd}$$

是一个前推方块，则 $\Phi_{L_f} = \Phi_{L_{f'}}$。

证明. 其证明与命题 3.2.5.11 的证明完全相同。 □

观察 3.2.5.16. 上面定义的前推构造给出了一个协变函子（covariant functor）

$$\mathsf{Nest} \longrightarrow \mathsf{Cat}$$

$$\begin{tikzcd}
S \arrow[r, mapsto] \arrow[d, "f"'] & \mathsf{Ref}(S) \arrow[d, "f_*"] & S'' \arrow[r] \arrow[d, mapsto] & S' \\
T \arrow[r, mapsto] & \mathsf{Ref}(T) & f_*S'' \arrow[r] & f_*S'
\end{tikzcd}$$

观察 3.2.5.17. 若 $f : S \to T$ 是嵌套元组之间的一个同构（isomorphism），则

$$\mathsf{Ref}(T) \xrightarrow{f^*} \mathsf{Ref}(S)$$

与

$$\mathsf{Ref}(S) \xrightarrow{f_*} \mathsf{Ref}(T)$$

是一对互逆的范畴同构（isomorphism of categories）。具体而言，

$$(f^{-1})^* = f_* \quad \text{and} \quad (f^{-1})_* = f^*.$$

观察 3.2.5.18. 若 S_1 与 S_2 是满足 $\mathsf{flat}(S_1) = \mathsf{flat}(S_2)$ 的嵌套元组，则存在一个典范（canonical）的嵌套元组同构 $S_1 \cong S_2$，从而存在一个典范的范畴同构

$$\mathsf{Ref}(S_1) \cong \mathsf{Ref}(S_2).$$

我们还需要再规定一个概念，称为*相互加细*（mutual refinement）。这一概念的重要性将在第 四 章变得清晰，届时我们会在布局复合算法中使用这一概念。

定义 3.2.5.19. 设 T 与 U 是嵌套元组。(T, U) 的一个*相互加细*是形如

$$\begin{tikzcd}
T' \arrow[r, tail] \arrow[d, two heads] & U' \arrow[d, two heads] \\
T & U
\end{tikzcd}$$

的图。确切地说，这是一对嵌套元组 (T', U')，满足

1. T' 加细 T，
2. U' 加细 U，且
3. T' 整除（divides）U'。

除相互加细的定义之外，我们还需要下面的事实。

引理 3.2.5.20. 设 T 与 U 是嵌套元组。则 (T,U) 的相互加细与 $(T^\flat,U^\flat)$ 的相互加细之间存在一一对应。

证明. 若 (T',U') 是 (T,U) 的一个相互加细，则沿去扁平化同构 $\mathsf{id}^{T}_{T^\flat}$ 与 $\mathsf{id}^{U}_{U^\flat}$ 作拉回，得到 $(T^\flat,U^\flat)$ 的一个相互加细

$$\begin{tikzcd}
(\mathsf{id}^{T}_{T^\flat})^* T' \arrow[r, tail] \arrow[d, two heads] & (\mathsf{id}^{U}_{U^\flat})^* U' \arrow[d, two heads] \\
T^\flat & U^\flat
\end{tikzcd}$$

反之，若 $((T^\flat)',(U^\flat)')$ 是 $T^\flat,U^\flat$ 的一个相互加细，则沿扁平化同构（flattening isomorphism）$\mathsf{id}^{T^\flat}_{T}$ 与 $\mathsf{id}^{U^\flat}_{U}$ 作拉回，得到 $(T^\flat,U^\flat)$ 的一个相互加细

$$\begin{tikzcd}
(\mathsf{id}^{T^\flat}_{T})^* (T^\flat)' \arrow[r, tail] \arrow[d, two heads] & (\mathsf{id}^{U^\flat}_{U})^* (U^\flat)' \arrow[d, two heads] \\
T & U
\end{tikzcd}$$

□

3.2.6　嵌套元组态射上的运算

我们的下一个任务是发展一套「嵌套元组态射（nested tuple morphism）代数」。由于我们已经为元组态射建立过这样一套「代数」，我们可以通过为各种运算的输出配备适当的轮廓（profile），把这套代数推广到嵌套情形。

3.2.6.1　拼接

接下来，我们定义嵌套元组态射上的一个拼接（concatenate）运算，它与布局（layout）的拼接相容，即

$$L_{(f,g)} = (L_f, L_g).$$

我们通过拼接 f 与 g 的定义域（domain）来拼接嵌套元组态射 f 与 g。为使这一定义良定义（well-defined），需要 f 与 g 满足一个不交性条件，其确切表述见下文。

定义 3.2.6.1. 设 f 与 g 是具有相同陪域（codomain）的嵌套元组态射。若 $f^\flat$ 与 $g^\flat$ 如定义 3.1.5.32 中那样具有不交的像，则称 f 与 g 具有不交的像（disjoint images）。

例 3.2.6.2. 若

$$f : (3, (512, 512)) \to (2, 512, 2, 512)$$

位于（lies over）$(*, 2, 4)$ 之上，且

$$g : (2, 2) \to (2, 512, 2, 512)$$

位于 $(1, 3)$ 之上，则 f 与 g 具有不交的像。

例 3.2.6.3. 若

$$f : (2, (32, 64)) \to (32, (2, 2, 2), 64)$$

位于 $\alpha = (3, 1, 5)$ 之上，且

$$g : ((2, 2)) \to (32, (2, 2, 2), 64)$$

位于 $\beta = (2, 4)$ 之上，则 f 与 g 具有不交的像。

构造 3.2.6.4. 设 $f : S \to T$ 与 $g : U \to T$ 是分别位于 α 与 β 之上的嵌套元组态射，且 f 与 g 具有不交的像。我们把 f 与 g 的拼接定义为嵌套元组态射

$$(f, g) : (S, U) \to T$$

它满足

$$\mathsf{flat}((f, g)) = f^\flat \star g^\flat.$$

更一般地，若 $f_i : S_i \to T$（$1 \leq i \leq k$）是嵌套元组态射，且 $f_1, \ldots, f_k$ 两两具有不交的像，则我们把拼接

$$(f_1, \ldots, f_k) : (S_1, \ldots, S_k) \to T.$$

定义为满足

$$(f_1, \ldots, f_k)^\flat = f_1^\flat \star \cdots \star f_k^\flat.$$

例 3.2.6.5. 例 3.2.6.2 中态射 f 与 g 的拼接是嵌套元组态射

$$(f, g) : ((3, (512, 512)), (2, 2)) \to (2, 512, 2, 512)$$

它位于 $\alpha \star \beta = (*, 2, 4, 1, 3)$ 之上。

例 3.2.6.6. 例 3.2.6.3 中态射 f 与 g 的拼接是嵌套元组态射

$$(f, g) : ((2, (32, 64)), ((2, 2))) \to (32, (2, 2, 2), 64)$$

它位于 $\alpha \star \beta = (3, 1, 5, 2, 4)$ 之上。

例 3.2.6.7. 若

$$f : (2, 2) \to (2, 3, 5, 2, 3, 5)$$

位于 $\alpha = (1, 4)$ 之上，

$$g : (3, 3) \to (2, 3, 5, 2, 3, 5)$$

位于 $\beta = (2, 5)$ 之上，且

$$h : (5, 5) \to (2, 3, 5, 2, 3, 5)$$

位于 $\gamma = (3, 6)$ 之上，则 f、g 与 h 两两具有不交的像，且拼接

$$(f, g, h) : ((2, 2), (3, 3), (5, 5)) \to (2, 3, 5, 2, 3, 5)$$

位于 $\alpha \star \beta \star \gamma = (1, 4, 2, 5, 3, 6)$ 之上。

例 3.2.6.8. 设 $f : S \to T$ 是嵌套元组态射，并设

$$S^\flat = (s_1, \ldots, s_m).$$

回顾例 3.2.3.4：对任意 $1 \leq i \leq m$，存在嵌套元组态射

$$f_i : s_i \to T.$$

称为 f 的第 i 个元素（entry）。这些态射两两具有不交的像，而拼接

$$(f_1, \ldots, f_m) : S^\flat \to T$$

是例 3.2.3.2 中的复合（composite）

$$(f_1, \ldots, f_m) = f \circ \mathsf{id}_{S^\flat}^{S}$$

例 3.2.6.9. 设 $f : S \to T$ 是嵌套元组态射，并设

$$S = (S_1, \ldots, S_r).$$

回顾例 3.2.3.6：对任意 $1 \le i \le r$，存在嵌套元组态射

$$f_i : S_i \to T.$$

称为 f 的第 i 个模式（mode）。这些态射两两具有不交的像，且拼接

$$(f_1, \ldots, f_r) : S \to T$$

等于 f。换言之，每个嵌套元组态射 f 都可以写成它的各模式的拼接：

$$f = (f_1, \ldots, f_r).$$

命题 3.2.6.10. 若 $f_1, \ldots, f_k$ 是具有相同陪域且两两具有不交像的嵌套元组态射，则

$$L_{(f_1,\ldots,f_k)} = (L_{f_1}, \ldots, L_{f_k}).$$

证明. 由构造（by construction）可知，我们有

$$\begin{aligned}\mathsf{shape}((L_{f_1}, \ldots, L_{f_k})) &= (\mathsf{shape}(L_{f_1}), \ldots, \mathsf{shape}(L_{f_k}))\\ &= \mathsf{shape}(L_{(f_1,\ldots,f_k)}).\end{aligned}$$

再利用命题 3.1.5.38，可得

$$\begin{aligned}(L_{f_1}, \ldots, L_{f_k})^\flat &= L_{f_1}^\flat \star \cdots \star L_{f_k}^\flat\\ &= L_{f_1^\flat} \star \cdots \star L_{f_k^\flat}\\ &= L_{f_1^\flat \star \cdots \star f_k^\flat}\\ &= L_{(f_1,\ldots,f_k)^\flat}\\ &= (L_{(f_1,\ldots,f_k)})^\flat.\end{aligned}$$

□

3.2.6.2　合并

若 f 是嵌套元组态射，我们本可以把合并（coalesce）$\mathsf{coal}(f)$ 定义为 $\mathsf{coal}^\flat(f^\flat)$。从理论上讲，这是一个健全的定义。然而，为了让我们的定义与 `cute` 实现相容，我们要对 $\mathsf{coal}(f)$ 的定义做一点小修改。

定义 3.2.6.11. 设 $f : S \to T$ 是嵌套元组态射，并记

$$\mathsf{coal}^\flat(f^\flat) : (s_1, \ldots, s_m) \to (t_1, \ldots, t_n).$$

- （情形 1）：若 $m>1$，我们定义
$$\mathsf{coal}(f)=\mathsf{coal}^\flat(f^\flat).$$
- （情形 2）：若 $m=1$，我们把 $\mathsf{coal}(f)$ 定义为复合
$$s_1 \xrightarrow[(1)]{} (s_1) \xrightarrow{\mathsf{coal}^\flat(f^\flat)} (t_1,\dots,t_n).$$
- （情形 3）：若 $m=0$，我们把 $\mathsf{coal}(f)$ 定义为复合
$$1 \xrightarrow[(*)]{} () \xrightarrow{\mathsf{coal}^\flat(f^\flat)} (t_1,\dots,t_n).$$

例 3.2.6.12. 若
$$f:((2,2),(3,3),(5,5))\to(5,5,3,3,2,2)$$
位于 $\alpha=(5,6,3,4,1,2)$ 之上，则
$$\mathsf{coal}(f):(4,9,25)\to(25,9,4)$$
位于 $\alpha'=(3,2,1)$ 之上。

命题 3.2.6.13. 若 $f:S\to T$ 是嵌套元组态射，则
$$\mathsf{coal}(L_f)=L_{\mathsf{coal}(f)}.$$

证明. 我们再次写出
$$(s_1,\dots,s_m)\xrightarrow[\alpha]{\mathsf{coal}^\flat(f^\flat)}(t_1,\dots,t_n).$$
需考虑三种情形。

- （情形 1）：设 $m>1$。则
$$\begin{aligned} L_{\mathsf{coal}(f)} &= L_{\mathsf{coal}^\flat(f^\flat)}\\ &= \mathsf{coal}^\flat(L_{f^\flat})\\ &= \mathsf{coal}((L_f)^\flat)\\ &= \mathsf{coal}(L_f).\end{aligned}$$
- （情形 2）：设 $m=1$。则
$$\begin{aligned} L_{\mathsf{coal}(f)} &= s_1 : t_1\dots,t_{\alpha(1)-1}\\ &= \mathsf{coal}((s_1):(t_1\cdots t_{\alpha(1)-1}))\\ &= \mathsf{coal}(L_{\mathsf{coal}^\flat(f^\flat)})\\ &= \mathsf{coal}(\mathsf{coal}^\flat(L_{f^\flat}))\\ &= \mathsf{coal}((L_f)^\flat)\\ &= \mathsf{coal}(L_f).\end{aligned}$$

- （情形 3）：设 $m = 0$。则

$$\begin{aligned} L_{\mathsf{coal}(f)} &= 1 : 0 \\ &= \mathsf{coal}(() : ()) \\ &= \mathsf{coal}(L_{\mathsf{coal}^\flat(f^\flat)}) \\ &= \mathsf{coal}(\mathsf{coal}^\flat(L_{f^\flat})) \\ &= \mathsf{coal}((L_f)^\flat) \\ &= \mathsf{coal}(L_f). \end{aligned}$$

□

3.2.6.3　求补

本节中，我们定义嵌套元组态射互补的概念。

定义 3.2.6.14. 设 $f : S \to T$ 与 $g : U \to T$ 是具有不交像的嵌套元组态射。若

$$(f, g) : (S, U) \to T$$

是一个同构（isomorphism），则称 g 为 f 的一个补（complement）。

注 3.2.6.15. 若 $f : S \to T$ 与 $g : U \to T$ 是嵌套元组态射，则 g 是 f 的补当且仅当 $g^\flat$ 是 $f^\flat$ 的补，因为 $(f, g)^\flat = f^\flat \star g^\flat$。

命题 3.2.6.16. *若 $f : S \to T$ 是嵌套元组态射，且 $g : U \to T$ 是 f 的补，则 L_g 是 L_f 的 $\mathsf{size}(T)$-补。*

证明. 观察 3.2.2.5 蕴含

$$\begin{aligned} (L_f)^\flat &= L_{f^\flat}, \text{ and} \\ (L_g)^\flat &= L_{g^\flat} \end{aligned}$$

而引理 2.3.6.2 使我们可以化归到扁平情形（命题 3.1.5.42）。 □

构造 3.2.6.17. 设 $f : S \to T$ 是嵌套元组态射。我们把 f 的补定义为复合

$$\begin{tikzcd} U \arrow[rr, "f^c"] \arrow[dr, "(f^\flat)^c"'] & & T \\ & T^\flat \arrow[ur, "\mathsf{id}^T_{T^\flat}"'] & \end{tikzcd}$$

其中 $(f^\flat)^c$ 如构造 3.1.5.46 中所定义，而 $\mathsf{id}^T_{T^\flat} : T^\flat \cong T$ 是去扁平化同构（unflattening isomorphism）。

例 3.2.6.18. 嵌套元组态射

$$((2,2),(5,5)) \xrightarrow[(1,4,2,5)]{f} ((2,5,7),(2,5,7))$$

的补是

$$(7,7) \xrightarrow[(3,6)]{f^c} ((2,5,7),(2,5,7)).$$

命题 3.2.6.19. *设 $f : S \to T$ 与 $g : U \to T$ 是嵌套元组态射。若 f 是单射且 g 是 f 的补，则 L_g 是 L_f 的 $\mathsf{size}(T)$-补。*

证明. 这由命题 3.1.5.42 与引理 2.3.6.2 即得，因为

$$\begin{aligned}(L_f)^\flat &= L_{f^\flat}\\ (L_g)^\flat &= L_{g^\flat}.\end{aligned}$$

□

命题 3.2.6.20. *若 $f : S \to T$ 是单射嵌套元组态射，则*

$$\mathsf{coal}(L_{f^c}) = \mathsf{comp}(L_f, \mathsf{size}(T)).$$

证明. 由于 f^c 是由 $(f^\flat)^c$ 通过与一个重新加括号同构作后复合（post-compose）而得到的，于是

$$L_{f^c} = L_{(f^\flat)^c}$$

故由命题 3.2.6.20 可得

$$\mathsf{coal}^\flat(L_{f^c}) = \mathsf{comp}^\flat(L_f, \mathsf{size}(T)).$$

对两边应用 $\mathsf{coal}(-)$ 即得结论。 □

3.2.6.4　复合

我们可以利用第 3.2.4 节的实现函子（realization functor）来证明嵌套元组态射的复合（composition）与相应布局的复合相容。

定理 3.2.6.21. *若 f 与 g 是非退化（non-degenerate）且可复合的嵌套元组态射，则*

$$L_{g\circ f} = L_g \circ L_f.$$

证明. 设 $f : S \to T$ 与 $g : T \to U$ 是非退化嵌套元组态射。我们需要验证

1. $\mathsf{shape}(L_{g\circ f})$ 加细（*refine*）$\mathsf{shape}(L_f)$：这成立，因为

$$\mathsf{shape}(L_f) = S = \mathsf{shape}(L_{g\circ f}).$$

2. $L_{g\circ f}$ 在 $\mathsf{shape}(L_f)$ 上是已合并（*coalesced*）的：这成立，因为嵌套元组态射 $g\circ f$ 是非退化的，从而布局 $L_{g\circ f}$ 也是非退化的。

3. $\Phi_{L_{g\circ f}} = \Phi_{L_g} \circ \Phi_{L_f}^{\mathsf{size}(L_g)}$：利用引理 3.2.4.2，我们有

$$\begin{aligned}\Phi_{L_{g\circ f}}^{\mathsf{size}(U)} &= |g\circ f| \\ &= |g| \circ |f| \\ &= \Phi_{L_g}^{\mathsf{size}(U)} \circ \Phi_{L_f}^{\mathsf{size}(T)}\end{aligned}$$

再与包含映射（inclusion）$[0, \mathsf{size}(U)) \subset \mathbb{Z}$ 作后复合，并注意到 $\mathsf{size}(T) = \mathsf{size}(L_g)$，即得结论。

□

3.2.6.5　逻辑切分

接下来，我们引入嵌套元组态射的逻辑切分（logical division）。这一构造（construction）由扁平切分（flat division）通过引入嵌套轮廓而得到，不带任何相容性约束。

定义 3.2.6.22. 设 f 与 g 是嵌套元组态射。若 g 与 f 可复合，则称 g 整除（divides）f。换言之，

$$\mathsf{codomain}(g) = \mathsf{domain}(f).$$

定义 3.2.6.23. 设 $g: S \to T$ 与 $f: T \to U$ 是嵌套元组态射。我们把 f 除以 g 的逻辑切分定义为嵌套元组态射

$$f \oslash g = f \circ (g, g^c).$$

例 3.2.6.24. 将

$$((2,2),2) \xrightarrow[(2,4,*)]{f} ((4,2),(4,2))$$

除以

$$(2,2) \xrightarrow[(1,3)]{g} ((2,2),2)$$

所得的逻辑切分是

$$((2,2),2) \xrightarrow[(2,*,4)]{f\oslash g} ((4,2),(4,2)).$$

例 3.2.6.25. 将

$$(8,8,512,512,512) \xrightarrow[(*,*,1,2,3)]{f} (512,512,512)$$

除以

$$(8,512) \xrightarrow[(1,5)]{g} (8,8,512,512,512)$$

所得的逻辑切分是

$$((8,512),(8,512,512)) \xrightarrow[(*,1,*,2,3)]{f \oslash g} ((4,2),(4,2)).$$

命题 3.2.6.26. *若 $g: S \to T$ 与 $f: T \to U$ 是非退化嵌套元组态射，则*

$$\mathsf{coal}(L_{f \oslash g}) = \mathsf{coal}(L_f \oslash L_g).$$

证明. 由命题 3.2.6.20，我们有

$$\mathsf{coal}(\mathsf{comp}(L_g, \mathsf{size}(L_f))) = \mathsf{coal}(L_{g^c})$$

并且我们计算

$$\begin{aligned}
\mathsf{coal}(L_f \oslash L_g) &= \mathsf{coal}(L_f \circ (L_g, \mathsf{comp}(L_g, \mathsf{size}(L_f)))) \\
&= \mathsf{coal}(L_f \circ (L_g, L_{g^c})) \\
&= \mathsf{coal}(L_f \circ L_{(g,g^c)}) \\
&= \mathsf{coal}(L_f \circ L_{(g,g^c)}) \\
&= \mathsf{coal}(L_{f \circ (g,g^c)}) \\
&= \mathsf{coal}(L_{f \oslash g}).
\end{aligned}$$

□

命题 3.2.6.27. *若 f 与 g 是嵌套元组（nested tuple）且 g 整除 f，则*

$$(f \oslash g)^\flat = f^\flat \oslash^\flat g^\flat.$$

证明. 我们计算

$$\begin{aligned}
(f \oslash g)^\flat &= (f \circ (g, g^c))^\flat \\
&= f^\flat \circ (g, g^c)^\flat \\
&= f^\flat \circ (g^\flat \star (g^c)^\flat) \\
&= f^\flat \circ (g^\flat \star (g^\flat)^c) \\
&= f^\flat \oslash^\flat g^\flat.
\end{aligned}$$

□

3.2.6.6　逻辑乘积

本节中，我们定义嵌套元组态射的逻辑乘积。

定义 3.2.6.28. 设 f 与 g 是嵌套元组态射。若 $\mathsf{codomain}(g) = \mathsf{domain}(f^c)$，则称 f 与 g 是乘积可许的（product admissible）。若 f 与 g 乘积可许，我们把 f 与 g 的逻辑乘积（logical product）定义为嵌套元组态射

$$f \otimes g = (f, f^c \circ g).$$

例 3.2.6.29. 嵌套元组态射

$$(8,8) \xrightarrow[(1,2)]{f} (8,8,16,16)$$

与

$$(16,16) \xrightarrow[(1,2)]{g} (16,16)$$

是乘积可许的，它们的逻辑乘积是

$$((8,8),(16,16)) \xrightarrow[(1,2,3,4)]{f\otimes g} (8,8,16,16).$$

例 3.2.6.30. 嵌套元组态射

$$(128,128) \xrightarrow[(3,4)]{f} (32,32,128,128)$$

与

$$(32) \xrightarrow[(2)]{g} (32,32)$$

是乘积可许的，它们的逻辑乘积是

$$((128,128),(32)) \xrightarrow[(3,4,2)]{f\otimes g} (32,32,128,128).$$

命题 3.2.6.31. 设 f 与 g 是非退化嵌套元组态射且 f 与 g 乘积可许。则

$$L_{f\otimes g} = L_f \otimes L_g.$$

证明. 设 $f : S \to T$ 与 $g : U \to V$ 乘积可许，并令

$$L_f^* = \mathsf{comp}(L_f, \mathsf{size}(L_f)\cdot \mathsf{cosize}(L_g))$$

由于 f 是单射且 $\mathsf{codomain}(g) = \mathsf{domain}(f^c)$，可知

$$\mathsf{size}(L_f)\cdot \mathsf{cosize}(L_g) \leq \mathsf{size}(S)\cdot \mathsf{size}(V) = \mathsf{size}(T).$$

利用这一事实，以及事实

$$\Phi_{\mathsf{comp}(L_f,\mathsf{size}(T))} = \Phi_{L_{f^c}},$$

我们有

$$\begin{aligned} L_f^* \circ L_g &= \mathsf{comp}(L_f, \mathsf{size}(T)) \circ L_g \\ &= L_{f^c} \circ L_g. \end{aligned}$$

再利用这一事实，我们计算

$$\begin{aligned} L_f \otimes L_g &= (L_f, L_f^* \circ L_g) \\ &= (L_f, L_{f^c} \circ L_g) \\ &= (L_f, L_{f^c \circ g}) \\ &= L_{(f, f^c \circ g)} \\ &= L_{f \otimes g} \end{aligned}$$

□

第四章　计算

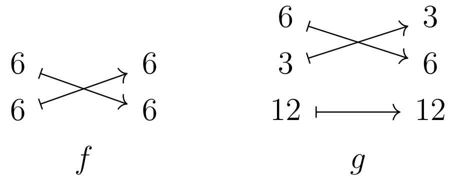
范畴（category）**Tuple** 与 **Nest** 为可处理布局（tractable layout）的计算提供了一个强大的框架。然而在实践中经常出现这样的情形：我们遇到的两个可处理布局 A 与 B 在 `cute` 的语境中是可复合（composable）的，但它们的标准表示在 **Tuple** 与 **Nest** 中都不可复合。本章专门阐明：如何仍然借助范畴 **Tuple** 与 **Nest**，利用**相互加细**（mutual refinement）这一概念，来计算可处理布局的复合、逻辑切分与逻辑乘积。我们将在 节 4.1.1 中引入这一概念，在算法 4.1.1 中给出一个计算相互加细的算法，并细致地演练许多具体例子。

4.1　可处理布局的复合

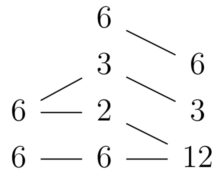
假设我们想要计算可处理布局

$$A = (6,6) : (6,1),$$
$$B = (12,3,6) : (1,72,12).$$

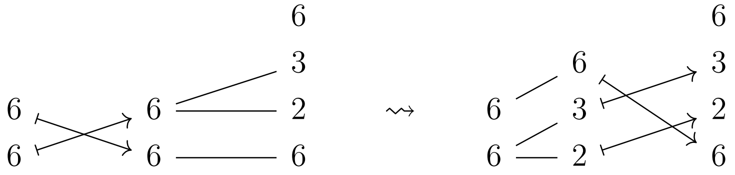
的复合 $B \circ A$。我们或许可以尝试通过计算 A 与 B 的标准表示 f 与 g 的复合来算出 $B \circ A$：



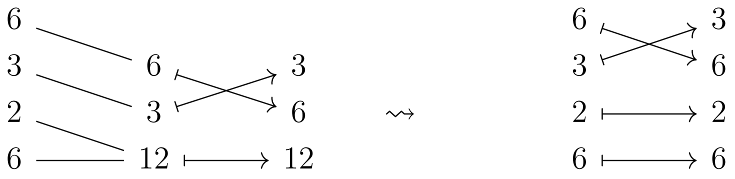
然而，这两个态射（morphism）并不可复合，因为 f 的陪域（codomain）$(6,6)$ 不等于 g 的定义域（domain）$(12,3,6)$。这意味着我们不能直接用态射 f 与 g 来计算复合 $B \circ A$。不过，我们可以通过求出 $(6,6)$ 与 $(12,3,6)$ 的一个相互加细来继续我们的计算，如下图所示



这是一个把 f 与 g 转换为可复合态射 f' 与 g' 的装置：

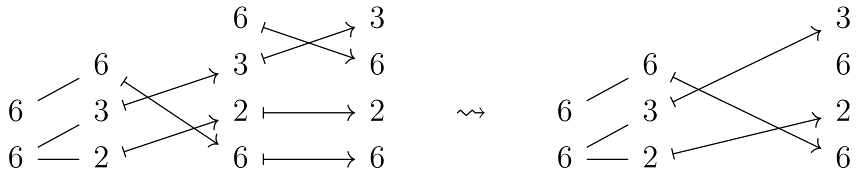

6
6
6
6
6
3
2
6
⇝
6
6
6
3
2
6
3
2
6

f f'


6
3
2
6
6
3
12
3
6
12
⇝
6
3
2
6
3
6
2
6

g g'

态射 f' 与 g' 是可复合的，因此我们可以作复合


6
6
6
3
2
6
3
2
6
3
6
2
6
⇝
6
6
6
3
2
3
6
2
6

f' g' $g' \circ f'$

而计算其编码的布局便得到

$$B \circ A = L_{g' \circ f'} = ((2,3),6) : ((6,72),1).$$

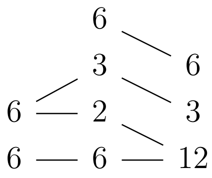
本节的目标是把这一计算过程形式化为一个计算可处理布局 A 与 B 之复合的算法（algorithm）。正如我们在例子中所看到的，我们计算中非平凡的步骤是

1. 求出某些（嵌套）元组的一个相互加细，以及
2. 利用该相互加细把 f 与 g 转换为可复合的态射 f' 与 g'。

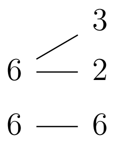
下面两节将分别阐释这两个步骤。

4.1.1 相互加细

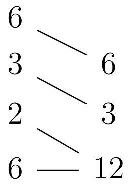
在利用第 三 章的范畴论框架给出相互加细的精确定义之前，我们先作一个非正式的概述。考虑我们那个引例中的元组 $(6,6)$ 与 $(12,3,6)$。我们断言图



是 $(6,6)$ 与 $(12,3,6)$ 的一个相互加细。我们可以对这一相互加细给出更精确的描述如下。图的左半表示加细（refinement）$(6,6) \twoheadleftarrow (6,(2,3))$，图的右半表示加细 $((6,2),3,6) \twoheadrightarrow (12,3,6)$：



$$\rightsquigarrow \qquad (6,6) \xtwoheadleftarrow{\quad} (6,(2,3))$$

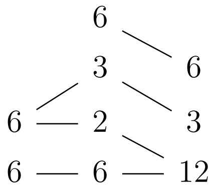


$$\rightsquigarrow \qquad ((6,2),3,6) \xtwoheadrightarrow{\quad} (12,3,6)$$

图的两半可以粘合在一起这一事实，对应于嵌套元组（nested tuple）$(6,(2,3))$ 整除（divides）$((6,2),3,6)$ 这一事实，我们把它记为

$$(6,(2,3)) \rightarrowtail ((6,2),3,6).$$

把这些观察合在一起，我们就可以把我们的相互加细精确地表达为



$$\rightsquigarrow \qquad \begin{tikzcd}(6,(2,3)) \arrow[r, tail] \arrow[d, two heads] & ((6,2),3,6) \arrow[d, two heads] \\ (6,6) & (12,3,6)\end{tikzcd}$$

其中我们选择把加细 $(6,6) \twoheadleftarrow (6,(2,3))$ 与 $((6,2),3,6) \twoheadrightarrow (12,3,6)$ 竖直地画出。现在我们可以给出相互加细的精确定义了。

定义 4.1.1.1. 设 T 与 U 是嵌套元组。(T,U) 的一个*相互加细*是形如

$$\begin{tikzcd}
T' \arrow[r, tail] \arrow[d, two heads] & U' \arrow[d, two heads] \\
T & U
\end{tikzcd}$$

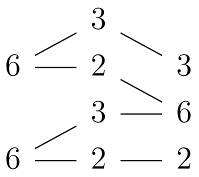
的图。确切地说，这是一对嵌套元组 (T',U')，使得

1. T' 加细 T，
2. U' 加细 U，且
3. T' 整除 U'。

例 4.1.1.2. $T=(6,6)$ 与 $U=(2,6,3)$ 的一个相互加细由

$$\begin{tikzcd}
((2,3),(2,3)) \arrow[r, tail] \arrow[d, two heads] & (2,(3,2),3) \arrow[d, two heads] \\
(6,6) & (2,6,3)
\end{tikzcd}$$

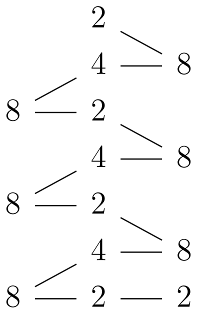
给出。我们把这一相互加细描绘如下。



例 4.1.1.3. $T=(8,8,8)$ 与 $U=(2,8,8,8)$ 的一个相互加细由

$$\begin{tikzcd}
((2,4),(2,4),(2,4)) \arrow[r, tail] \arrow[d, two heads] & (2,(4,2),(4,2),(4,2)) \arrow[d, two heads] \\
(8,8,8) & (2,8,8,8)
\end{tikzcd}$$

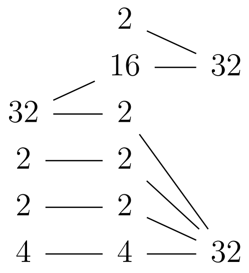
给出。我们把这一相互加细描绘如下。



例 4.1.1.4. $T=(4,2,2,32)$ 与 $U=(32,32)$ 的一个相互加细由

$$\begin{tikzcd}
(4,2,2,(2,16)) \arrow[r, tail] \arrow[d, two heads] & ((4,2,2,2),(16,2)) \arrow[d, two heads] \\
(4,2,2,32) & (32,32)
\end{tikzcd}$$

给出。我们把这一相互加细描绘如下。



例 4.1.1.5. 若 $T=(8,8)$ 且 $U=(3,8,8)$，则不存在 T 与 U 的相互加细。

例 4.1.1.6. 若 T 与 U 是满足 $\mathsf{size}(T)=2^k$ 与 $\mathsf{size}(L)=2^\ell$（其中 $k\leq\ell$）的元组，则存在 T 与 U 的相互加细。更一般地，若 T 与 U 是元组，且 $\mathsf{size}(T)\leq\mathsf{size}(U)$ 均为某个固定整数的幂，则存在 T 与 U 的相互加细。

观察 4.1.1.7. 在前面的每个例子中，我们考虑的都是*扁平*（flat）元组 T 与 U 的相互加细。不过，相互加细的定义允许 T 与 U 是任意嵌套元组。无论如何，限制到扁平情形并不损失一般性，因为一对嵌套元组 (T,U) 的相互加细与它们的扁平化 (flattening) $(T^\flat,U^\flat)$ 的相互加细之间存在一一对应 (见引理 3.2.5.20)。特别地，(T,U) 存在相互加细，当且仅当 $(T^\flat,U^\flat)$ 存在相互加细。

在给出适当的定义之后，我们提供一个计算 (T,U) 的相互加细的算法。

算法 4.1.1：相互加细算法

```
输入：嵌套元组 $T$ 与 $U$。
输出：$(T,U)$ 的一个相互加细 $(T',U')$（若存在），否则 None。
────────────────────────────────────────
$X \leftarrow T$;  $Y \leftarrow U$
$X'$, $Y'$, $X_{\mathsf{mode}}$, $Y_{\mathsf{mode}} \leftarrow ()$
$i \leftarrow 1$;  $j \leftarrow 1$
while $i \leq \mathsf{len}(X)$ 且 $j \leq \mathsf{len}(Y)$ do
    if $\mathsf{entry}_i(X) = \mathsf{entry}_j(Y)$ then
        将 $\mathsf{entry}_i(X)$ 追加到 $X_{\mathsf{mode}}$; 将 $X_{\mathsf{mode}}$ 追加到 $X'$; $X_{\mathsf{mode}} \leftarrow ()$
```

算法 4.1.1（续）：相互加细算法

```
        将 $\mathsf{entry}_j(Y)$ 追加到 $Y_{\mathsf{mode}}$;
        将 $Y_{\mathsf{mode}}$ 追加到 $Y'$;
        $Y_{\mathsf{mode}} \leftarrow ()$
        $i \leftarrow i + 1$;
        $j \leftarrow j + 1$
    else if $\mathsf{entry}_i(X)$ 整除 $\mathsf{entry}_j(Y)$ then
        将 $\mathsf{entry}_i(X)$ 追加到 $X_{\mathsf{mode}}$;
        将 $X_{\mathsf{mode}}$ 追加到 $X'$;
        $X_{\mathsf{mode}} \leftarrow ()$
        将 $\mathsf{entry}_i(X)$ 追加到 $Y_{\mathsf{mode}}$
        $\mathsf{entry}_j(Y) \leftarrow \mathsf{entry}_j(Y)/\mathsf{entry}_i(X)$;
        $i \leftarrow i + 1$
    else if $\mathsf{entry}_j(Y)$ 整除 $\mathsf{entry}_i(X)$ then
        将 $\mathsf{entry}_j(Y)$ 追加到 $X_{\mathsf{mode}}$;
        将 $\mathsf{entry}_j(Y)$ 追加到 $Y_{\mathsf{mode}}$;
        将 $Y_{\mathsf{mode}}$ 追加到 $Y'$;
        $Y_{\mathsf{mode}} \leftarrow ()$
        $\mathsf{entry}_i(X) \leftarrow \mathsf{entry}_i(X)/\mathsf{entry}_j(Y)$;
        $j \leftarrow j + 1$
    else
        return None
    end if
end while
if $Y_{\mathsf{mode}} \neq ()$ then
    将 $\mathsf{entry}_j(Y)$ 追加到 $Y_{\mathsf{mode}}$;
    将 $Y_{\mathsf{mode}}$ 追加到 $Y'$;
    $j \leftarrow j + 1$
end if
while $j < \mathsf{len}(Y)$ do
    将 $\mathsf{entry}_j(Y)$ 追加到 $Y'$;
    $j \leftarrow j + 1$
end while
$T' \leftarrow (X')_{\mathsf{prof}(T)}$;
$U' \leftarrow (Y')_{\mathsf{prof}(U)}$
return $(T', U')$
```

4.1.2　从相互加细到可复合态射

回顾：为了计算

$$
\begin{aligned}
A &= (6,6) : (6,1) \text{ and} \\
B &= (12,3,6) : (1,72,12),
\end{aligned}
$$

的复合 $B \circ A$，我们构造了元组态射（tuple morphism）

$$\begin{tikzcd}
 & & & 6 \arrow[dr, mapsto] & 3 \\
6 \arrow[dr, mapsto] & 6 & & 3 \arrow[ur, mapsto] & 6 \\
6 \arrow[ur, mapsto] & 6 & & 12 \arrow[r, mapsto] & 12 \\
 & f & & g &
\end{tikzcd}$$

以及一个相互加细。

$$\begin{tikzcd}
 & 6 \arrow[dr, dash] & \\
 & 3 \arrow[dr, dash] & 6 \\
6 \arrow[r, dash] \arrow[ur, dash] & 2 \arrow[dr, dash] & 3 \\
6 \arrow[r, dash] & 6 \arrow[r, dash] & 12
\end{tikzcd}$$

我们计算的下一步是利用我们的相互加细把 f 与 g 转换为可复合的态射 f' 与 g'。在给出这一过程形式化的范畴论定义之前，我们先通过一个例子来演示该过程。

我们由 f 与我们相互加细的左半构造 f'：

$$\begin{tikzcd}
 & & 6 & & & & 6 \\
 & & 3 & & & 6 \arrow[ddr, mapsto] & 3 \\
6 \arrow[dr, mapsto] & 6 \arrow[ur, dash] \arrow[r, dash] & 2 & \rightsquigarrow & 6 \arrow[ur, dash] & 3 \arrow[ur, mapsto] & 2 \\
6 \arrow[ur, mapsto] & 6 \arrow[r, dash] & 6 & & 6 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[ur, mapsto] & 6 \\
 & f & & & & f' &
\end{tikzcd}$$

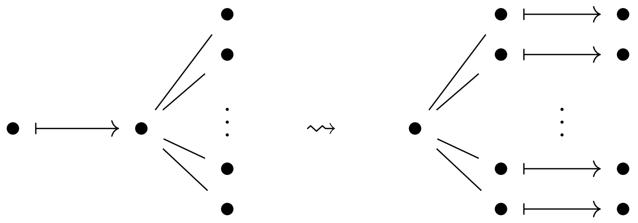
这一构造是通过作替换

$$\begin{tikzcd}
6 \arrow[dr, mapsto] & & & \rightsquigarrow & 6 \arrow[r, dash] & 6 \arrow[dr, mapsto] & \\
 & 6 \arrow[r, dash] & 6 & & & & 6
\end{tikzcd}$$

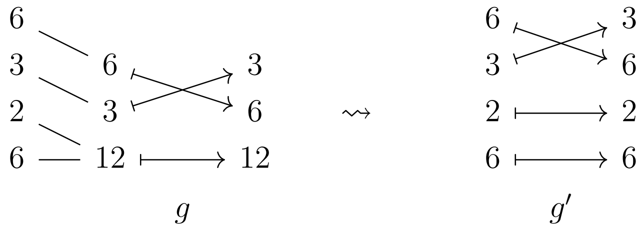
以及作替换

$$\begin{tikzcd}
 & & 3 & & & & 3 \\
 & 6 \arrow[ur, dash] \arrow[r, dash] & 2 & \rightsquigarrow & & 3 \arrow[ur, mapsto] & 2 \\
6 \arrow[ur, mapsto] & & & & 6 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[ur, mapsto] &
\end{tikzcd}$$

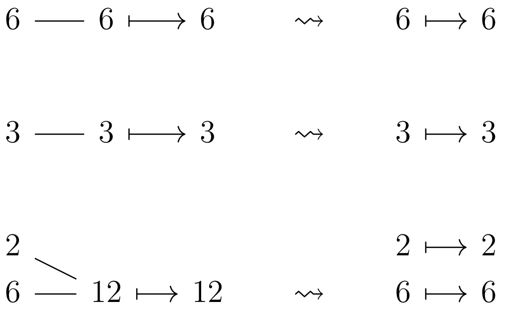
完成的。更一般地，我们作替换



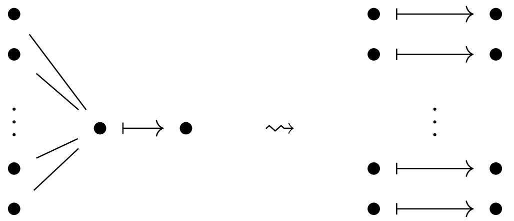
由 g 与我们相互加细的右半构造 g' 的过程是类似的。



这一构造是通过作替换



完成的。更一般地，我们作替换



在给出我们流程的非正式描述之后，我们现在把事情精确化如下。

构造 4.1.2.1. 设 $f: S \to T$ 与 $g: U \to V$ 是嵌套元组态射（nested tuple morphism），且 (T', U') 是 (T, U) 的一个相互加细。于是我们可以利用 3.2.5 节中的拉回（pullback）与前推（pushforward）构造来作出下图：

$$\begin{tikzcd}
S' \arrow[r, "\tilde{f}"] \arrow[d, two heads] \arrow[dr, phantom, "\lrcorner", very near start] & T' \arrow[r, tail, "i"] \arrow[d, two heads] & U' \arrow[r, "\tilde{g}"] \arrow[d, two heads] & V' \arrow[d, two heads] \arrow[dl, phantom, "\llcorner", very near start] \\
S \arrow[r, "f"'] & T & U \arrow[r, "g"'] & V
\end{tikzcd}$$

若令 $f' = i \circ \tilde{f}$ 且 $g' = \tilde{g}$，则

$$\begin{tikzcd}S' \arrow[r, "f'"] & U' \arrow[r, "g'"] & V'\end{tikzcd}$$

是可复合的嵌套元组态射。

4.1.3　复合算法

算法 4.1.2：可处理布局复合算法

```
输入: 可处理布局 $A$ 与 $B$。
输出: $A$ 与 $B$ 的一个弱复合（weak composite）$C$（若存在），否则 None..
------------------------------------------------------------
分别取 $A$ 与 $\mathsf{coal}(B)$ 的标准表示
    $$\begin{tikzcd}S \arrow[r, "f"] & T & U \arrow[r, "g"] & V\end{tikzcd}$$
用算法 4.1.1 求出 $(T, U)$ 的一个相互加细
    $$\begin{tikzcd}T' \arrow[r, tail] \arrow[d, two heads] & U' \arrow[d, two heads] \\ T & U\end{tikzcd}$$
若 $(T, U)$ 不存在相互加细，则返回 None。
用构造 4.1.2.1 得到可复合的嵌套元组态射
    $$\begin{tikzcd}S' \arrow[r, "f'"] & U' \arrow[r, "g'"] & V'\end{tikzcd}$$
复合 $f'$ 与 $g'$，并计算编码的布局
    $$C = L_{g' \circ f'}$$
return $C$
```

定理 4.1.3.1. 若 A 与 B 是可处理布局，则前一算法的输出 C 是 A 与 B 的一个弱复合。于是，

$$B \circ A = \mathsf{coal}(C, \mathsf{shape}(A)).$$

证明. 命题 3.2.5.15 告诉我们

$$\Phi_{L_{g'}} = \Phi_{L_g} = \Phi_{\mathsf{coal}(B)} = \Phi_B,$$

而命题 3.2.5.11 与例 3.1.3.6 告诉我们

$$\Phi_{L_{f'}} = \Phi_{L_f} = \Phi_A.$$

于是定理 3.2.6.21 蕴含

$$\begin{aligned} \Phi_C = \Phi_{L_{g' \circ f'}} &= \Phi_{g'} \circ \Phi_{f'}^{\mathsf{size}(U')} \\ &= \Phi_B \circ \Phi_A^{\mathsf{size}(B)}. \end{aligned}$$

按构造，$L_{f'}$ 的形状 S' 加细 A 的形状 S，因此我们断定 C 是 A 与 B 的一个弱复合。 □

4.1.4 例子

本节中，我们展示如何用算法 4.1.3 来计算可处理布局 A 与 B 的复合 $B \circ A$。

例 4.1.4.1. 设 $A = (4) : (1)$，$B = (2, 2) : (2, 1)$。

1. 取 A 与 $\mathsf{coal}(B) = B$ 的标准表示。

$$\begin{tikzcd} & & 2 \arrow[dr, mapsto] & 2 \\ 4 \arrow[r, mapsto, "f"'] & 4 & 2 \arrow[ur, mapsto, "g"'] & 2 \end{tikzcd}$$

2. 应用算法 4.1.1 得到相互加细

$$\begin{tikzcd} & 2 \arrow[r, dash] & 2 \\ 4 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[r, dash] & 2 \end{tikzcd}$$

3. 作图

$$\begin{tikzcd} & & 2 \arrow[r, dash] & 2 \arrow[dr, mapsto] & 2 \\ 4 \arrow[r, mapsto, "f"'] & 4 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[r, dash] & 2 \arrow[ur, mapsto, "g"'] & 2 \end{tikzcd}$$

4. 解出该图表

$$\begin{tikzcd} & 2 \arrow[r, mapsto] & 2 \arrow[dr, mapsto] & 2 \\ 4 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[r, mapsto, "f'"'] & 2 \arrow[ur, mapsto, "g'"'] & 2 \end{tikzcd}$$

5. 复合 f' 与 g'，得到

$$\begin{tikzcd} & 2 \arrow[dr, mapsto] & 2 \\ 4 \arrow[ur, dash] \arrow[r, dash] & 2 \arrow[ur, mapsto, "g' \circ f'"'] & 2 \end{tikzcd}$$

6. 计算关联的布局

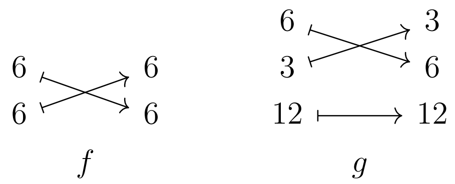
$$L_{g'\circ f'} = ((2,2)) : ((2,1)).$$

7. $L_{g'\circ f'}$ 相对于 (4) 已合并，所以

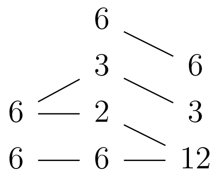
$$B \circ A = ((2,2)) : ((2,1)).$$

例 4.1.4.2. 设 $A = (6,6) : (6,1)$，$B = (12,3,6) : (1,72,12)$。

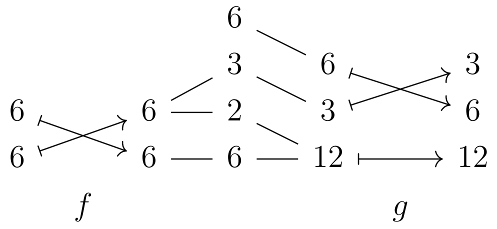
1. 取 A 与 $\mathsf{coal}(B) = B$ 的标准表示。


6
6
6
6
f
6
3
12
3
6
12
g

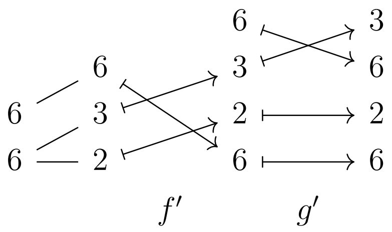
2. 应用算法 4.1.1 得到相互加细


6
3
2
6
6
6
6
3
12

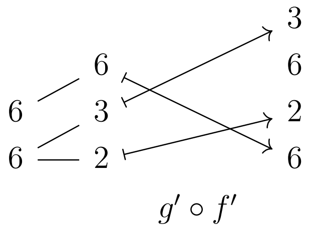
3. 作图


6
6
f
6
6
6
3
2
6
6
3
12
3
6
12
g

4. 解出该图表，得到


6
6
6
3
2
6
3
2
6
3
6
2
6
f'
g'

5. 复合 f' 与 g'，得到


3
6
6
3
6
2
6
2
6
$g' \circ f'$

6. 计算关联的布局

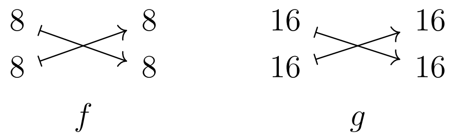
$$L_{g'\circ f'} = ((2,3),6) : ((6,72),1).$$

7. $L_{g'\circ f'}$ 相对于 $(6,6)$ 已合并，因此

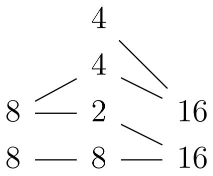
$$B \circ A = ((2,3),6) : ((6,72),1).$$

例 4.1.4.3. 设 $A = (8,8) : (8,1)$，$B = (16,16) : (16,1)$。

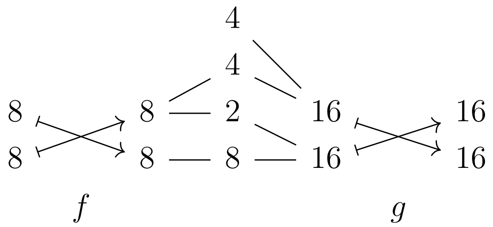
1. 取 A 与 $\mathsf{coal}(B) = B$ 的标准表示。


8
8
8
8
f
16
16
16
16
g

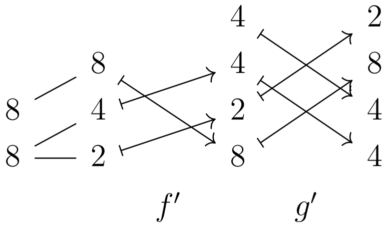
2. 应用算法 4.1.1 得到相互加细


4
4
8
2
16
8
8
16

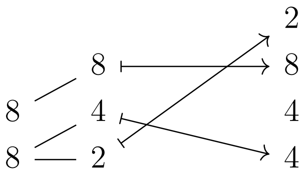
3. 作图


8
8
f
4
4
8
2
16
8
8
16
16
16
g

4. 解出该图表，得到


8
8
8
4
2
f'
4
4
2
8
2
8
4
4
g'

5. 复合 f' 与 g'，得到



$g' \circ f'$

6. 计算关联的布局

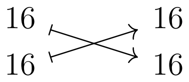
$$L_{g' \circ f'} = ((2,4),8) : ((128,1),16)$$

7. $L_{g' \circ f'}$ 相对于 $(8,8)$ 已合并，因此

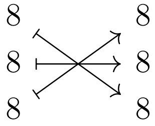
$$B \circ A = ((2,4),8) : ((128,1),16)$$

例 4.1.4.4. 设 $A = (16,16) : (16,1)$，$B = (8,8,8) : (64,8,1)$。

1. 取 A 与 $\mathsf{coal}(B) = B$ 的标准表示。

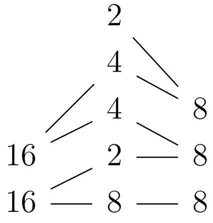


f

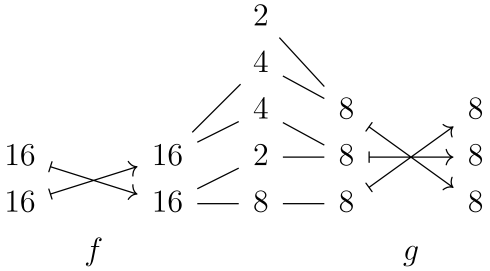


g

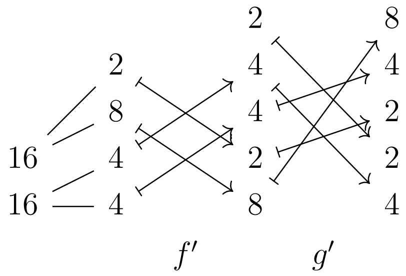
2. 应用算法 4.1.1 得到相互加细


2
4
4
2
8
16
16
8
8
8

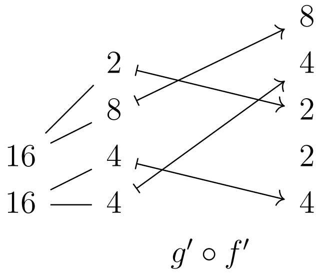
3. 作图


16
16
16
16
2
4
4
2
8
8
8
8
8
8
8
f
g

4. 解出该图表，得到


2
4
4
2
8
8
4
2
2
4
16
16
2
8
4
4
f′
g′

5. 复合 f' 与 g'，得到


16
16
2
8
4
4
8
4
2
2
4
g′ ∘ f′

6. 计算关联的布局

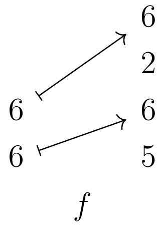
$$L_{g'\circ f'} = ((4,4),(8,2)) : ((16,1),(64,8)).$$

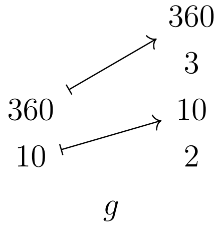
7. $L_{g'\circ f'}$ 相对于 $(16,16)$ 已合并，因此

$$B \circ A = ((4,4),(8,2)) : ((16,1),(64,8)).$$

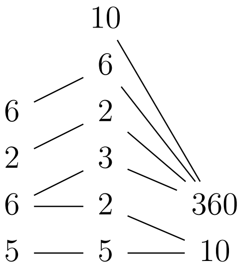
例 4.1.4.5. 设 $A = (6,6) : (5,60)$，$B = (10,360) : (2,60)$。

1. 取 A 与 $\mathsf{coal}(B) = B$ 的标准表示。

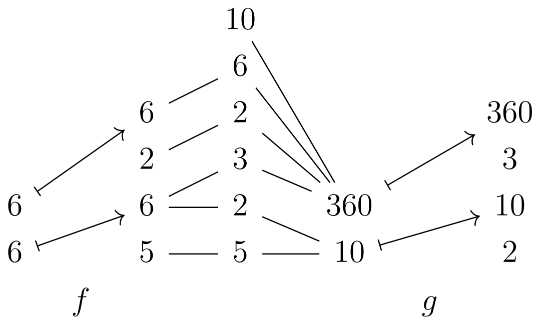



360
10
360
3
10
2
g

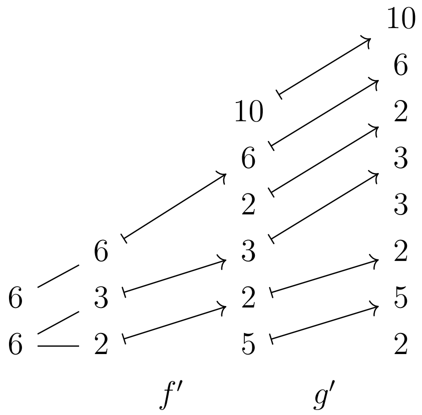
2. 应用算法 4.1.1 得到相互加细


6
2
6
5
10
6
2
3
2
5
360
10

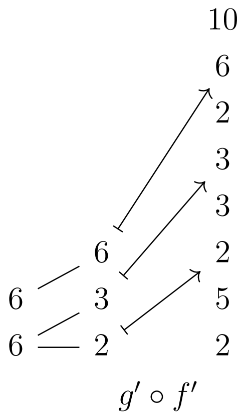
3. 作图


6
6
6
2
6
5
10
6
2
3
2
5
360
10
360
3
10
2
f
g

4. 解出该图表，得到


6
6
6
3
2
10
6
2
3
2
5
10
6
2
3
3
2
5
2
f′
g′

5. 复合 f' 与 g'，得到


6
6
6
3
2
10
6
2
3
3
2
5
2
g′ ∘ f′

6. 计算关联的布局

$$L_{g'\circ f'} = ((2,3),6) : ((10,60),360).$$

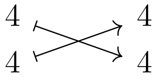
7. 布局 $L_{g'\circ f'}$ 相对于 $(6,6)$ 已合并，所以

$$B \circ A = ((2,3),6) : ((10,60),360).$$

4.1.5　更一般的复合

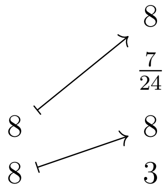
我们所发展的图形演算（graphical calculus）可以自然地推广，用来计算可处理布局 A 与任意 CuTe 布局 B 的复合 $B \circ A$。非正式地说，我们的做法是允许元组的元素取值于 $\mathbb{Q}_{>0} \supset \mathbb{Z}_{>0}$。我们用一个具体的计算例子来说明这一推广。

考虑布局 $A = (4,4):(4,1)$ 与 $B = (8,8):(3,7)$。布局 A 是可处理的，其标准表示是下图所示的元组态射 f。



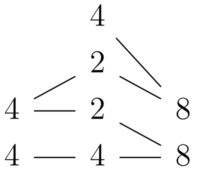
f

布局 B 不是可处理的，但我们仍可以用下图来描绘 B

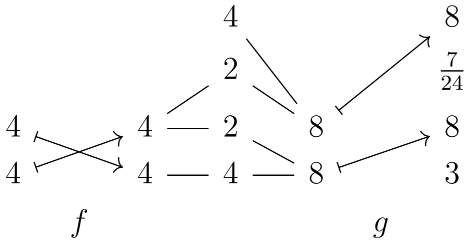


g

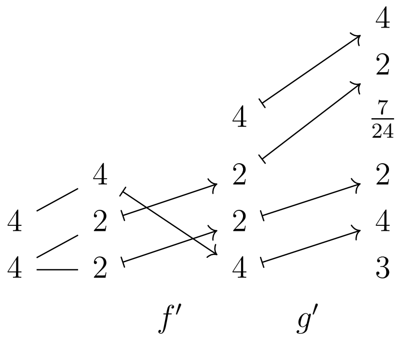
这个图并不对应于一个真正的元组态射，因为「陪域元组」$(3,8,\frac{7}{24},8)$ 含有非整数元素。然而，它仍然通过通常的前缀积（prefix product）公式编码布局 B，并且仍可作为我们复合算法的合法输入：我们可以应用算法 4.1.1 得到相互加细



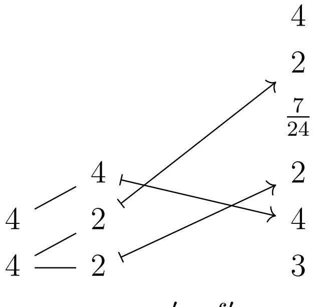
作图


4
4
4
4
4
2
2
4
8
8
8
7/24
8
3
f
g

解出该图表，得到



再复合 f' 与 g'，得到图



$g' \circ f'$

编码的布局为 $((2,2),4):((12,7),3)$，它相对于 $(4,4)$ 已合并，因此我们断定

$$B \circ A = ((2,2),4):((12,7),3).$$

4.1.6　复合可许性

在 [16] 中，作者引入了复合可许（admissibility for composition）的概念，它是布局 A 与 B 的复合 $B \circ A$ 存在的一个充分条件。我们来回顾复合可许的定义。与 [16] 一样，我们把注意力限制在不含等于 1 的形状元素的扁平布局上，并假设我们复合中的第一个布局不含等于 0 的步长。

定义 4.1.6.1. 设

$$A = (s_1, \ldots, s_m) : (d_1, \ldots, d_m)$$

是没有 $s_i = 1$ 且没有 $d_i = 0$ 的扁平布局。设 B 是满足

$$\mathsf{shape}(B) = (u_1, \ldots, u_p).$$

的扁平布局。若下列条件成立，则称 A 与 B 是复合可许的。

1. 对每个 $1 \leq i \leq m$，存在 $1 \leq k \leq \ell \leq p$ 使得
   (a) $u_1 \cdots u_{k-1}$ 整除 d_i，

   (b) d_i 整除 $u_1 \cdots u_k$（若 $k < p$，则为真整除（properly divides）），
   (c) $u_1 \cdots u_{\ell-1}$ 整除 $s_i d_i$，
   (d) $s_i d_i$ 整除 $u_1 \cdots u_\ell$（若 $\ell < p$，则为真整除）。

2. 区间

$$[d_i, d_i(s_i - 1)] \cap [1, s_1 \cdots s_{m-1})$$

   两两不交。

注 4.1.6.2. 在上面的定义中，指标 k, ℓ 在 [16] 中被称为「切分指标」(division indices)。

注 4.1.6.3. 在 [16] 中，作者使用的是布局的「扩充布局函数」(extended layout function)，它可以看作把布局的最后一个形状元素 s_m 替换为 ∞ 后所得布局的布局函数。我们这里给出的定义是采用普通布局函数工作时的恰当类似物。

引理 4.1.6.4. *设 $T = (t_1, \ldots, t_n)$ 与 $U = (u_1, \ldots, u_p)$ 是正整数元组，并设 (T, U) 容许一个相互加细。则对 T 与 U 的任意前缀积 $t_1 \cdots t_j$ 与 $u_1 \cdots u_k$，或者*

1. *$t_1 \cdots t_j$ 大于 $u_1 \cdots u_k$，或者*
2. *$t_1 \cdots t_j$ 整除 $u_1 \cdots u_k$。*

证明. 选取 (T, U) 的某个相互加细 (T', U')，并记 $(u'_1, \ldots, u'_{p'})$ 为 U' 的扁平化。T 或 U 的任何前缀积也是 U' 的前缀积，而固定正整数元组的前缀积满足 $x \leq y \Rightarrow x \mid y$，结论由此得证。 □

定理 4.1.6.5. *设 A 是不含等于 1 的形状元素、也不含等于 0 的步长元素的扁平可处理布局。设 B 是扁平可处理布局。令 $f : S \to T$ 与 $g : U \to V$ 分别表示 A 与 $\mathsf{coal}(B)$ 的标准表示。若 T 与 U 容许一个相互加细，则 A 与 B 是复合可许的。*

证明. 记 $S = (s_1, \ldots, s_m)$，$T = (t_1, \ldots, t_n)$，$U = (u_1, \ldots, u_p)$，并记 $\alpha : \langle m \rangle_* \to \langle n \rangle_*$ 为 f 所位于（lies over）其上的映射。我们需要验证定义 4.1.6.1 中的条件成立。

1. 设 $1 \leq i \leq m$。则对某个 j 有 $d_i = t_1 \cdots t_{j-1}$，即 $j = \alpha(i)$。设我们有 (T, U) 的一个相互加细 (T', U')，并记 $(T')^\flat = (t'_1, \ldots, t'_{n'})$ 与 $(U')^\flat = (u'_1, \ldots, u'_{p'})$。
   - (a) 与 (b)：由于 T' 加细 T，存在某个 $1 \leq a \leq n'$ 使得

$$d_i = t_1 \cdots t_{j-1} = t'_1 \cdots t'_a = u'_1 \cdots u'_a.$$

     取极大的 $k \in \langle p \rangle$ 使得 $u_1 \cdots u_{k-1} \leq u'_1 \cdots u'_a$。
     - 设 $k < p$。我们观察到

$$u_1 \cdots u_{k-1} \leq d_i < u_1 \cdots u_k.$$

       其中第二个不等式由 $k \in \langle p \rangle$ 的极大性得出。引理 4.1.6.4 蕴含 $u_1 \cdots u_{k-1}$ 整除 d_i，且 d_i 真整除 $u_1 \cdots u_k$。

  - 设 $k = p$。我们观察到

$$u_1 \cdots u_{k-1} \leq d_i = t_1 \cdots t_{j-1} < t_1 \cdots t_n \leq u_1 \cdots u_p = u_1 \cdots u_k.$$

    引理 4.1.6.4 蕴含 $u_1 \cdots u_{k-1}$ 整除 d_i，且 d_i 整除 $u_1 \cdots u_k$（虽为真整除，但我们并不需要这一点）。

- (c) 与 (d)：同样，由于 T' 加细 T，存在某个 $1 \leq b \leq n'$ 使得

$$s_i d_i = t_1 \cdots t_j = t'_1 \cdots t'_b = u'_1 \cdots u'_b.$$

  取极大的 $\ell \in \langle p \rangle$ 使得 $u_1 \cdots u_{\ell-1} \leq u'_1 \cdots u'_b$。

  - 设 $\ell < p$。我们观察到

$$u_1 \cdots u_{\ell-1} \leq s_i d_i < u_1 \cdots u_\ell.$$

    其中第二个不等式由 $\ell \in \langle p \rangle$ 的极大性得出。引理 4.1.6.4 蕴含 $u_1 \cdots u\ell - 1$ 整除 $s_i d_i$，且 $s_i d_i$ 真整除 $u_1 \cdots u_\ell$。

  - 设 $\ell = p$。我们观察到

$$u_1 \cdots u_{\ell-1} \leq s_i d_i = t_1 \cdots t_j \leq t_1 \cdots t_n \leq u_1 \cdots u_p = u_1 \cdots u_k.$$

    引理 4.1.6.4 蕴含 $u_1 \cdots u_{k-1}$ 整除 d_i，且 d_i 整除 $u_1 \cdots u_k$。

2. 对 $\langle m \rangle$ 中任意 $i \neq i'$，有 $d_i = t_1 \cdots t_{j-1}$，$s_i = t_j$，$d_{i'} = t_1 \cdots t_{j'-1}$，$s_{i'} = t_{j'}$，其中 $j = \alpha(i)$，$j' = \alpha(i')$。于是

$$[d_i, d_i(s_i - 1)] = [t_1 \cdots t_{j-1}, t_1 \cdots t_{j-1}(t_j - 1)]$$

   且

$$[d_{i'}, d_{i'}(s_{i'} - 1)] = [t_1 \cdots t_{j'-1}, t_1 \cdots t_{j'-1}(t_{j'} - 1)]$$

   若 $j' > j$，则

$$t_1 \cdots t_{j-1}(t_j - 1) < t_1 \cdots t_{j'-1}$$

   故这些区间互不重叠，$j < j'$ 的情形同理。

□

4.2　逻辑切分与逻辑乘积

本节展示如何利用复合(composition)算法 4.1.3 来计算逻辑切分(logical division)与逻辑乘积（logical product)。

4.2.1 逻辑切分的例子

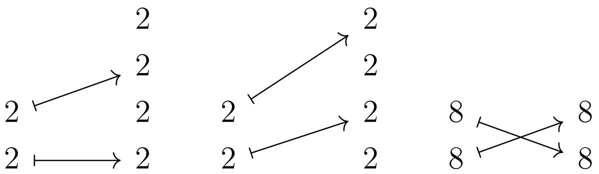
回顾：若 A 与 B 是布局（layout），则逻辑切分 $A \oslash B$ 定义为

$$A \oslash B = A \circ (B, B^c)$$

其中

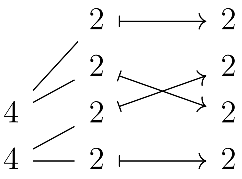
$$B^c = \mathsf{comp}(B, \mathsf{size}(A)).$$

例 4.2.1.1. 设我们想要计算逻辑切分 $A \oslash B$，其中 $A = (8, 8) : (8, 1)$，$B = (2, 2) : (1, 4)$。于是可以把它们写作 $A = L_g$、$B = L_h$ 与 $B^c = L_{h^c}$，其中 f 与 f^c 是如下所示的元组态射（tuple morphism）。


2 2 2 2
2 2
2 2 2 2
2 2
8 8
8 8

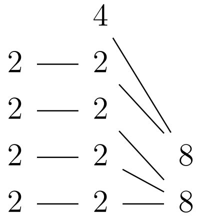
h　　h^c　　g

由此可知，(B, B^c) 由如下所示的嵌套元组态射（nested tuple morphism）$f = (h, h^c)$ 编码。

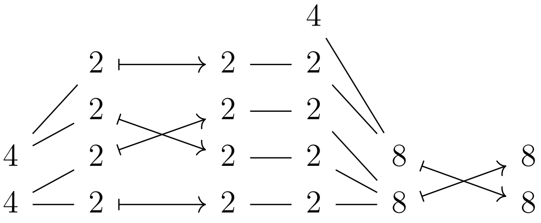

4
4
2 2 2 2
2 2 2 2

f

接下来，我们像之前那样执行复合算法。我们用算法 4.1.1 求出相互加细（mutual refinement）

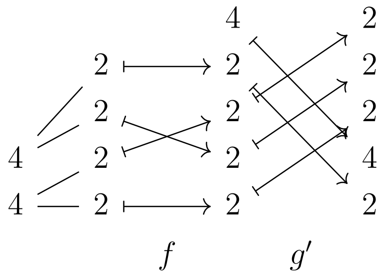

2 2 2 2
4 2 2 2 2
8
8

构成图表

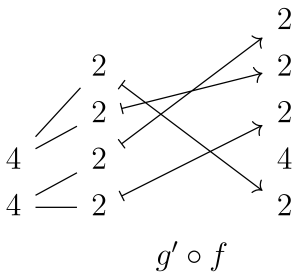

4
4
2 2 2 2
2 2 2 2
4 2 2 2 2
8
8
8
8

f　　g

解出该图表，得到



并把 f 与 g' 复合，得到



该嵌套元组态射编码的布局是

$$L_{g'\circ f} = ((2,2),(2,2)) : ((8,32),(16,1))$$

它在 $((2,2),(2,2))$ 上已合并（coalesced），因此我们断定

$$A \oslash B = ((2,2),(2,2)) : ((8,32),(16,1)).$$

4.2.2　逻辑乘积的例子

回顾：若 A 与 B 是布局，则逻辑乘积 $A \otimes B$ 定义为

$$A \otimes B = (A, A^c \circ B)$$

其中

$$A^c = \mathsf{comp}(A, \mathsf{size}(A) \cdot \mathsf{cosize}(B)).$$

特别地，若想手工计算 $A \otimes B$，只需算出 $A^c \circ B$，再把所得结果与 A 拼接（concatenate）即可。

例 4.2.2.1. 设我们想要计算逻辑乘积 $A \otimes B$，其中 $A = (2,2) : (1,2)$，$B = (5,5) : (5,1)$。于是

$$\begin{aligned} A^c &= \mathsf{comp}(A, \mathsf{size}(A) \cdot \mathsf{cosize}(B)) \\ &= \mathsf{comp}(A, 100) \\ &= (25) : (4). \end{aligned}$$

我们按上一节的方式进行。

1. 取 B 与 $\mathsf{coal}(A^c) = A^c$ 的标准表示（standard representation）。

$$\begin{tikzcd}
5 \arrow[dr, mapsto] & 5 & & 25 \\
5 \arrow[ur, mapsto] & 5 & 25 \arrow[ur, mapsto] & 4 \\
 \arrow[r, phantom, "f"] & & \arrow[r, phantom, "g"] &
\end{tikzcd}$$

2. 应用算法 4.1.1，得到相互加细

$$\begin{tikzcd}
5 \arrow[r, dash] & 5 \arrow[dr, dash] & \\
5 \arrow[r, dash] & 5 \arrow[r, dash] & 25
\end{tikzcd}$$

3. 构成图表

$$\begin{tikzcd}
5 \arrow[dr, mapsto] & 5 \arrow[r, dash] & 5 \arrow[dr, dash] & & 25 \\
5 \arrow[ur, mapsto] & 5 \arrow[r, dash] & 5 \arrow[r, dash] & 25 \arrow[ur, mapsto] & 4 \\
 \arrow[r, phantom, "f"] & & & \arrow[r, phantom, "g"] &
\end{tikzcd}$$

4. 解出该图表，得到

$$\begin{tikzcd}
& & 5 \\
5 \arrow[dr, mapsto] & 5 \arrow[ur, mapsto] & 5 \\
5 \arrow[ur, mapsto] & 5 \arrow[ur, mapsto] & 4 \\
 \arrow[r, phantom, "f'"] & \arrow[r, phantom, "g'"] &
\end{tikzcd}$$

5. 把 f' 与 g' 复合，得到

$$\begin{tikzcd}
& 5 \\
5 \arrow[r, mapsto] & 5 \\
5 \arrow[uur, mapsto] & 4 \\
 \arrow[r, phantom, "g' \circ f'"] &
\end{tikzcd}$$

6. 计算所编码的布局

$$L_{g' \circ f'} = (5, 5) : (20, 4).$$

7. 布局 $L_{g' \circ f'}$ 在 $(5, 5)$ 上已合并，因此

$$A^c \circ B = (5, 5) : (20, 4).$$

我们断定

$$A \otimes B = ((2, 2), (5, 5)) : ((1, 2), (20, 4)).$$

附录 A　范畴论入门

在本书中，我们自由地使用范畴（category）的语言；范畴是一类数学对象，它把态射（morphism）及其复合（composition）的概念抽象出来。本附录的目的，是对范畴的基础知识给出一份简明而友好的导引。特别地，我们旨在回答以下问题：

1. 什么是范畴?
2. 什么是函子（functor)?

能够自信地回答这些问题、并且脑中备有例子的读者，将能够理解本书中最重要的概念与构造。对于有意学习范畴论中更进阶概念（例如自然变换（natural transformation)、伴随（adjunction）与（余）极限（(co)limit)）的读者，我们推荐 [13]。

A.1　什么是范畴？

我们从第一个问题开始。在给出定义之前，先考虑一个具有启发性的例子。设 X 与 Y 是集合（set）。映射（function）$f : X \to Y$ 把每个元素 $x \in X$ 指派为某个元素 $f(x) \in Y$。我们把 X 称为 f 的定义域（domain），把 g 称为 Y 的陪域（codomain）。

例 A.1.0.1. 存在映射 $f : \mathbb{Z} \to \mathbb{Z}$，由

$$f(x) = 2x.$$

给出。

例 A.1.0.2. 存在映射 $g : \mathbb{Z} \to \mathsf{Bool}$，其中 $\mathsf{Bool} = \{\textbf{True}, \textbf{False}\}$，由

$$g(x) = \begin{cases} \textbf{True} & x \text{ 为偶数，} \\ \textbf{False} & x \text{ 为奇数。} \end{cases} .$$

给出。

若 $f : X \to Y$ 与 $g : Y \to Z$ 是映射，则我们可以复合 f 与 g：f 与 g 的复合（composite）是映射 $g \circ f : X \to Z$，由

$$(g \circ f)(x) = g(f(x)).$$

给出。

例 A.1.0.3. 若 f 与 g 是例 A.1.0.1 与例 A.1.0.2 中的映射，则复合 $g \circ f : \mathbb{Z} \to \mathsf{Bool}$ 由

$$(g \circ f)(x) = \mathbf{True}.$$

给出。

映射的复合满足两条本质性质。其一，复合是结合（associative）的：若 f 与 g 可复合，且 g 与 h 可复合，则

$$h \circ (g \circ f) = (h \circ g) \circ f.$$

其二，每个集合 X 都有一个恒等映射（identity function）$\mathsf{id}_X : X \to X$，由

$$\mathsf{id}_X(x) = x.$$

给出。若 $f : X \to Y$ 是任意映射，则与 id_X 预复合或与 id_Y 后复合都不改变映射 f：

$$f \circ \mathsf{id}_X = f = \mathsf{id}_Y \circ f.$$

在纯数学与应用数学中，有许多这样的情形：我们有一个由**对象**（object）组成的类（collection），以及这些对象之间的**态射**，它们具有与集合和映射相同的形式行为：态射可以按结合的方式复合，且对象拥有恒等态射。虽然集合之间的映射是其原型性的例子，但范畴中的对象不必是集合，范畴中的态射也不必是映射。我们后面会看到许多这样的例子。为了捕捉这种反复出现的结构，我们定义**范畴**的概念。

定义 A.1.0.4. 一个范畴 C 由以下内容组成：

1. 一个由对象组成的类：

   $$\mathsf{ob}(\mathsf{C}) = \{X, Y, Z, \ldots\}.$$

   这些对象可以是集合、元组（tuple）、数、向量空间、矩阵（matrix），或某种别的数学结构，取决于范畴 C。

2. 这些对象之间的态射所组成的类：

   $$\mathsf{mor}(\mathsf{C}) = \{f, g, h, \ldots\}.$$

   C 中每个态射 $f : X \to Y$ 都有一个定义域 X 和一个陪域 Y，它们都是 C 中的对象。

3. 一条复合法则（composition rule）：若 $f : X \to Y$ 与 $g : Y \to Z$ 是 C 中的态射，则存在态射

   $$g \circ f : X \to Z$$

   称为 f 与 g 的复合。C 中态射的复合是结合的，即

   $$h \circ (g \circ f) = (h \circ g) \circ f,$$

   只要各式有定义。

4. 恒等态射（identity morphism）：若 X 是 C 中的对象，则存在态射

$$\mathsf{id}_X : X \to X$$

称为 X 上的恒等态射。若 $f : X \to Y$ 是 C 中的任意态射，则

$$f \circ \mathsf{id}_X = f = \mathsf{id}_Y \circ f.$$

我们来看范畴的一些重要例子。先从前面那个具有启发性的例子开始。

例 A.1.0.5. 存在一个范畴 **Set**，其对象是集合，其态射是映射。态射的复合由函数复合给出：

$$(g \circ f)(x) = g(f(x))$$

而集合 X 上的恒等态射是恒等映射

$$\mathsf{id}_X(x) = x.$$

例 A.1.0.6. 存在一个范畴 **Vect**，其对象是向量空间 $\mathbb{R}^n$（$n \geq 0$），其态射是矩阵。具体地说，**Vect** 中的态射

$$A : \mathbb{R}^n \to \mathbb{R}^m$$

是一个 $m \times n$ 矩阵 A。**Vect** 中的复合由矩阵乘积给出：

$$B \circ A = BA,$$

而 $\mathbb{R}^n$ 上的恒等态射是 $n \times n$ 矩阵

$$\mathsf{id}_{\mathbb{R}^n} = I_n = \begin{bmatrix} 1 & 0 & \cdots & & 0 \\ 0 & 1 & & & \\ \vdots & & \ddots & & \vdots \\ & & & 1 & 0 \\ 0 & & \cdots & 0 & 1 \end{bmatrix}.$$

例 A.1.0.7. 存在一个范畴 **Div**，其对象是整数 $a \geq 1$；并且当 a 整除 b 时，其中存在唯一的态射

$$\mathsf{div}_a^b : a \to b$$

若 a 整除 b 且 b 整除 c，则 a 整除 c，这意味着我们有一条良定义（well-defined）的复合法则

$$\mathsf{div}_b^c \circ \mathsf{div}_a^b = \mathsf{div}_a^c,$$

而恒等态射

$$\mathsf{id}_a = \mathsf{div}_a^a$$

存在，因为每个正整数 a 都整除它自身。

除范畴的定义之外，还有几个重要的范畴学概念需要我们理解。例如，理解同构（isomorphism）的概念就很重要，它推广了集合的双射（bijection）这一概念。

定义 A.1.0.8. 设 C 是一个范畴，并设 $f : X \to Y$ 是 C 中的一个态射。若存在 C 中的态射 $f^{-1} : Y \to X$，使得

1. $f^{-1} \circ f = \mathsf{id}_X$，且
2. $f \circ f^{-1} = \mathsf{id}_Y$，

则称 f 为同构。

例 A.1.0.9. 在范畴 **Set** 中，同构就是双射，即这样的映射 $f : X \to Y$：对每个 $y \in Y$，存在唯一的 $x \in X$ 使得 $f(x) = y$。例如，映射 $f : \mathbb{Z} \to \mathbb{Z}$

$$f(x) = x + 10$$

是一个双射，其逆（inverse）$f^{-1} : \mathbb{Z} \to \mathbb{Z}$ 由

$$f^{-1}(x) = x - 10.$$

给出。

例 A.1.0.10. 在范畴 **Vect** 中，同构是可逆矩阵（invertible matrix）。例如，矩阵

$$A = \begin{bmatrix} 3 & 2 \\ 1 & 1 \end{bmatrix}$$

可逆，其逆为

$$A^{-1} = \begin{bmatrix} 1 & -2 \\ -1 & 3 \end{bmatrix}$$

因为

$$A^{-1}A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = AA^{-1}$$

例 A.1.0.11. 在范畴 **Div** 中，仅有的同构是恒等态射

$$\mathsf{id}_a = \mathsf{div}_a^a.$$

这是因为若 a 整除 b 且 b 整除 a，则 $a = b$。

A.2　什么是函子？

接下来，我们把注意力转向第二个问题。

定义 A.2.0.1. 设 C 与 D 是范畴。一个*函子* $F:\mathsf{C}\to\mathsf{D}$ 由以下内容组成：

1. 对 C 中的每个对象 X，指定 D 中的一个对象 FX；以及
2. 对 C 中的每个态射 $f:X\to Y$，指定 D 中的一个态射

$$Ff:FX\to FY$$

并要求满足以下性质：

1. F 与复合相容：若 f 与 g 是 C 中可复合的态射，则

$$F(g\circ f)=Fg\circ Ff.$$

2. F 与恒等态射相容：若 X 是 C 中的对象，则

$$F\mathsf{id}_X=\mathsf{id}_{FX}.$$

例 A.2.0.2. 存在一个函子 $F:\mathsf{Div}\to\mathsf{Set}$，其定义如下。在对象上，$F$ 由

$$Fa=[0,a]=\{x\in\mathbb{R}\mid 0\le x\le a\}.$$

给出；在态射上，F 由

$$F\mathsf{div}_a^b(x)=\tfrac{b}{a}\cdot x.$$

给出。我们来验证 F 是一个函子。

1. F 与复合相容：若 a 整除 b 且 b 整除 c，则

$$(F\mathsf{div}_b^c\circ F\mathsf{div}_a^b)(x)=F\mathsf{div}_b^c(F\mathsf{div}_a^b(x))=\frac{c}{b}\cdot(\frac{b}{a}\cdot x)=\frac{c}{a}\cdot x=F\mathsf{div}_a^c(x).$$

2. F 与恒等态射相容：若 $a\ge 1$，则

$$F\mathsf{id}_a(x)=F\mathsf{div}_a^a(x)=\frac{a}{a}\cdot x=\mathsf{id}_{Fa}(x).$$

Bibliography

[1] Somashekaracharya G. Bhaskaracharya et al. *Modeling Layout Abstractions Using Integer Set Relations*. 2025. arXiv: 2511.10374. URL: https://arxiv.org/abs/2511.10374.

[2] Uday Bondhugula et al. "A Practical Automatic Polyhedral Parallelizer and Locality Optimizer". In: *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation*. PLDI '08. ACM, 2008. DOI: 10.1145/1375581.1375595.

[3] Cris Cecka, Vijay Thakkar, and Tejash Shah. *CUTLASS: Principled Abstractions for Handling Multidimensional Data Through Tensors and Spatial Microkernels*. NVIDIA Technical Blog. 2025. URL: https://developer.nvidia.com/blog/cutlass-principled-abstractions-for-handling-multidimensional-data-through-tensors-and-spatial-microkernels/.

[4] Zhaodong Chen et al. "EVT: Accelerating Deep Learning Training with Epilogue Visitor Tree". In: *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Volume 3*. ACM, 2024, pp. 301–316. DOI: 10.1145/3620666.3651369.

[5] NVIDIA Corporation. *CuTe —NVIDIA CuTe Documentation*. 2024. URL: https://docs.nvidia.com/cutlass/media/docs/cpp/cute/index.html.

[6] NVIDIA Corporation. *CuTe DSL —NVIDIA CUTLASS Documentation*. 2025. URL: https://docs.nvidia.com/cutlass/media/docs/pythonDSL/cute_dsl.html.

[7] Tri Dao. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning". In: *International Conference on Learning Representations (ICLR)*. 2024.

[8] Oliver J. Sharp David F. Bacon Susan L. Graham. "Compiler Transformations for High-Performance Computing". In: *ACM Computing Surveys* 26.4 (Dec. 1994), pp. 345–420. DOI: 10.1145/197405.197406.

[9] Tobias Grosser, Armin Größlinger, and Christian Lengauer. "Polly—Performing Polyhedral Optimizations on a Low-Level Intermediate Representation". In: *Parallel Processing Letters* 22.4 (2012).

[10] Wentao Guo et al. *SonicMoE: Accelerating MoE with IO and Tile-aware Optimizations*. 2025. arXiv: `2512.14080 [cs.LG]`. URL: `https://arxiv.org/abs/2512.14080`.

[11] Yong-Jae Ju and Henry Dietz. "Reduction of Cache Coherence Overhead by Compiler Data Layout and Loop Transformation". In: *Languages and Compilers for Parallel Computing*. Ed. by Utpal Banerjee et al. Berlin, Heidelberg: Springer, 1992. DOI: `10.1007/BFb0038675`.

[12] Tamara G. Kolda and Brett W. Bader. "Tensor Decompositions and Applications". In: *SIAM Review* 51.3 (2009), pp. 455–500. DOI: `10.1137/07070111X`. URL: `https://doi.org/10.1137/07070111X`.

[13] Saunders Mac Lane. *Categories for the Working Mathematician*. 2nd ed. Vol. 5. Graduate Texts in Mathematics. Springer, 1998. ISBN: 978-0-387-98403-0.

[14] OpenAI. *Linear Layouts in Triton*. 2025. URL: `https://github.com/triton-lang/triton/blob/main/include/triton/Tools/LinearLayout.h`.

[15] Easwaran Raman, Robert Hundt, and Sandya Mannarswamy. "Structure Layout Optimization for Multithreaded Programs". In: *Proceedings of the International Symposium on Code Generation and Optimization*. CGO '07. Washington, DC, USA: IEEE Computer Society, 2007, pp. 271–282. DOI: `10.1109/CGO.2007.18`.

[16] Jay Shah. *A Note on the Algebra of CuTe Layouts*. Tech. rep. Colfax Research, Jan. 2024. URL: `https://research.colfax-intl.com/wp-content/uploads/2024/01/layout_algebra.pdf`.

[17] Jay Shah, Paul VanKoughnett, and Ryo Asai. *CUTLASS Tutorial: GEMM with Thread Block Clusters on NVIDIA Blackwell GPUs*. Colfax Research. 2025. URL: `https://research.colfax-intl.com/cutlass-tutorial-gemm-with-thread-block-clusters-on-nvidia-blackwell-gpus/`.

[18] Jay Shah, Paul VanKoughnett, and Ryo Asai. *CUTLASS Tutorial: Persistent Kernels and Stream-K*. Colfax Research. 2024. URL: `https://research.colfax-intl.com/cutlass-tutorial-persistent-kernels-and-stream-k/`.

[19] Jay Shah, Paul VanKoughnett, and Ryo Asai. *CUTLASS Tutorial: Sub-byte GEMM on NVIDIA Blackwell GPUs*. Colfax Research. 2025. URL: `https://research.colfax-intl.com/cutlass-tutorial-sub-byte-gemm-on-nvidia-blackwell-gpus/`.

[20] Jay Shah, Paul VanKoughnett, and Ryo Asai. *CUTLASS Tutorial: Writing GEMM Kernels Using Tensor Memory For NVIDIA Blackwell GPUs*. Colfax Research. 2025. URL: https://research.colfax-intl.com/cutlass-tutorial-writing-gemm-kernels-using-tensor-memory-for-nvidia-blackwell-gpus/.

[21] Jay Shah et al. "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2024.

[22] Kamal Sharma et al. "Data Layout Optimization for Portable Performance". In: *European Conference on Parallel Processing*. Euro-Par 2015. Berlin, Heidelberg: Springer, 2015. DOI: 10.1007/978-3-662-48096-0_20.

[23] Yang Shi et al. "Tensor Contractions with Extended BLAS Kernels on CPU and GPU". In: *arXiv preprint* (2016). eprint: arXiv:1606.05696. URL: https://arxiv.org/abs/1606.05696.

[24] Brandon Sun et al. *Achieve CUTLASS C++ Performance with Python APIs Using CuTe DSL*. NVIDIA Technical Blog. Nov. 2025. URL: https://developer.nvidia.com/blog/achieve-cutlass-c-performance-with-python-apis-using-cute-dsl/.

[25] Vijay Thakkar et al. *CUTLASS 3.x: Orthogonal, Reusable, and Composable Abstractions for GEMM Kernel Design*. NVIDIA Technical Blog. 2025. URL: https://developer.nvidia.com/blog/cutlass-3-x-orthogonal-reusable-and-composable-abstractions-for-gemm-kernel-design/.

[26] Arun Thangamani, Vincent Loechner, and Stéphane Genaud. "A Survey of General-purpose Polyhedral Compilers". In: *ACM Transactions on Architecture and Code Optimization* 21.4 (2024), pp. 1–26. DOI: 10.1145/3674735.

[27] Nicolas Vasilache et al. "Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions". In: *arXiv preprint arXiv:1802.04730* (2018). Facebook AI Research Technical Report.

[28] Sven Verdoolaege. "isl: An Integer Set Library for the Polyhedral Model". In: *Mathematical Software – ICMS 2010*. Ed. by Komei Fukuda et al. Vol. 6327. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2010. DOI: 10.1007/978-3-642-15582-6_49.

[29] Sven Verdoolaege. *Presburger Formulas and Polyhedral Compilation*. Technical Report / Tutorial. Version v0.02-13-g53eb23d, August 21, 2021. Polly Labs and KU Leuven, 2021.

[30] Zhiying Xu et al. “ALT: Breaking the Wall between Data Layout and Loop Optimizations for Deep Learning Compilation”. In: *Proceedings of the Eighteenth European Conference on Computer Systems*. EuroSys ’23. New York, NY, USA: Association for Computing Machinery, 2023. DOI: 10.1145/3552326.3587440.

[31] Tuowen Zhao et al. “Polyhedral Specification and Code Generation of Sparse Tensor Contraction with Co-Iteration”. In: *ACM Transactions on Architecture and Code Optimization* 20.1 (Dec. 2022). Article 16, pp. 1–26. DOI: 10.1145/3566054.

[32] Keren Zhou et al. “Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using F2”. In: *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. ASPLOS ’26. Pittsburgh, PA, USA: ACM, Mar. 2026, pp. 1–18. DOI: 10.1145/3760250.3762221.